The OpenShift Platform Engineer's Interview Book

250 questions, answered the way a senior engineer actually talks - structured from first principles to architecture

OpenShift 4.x · Kubernetes · RHCOS · ACM 2.x · Ansible · HPC · HA/DR

Edition V5 - restructured, expanded and humanised

Reading route: seventeen parts, three gears

Parts are ordered so each one only depends on the ones before it. Inside every part the questions climb from Foundation to Architect, so you can stop at your level and come back later.

Gear	Parts	What you are building	Stop when you can
Basics	1-4	Linux and container primitives, the control plane, workload objects, scheduling and resources	Explain what happens between oc apply and a running pod without notes
Intermediate	5-11	Networking, storage, security, operators, cluster lifecycle and observability	Trace a request end to end and read a failure from the owning controller's conditions
Senior	12-17	Troubleshooting under pressure, HA and DR, HPC, automation, fleet operations and architecture	Run an incident, defend a design trade-off and describe prevention, not just a fix

Every question carries a spoken answer, an analogy, production context, a step-by-step procedure, the evidence you would quote, the red flag that costs you the offer, and the follow-up question the interviewer is already holding.

Contents

250 questions across 17 parts. Parts are ordered so that nothing depends on a concept you have not met yet, and inside each part the questions climb from Foundation to Architect.

Read this page first

This book exists because most interview prep fails in the same way: you can recite the definition, the interviewer asks "and how would you prove that?", and the conversation stops. Definitions are table stakes. What gets scored is whether you can name the owning controller, quote the evidence, choose the smallest safe action, and say what you would change afterwards so it never happens again.

So every one of the 250 questions here is answered in six layers. Read the first layer if you are short on time. Read all six if this is a topic you would rather not be surprised by.

How every answer in this book is built

Six layers per question. Under pressure you speak layers 1-2; when the interviewer digs, you already have layers 3-6 loaded.

1. Say this	The 45-90 second spoken answer. Plain words, no padding, leads with the point.
2. Think of it like	One analogy that makes the mechanism obvious and buys you thinking time.
3. Why it matters	The production context: what breaks, who notices, what it costs.
4. Step by step	The ordered procedure or diagnostic path you would actually follow.
5. Evidence	The commands and signals that prove your claim instead of asserting it.
6. Red flag	The answer that quietly loses the offer, plus the follow-up you should expect.

Levels, and why they are marked

Each question is tagged Foundation, Intermediate, Senior or Architect. The tag is not about how hard the topic is - it is about how deep the expected answer goes. A Foundation question about etcd quorum and an Architect question about etcd quorum are the same subject scored on completely different scales.

The answer ladder: what each level actually sounds like

Same question - "What is a PodDisruptionBudget?" - answered four ways. Interviewers score the rung you land on, not the number of facts you list.

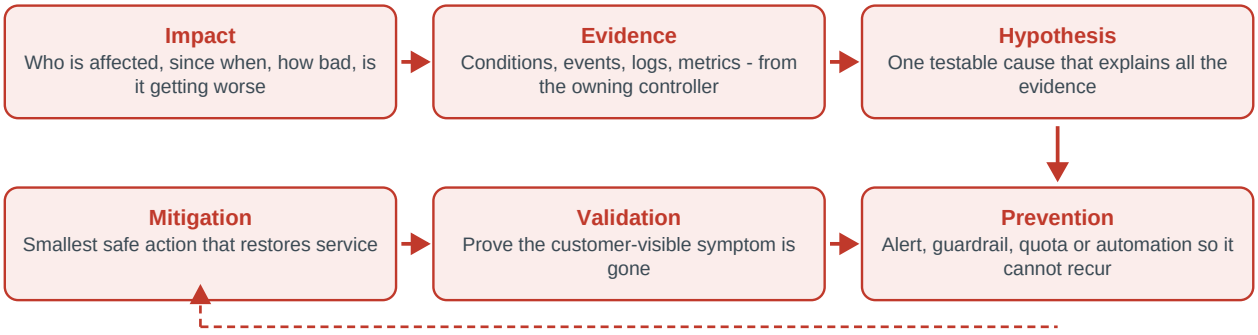
Foundation	**Names the object** "A PDB controls how many pods can be disrupted." Correct, forgettable, scores one point.
Intermediate	**Explains the mechanism** "It caps voluntary disruptions so a minimum number of replicas stays available during a drain."
Senior	**Adds failure modes and evidence** "It only covers voluntary eviction, not node crashes. I check the selector and healthy replica count before maintenance, because a badly sized PDB blocks drains and stalls MCO rollout."
Architect	**Adds trade-off, standard and prevention** "I pair PDBs with replica minimums, topology spread and tested maintenance headroom, and I validate them in a game day so the first real drain is not the first test."

Two frameworks that carry most of the interview

Roughly a third of a senior platform interview is troubleshooting and another third is design. If you only memorise two things from this book, memorise these two loops. They give you a structure to speak into when a question catches you cold, which is worth more than any individual fact.

The evidence-first loop - use this for every troubleshooting question

Say these six words out loud before you say anything technical. Candidates who jump straight to "I would restart the pod" lose the point even when restarting works.



For design and architecture questions, swap the loop for: Requirement, Options, Trade-off, Decision, Risk, Verification. Say the requirement back before you propose anything - it is the single fastest way to sound like someone who has shipped a platform rather than read about one.

The scorecard your interviewer is probably using

Score yourself 0-3 on each row after every mock answer. Anything below 2 is where your next hour of study should go.

Dimension	What a 3 sounds like
Correctness	Names the right object, the owning controller and the control loop involved
Evidence	Quotes conditions, events, metrics or a specific command instead of asserting
Safety	Controls blast radius, uses supported procedures, says what could go wrong
Validation	Checks the customer-visible symptom, not just that a resource is green
Prevention	Adds an alert, guardrail or automation so the incident cannot repeat
Communication	States impact and uncertainty plainly, and flags trade-offs without hedging

A three-day, twenty-hour run

If you have three days, do not read this book front to back at a constant speed. Spend day one on Parts 1 to 6 for breadth and to find your weak spots. Spend day two on Parts 7 to 13, which is where most senior interviews actually live: storage, security, lifecycle, observability, troubleshooting and recovery. Spend day three on Parts 14 to 17 plus spoken rehearsal - answer thirty questions out loud, timed, without notes.

In the final hour, stop reading new material. Re-read only the red flags and the analogies. The red flags stop you from losing points you have already earned, and the analogies are what you will reach for when you are asked something you did not prepare.

Honest scope note

The healthcare and enterprise examples in this book are illustrative. They are not descriptions of any specific customer environment or confidential architecture. Product behaviour is described against current OpenShift 4.x and ACM 2.x documentation; a real project may run an earlier supported or EUS release. In the interview, state the principle first, then ask which version and infrastructure provider the project runs on. That question reads as experience, not ignorance.

PART 1 · Q1-Q14 · 14 QUESTIONS

Linux, containers and the Kubernetes idea

The layer everything else sits on - and the layer most candidates skip

Almost every hard OpenShift problem turns out to be a Linux problem wearing a Kubernetes costume. A pod that will not start is a runtime, image or SELinux problem. A pod that is slow is a cgroup, NUMA or filesystem problem. Interviewers know this, which is why a surprising number of senior interviews open with something that sounds basic - "what actually is a container?" - and then quietly measure how deep you can go before you run out of road. This part builds that floor properly so the rest of the book has something to stand on.

Level mix - Foundation: 8 · Intermediate: 4 · Senior: 1 · Architect: 1

A container is a Linux process wearing five costumes

There is no "container" object in the kernel. Every isolation feature you see is a separate Linux mechanism, which is exactly why container problems are usually Linux problems.

Namespaces	What the process can see: PID, network, mount, UTS, IPC, user
cgroups v2	What the process can use: CPU shares and quota, memory limit, IO and PID caps
Union filesystem	What the process reads: stacked read-only image layers plus a thin writable layer
SELinux + seccomp	What the process may do: MCS labels on files, syscall filtering, capability set
Image + registry	What the process is: a manifest of layers addressed by an immutable digest
Pod (Kubernetes)	The shared unit: one network namespace and shared volumes for one or more containers

Q1 FOUNDATION

What actually is a container?

SAY THIS

A container is just a normal Linux process that the kernel has been told to isolate. There is no container object in the kernel. Namespaces decide what the process can see, cgroups decide what it can consume, a union filesystem gives it its own root filesystem from stacked image layers, and SELinux, seccomp and capabilities decide what it is allowed to do. Package those together and you get something that feels like a lightweight machine but is really one process tree sharing the host kernel.

THINK OF IT LIKE

It is an open-plan office, not a separate building. Everyone shares the same air conditioning - the kernel - but each team gets its own desk, its own filing cabinet and a badge that only opens certain doors.

WHY IT MATTERS IN PRODUCTION

This matters the moment something misbehaves. Because containers share the host kernel, a kernel-level problem - an exhausted PID limit, a saturated inode table, a conntrack table that is full - shows up as several unrelated pods failing on the same node at once. If you think of a container as a tiny virtual machine you will keep looking inside the pod, and you will keep missing the node.

STEP BY STEP

1. Name the four mechanisms out loud: namespaces, cgroups, union filesystem, security context.
2. Say what each one buys you: visibility, consumption, filesystem, permitted actions.

3 Point out the shared kernel, because that is where the failure modes and the security boundary questions come from.
4 Land it with an operational consequence - node-level limits are shared, so one greedy pod can hurt its neighbours unless requests and limits are set.

EVIDENCE YOU WOULD QUOTE

```
oc debug node/<node> -- chroot /host crictl ps

cat /proc/<pid>/cgroup ; ls -l /proc/<pid>/ns

oc adm top node ; oc describe node <node> | sed -n '/Allocated/,$p'
```

RED FLAG - DO NOT SAY THIS

Do not say "a container is a lightweight VM". It is the fastest way to signal that you have used containers but never debugged one from the node.

EXPECT THIS FOLLOW-UP

So where exactly is the security boundary, and why is that weaker than a VM?

Q2 FOUNDATION

How is a container different from a virtual machine?

SAY THIS

A VM virtualises hardware and runs its own kernel, so the isolation boundary is the hypervisor. A container virtualises the operating system view and shares the host kernel, so the boundary is kernel features. That makes containers start in milliseconds and cost almost nothing in memory overhead, but it also means a kernel vulnerability is a shared risk. In practice you choose VMs when you need a different kernel or a hard security boundary, and containers when you need density and fast, repeatable deployment.

THINK OF IT LIKE

A VM is a detached house with its own boiler. A container is a flat in a block - cheaper, faster to move into, but if the building's boiler fails, everyone is cold.

WHY IT MATTERS IN PRODUCTION

The trade-off shows up in real designs. Multi-tenant clusters running untrusted code often keep hard tenancy at the cluster or node level rather than relying on namespaces alone, because a container escape crosses namespaces but a hypervisor escape is a different class of problem. It is also why OpenShift Virtualization exists: some workloads genuinely need a kernel of their own and you should not pretend otherwise.

STEP BY STEP

1 State the boundary difference first: hypervisor versus shared kernel.
2 Give the practical consequence: startup time and density versus isolation strength.
3 Name when you would still choose a VM - legacy kernel modules, untrusted tenants, compliance requirements for hard isolation.
4 Mention that OpenShift can run both, so the answer is a placement decision rather than a religious one.

EVIDENCE YOU WOULD QUOTE

```
oc get nodes -o wide # kernel and container runtime version per node

oc get kubevirt -A # is OpenShift Virtualization in play for VM workloads
```

RED FLAG - DO NOT SAY THIS

Do not claim containers are "just as isolated as VMs". Interviewers hear that as a security answer you have not thought through.

EXPECT THIS FOLLOW-UP

Where would you draw the tenancy boundary in a cluster shared by three business units?

Q3 FOUNDATION

What is a pod, and why not just schedule containers?

SAY THIS

A pod is the smallest thing Kubernetes will schedule, and it is a group of containers that share a network namespace and can share volumes. Sharing the network namespace means the containers reach each other on localhost and present a single pod IP to the cluster. That gives you the sidecar pattern - a proxy, a log shipper, a credential helper - without rebuilding the application image. Kubernetes schedules pods rather than containers because co-located helpers must land on the same node to be useful at all.

THINK OF IT LIKE

A pod is a taxi, not a passenger. Everyone in the taxi shares the same address and arrives at the same time; you book the taxi, not each individual seat.

WHY IT MATTERS IN PRODUCTION

The shared namespace is also a shared fate. If an init container never completes, nothing else in that pod starts. If one container in the pod eats the memory limit, the whole pod can be OOM-killed. And because the pod IP is per-pod rather than per-container, two containers in the same pod cannot both bind port 8080 - a genuinely common cause of a sidecar that will not start.

STEP BY STEP

1. Define the pod as the scheduling unit, not as "a wrapper around a container".
2. Explain the shared network namespace and shared volumes, and what they enable.
3. Give one concrete pattern - sidecar or init container - and why it needs co-location.
4. Close with the shared-fate consequence, which is what senior candidates add and juniors miss.

EVIDENCE YOU WOULD QUOTE

```
oc get pod <pod> -o jsonpath='{.status.podIP}{"\n"}'

oc describe pod <pod> | sed -n '/Init Containers/,/Conditions/p'

oc logs <pod> -c <container> --previous
```

RED FLAG - DO NOT SAY THIS

Do not describe a pod as "one container". You will be asked about sidecars within thirty seconds and the answer will unravel.

EXPECT THIS FOLLOW-UP

An init container is stuck. How do you tell whether it is the image, the config or a dependency?

Q4 FOUNDATION

What is a container image, and why do we care about digests instead of tags?

SAY THIS

An image is a manifest plus a set of read-only filesystem layers, all content-addressed. A tag such as latest is a mutable pointer - someone can move it - whereas a digest is a SHA-256 hash of the manifest and can only ever refer to one exact set of bytes. In production I pin by digest so that what I tested is what runs, and so an incident investigation can say with certainty which code was on the node.

THINK OF IT LIKE

A tag is a nickname and a digest is a fingerprint. Nicknames get reassigned; fingerprints do not.

WHY IT MATTERS IN PRODUCTION

This is not academic. The classic outage is a Deployment with image:app:latest that has run fine for six weeks, a node reboots, the image is re-pulled, and a newer latest arrives that nobody approved. Pinning by digest turns that from an outage into a pull request. It is also the foundation of everything else in supply chain security - signing, SBOMs and admission policy all key off an immutable reference.

STEP BY STEP

1. Describe the structure: manifest, layers, content addressing.
2. Contrast the mutable tag with the immutable digest.
3. Give the failure story - a re-pull silently changing the running version.
4. Connect it forward to signing and admission control, which only work on stable references.

EVIDENCE YOU WOULD QUOTE

```
oc get pod <pod> -o jsonpath='{.status.containerStatuses[*].imageID}{"\n"}'

skopeo inspect docker://registry.example.com/app:1.4 | head -20

oc get is <stream> -o jsonpath='{.spec.tags[*].referencePolicy.type}'
```

RED FLAG - DO NOT SAY THIS

Do not defend using latest in production because "it is convenient". Convenience is the reason the outage was hard to explain.

EXPECT THIS FOLLOW-UP

How would you enforce digest pinning across every namespace in the cluster?

Q5 FOUNDATION

What do namespaces and cgroups actually do?

SAY THIS

Linux namespaces partition what a process can see - its own PID tree, network stack, mount table, hostname and users. Cgroups, version 2 on RHCOS, partition what a process can consume: CPU weight and quota, a memory limit, IO throughput and a PID ceiling. Namespaces are about visibility, cgroups are about accounting and enforcement. Kubernetes requests and limits are ultimately just values written into cgroup files.

THINK OF IT LIKE

Namespaces are the walls between offices; cgroups are the electricity meter and the fuse. One stops you seeing the neighbours, the other stops you tripping the building.

WHY IT MATTERS IN PRODUCTION

Knowing this collapses two mysteries into one. "Why was my pod killed with exit code 137?" is the memory cgroup enforcing a hard limit - the kernel OOM killer, not Kubernetes, made that decision. "Why is my app slow when CPU usage looks low?" is CPU quota throttling within the cgroup period. Both answers live in cgroup accounting, and both are invisible if you only look at pod status.

STEP BY STEP

1. Separate the two concepts cleanly: visibility versus consumption.
2. Name a couple of namespace types and a couple of cgroup controllers so it is concrete.
3. Map Kubernetes concepts onto them - requests to CPU weight, limits to quota and memory caps.
4. Finish with the two symptoms they explain: OOMKilled and CPU throttling.

EVIDENCE YOU WOULD QUOTE

```
oc debug node/<node> -- chroot /host cat /sys/fs/cgroup/kubepods.slice/.../memory.max

container_cpu_cfs_throttled_seconds_total # Prometheus

oc describe pod <pod> | grep -A3 'Last State'
```

RED FLAG - DO NOT SAY THIS

Do not say Kubernetes "kills the pod when it uses too much memory". The kernel does it, and the distinction is exactly what the question is testing.

EXPECT THIS FOLLOW-UP

A container is being OOMKilled but node memory looks fine. What is happening?

Q6 FOUNDATION

What does the kubelet do?

SAY THIS

The kubelet is the node's agent. It watches the API server for pods bound to its node, asks the container runtime to create the sandbox and containers, mounts volumes, pulls images using the node's pull secrets, runs the probes, and reports node and pod status back. It does not decide where pods go - that is the scheduler - and it does not create pods for Deployments. It is the thing that turns a scheduling decision into running processes.

THINK OF IT LIKE

The kubelet is the site foreman. Head office decides what gets built and where; the foreman makes it happen on that plot and phones in progress.

WHY IT MATTERS IN PRODUCTION

Most "the node is broken" incidents are kubelet-shaped. If the kubelet cannot reach the API server, the node goes NotReady even though its pods are still running and serving traffic. If the kubelet's disk fills, it starts evicting pods under DiskPressure. Knowing which decisions belong to the kubelet stops you from restarting a control-plane component when the problem was local certificate expiry or a full /var.

STEP BY STEP

1. State its scope: everything on this node, nothing about placement.
2. List the responsibilities in order - watch, sandbox, volumes, images, probes, status.
3. Name the boundary explicitly: the scheduler binds, the kubelet executes.
4. Add the failure signature - a kubelet that cannot talk to the API server produces NotReady while workloads may still be serving.

EVIDENCE YOU WOULD QUOTE

```
oc debug node/<node> -- chroot /host journalctl -u kubelet -n 200 --no-pager

oc get node <node> -o jsonpath='{.status.conditions}' | python3 -m json.tool

oc adm node-logs <node> --path=kubelet/
```

RED FLAG - DO NOT SAY THIS

Do not say the kubelet schedules pods. It is a small slip that tells the interviewer you have not read the control flow.

EXPECT THIS FOLLOW-UP

A node is NotReady but its pods are still answering traffic. What do you check first?

Q7 FOUNDATION

What is CRI-O and where does it sit?

SAY THIS

CRI-O is the container runtime OpenShift ships, and it exists to do exactly what the Kubernetes Container Runtime Interface asks and nothing more. The kubelet speaks CRI to CRI-O, CRI-O pulls images, sets up the sandbox, and calls runc or crun to start the actual process. Its deliberately narrow scope is the point - it tracks Kubernetes releases and carries no daemon-level features you did not ask for.

THINK OF IT LIKE

CRI-O is a specialist contractor with one trade licence. It does not try to also be the architect, the delivery service and the site office the way a general-purpose container daemon does.

WHY IT MATTERS IN PRODUCTION

You touch CRI-O directly when the kubelet's view and reality disagree - a pod that shows as running but has no process, an image pull that fails only on one node, a sandbox that will not delete. That is when you drop to the node and use crictl, which speaks the same CRI socket. Being comfortable saying "I would check with crictl on the node" is a genuine seniority signal.

STEP BY STEP

1. Place it in the chain: kubelet, CRI, CRI-O, runc or crun, Linux process.
2. Explain the design intent - minimal, Kubernetes-versioned, no extra daemon surface.
3. Say when you would inspect it directly rather than through the API.
4. Name the tool: crictl, from oc debug node.

EVIDENCE YOU WOULD QUOTE

```
oc debug node/<node> -- chroot /host crictl ps -a

oc debug node/<node> -- chroot /host crictl images | head

oc debug node/<node> -- chroot /host journalctl -u crio -n 100 --no-pager
```

RED FLAG - DO NOT SAY THIS

Do not say "OpenShift uses Docker". It has not for many major versions and it dates you immediately.

EXPECT THIS FOLLOW-UP

An image pulls on three nodes and fails on the fourth. Where do you look?

Q8 FOUNDATION

What is Kubernetes actually doing, in one sentence?

SAY THIS

Kubernetes stores your desired state and runs control loops that continuously try to make observed state match it. You do not tell it to start a container; you declare that three replicas should exist, and a controller notices the gap and closes it. Everything else - self-healing, rolling updates, autoscaling - is that same loop applied to a different object. Once you see the pattern, unfamiliar objects stop being scary because they all behave the same way.

THINK OF IT LIKE

It is a thermostat, not a light switch. You set twenty-one degrees and walk away; the thermostat keeps checking and correcting forever.

WHY IT MATTERS IN PRODUCTION

This is the single most load-bearing idea in the whole interview. It tells you where to look when something is wrong: find the controller that owns the object and read its status conditions, because that controller is writing down exactly why it cannot reach the desired state. It also explains why manually deleting a pod is usually pointless - the loop will just recreate it, and you have destroyed your evidence.

STEP BY STEP

1. State the loop: desired state in etcd, controller observes, controller acts, status reported.
2. Give one worked example - three replicas, one node dies, ReplicaSet creates a replacement.
3. Draw the operational conclusion: read conditions on the owner, not just pod status.
4. Add the anti-pattern: deleting pods to fix things destroys evidence and fixes nothing.

EVIDENCE YOU WOULD QUOTE

```
oc get deploy <name> -o jsonpath='{.status.conditions}' | python3 -m json.tool

oc get events -n <ns> --sort-by=.lastTimestamp | tail -20

oc get <kind> <name> -o yaml | sed -n '/status:/,$p'
```

RED FLAG - DO NOT SAY THIS

Do not describe Kubernetes as "a tool that runs containers". Every follow-up question in the interview is really about reconciliation.

EXPECT THIS FOLLOW-UP

A Deployment says 3/3 ready but users get errors. Where has the loop misled you?

Q9 INTERMEDIATE

What does OpenShift add on top of Kubernetes?

SAY THIS

OpenShift is Kubernetes with the operational problem solved rather than left as an exercise. It adds a lifecycle-managed release - the Cluster Version Operator drives a tested payload of Cluster Operators - plus node configuration through the Machine Config Operator, integrated OAuth and identity, Routes and a supported ingress layer, Security Context Constraints that make restricted-by-default real, an integrated registry and build system, and monitoring and logging that are part of the product rather than a project you assemble.

THINK OF IT LIKE

Upstream Kubernetes is an engine. OpenShift is the certified car around it - the engine is the same, but somebody has already solved brakes, airbags, servicing intervals and who you call when it stops.

WHY IT MATTERS IN PRODUCTION

The reason this matters operationally is supportability. Because the release is a single digest-addressed payload, an upgrade moves a known set of components together and the cluster can tell you whether it is Upgradeable. Because node config is declarative through MCO, a node that drifts gets corrected rather than becoming a snowflake. Those two properties are what make a fleet of clusters manageable by a small team.

STEP BY STEP

1. Start with the framing: same Kubernetes API, plus lifecycle and enterprise defaults.
2. Group the additions - lifecycle, node config, identity, ingress, security defaults, developer tooling, observability.
3. Say why each grouping exists operationally rather than listing product names.
4. Finish with supportability: a tested payload and declarative nodes are what make fleets possible.

EVIDENCE YOU WOULD QUOTE

```
oc get clusterversion ; oc get co

oc get mcp ; oc get scc

oc get oauth cluster -o yaml | head -30
```

RED FLAG - DO NOT SAY THIS

Do not answer "OpenShift is Kubernetes with a nicer UI". The console is the least interesting thing on the list.

EXPECT THIS FOLLOW-UP

Which of those additions would you miss most on day two, and why?

Q10 INTERMEDIATE

How does administering RHCOS differ from administering RHEL?

SAY THIS

RHCOS is immutable and machine-managed. You do not log in and yum install; the filesystem is largely read-only, packages are layered with rpm-ostree, first boot is configured by Ignition, and ongoing configuration comes from MachineConfig objects rendered and applied by the Machine Config Operator. The node is treated as cattle: if it drifts or breaks, you replace it through the Machine API rather than repairing it by hand.

THINK OF IT LIKE

RHEL is a house you renovate. RHCOS is a hotel room - you change the booking, not the wallpaper, and housekeeping resets anything you left behind.

WHY IT MATTERS IN PRODUCTION

The practical consequence is that any manual node change is temporary and invisible to the next person. Someone edits sysctl by hand, the node reboots into a rendered MachineConfig, and the fix vanishes at the worst possible moment. Every durable node change - kernel arguments, chrony, extra certificates, kubelet config - has to become a MachineConfig or a PerformanceProfile so it survives reboots and applies to every node in the pool.

STEP BY STEP

1. State the principle: immutable, declaratively configured, replaceable.
2. Name the mechanisms - Ignition at first boot, MachineConfig plus MCO afterwards, rpm-ostree for layering.
3. Explain the operational rule: no durable change without a MachineConfig.
4. Add the debugging path - oc debug node gives you a shell without making the change durable.

EVIDENCE YOU WOULD QUOTE

```
oc debug node/<node> -- chroot /host rpm-ostree status

oc get mc ; oc get mcp -o wide

oc debug node/<node> -- chroot /host journalctl -u machine-config-daemon -n 100 --no-pager
```

RED FLAG - DO NOT SAY THIS

Do not describe SSHing to nodes to fix things as your normal workflow. It is the clearest signal that you would create snowflakes in a fleet.

EXPECT THIS FOLLOW-UP

You need a custom kernel argument on GPU nodes only. Walk me through it.

Q11 INTERMEDIATE

How do you troubleshoot at the node level in OpenShift?

SAY THIS

I use oc debug node to get a privileged pod on the node, chroot to /host, and then work with familiar Linux tools: journalctl for kubelet and CRI-O, crictl for the runtime's own view, df and lsblk for disk, ip and ss for networking, and dmesg for kernel events such as OOM kills. I prefer this to SSH because it goes through the API, is audited, and works identically on every node without managing keys.

THINK OF IT LIKE

It is the difference between having a key cut for the building and signing in at reception. Both get you inside; only one leaves a record and works when you change buildings.

WHY IT MATTERS IN PRODUCTION

The order you look in matters more than the commands. Node problems cluster into four families - disk, memory, runtime and network - and each has a fast test. DiskPressure shows in df and in the node conditions. Memory shows in dmesg as OOM kills. Runtime shows as crictl and kubelet disagreeing. Network shows as failing DNS or a hung endpoint from the node itself. Running those four checks takes two minutes and usually ends the guessing.

STEP BY STEP

1. Read the node conditions first - the kubelet has often already told you which family it is.
2. Get a shell: oc debug node/<node>, then chroot /host.
3. Run the four fast checks - disk, kernel messages, runtime state, node-local network.

4 Correlate with events and metrics before changing anything, and capture must-gather if it may become a support case.

EVIDENCE YOU WOULD QUOTE

```
oc get node <node> -o jsonpath='{.status.conditions}' | python3 -m json.tool

oc debug node/<node> -- chroot /host sh -c 'df -h /var; dmesg -T | tail -30'

oc debug node/<node> -- chroot /host crictl pods --state notready
```

RED FLAG - DO NOT SAY THIS

Do not start by rebooting the node. It destroys the evidence and often just relocates the problem to a different node.

EXPECT THIS FOLLOW-UP

Everything on the node looks healthy but pods there fail DNS. Now what?

Q12 INTERMEDIATE

What is SELinux doing in an OpenShift cluster?

SAY THIS

SELinux is mandatory access control on the node, enforcing on RHCOS by default. Each pod gets a Multi-Category Security label, and the files it writes are labelled to match, so even a process running as root inside a container cannot read another container's data or touch host paths it was not granted. It is the layer that turns a container escape attempt into a denied syscall rather than a lateral move.

THINK OF IT LIKE

It is the hotel key card system. Being inside the building does not mean every door opens - your card is coded for your floor and your room only.

WHY IT MATTERS IN PRODUCTION

SELinux surfaces in interviews as a mount failure. A pod mounts a hostPath or an NFS share, the process gets permission denied even though the UNIX permissions look right, and the AVC denial is sitting in the node's audit log. The correct answer is to fix the label or the volume's SELinux options - not to set the node to permissive, which is an unsupported configuration and a security finding waiting to happen.

STEP BY STEP

1 Define it as mandatory access control layered on top of normal permissions.
2 Explain per-pod MCS labels and why they isolate pods from each other.
3 Describe the diagnosis: look for AVC denials in the node audit log, matched by timestamp.
4 State the fix hierarchy - correct labelling or a supported volume option first, never disabling enforcement.

EVIDENCE YOU WOULD QUOTE

```
oc debug node/<node> -- chroot /host ausearch -m avc -ts recent

oc get pod <pod> -o jsonpath='{.spec.securityContext.seLinuxOptions}'

oc debug node/<node> -- chroot /host ls -Z /var/lib/kubelet/pods/<uid>/volumes
```

RED FLAG - DO NOT SAY THIS

Do not suggest setenforce 0 as a fix. In a regulated environment that single sentence can end the interview.

EXPECT THIS FOLLOW-UP

A pod mounting an NFS share gets permission denied. Walk me through your diagnosis.

Q13 SENIOR

How would you build a secure, minimal container image for production?

SAY THIS

I start from a small, supported base such as a Red Hat Universal Base Image, build with a multi-stage Containerfile so compilers and build secrets never reach the final layer, run as an arbitrary non-root UID with a group-writable filesystem so it satisfies restricted-v2, drop all capabilities, and pin everything by digest. Then the pipeline scans the image, generates an SBOM, signs it, and promotes the same digest through environments instead of rebuilding per stage.

THINK OF IT LIKE

Ship the finished cake, not the whole kitchen. Nobody needs the mixer, the raw eggs or the recipe notes in the box you hand to the customer.

WHY IT MATTERS IN PRODUCTION

The arbitrary-UID requirement catches people out constantly. OpenShift's default SCC assigns a random high UID per namespace, so an image that hardcodes USER 1001 and writes to a directory owned by 1001 will fail with permission denied. The fix is to make paths group-writable by the root group, which is what UBI images already do. Getting this right once removes an entire category of "it works on my laptop" tickets.

STEP BY STEP

1. Choose a supported minimal base and pin it by digest.
2. Use multi-stage builds so build tooling and secrets stay out of the runtime layer.
3. Make the image arbitrary-UID safe: no hardcoded UID assumptions, group-writable paths, no root requirement.
4. Wire the pipeline: scan, SBOM, sign, then promote the identical digest across environments with admission policy enforcing signed images.

EVIDENCE YOU WOULD QUOTE

```
podman build --squash-all -t app:1.4 . ; podman inspect app:1.4 --format '{{.User}}'

syft app:1.4 -o spdx-json > sbom.json ; cosign sign --key <key> <digest>

oc get scc restricted-v2 -o jsonpath='{.runAsUser.type}{"\n"}'
```

RED FLAG - DO NOT SAY THIS

Do not solve a permissions problem by granting the anyuid SCC. That is trading a five-minute image fix for a permanent audit finding.

EXPECT THIS FOLLOW-UP

The vendor image insists on running as root. What are your options, ranked?

Q14 ARCHITECT

How do you decide what belongs in the image versus in configuration?

SAY THIS

The image holds anything that must be identical everywhere and is safe to be public inside the organisation: the runtime, dependencies, the application binary. Configuration that varies by environment goes in ConfigMaps, and anything secret goes in Secrets or an external vault injected at runtime. My rule is that one digest should be promotable from development to production unchanged - if a rebuild is required to move an artefact between environments, the boundary is drawn in the wrong place.

THINK OF IT LIKE

The image is the aircraft; configuration is the flight plan. You do not build a new aircraft because you are flying a different route today.

WHY IT MATTERS IN PRODUCTION

Getting this wrong produces two distinct failures. Baking configuration into images means every environment runs different bytes, so testing proves nothing and rollbacks are ambiguous. Pushing too much into runtime configuration produces pods that cannot start without a dozen ConfigMaps existing first, which turns namespace creation into a fragile ritual. The middle ground is a small, well-documented set of environment inputs with sane defaults and schema validation in CI.

STEP BY STEP

1. State the promotion rule: the same digest moves across environments untouched.
2. Classify inputs - invariant into the image, environmental into ConfigMaps, sensitive into Secrets or an external vault.
3. Validate configuration in CI against a schema so a bad value fails a pipeline rather than a rollout.
4. Make the contract explicit in the golden path template so every team draws the line the same way.

EVIDENCE YOU WOULD QUOTE

```
oc set env deploy/<name> --list

oc get cm,secret -n <ns> --show-labels

oc rollout history deploy/<name> # does a config change show as a new revision
```

RED FLAG - DO NOT SAY THIS

Do not defend building a separate image per environment. It quietly destroys the value of everything you tested.

EXPECT THIS FOLLOW-UP

How do you roll out a configuration change safely if it does not create a new pod revision?

PART 2 · Q15-Q31 · 17 QUESTIONS

The control plane and who owns what

Where desired state lives, who reconciles it, and where to look when it stops

If you can only prepare one thing properly, prepare this. Every serious OpenShift interview comes back to a single skill: given a symptom, can you name the component that owns it and read its status rather than guessing? The control plane is not a list of daemons to memorise, it is a chain of ownership. The API server is the front door, etcd is the record, controllers close the gap between record and reality, and OpenShift layers its own operators on top so that the cluster can upgrade and repair itself. Learn the chain and most troubleshooting questions answer themselves.

Level mix - Foundation: 7 · Intermediate: 5 · Senior: 4 · Architect: 1

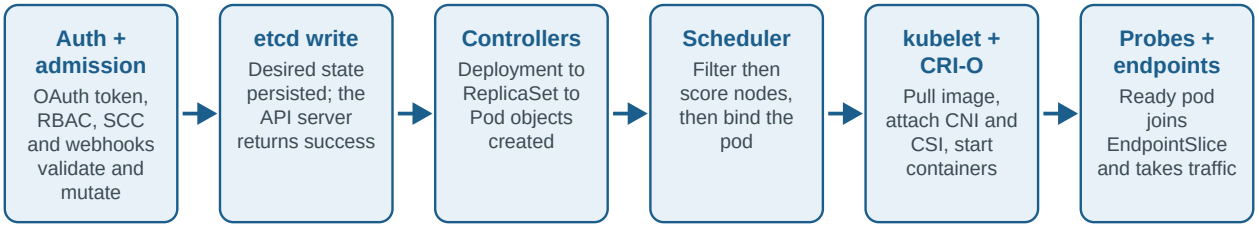
Who owns what in an OpenShift cluster

When something is broken, the fastest question is not "what is wrong" but "who owns this". Read the conditions on the owner, not the symptom at the bottom.

CVO	Owns the release payload. Drives every Cluster Operator toward the version you asked for.
Cluster Operators	Own core capabilities: authentication, ingress, storage, monitoring, network, etcd.
OLM	Owns optional add-on operators through CatalogSource, Subscription, InstallPlan and CSV.
MCO	Owns node configuration. Renders MachineConfigs and rolls them out pool by pool.
Machine API	Owns machine existence: MachineSet, Machine, MachineHealthCheck, autoscaling.
kube-controller-manager	Owns workload reconciliation: ReplicaSets, endpoints, node lifecycle, PV binding.
kubelet + CRI-O	Owns what actually runs: pod sandboxes, containers, probes, image pulls, node status.

What really happens after oc apply

The single most asked OpenShift interview question. Walk it as a relay race: every stage hands off to the next, and every stage can be the one that fails.



Q15 FOUNDATION

What does the Kubernetes API server do?

SAY THIS

The API server is the only component that talks to etcd, and it is the single front door for every request. It authenticates the caller, authorises them through RBAC, runs mutating and then validating admission - which in OpenShift includes Security Context Constraints - validates the object against its schema, and persists it. It also serves watches, which is how every controller and kubelet learns that something changed.

THINK OF IT LIKE

It is the registry office. Nothing is legally true until it is written in the ledger there, and everyone else finds out by subscribing to the ledger's updates.

WHY IT MATTERS IN PRODUCTION

Because it is the only writer to etcd and the source of every watch, API server health is cluster health. When it is slow, symptoms appear everywhere at once and look unrelated: controllers lag, kubelets are late reporting status, nodes flap between Ready and NotReady, and oc commands feel sticky. Recognising that pattern - many unrelated symptoms, one shared dependency - is what separates a fast diagnosis from an hour of chasing pods.

STEP BY STEP

1. State the request pipeline in order: authenticate, authorise, mutate, validate, persist.
2. Point out that it is the only component with etcd access - everything else goes through it.
3. Mention watches, because that is how the declarative model actually propagates.
4. Give the failure signature: broad, simultaneous, unrelated-looking symptoms mean you should check the API server before the workloads.

EVIDENCE YOU WOULD QUOTE

```
oc get --raw /readyz?verbose | head

apiserver_request_duration_seconds_bucket # Prometheus, by verb and resource

oc get co kube-apiserver openshift-apiserver
```

RED FLAG - DO NOT SAY THIS

Do not describe the API server as "where you run kubectl". It is an admission and persistence pipeline, and that pipeline is what interviewers probe.

EXPECT THIS FOLLOW-UP

Which admission step would reject a pod asking to run as root, and why that one?

Q16 FOUNDATION

What is etcd and what does it actually store?

SAY THIS

etcd is the cluster's consistent key-value store and it holds every API object: Deployments, Secrets, ConfigMaps, node registrations, RBAC, custom resources. It is the desired and recorded state of the cluster. What it does not hold is application data - your database contents live on a persistent volume in the storage backend, not in etcd. That distinction drives the whole backup and recovery conversation.

THINK OF IT LIKE

etcd is the building's blueprint and tenancy register. Restoring it rebuilds who is supposed to live where; it does not restore the furniture inside the flats.

WHY IT MATTERS IN PRODUCTION

The consequence is one interviewers ask about constantly: an etcd restore recovers cluster state to a point in time, but everything that happened since - new objects, new secrets, new certificates - is gone, and persistent volume data is untouched. So an etcd backup is a cluster-recovery tool, not an application-recovery tool, and you need both. It also explains why etcd performance is so sensitive to disk latency: every write is a consensus round trip that must be fsynced.

STEP BY STEP

1. Define it as the consistent store behind the API server.
2. Say what is in it - all API objects - and what is not: PV contents and application state.
3. Note the disk-latency sensitivity, because that is the number one etcd production problem.
4. Draw the recovery conclusion: etcd backup and application backup solve different failures.

EVIDENCE YOU WOULD QUOTE

```
oc get etcd cluster -o jsonpath='{.status.conditions}' | python3 -m json.tool

etcd_disk_wal_fsync_duration_seconds_bucket # p99 should stay in low milliseconds

oc -n openshift-etcd rsh <etcd-pod> etcdctl endpoint status -w table
```

RED FLAG - DO NOT SAY THIS

Do not say "we back up etcd so our applications are protected". That single sentence is a well-known senior-level trap.

EXPECT THIS FOLLOW-UP

You restore etcd from twelve hours ago. What is now broken that was fine before?

Q17 FOUNDATION

What is etcd quorum, and why an odd number of members?

SAY THIS

etcd uses Raft, which requires a majority of members to agree before a write is committed. With three members the quorum is two, so you survive one failure; with five it is three, so you survive two. Odd numbers are used because adding an even member increases the quorum requirement without increasing fault tolerance - four members still only tolerate one failure, but give you one more thing that can break.

THINK OF IT LIKE

It is a committee that needs a majority to sign off. Adding a fourth member to a committee of three does not make decisions easier; it just adds another person who can be off sick.

WHY IT MATTERS IN PRODUCTION

Lose quorum and the API server goes read-only or fails entirely - the cluster stops accepting changes, though already-running pods keep serving traffic. This is why control plane nodes must sit in separate failure domains and why you never take two of three down for maintenance at the same time. In stretched or three-zone designs, quorum placement is the whole design conversation: two zones cannot give you a safe majority.

STEP BY STEP

1. Define quorum as a strict majority under Raft.
2. Do the arithmetic out loud: 3 tolerates 1, 5 tolerates 2, 4 still only tolerates 1.

3 State the failure behaviour - no quorum means no writes, but existing workloads keep running.
4 Connect it to placement: separate failure domains, and never drain two control plane nodes together.

EVIDENCE YOU WOULD QUOTE

```
oc -n openshift-etcd rsh <etcd-pod> etcdctl endpoint health --cluster

oc get nodes -l node-role.kubernetes.io/master -o wide

oc get co etcd -o jsonpath='{.status.conditions}' | python3 -m json.tool
```

RED FLAG - DO NOT SAY THIS

Do not say more etcd members means more resilience without qualification. Beyond five, write latency gets worse and resilience does not.

EXPECT THIS FOLLOW-UP

Two of three control plane nodes are lost. What is your recovery sequence?

Q18 FOUNDATION

What is a controller, and what does the controller manager do?

SAY THIS

A controller is a loop that watches one kind of object, compares desired state with observed state, and acts to close the gap. The kube-controller-manager bundles many of these together - the Deployment controller creating ReplicaSets, the ReplicaSet controller creating pods, the node lifecycle controller marking nodes unhealthy, the PV binder matching claims to volumes. Each one writes what it thinks into the object's status, which is why status conditions are the best diagnostic in the cluster.

THINK OF IT LIKE

Each controller is a shopkeeper who keeps glancing at the shelf. Two loaves missing, so bake two more. Nobody instructed them; they are simply comparing shelf to plan, continuously.

WHY IT MATTERS IN PRODUCTION

This is why chasing pods is usually the wrong instinct. If pods are missing, the interesting object is the ReplicaSet or the Deployment, because its conditions will say whether it is blocked by quota, by an admission webhook, or by a failing image. Reading down the ownership chain - Deployment, ReplicaSet, Pod - takes three commands and replaces a lot of guessing.

STEP BY STEP

1 Define the loop: watch, diff, act, report status.
2 Name three concrete controllers so it is not abstract.
3 Explain that status conditions are the controller's own explanation of what is blocking it.
4 Show the diagnostic habit: walk the ownership chain from the top object down to the pod.

EVIDENCE YOU WOULD QUOTE

```
oc get deploy <name> -o jsonpath='{.status.conditions}' | python3 -m json.tool

oc get rs -n <ns> --sort-by=.metadata.creationTimestamp | tail -3

oc get events -n <ns> --field-selector type=Warning --sort-by=.lastTimestamp
```

RED FLAG - DO NOT SAY THIS

Do not say Kubernetes "just runs your containers". The whole interview is about reconciliation loops.

EXPECT THIS FOLLOW-UP

A Deployment reports ReplicaFailure. Which controller wrote that, and what would it mean?

Q19 FOUNDATION

What is a CustomResourceDefinition?

SAY THIS

A CRD extends the Kubernetes API with a new object type. Once it is registered you get a first-class resource - it can be created with oc apply, protected with RBAC, watched, and validated against an OpenAPI schema, exactly like a built-in object. On its own a CRD does nothing; it is just a data shape. It becomes useful when a controller watches those objects and acts on them, which is what an operator is.

THINK OF IT LIKE

A CRD is a new form in the filing system. Printing the form changes nothing - it matters once there is a clerk whose job is to read it and act.

WHY IT MATTERS IN PRODUCTION

In production the CRD and its controller can fail independently, and that produces a confusing symptom: your custom resource applies successfully and then nothing happens. That is almost always a controller that is not running, is missing RBAC for the resource, or is watching a different API version. Knowing to check the operator's own logs and RBAC first makes you look like you have actually run operators rather than only installed them.

STEP BY STEP

1. Define it as an API extension with a schema, not a piece of logic.
2. Point out what you get for free: RBAC, validation, watch, oc tooling.
3. Separate the CRD from its controller, and say why that separation causes silent no-ops.
4. Give the diagnostic order - is the CR accepted, is the controller running, does it have RBAC, does the version match.

EVIDENCE YOU WOULD QUOTE

```
oc get crd | grep <domain>

oc explain <kind>.spec --recursive | head -30

oc -n <operator-ns> logs deploy/<operator> --tail=100
```

RED FLAG - DO NOT SAY THIS

Do not conflate a CRD with an operator. The CRD is the noun, the operator is the verb.

EXPECT THIS FOLLOW-UP

Your custom resource applies cleanly but nothing happens. Diagnose it.

Q20 FOUNDATION

What is an ownerReference and why does it matter?

SAY THIS

An ownerReference records that one object was created by and belongs to another - a pod points at its ReplicaSet, which points at its Deployment. Garbage collection uses this: delete the owner and the dependants are cleaned up automatically. It also gives you the ownership chain you walk during troubleshooting, and it is why deleting a pod directly gets you a new pod instead of an empty namespace.

THINK OF IT LIKE

It is a parent's name on a school form. Remove the family from the register and the children's records go with them - nobody has to tidy up each one by hand.

WHY IT MATTERS IN PRODUCTION

Two operational consequences come up regularly. First, cascading deletes are the reason removing a Deployment is safe and tidy, while orphaning a ReplicaSet by hand leaves pods nobody manages. Second, when a namespace refuses to delete, ownership and finalizers are the pair you investigate together - something is either waiting on a dependant or waiting on a finalizer that no controller is left to clear.

STEP BY STEP

1. Define it as a parent pointer used by garbage collection.
2. Walk one chain out loud: Deployment owns ReplicaSet owns Pod.
3. Explain cascading deletion and what orphaning does instead.
4. Link it to troubleshooting: the chain tells you which controller to interrogate.

EVIDENCE YOU WOULD QUOTE

```
oc get pod <pod> -o jsonpath='{.metadata.ownerReferences}' | python3 -m json.tool

oc delete deploy <name> --cascade=orphan # know what this leaves behind

oc get rs -n <ns> -o custom-columns=NAME:.metadata.name,OWNER:.metadata.ownerReferences[0].name
```

RED FLAG - DO NOT SAY THIS

Do not say pods are deleted "automatically by Kubernetes" without naming the mechanism. The mechanism is the answer.

EXPECT THIS FOLLOW-UP

What happens to the pods if you delete a ReplicaSet with --cascade=orphan?

Q21 FOUNDATION

What is a finalizer?

SAY THIS

A finalizer is a string on an object that blocks deletion until a controller has finished its cleanup and removes that string. When you delete such an object, the API server sets a deletionTimestamp and the object stays visible in a Terminating state until every finalizer is gone. It exists so that external resources - a cloud load balancer, a storage volume, a DNS record - can be released before the record disappears.

THINK OF IT LIKE

It is a checkout hold on a hotel room. You have handed back the key, but the room stays flagged until housekeeping confirms the minibar and the safe are clear.

WHY IT MATTERS IN PRODUCTION

This is the mechanism behind namespaces stuck in Terminating forever. The usual cause is a finalizer belonging to a controller that no longer exists - an operator was uninstalled before its custom resources were removed, so nobody is left to do the cleanup. Force-editing the finalizer out works, but it silently leaks whatever the finalizer was going to release, so it is a last resort after you have identified what is missing.

STEP BY STEP

1. Define it as a deletion guard owned by a controller.
2. Explain the Terminating state and deletionTimestamp.
3. Give the common failure - orphaned finalizer after an operator was removed.
4. State the safe order: restore or reinstall the controller first, remove the finalizer only as a documented last resort.

EVIDENCE YOU WOULD QUOTE

```
oc get ns <ns> -o jsonpath='{.spec.finalizers}{"\n"}'

oc api-resources --verbs=list --namespaced -o name | xargs -n1 oc get -n <ns> --ignore-not-found

oc get <kind> <name> -o jsonpath='{.metadata.finalizers}{"\n"}'
```

RED FLAG - DO NOT SAY THIS

Do not offer "just patch the finalizers out" as your first move. It works, and it is how clusters end up leaking cloud resources nobody can account for.

EXPECT THIS FOLLOW-UP

A namespace has been Terminating for an hour. What is your sequence?

Q22 INTERMEDIATE

What does the Cluster Version Operator do?

SAY THIS

The CVO owns the cluster's version. It takes a release image - a single digest that describes every component in that OpenShift release - and drives each Cluster Operator toward the manifests in that payload, in dependency order. It reports overall progress on the ClusterVersion object, including whether the cluster is Upgradeable. It is what makes an OpenShift upgrade a single supported transaction rather than a set of independent component upgrades.

THINK OF IT LIKE

The CVO is the conductor with the only copy of the score. Individual sections do not decide their own tempo; they follow the release the conductor is playing.

WHY IT MATTERS IN PRODUCTION

Understanding this reframes upgrade troubleshooting. When an upgrade stalls, the ClusterVersion status names the operator it is waiting on, and that operator's conditions name the reason - a degraded dependency, a node that will not drain, an unavailable API. You do not fix the CVO, you fix what it is blocked on. It also explains why you should not hand-edit resources the CVO owns: it will simply put them back.

STEP BY STEP

1. Define the release image as a digest-addressed payload of all components.
2. Explain that the CVO reconciles Cluster Operators toward that payload in dependency order.
3. Show where to read progress and blockage - ClusterVersion conditions, then the named operator's conditions.
4. Add the rule: do not hand-edit CVO-managed resources, because reconciliation reverts them.

EVIDENCE YOU WOULD QUOTE

```
oc get clusterversion version -o jsonpath='{.status.conditions}' | python3 -m json.tool

oc adm upgrade # available targets and current state

oc get co --sort-by=.metadata.name | grep -v 'True.*False.*False'
```

RED FLAG - DO NOT SAY THIS

Do not talk about upgrading individual OpenShift components independently. That is not how the product is supported.

EXPECT THIS FOLLOW-UP

An upgrade has sat at 62% for an hour. What is your first command and why?

Q23 INTERMEDIATE

What is the difference between a Cluster Operator and an OLM-managed operator?

SAY THIS

A Cluster Operator is part of the OpenShift release payload and is managed by the CVO - authentication, ingress, storage, monitoring, etcd. You do not install or uninstall them, and their version moves with the cluster. An OLM-managed operator is an optional add-on you chose to install from a catalog - it has a Subscription, an InstallPlan and a CSV, and it has its own update channel independent of the cluster version.

THINK OF IT LIKE

Cluster Operators are the organs you were born with. OLM operators are the apps you installed - both essential to how the system behaves, but only one set has an uninstall button and its own update schedule.

WHY IT MATTERS IN PRODUCTION

The distinction determines where you look and what you are allowed to do. A degraded Cluster Operator is a cluster health and upgrade blocker, diagnosed through oc get co and resolved through supported procedures. A stuck OLM operator is diagnosed by walking CatalogSource to Subscription to InstallPlan to CSV. Mixing these up sends candidates looking for a Subscription that will never exist.

STEP BY STEP

1. Split them by lifecycle owner: CVO versus OLM.
2. Give examples of each so the boundary is concrete.
3. State the different diagnostic entry points - oc get co versus the OLM object chain.
4. Mention the upgrade implication: an OLM operator's channel must be compatible with the target cluster version before you upgrade.

EVIDENCE YOU WOULD QUOTE

```
oc get co ; oc get csv -A | head

oc get subscription -A -o
custom-columns=NS:.metadata.namespace,NAME:.metadata.name,CHANNEL:.spec.channel

oc get installplan -A
```

RED FLAG - DO NOT SAY THIS

Do not call everything "an operator" without saying who manages its lifecycle. That distinction is the whole question.

EXPECT THIS FOLLOW-UP

You are about to upgrade the cluster. What do you check about your OLM operators first?

Q24 INTERMEDIATE

What does the Machine Config Operator do, and what is a rendered MachineConfig?

SAY THIS

The MCO owns node configuration. You write MachineConfig objects - kernel arguments, files, systemd units, kubelet settings - and label them for a pool. The MCO merges every MachineConfig that applies to a pool, in name order, into a single rendered MachineConfig, and then the Machine Config Daemon on each node applies it, draining and rebooting the node one at a time according to the pool's maxUnavailable.

THINK OF IT LIKE

It is a building's standard fit-out spec. You do not renovate each flat individually; you publish one merged spec and the contractor works through the block, one flat at a time.

WHY IT MATTERS IN PRODUCTION

Two things make this interview-relevant. First, the rendered config is the source of truth - when a node's configuration looks wrong, you compare its current and desired rendered config rather than reading individual MachineConfigs. Second, MCO rollout is where PodDisruptionBudgets and drains collide with change management: a workload with a badly sized PDB will stall a node update and therefore stall a cluster upgrade, and the pool will sit Degraded telling you exactly that.

STEP BY STEP

1. Describe the object flow: MachineConfig, merged into rendered config per pool, applied by MCD.
2. Explain the rollout: one node at a time, cordon, drain, apply, reboot, uncordon.
3. Name where to read state - MachineConfigPool conditions and the node's current versus desired config annotations.
4. Call out the classic blocker: a drain stuck on a PDB or a pod with no controller.

EVIDENCE YOU WOULD QUOTE

```
oc get mcp -o wide

oc get node <node> -o
jsonpath='{.metadata.annotations.machineconfiguration\.openshift\.io/currentConfig}{"\n"}'

oc -n openshift-machine-config-operator logs ds/machine-config-daemon -c machine-config-daemon
--tail=100
```

RED FLAG - DO NOT SAY THIS

Do not describe editing files on nodes by hand as an alternative. The MCD will revert it and may flag the node as degraded.

EXPECT THIS FOLLOW-UP

A MachineConfigPool is Degraded with one node stuck. Walk me through it.

Q25 INTERMEDIATE

Walk me through everything that happens after oc apply.

SAY THIS

The client sends the object to the API server, which authenticates the token, authorises via RBAC, runs mutating admission - including SCC deciding what the pod may be - then validating admission and schema validation, and writes to etcd. Controllers watching that type wake up: the Deployment controller creates a ReplicaSet, which creates pods. The scheduler filters and scores nodes and binds the pod. The kubelet on that node pulls the image, sets up the network through CNI and storage through CSI, and starts containers. Probes pass, the pod goes Ready, and the endpoint controller adds it to the EndpointSlice so it takes traffic.

THINK OF IT LIKE

It is a relay race with six batons. Reception, filing, planning, allocation, construction, opening. Any runner can drop the baton, and the whole point is knowing which one did.

WHY IT MATTERS IN PRODUCTION

This question is really a diagnostic map in disguise. Rejected at admission means RBAC, SCC or a webhook. Object exists but no pods means a controller problem or quota. Pod Pending means the scheduler found no fit. Pod ContainerCreating means image, CNI or CSI. Pod Running but not Ready means probes. Ready but no traffic means EndpointSlice, Service selector or NetworkPolicy. If you can recite the stages, you can localise almost any workload failure in under a minute.

STEP BY STEP

1. Say the six stages in order, briefly, without drowning in detail.
2. For each stage, name the observable symptom when it fails.
3. Highlight the OpenShift-specific step: SCC admission deciding the pod's security context.
4. Close by using it as a triage map rather than trivia - that is the point of knowing it.

EVIDENCE YOU WOULD QUOTE

```
oc get events -n <ns> --sort-by=.lastTimestamp | tail -20

oc describe pod <pod> | sed -n '/Events/,$p'

oc get endpointslice -n <ns> -l kubernetes.io/service-name=<svc>
```

RED FLAG - DO NOT SAY THIS

Do not stop at "the scheduler places it and it runs". The value of this answer is in the stages you skipped.

EXPECT THIS FOLLOW-UP

At which stage would a Security Context Constraint reject the pod, and what would you see?

Q26 INTERMEDIATE

What is API Priority and Fairness, and why does it matter?

SAY THIS

APF replaces the old global max-in-flight limits with fair queueing. Requests are sorted into flow schemas and priority levels, each with its own concurrency share, so one noisy client - a runaway controller, a mis-tuned CI job listing every pod in the cluster - gets throttled in its own queue instead of starving the whole API server. Critical traffic like leader election has a protected level.

THINK OF IT LIKE

It is separate queues at the airport rather than one line. The person with twelve suitcases still gets served, but they do not hold up everyone behind them.

WHY IT MATTERS IN PRODUCTION

You meet APF when clients start getting 429s and someone asks whether the API server is "broken". Usually it is doing its job: something is generating an enormous request rate and APF is containing the damage. The right response is to find the offending flow, fix the client - add a resourceVersion, use informers instead of polling, filter by label - and only then consider adjusting concurrency shares.

STEP BY STEP

1. Explain the model: flow schemas map requests to priority levels with concurrency shares.
2. Say what problem it solves - isolating a noisy client from everyone else.
3. Describe the symptom: 429 responses and rising request wait duration for one flow.
4. Give the fix order - identify the flow, fix the client's request pattern, tune shares last.

EVIDENCE YOU WOULD QUOTE

```
oc get flowschema ; oc get prioritylevelconfiguration

apiserver_flowcontrol_rejected_requests_total # by flow schema

apiserver_flowcontrol_request_wait_duration_seconds_bucket
```

RED FLAG - DO NOT SAY THIS

Do not immediately propose raising limits. Raising the ceiling for a client that is polling in a loop just moves the outage.

EXPECT THIS FOLLOW-UP

A team says their operator is being rate limited. How do you confirm and what do you advise?

Q27 SENIOR

How do you investigate a degraded Cluster Operator?

SAY THIS

I start at the operator itself: oc get co to see which conditions are set, then read the message on Degraded or Progressing, because it usually names the dependency. Then I look at the operator's namespace - its pods, events and logs - and at what it depends on: nodes, storage, DNS, certificates, the network. I state impact first, gather that evidence, and only then take the smallest action, because most degraded operators are symptoms of an infrastructure problem rather than causes.

THINK OF IT LIKE

The warning light on the dashboard is not the fault. You read the code, then check the system it points at - you do not replace the dashboard.

WHY IT MATTERS IN PRODUCTION

The most common trap is treating the operator as the problem and restarting its pods. That clears the evidence and, in the common cases, changes nothing: the ingress operator is degraded because a load balancer health check is failing, the storage operator because a CSI driver cannot reach its backend, the authentication operator because the OAuth route is unreachable or an IdP certificate expired. The message on the condition almost always names the real subject.

STEP BY STEP

1. Establish impact and scope: is a customer-facing capability affected, or only cluster management?
2. Read the operator's conditions and take the message literally - it names the dependency.
3. Inspect the operator's own namespace: pod state, recent events, logs at the time of change.
4. Check the named dependency - node, certificate, load balancer, storage backend, DNS - and fix there; capture must-gather if this may become a support case.

EVIDENCE YOU WOULD QUOTE

```
oc get co ; oc describe co <name>

oc get pods -n openshift-<component> ; oc logs -n openshift-<component> <pod> --tail=200

oc adm must-gather -- /usr/bin/gather_<component>
```

RED FLAG - DO NOT SAY THIS

Do not delete operator pods as a first step. It is the platform equivalent of turning it off and on again, and it destroys the evidence you would need for support.

EXPECT THIS FOLLOW-UP

The authentication operator is degraded but users can still log in. What does that tell you?

Q28 SENIOR

How do you investigate API server latency?

SAY THIS

I confirm the symptom with the request duration metrics broken down by verb and resource, because a slow LIST on one custom resource is a very different problem from broad slowness. Then I check etcd, since API latency is usually etcd disk latency - fsync and backend commit durations. In parallel I look for a client generating heavy uncached LIST or WATCH traffic, and at APF rejections. Only after that do I consider control plane sizing.

THINK OF IT LIKE

The queue at the counter can be long for three different reasons: the clerk is slow, the filing cabinet is slow, or one customer is ordering for the entire street. You check which before hiring another clerk.

WHY IT MATTERS IN PRODUCTION

The single most common root cause in real clusters is disk. etcd needs consistently low write latency, and a control plane on shared or throttled storage will produce API slowness that looks like a Kubernetes problem and is actually an infrastructure one. The second most common is a badly written controller that lists all pods cluster-wide every few seconds. Naming both, in that order, is what a senior answer sounds like.

STEP BY STEP

1. Quantify it: request duration by verb and resource, plus which clients are affected.
2. Check etcd health first - fsync and backend commit p99, database size, leader elections.
3. Look for abusive clients and APF rejections; identify the flow schema being throttled.
4. Then consider capacity - control plane CPU, memory and disk class - and change one thing with a measured before and after.

EVIDENCE YOU WOULD QUOTE

```
histogram_quantile(0.99, sum by (le,verb) (rate(apiserver_request_duration_seconds_bucket[5m])))

etcd_disk_backend_commit_duration_seconds_bucket ; etcd_server_leader_changes_seen_total

oc get --raw /metrics | grep apiserver_current_inflight_requests
```

RED FLAG - DO NOT SAY THIS

Do not jump to "add more control plane nodes". More etcd members usually makes write latency worse, not better.

EXPECT THIS FOLLOW-UP

etcd fsync p99 is 90ms. What does that tell you and what would you do?

Q29 SENIOR

What are etcd fragmentation and defragmentation, and when do you care?

SAY THIS

etcd keeps historical revisions, and compaction removes old ones logically but leaves free space inside the database file. Defragmentation reclaims that space and shrinks the file. You care when the database approaches its size quota, because hitting it puts the cluster into a no-space alarm and the API server goes read-only. Defragmentation is done one member at a time, during a maintenance window, because the member being defragmented blocks.

THINK OF IT LIKE

It is a filing cabinet where removing documents leaves the gaps behind. Compaction shreds the old copies; defragmentation is the tidy-up that actually closes the gaps.

WHY IT MATTERS IN PRODUCTION

This appears in interviews as a scale question - what happens to a cluster with heavy object churn, such as thousands of short-lived Jobs or an operator rewriting status constantly. The senior answer is not just "defragment", it is to reduce the churn: fix the controller that is hot-looping, add TTLs to Jobs, and monitor database size with an alert well before the quota. Defragmentation treats the symptom.

STEP BY STEP

1. Separate compaction, which is automatic and logical, from defragmentation, which reclaims file space.
2. State the risk: approaching the space quota makes the API server read-only.
3. Describe the safe procedure - one member at a time, watch for leader changes, verify health before moving on.
4. Add the real fix: find and stop the churn, and alert on database size ahead of the quota.

EVIDENCE YOU WOULD QUOTE

```
etcd_mvcc_db_total_size_in_bytes ; etcd_mvcc_db_total_size_in_use_in_bytes

oc -n openshift-etcd rsh <etcd-pod> etcdctl endpoint status -w table

oc get events -A --field-selector reason=Unhealthy -n openshift-etcd
```

RED FLAG - DO NOT SAY THIS

Do not defragment all members at once. You will take the control plane down while trying to protect it.

EXPECT THIS FOLLOW-UP

Which workload patterns cause the most etcd churn, and how would you find them?

Q30 SENIOR

Why are ClusterVersion overrides risky?

SAY THIS

An override tells the CVO to stop managing a specific component. That immediately makes the cluster unsupported for upgrades in most cases, because the CVO can no longer guarantee the payload is consistent, and it silently freezes that component at whatever state it was in - including missing security fixes. Overrides exist for narrow, time-boxed, support-directed situations, not as a way to keep a resource edited the way you like it.

THINK OF IT LIKE

It is disabling one smoke detector because it keeps chirping. The noise stops, the protection stops, and everyone forgets it was ever disabled.

WHY IT MATTERS IN PRODUCTION

The realistic scenario is someone edited a CVO-managed resource, the CVO reverted it, and the team's instinct is to add an override so their change sticks. The senior response is to ask why the change is needed and find the supported knob - a Custom Resource on the owning operator, an IngressController field, a MachineConfig - because those exist for almost every legitimate case. If an override truly is required, it should be a documented, expiring decision with a ticket attached.

STEP BY STEP

1. Explain what an override actually does: CVO stops reconciling that manifest.
2. State the consequences - support posture, upgrade blocking, frozen component.
3. Redirect to the supported alternative: the owning operator's own API almost always exposes the setting.
4. If unavoidable, time-box it, document it, alert on it and remove it as part of the fix.

EVIDENCE YOU WOULD QUOTE

```
oc get clusterversion version -o jsonpath='{.spec.overrides}' | python3 -m json.tool

oc get co <component> -o jsonpath='{.status.conditions}' | python3 -m json.tool

oc adm upgrade # will report why the cluster is not Upgradeable
```

RED FLAG - DO NOT SAY THIS

Do not present overrides as a normal customisation technique. It reads as someone who fights the platform instead of using its API.

EXPECT THIS FOLLOW-UP

A team wants to change a CVO-managed Deployment's replica count. What do you tell them?

Q31 ARCHITECT

How would you choose a control plane topology for a new cluster?

SAY THIS

I start from the availability requirement and the failure domains actually available, not from a preferred number. Three control plane nodes across three independent domains is the default because it gives quorum with one failure. If only two domains exist, I say plainly that a stretched cluster cannot give a safe majority and propose two clusters with application-level replication instead. For small edge sites I consider compact three-node or single-node OpenShift, and I make the recovery expectation explicit up front.

THINK OF IT LIKE

You do not decide how many lifeboats to carry by preference. You count passengers, then count the exits the ship actually has.

WHY IT MATTERS IN PRODUCTION

The mistake this question is designed to catch is answering with a number instead of a requirement. Every topology carries an implied recovery story: single-node OpenShift means the site is down while it rebuilds, compact clusters mean control plane and workload contention, stretched clusters mean latency-sensitive quorum. Stating the recovery consequence alongside the topology is what turns this from a fact into a design answer.

STEP BY STEP

1. Capture requirements first: RTO, RPO, latency between sites, regulatory placement, expected scale.
2. Enumerate real failure domains, then choose a topology that gives quorum across them.
3. Name the trade-off explicitly - cost and operational complexity versus tolerated failures.
4. Write down the recovery expectation for the chosen topology and schedule a test that proves it.

EVIDENCE YOU WOULD QUOTE

```
oc get nodes -L topology.kubernetes.io/zone -l node-role.kubernetes.io/master

oc get infrastructure cluster -o jsonpath='{.status.controlPlaneTopology}{"\n"}'

Documented DR test results with measured RTO against the stated target
```

RED FLAG - DO NOT SAY THIS

Do not propose a two-zone stretched control plane. It looks highly available and cannot survive losing the wrong zone.

EXPECT THIS FOLLOW-UP

The customer only has two data centres and wants active-active. What do you propose?

PART 3 · Q32-Q47 · 16 QUESTIONS

Workloads, controllers and the application contract

Choosing the right object, and the behaviours that make a workload survivable

Workload questions look easy and are scored hard. Nobody is impressed that you know what a Deployment is; they want to know which object you would choose for a given requirement, what happens to it during a node drain, and what contract you expect application teams to meet before you will call something production ready. That contract - probes, graceful shutdown, resource requests, replica minimums - is the difference between a platform that survives routine maintenance and one where every upgrade becomes an incident.

Level mix - Foundation: 7 · Intermediate: 5 · Senior: 2 · Architect: 2

Choosing the right workload object

Interviewers rarely ask "what is a DaemonSet". They ask "which one would you use here, and why".

Object	Use when	Identity	Watch out for
Deployment	Interchangeable stateless replicas	Random pod names	Rolling update surge needs spare capacity
StatefulSet	Ordered, sticky identity and per-pod storage	Stable ordinal names and PVCs	Scale-down leaves PVCs behind on purpose
DaemonSet	One agent per node: logging, CNI, storage, monitoring	One pod per matching node	Tolerations decide whether it lands on tainted nodes
Job	Run to completion once	Pod per attempt	backoffLimit and cleanup policy
CronJob	Run to completion on a schedule	Job per firing	concurrencyPolicy and missed-schedule pileups

Q32 FOUNDATION

What is a ReplicaSet?

SAY THIS

A ReplicaSet keeps a specified number of identical pods running. It watches pods matching its selector, and if there are too few it creates more, if too many it deletes some. You rarely create one directly - a Deployment creates and owns ReplicaSets on your behalf, one per revision, which is how rollbacks work.

THINK OF IT LIKE

It is a shift manager who only counts heads. Three people should be on the floor; someone leaves, so someone is called in. No judgement about who, just the count.

WHY IT MATTERS IN PRODUCTION

The reason it matters operationally is that the ReplicaSet is where rollout failures become visible. If pods are not appearing, the ReplicaSet's events and conditions will tell you why - quota exceeded, a failing admission webhook, an image that cannot be pulled - long before you find it by staring at the Deployment. Old ReplicaSets sticking around with zero replicas is normal and deliberate: they are the previous revisions you can roll back to.

STEP BY STEP

1. Define it as a count-keeping controller driven by a label selector.
2. Explain its relationship to the Deployment: one ReplicaSet per revision.
3. Say why the old zero-replica ReplicaSets exist - rollback history.
4. Note the diagnostic value: ReplicaSet events explain why pods are not being created.

EVIDENCE YOU WOULD QUOTE

```
oc get rs -n <ns> --sort-by=.metadata.creationTimestamp

oc describe rs <rs> | sed -n '/Events/,$p'

oc get rs <rs> -o jsonpath='{.status.conditions}' | python3 -m json.tool
```

RED FLAG - DO NOT SAY THIS

Do not say you would manage ReplicaSets directly. It suggests you have not used rollout history or rollbacks.

EXPECT THIS FOLLOW-UP

Your Deployment says 3 desired but 0 available, and no pods exist. Where do you look?

Q33 FOUNDATION

What is a Deployment, and how does a rolling update actually work?

SAY THIS

A Deployment manages stateless replicas through ReplicaSets and gives you declarative updates. On a change it creates a new ReplicaSet and shifts replicas across gradually, governed by maxSurge and maxUnavailable: maxSurge is how many extra pods it may create above the desired count, maxUnavailable is how many it may take away. A new pod only counts as available once its readiness probe passes, which is what stops a broken image from replacing a working one.

THINK OF IT LIKE

It is replacing the tyres on a moving lorry, one at a time. You need at least one spare position free, and you do not remove the next tyre until the new one is proven to grip.

WHY IT MATTERS IN PRODUCTION

Two production consequences follow. First, if maxSurge is zero and maxUnavailable is zero, the rollout deadlocks - it cannot add or remove anything. Second, if readiness is defined badly, for example returning healthy before dependencies are reachable, the rollout happily replaces every working pod with a broken one and the Deployment reports success. Rollout safety is entirely a function of honest readiness probes.

STEP BY STEP

1. Describe the ReplicaSet swap and the two knobs that govern its pace.
2. Stress that readiness gates availability, so probes control rollout safety.
3. Mention progressDeadlineSeconds - the rollout gives up and reports failure rather than hanging forever.
4. State the rollback path and that it works by scaling the previous ReplicaSet back up.

EVIDENCE YOU WOULD QUOTE

```
oc rollout status deploy/<name> --timeout=5m

oc rollout history deploy/<name> ; oc rollout undo deploy/<name>

oc get deploy <name> -o jsonpath='{.spec.strategy.rollingUpdate}{"\n"}'
```

RED FLAG - DO NOT SAY THIS

Do not describe a rolling update as "it just restarts the pods". The surge and readiness mechanics are the whole answer.

EXPECT THIS FOLLOW-UP

Your rollout has been Progressing for twenty minutes with no new ready pods. What now?

Q34 FOUNDATION

What is a StatefulSet and when do you need one?

SAY THIS

A StatefulSet gives pods stable identity: predictable ordinal names, stable DNS through a headless Service, and a persistent volume per pod that follows that identity across restarts. It also creates and deletes pods in order by default. You need it when the workload cares which member it is - databases, message brokers, quorum-based systems - rather than treating replicas as interchangeable.

THINK OF IT LIKE

A Deployment is a pool of taxi drivers; any one will do. A StatefulSet is a set of numbered hotel rooms - room three always has room three's safe and room three's key.

WHY IT MATTERS IN PRODUCTION

The behaviour that catches people out is scale-down: the PersistentVolumeClaims are deliberately left behind, because deleting a database replica's data on a scale-down would be catastrophic. So shrinking a StatefulSet leaves orphaned PVCs consuming quota until someone cleans them up intentionally. The other one is ordered rollout - a StatefulSet update that stalls on pod zero will never reach pod one, which looks like a hang.

STEP BY STEP

1. Name the three guarantees: stable identity, stable storage, ordered operations.
2. Say when you need them - quorum members, primary/replica databases, brokers with partitions.
3. Call out the PVC retention behaviour on scale-down, and why it is intentional.
4. Mention that ordered updates mean one stuck pod blocks the rest, and where to see that.

EVIDENCE YOU WOULD QUOTE

```
oc get sts <name> -o wide ; oc get pvc -n <ns>

oc get sts <name> -o jsonpath='{.spec.updateStrategy}{"\n"}'

oc describe pod <name>-0 | sed -n '/Events/,$p'
```

RED FLAG - DO NOT SAY THIS

Do not say a StatefulSet is "a Deployment with storage". Identity and ordering are the point, not the volume.

EXPECT THIS FOLLOW-UP

You scale a StatefulSet from five to three. What is left behind and why?

Q35 FOUNDATION

What is a DaemonSet?

SAY THIS

A DaemonSet runs one pod on every node that matches its selector, and it automatically places a pod on any node that joins later. It is the pattern for node-level agents: log collectors, monitoring exporters, CNI and CSI components, security agents. Because infrastructure agents need to run even on nodes that are tainted for dedicated workloads, DaemonSets usually carry broad tolerations.

THINK OF IT LIKE

It is the fire extinguisher rule - one on every floor, and when a new floor is built, one appears there too without anyone filing a request.

WHY IT MATTERS IN PRODUCTION

The interview angle is usually placement and upgrade behaviour. If your logging agent is missing from GPU nodes, the cause is nearly always a missing toleration for the taint that protects those nodes. And during node maintenance, DaemonSet pods are not evicted by a normal drain - they are ignored - which is why a drain can succeed while a node-level agent is still running.

STEP BY STEP

1. Define it as one pod per matching node, including future nodes.
2. Give the canonical use cases so it is clearly infrastructure, not application, workload.
3. Explain the taint and toleration interaction, which is the usual reason one is missing.
4. Note drain behaviour: DaemonSet pods are skipped, which surprises people during maintenance.

EVIDENCE YOU WOULD QUOTE

```
oc get ds -A -o wide

oc get ds <name> -o jsonpath='{.spec.template.spec.tolerations}' | python3 -m json.tool

oc get nodes -o custom-columns=NAME:.metadata.name,TAINTS:.spec.taints
```

RED FLAG - DO NOT SAY THIS

Do not use a DaemonSet to get "one replica per node" for an application. That is a scheduling requirement, and topology spread constraints are the right tool.

EXPECT THIS FOLLOW-UP

Your log collector is missing on three nodes. Diagnose it.

Q36 FOUNDATION

What are Jobs and CronJobs, and what goes wrong with them?

SAY THIS

A Job runs pods until a specified number complete successfully, retrying failures up to backoffLimit. A CronJob creates Jobs on a schedule. What goes wrong is usually accumulation and overlap: without ttlSecondsAfterFinished, completed Jobs and their pods pile up and pressure etcd, and without a concurrencyPolicy a slow run overlaps the next trigger and you get two copies doing the same work.

THINK OF IT LIKE

A Job is a task with a tick box. A CronJob is a recurring calendar reminder. If nobody clears finished tasks and the reminder fires while you are still working, your desk disappears under duplicates.

WHY IT MATTERS IN PRODUCTION

These are a real source of cluster-level pain, not just application annoyance. Thousands of completed Job objects are thousands of etcd records and a slow LIST for anything that enumerates pods. And overlapping runs against a shared database are the classic cause of a batch job that works for months and then corrupts data the first time it runs long. Setting TTLs and concurrencyPolicy in your golden path template prevents both by default.

STEP BY STEP

1. Define completion semantics: completions, parallelism, backoffLimit.
2. Set ttlSecondsAfterFinished so finished Jobs clean themselves up.
3. Choose a concurrencyPolicy deliberately - Forbid for anything that mutates shared state.
4. Add startingDeadlineSeconds and alert on missed schedules rather than discovering them monthly.

EVIDENCE YOU WOULD QUOTE

```
oc get jobs -n <ns> --sort-by=.status.startTime | tail

oc get cronjob <name> -o jsonpath='{.spec.concurrencyPolicy}
{.spec.startingDeadlineSeconds}{"\n"}'

oc get pods -n <ns> --field-selector status.phase=Succeeded | wc -l
```

RED FLAG - DO NOT SAY THIS

Do not leave concurrencyPolicy at its default for a job that writes to a database, and then call the resulting corruption an application bug.

EXPECT THIS FOLLOW-UP

A nightly CronJob occasionally runs twice. How do you prove it and how do you fix it?

Q37 FOUNDATION

What is a Namespace and what does it actually isolate?

SAY THIS

A namespace is a scope for names, RBAC, quotas and network policy. It isolates API objects and gives you a boundary for permissions, resource limits and default network rules. It does not isolate the kernel, the node, or the network by default - pods in different namespaces can reach each other unless a NetworkPolicy says otherwise, and they share the same nodes and the same kernel.

THINK OF IT LIKE

It is a floor in an office building, not a separate building. Different teams, different door codes, different budgets - same air conditioning, same fire risk.

WHY IT MATTERS IN PRODUCTION

This is the foundation of every multi-tenancy question. Soft tenancy - namespaces plus RBAC, quota, LimitRange and default-deny NetworkPolicy - is right for teams inside one organisation. Hard tenancy, where you genuinely cannot trust the workload, needs a stronger boundary: dedicated node pools at minimum, and often separate clusters. Being clear about which one a namespace gives you is the difference between a confident answer and a hand-wave.

STEP BY STEP

1. List what it scopes: names, RBAC, quota, LimitRange, NetworkPolicy, some admission behaviour.
2. State plainly what it does not isolate: kernel, node resources, network by default.
3. Describe the standard hardening set applied at namespace creation.
4. Say when you would escalate from soft to hard tenancy, and what that costs.

EVIDENCE YOU WOULD QUOTE

```
oc get resourcequota,limitrange -n <ns>

oc get networkpolicy -n <ns>

oc auth can-i --list -n <ns> --as=system:serviceaccount:<ns>:default
```

RED FLAG - DO NOT SAY THIS

Do not claim namespaces provide security isolation on their own. Without NetworkPolicy they do not even isolate traffic.

EXPECT THIS FOLLOW-UP

What exactly would you apply to every new namespace, automatically?

Q38 FOUNDATION

What is a ServiceAccount and how do pods use it?

SAY THIS

A ServiceAccount is the identity a pod uses to talk to the API server. Every pod gets one - the namespace's default if you do not specify - and receives a short-lived, audience-bound projected token. RBAC is then granted to that ServiceAccount rather than to a person, and SCC is also matched against it, so the ServiceAccount determines both what API calls the pod can make and what the pod is allowed to be.

THINK OF IT LIKE

It is the contractor's site pass rather than a personal one. It says which areas this job may enter, it expires, and it does not belong to any individual.

WHY IT MATTERS IN PRODUCTION

The habit that matters is one ServiceAccount per workload with only the permissions that workload needs. The anti-pattern is everything running as default with a cluster-admin binding somewhere in its history, which means a single compromised pod inherits the keys to the cluster. Modern tokens are projected, time-limited and audience-scoped, so a leaked token is far less useful than the old permanent secrets - as long as you are not still creating long-lived token secrets by hand.

STEP BY STEP

1. Define it as workload identity, distinct from user identity.
2. Explain projected, short-lived, audience-bound tokens and why that is better than static secrets.
3. Say that both RBAC and SCC bind to it, so it controls capability and permission together.
4. Give the practice: one ServiceAccount per workload, least privilege, never reuse default for anything privileged.

EVIDENCE YOU WOULD QUOTE

```
oc get sa -n <ns> ; oc describe sa <name> -n <ns>

oc auth can-i --list --as=system:serviceaccount:<ns>:<sa>

oc get rolebinding,clusterrolebinding -A -o wide | grep <sa>
```

RED FLAG - DO NOT SAY THIS

Do not run workloads as the default ServiceAccount with elevated permissions. It is the single most common finding in a cluster security review.

EXPECT THIS FOLLOW-UP

How would you find every ServiceAccount in the cluster with cluster-admin?

Q39 INTERMEDIATE

When would you use a Deployment rather than a DeploymentConfig?

SAY THIS

Deployment is the Kubernetes-native, portable choice and it is what I use by default. DeploymentConfig is the older OpenShift-specific object with features Deployments did not originally have - image change triggers, config change triggers, and lifecycle hooks. It is deprecated, so new work should use Deployments, and the OpenShift-specific behaviours are replaced by pipelines, GitOps or explicit init containers.

THINK OF IT LIKE

It is the difference between a manufacturer's proprietary charging cable and USB-C. The old one still works on your device; you would not design a new product around it.

WHY IT MATTERS IN PRODUCTION

The interesting part of this question is what you do about the lost features. Image change triggers were genuinely convenient, and the modern replacement is either a pipeline that updates the image digest in Git, or an image updater controller. Lifecycle hooks become init containers or Jobs. Saying that out loud shows you understand the migration rather than just knowing which object is newer.

STEP BY STEP

1. State the default clearly: Deployment for anything new.
2. Name what DeploymentConfig gave you that is not native - triggers and hooks.
3. Give the modern replacement for each of those, so migration is concrete.
4. Mention portability: Deployments move between Kubernetes distributions unchanged.

EVIDENCE YOU WOULD QUOTE

```
oc get dc -A # anything still here is migration backlog

oc get deploy <name> -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'

oc rollout status deploy/<name>
```

RED FLAG - DO NOT SAY THIS

Do not recommend DeploymentConfig for new applications. It signals that your OpenShift knowledge stopped several releases ago.

EXPECT THIS FOLLOW-UP

A team relies on image change triggers. How do you migrate them?

Q40 INTERMEDIATE

Why does a StatefulSet usually need a headless Service?

SAY THIS

Because clients of a stateful system need to reach a specific member, not a random one. A headless Service - clusterIP set to None - makes DNS return the individual pod addresses instead of a single virtual IP, and combined with the StatefulSet's stable ordinal names that gives every member a predictable DNS name. Cluster members use those names to find each other for replication and quorum.

THINK OF IT LIKE

A normal Service is a call centre number - you get whoever is free. A headless Service is the internal directory, where you can dial the specific person you need.

WHY IT MATTERS IN PRODUCTION

This is why a database StatefulSet that appears healthy can still fail to form a cluster: the pods are running, but if serviceName is wrong or the headless Service is missing, members cannot resolve each other and each one sits waiting to join a cluster that never forms. The symptom is peer resolution errors in the application logs, not anything visible in pod status.

STEP BY STEP

1. Explain what headless means: no ClusterIP, DNS returns pod records.
2. Connect it to stable ordinal names giving predictable per-member DNS.
3. Give the failure symptom - members cannot discover peers, cluster never forms.
4. Check serviceName on the StatefulSet matches the headless Service's name.

EVIDENCE YOU WOULD QUOTE

```
oc get svc <name> -o jsonpath='{.spec.clusterIP}{"\n"}' # expect None

oc exec <pod> -- getent hosts <name>-0.<svc>.<ns>.svc.cluster.local

oc get sts <name> -o jsonpath='{.spec.serviceName}{"\n"}'
```

RED FLAG - DO NOT SAY THIS

Do not suggest putting a load balancer in front of database replicas that need per-member addressing. It will work until it very badly does not.

EXPECT THIS FOLLOW-UP

Pods are Running but the cluster never forms. Where do you start?

Q41 INTERMEDIATE

How should startup, readiness and liveness probes be designed?

SAY THIS

Startup probes cover slow initialisation so the liveness probe does not kill a container that is legitimately still booting. Readiness gates traffic - a pod that is not ready is removed from the EndpointSlice but left running. Liveness restarts a container that is genuinely wedged. The rule I apply is that readiness may depend on downstream dependencies, liveness must not, because a shared dependency failure would otherwise restart every pod in the fleet simultaneously.

THINK OF IT LIKE

Readiness is a shop turning its Open sign around. Liveness is the fire alarm. You flip the sign often; you do not evacuate the building because the supplier is late.

WHY IT MATTERS IN PRODUCTION

Badly designed probes cause more self-inflicted outages than almost anything else. A liveness probe that checks a database connection turns a slow database into a cluster-wide restart storm, which adds load and makes the database slower. A readiness probe that is too shallow lets a broken pod take traffic and turns a safe rolling update into an outage. Timeouts matter too: a probe timeout shorter than the endpoint's real latency under load produces restarts that only happen at peak.

STEP BY STEP

1. Assign each probe its job: startup for boot time, readiness for traffic, liveness for deadlock.
2. Keep liveness local and cheap - no downstream dependency checks.
3. Make readiness honest: it should fail when the pod genuinely cannot serve.
4. Tune timing against measured p99 latency under load, not against a default you copied.

EVIDENCE YOU WOULD QUOTE

```
oc get pod <pod> -o jsonpath='{.spec.containers[0].livenessProbe}' | python3 -m json.tool

oc describe pod <pod> | grep -E 'Liveness|Readiness|Startup|Restart Count'

kube_pod_container_status_restarts_total # correlate restarts with dependency latency
```

RED FLAG - DO NOT SAY THIS

Do not put a dependency check in a liveness probe. It converts a partial outage into a total one, automatically.

EXPECT THIS FOLLOW-UP

Every pod in a service restarted at once. How do probes explain that?

Q42 INTERMEDIATE

How do you implement graceful shutdown?

SAY THIS

When a pod is terminating, two things happen in parallel: it is removed from EndpointSlices, and it receives SIGTERM. Those are not synchronised, so the application must keep serving for a short while after SIGTERM to drain in-flight requests and let the routing layer catch up. I fail readiness first, add a small preStop sleep, handle SIGTERM to stop accepting new work while finishing current work, and set terminationGracePeriodSeconds longer than the worst-case drain.

THINK OF IT LIKE

It is closing a shop properly. You stop letting new customers in, serve the people already inside, then lock the door - you do not turn the lights off with people at the till.

WHY IT MATTERS IN PRODUCTION

This is the invisible cause of "we get 502s during every deploy". The pod exits instantly on SIGTERM while the router is still sending it requests for another second or two. Nobody notices in staging because there is no traffic. The fix is entirely in the application contract, which is why platform teams put it in the golden path template rather than discovering it per team.

STEP BY STEP

1. Explain the race: endpoint removal and SIGTERM are concurrent, not ordered.
2. Fail readiness immediately so new traffic stops being routed.

3 Use a preStop hook with a short sleep to cover propagation delay.
4 Handle SIGTERM to drain, and set terminationGracePeriodSeconds above the real worst case.

EVIDENCE YOU WOULD QUOTE

```
oc get deploy <name> -o jsonpath='{.spec.template.spec.terminationGracePeriodSeconds}{"\n"}'

oc get pod <pod> -o jsonpath='{.spec.containers[0].lifecycle}' | python3 -m json.tool

Router or ingress 5xx rate during a rollout window
```

RED FLAG - DO NOT SAY THIS

Do not blame the router for deploy-time 502s without checking the application's shutdown behaviour first. It is almost always the application.

EXPECT THIS FOLLOW-UP

You cannot change the application. What can the platform do to reduce the 502s?

Q43 INTERMEDIATE

How do you run batch and scheduled work safely at cluster scale?

SAY THIS

I treat batch as a first-class tenant rather than an afterthought. That means TTLs on finished Jobs so they do not accumulate, an explicit concurrencyPolicy, resource requests so the scheduler can place them honestly, a PriorityClass below interactive workloads so batch yields under pressure, and quota on the namespace so a runaway loop cannot consume the cluster. Failures should alert, not just retry forever.

THINK OF IT LIKE

Batch work is the delivery lorry. It belongs on the road, but it uses the service entrance, off-peak, and it does not get to block the ambulance.

WHY IT MATTERS IN PRODUCTION

Batch is where clusters quietly fall over. Uncapped parallelism plus BestEffort QoS means a single misconfigured Job can evict production pods from a dozen nodes. Unbounded retries against a failing dependency turn one broken integration into a permanent load generator. Setting priority, quota and backoff limits as defaults - not as a per-team decision - is what keeps this boring.

STEP BY STEP

1 Give every Job requests and limits, and never leave batch at BestEffort.
2 Set ttlSecondsAfterFinished, backoffLimit and activeDeadlineSeconds so failures end.
3 Assign a lower PriorityClass than interactive workloads so preemption favours users.
4 Apply namespace quota and alert on failed and missed schedules, not just on Job existence.

EVIDENCE YOU WOULD QUOTE

```
oc get jobs -A --field-selector status.successful=0

kube_cronjob_status_last_schedule_time # compare against expected cadence

oc get resourcequota -n <batch-ns> -o yaml
```

RED FLAG - DO NOT SAY THIS

Do not let batch workloads run without resource requests. They will be BestEffort and the first thing evicted - or the thing that evicts everyone else.

EXPECT THIS FOLLOW-UP

A batch namespace is causing evictions in production namespaces. What is your fix?

Q44 SENIOR

How would you choose between rolling, blue-green and canary deployments on OpenShift?

SAY THIS

Rolling is the default and it is right when the change is backward compatible and you can tolerate mixed versions briefly. Blue-green is right when you cannot tolerate mixed versions - an incompatible schema change, for example - and you can afford double capacity for the cutover. Canary is right when you want production traffic to be the test, and you have the observability to detect a regression in a small slice. On OpenShift I usually implement canary with weighted Routes or a service mesh, and blue-green by switching the Route target.

THINK OF IT LIKE

Rolling is repainting a room wall by wall. Blue-green is building the new room and moving everyone across in one step. Canary is inviting five people into the new room first and watching their faces.

WHY IT MATTERS IN PRODUCTION

The part interviewers listen for is the rollback story, because that is what the strategy is really buying. Rolling rollback means another rolling update, so recovery time equals rollout time. Blue-green rollback is a Route switch, measured in seconds, which is why it suits high-risk changes. Canary limits blast radius but only works if you have per-version metrics - otherwise you are shipping to a small group and hoping.

STEP BY STEP

1. Start from the constraint: is a mixed-version state acceptable, and what is the required rollback time?
2. Match the strategy to that constraint, and state the capacity cost of each.
3. Describe the OpenShift mechanism - rollout parameters, weighted Routes, or a mesh.
4. Define the abort criteria in advance: which metric, what threshold, who decides, how fast.

EVIDENCE YOU WOULD QUOTE

```
oc get route <name> -o jsonpath='{.spec.alternateBackends}' | python3 -m json.tool

oc set route-backends <route> app-v1=90 app-v2=10

Error rate and latency split by version label in Prometheus
```

RED FLAG - DO NOT SAY THIS

Do not propose canary without saying which metric decides success. Without that it is just a slower way to ship a bug.

EXPECT THIS FOLLOW-UP

Your canary shows a 0.3% error increase. Roll back or continue? Justify it.

Q45 SENIOR

How do you design namespace tenancy for a shared cluster?

SAY THIS

I define a tenancy unit - usually team plus environment - and make every namespace come with the same guardrails automatically: RBAC bound to a group rather than individuals, ResourceQuota, LimitRange to give sane defaults, a default-deny NetworkPolicy with explicit allows, and labels that drive policy, monitoring and chargeback. Namespaces are created through automation from a template, never by hand, so no namespace can exist without its guardrails.

THINK OF IT LIKE

It is renting serviced offices, not empty units. Every tenant gets the same locks, the same meter and the same fire policy on day one - you do not negotiate them per tenant.

WHY IT MATTERS IN PRODUCTION

The failure mode of hand-created namespaces is invisible until it hurts: one namespace with no quota consumes a node's memory during an incident, another has no NetworkPolicy and becomes the lateral movement path in a security review, a third has a RoleBinding to a person who left. Templating removes the whole class. It also makes the platform's offer legible to teams, which reduces the number of one-off exception requests.

STEP BY STEP

1. Define the tenancy unit and the labelling scheme that drives everything else.
2. Bundle the guardrails - RBAC to groups, quota, LimitRange, default-deny NetworkPolicy.
3. Automate creation through GitOps or a template so guardrails cannot be skipped.
4. Decide the escalation path to hard tenancy - dedicated nodes or a separate cluster - and the criteria that trigger it.

EVIDENCE YOU WOULD QUOTE

```
oc get ns --show-labels

oc get resourcequota,limitrange,networkpolicy -A | head -30

oc get rolebinding -A -o wide | grep -v ServiceAccount # user bindings to review
```

RED FLAG - DO NOT SAY THIS

Do not bind roles to individual users. When they change teams, nobody remembers, and the access outlives the reason for it.

EXPECT THIS FOLLOW-UP

A team asks for cluster-scoped permissions to install a CRD. How do you handle it?

Q46 ARCHITECT

What does your platform's golden path actually contain?

SAY THIS

A golden path is the paved route from repository to running service: a templated project with a Containerfile that satisfies restricted-v2, a pipeline that builds, scans, signs and produces an SBOM, GitOps manifests with probes, requests, limits, a PDB and topology spread already set, a default dashboard and alert set, and documented SLOs. Teams may leave the path, but then they own the parts they replaced, and that trade is explicit.

THINK OF IT LIKE

It is the paved footpath across the park. People are free to walk on the grass, but if the path goes where they need to go, almost nobody does - and the grass survives.

WHY IT MATTERS IN PRODUCTION

The purpose is to move the platform team from reviewing every deployment to reviewing exceptions. It works because it makes the safe option also the fast option: a team using the template gets probes, security context, monitoring and a working pipeline for free, while a team going their own way has to build all of it. The measure of success is the percentage of services on the path and the number of production incidents traced to off-path configuration.

STEP BY STEP

1. Enumerate the artefacts: project template, pipeline, manifests, dashboards, alerts, SLO doc.
2. Encode the non-negotiables as defaults in the template, not as review checklists.
3. Publish the exception process so leaving the path is possible but visible and owned.
4. Measure adoption and off-path incident rate, and use those numbers to prioritise improvements.

EVIDENCE YOU WOULD QUOTE

```
Percentage of Deployments carrying required labels, probes, requests and PDBs

oc get deploy -A -o json | jq '[.items[] |
select(.spec.template.spec.containers[0].resources.requests == null)] | length'

Pipeline compliance report: signed images, SBOM present, scan passed
```

RED FLAG - DO NOT SAY THIS

Do not describe a golden path as documentation. If it is a wiki page rather than a working template, adoption will be near zero.

EXPECT THIS FOLLOW-UP

How would you migrate fifty existing services onto the golden path without a big bang?

Q47 ARCHITECT

What is your definition of production ready for a workload?

SAY THIS

Concretely: more than one replica with topology spread across failure domains, a PodDisruptionBudget that permits maintenance, resource requests and limits that reflect measured usage, honest readiness and safe liveness probes, graceful shutdown, no hardcoded secrets, images pinned by digest and signed, a documented SLO with an alert tied to it, a tested rollback path, and a runbook that a person on call at 3am can follow without context.

THINK OF IT LIKE

It is an aircraft's pre-flight checklist. Not one of the items is clever; skipping any of them is how routine flights become incidents.

WHY IT MATTERS IN PRODUCTION

The value of writing this down is that it converts arguments into checks. Instead of debating whether a service is ready, the pipeline reports which items are missing, and the conversation becomes about the two that failed rather than about opinions. It also gives the platform team a defensible position: the answer to "can we go live without probes?" is a published standard rather than a personal judgement.

STEP BY STEP

1. Publish the checklist as a short, testable list - each item must be machine-checkable.
2. Enforce what you can in CI and admission policy; report the rest on a dashboard.
3. Require evidence for the untestable ones: a rollback that was actually performed, a runbook that was actually used in a game day.
4. Review the list after every incident and add the item that would have prevented it.

EVIDENCE YOU WOULD QUOTE

```
Readiness report per namespace: replicas, PDB, requests, probes, spread constraints

oc get pdb -A ; oc get deploy -A -o wide

Alert-to-SLO mapping: every page traces to a user-visible objective
```

RED FLAG - DO NOT SAY THIS

Do not make the list so long that nobody completes it. A checklist people bypass is worse than a short one they follow.

EXPECT THIS FOLLOW-UP

A critical service fails two checklist items and the launch is tomorrow. What do you do?

PART 4 · Q48-Q63 · 16 QUESTIONS

Scheduling, resources and capacity

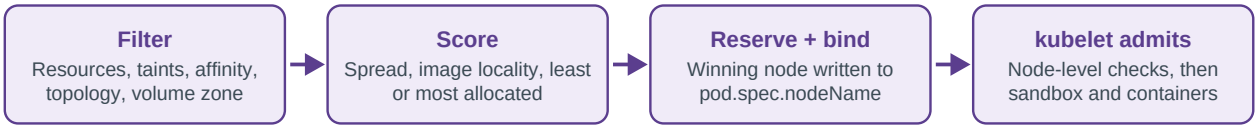
Where pods go, what they are allowed to consume, and who loses when the node runs out

Scheduling is the part of Kubernetes that most directly shapes reliability, and it is also where the most expensive misunderstandings live. Requests are not limits. A limit is not a reservation. Pending is not a bug. Autoscaling does not create capacity that does not exist. Interviewers probe here because the answers reveal whether you have run a busy cluster or only a demo one: on a quiet cluster every scheduling decision looks correct, and on a full one every shortcut becomes an incident.

Level mix - Foundation: 7 · Intermediate: 5 · Senior: 3 · Architect: 1

How a pod finds a node

Pending is not a scheduler bug, it is the scheduler telling you no node survived the filters. The event message names the exact filter that rejected each node.



Requests, limits and who dies first

Requests buy you a scheduling guarantee. Limits buy you a ceiling. QoS class decides eviction order when a node runs out of memory.

QoS class	How you get it	CPU behaviour	Under node memory pressure
Guaranteed	requests == limits for every container, CPU and memory	Eligible for exclusive CPUs under the static policy	Evicted last; safest for latency-sensitive work
Burstable	Requests set, limits higher or absent	Throttled at the limit, can borrow idle CPU	Evicted after BestEffort, ranked by usage over request
BestEffort	No requests and no limits anywhere	First to be starved of CPU	Evicted first; never use it for anything a customer notices

Q48 FOUNDATION

How does the Kubernetes scheduler decide where a pod goes?

SAY THIS

It runs in two phases. Filtering removes every node that cannot host the pod - not enough allocatable resources, an untolerated taint, a failed node selector or affinity rule, a volume that is bound to another zone. Scoring then ranks the survivors using plugins such as spreading across zones, image locality and how balanced the resulting allocation would be. The highest scoring node wins and the pod is bound to it by writing nodeName.

THINK OF IT LIKE

It is seating people in a restaurant. First you exclude tables that are too small, reserved or in the wrong section. Then you pick the best of what is left - near a window, away from the kitchen door.

WHY IT MATTERS IN PRODUCTION

The practical payoff is that Pending is never mysterious. The scheduler records exactly why each node was rejected, in the pod's events, in the form "3 node(s) had untolerated taint, 2 Insufficient memory". Reading that line and translating it is a thirty-second diagnosis. It also explains why scheduling is a point-in-time decision: the scheduler does not move a pod later just because the cluster changed.

STEP BY STEP

1. Say the two phases: filter for feasibility, score for preference.
2. Name a few real filters and a couple of scoring plugins so it is concrete.
3. Explain binding - the decision is recorded on the pod, and the kubelet acts on it.
4. Point out that the scheduler never revisits the decision; that is the descheduler's job.

EVIDENCE YOU WOULD QUOTE

```
oc describe pod <pod> | sed -n '/Events/,$p'

oc get events -n <ns> --field-selector reason=FailedScheduling

oc describe node <node> | sed -n '/Allocated resources/,$p'
```

RED FLAG - DO NOT SAY THIS

Do not say the scheduler "picks a node with space". Filtering versus scoring is precisely what is being asked.

EXPECT THIS FOLLOW-UP

A pod is Pending with 'Insufficient cpu' on every node. Is adding nodes the right fix?

Q49 FOUNDATION

Explain requests, limits and QoS classes.

SAY THIS

A request is what the scheduler reserves for the pod and it is the only number that influences placement. A limit is the hard ceiling the kernel enforces at runtime - CPU is throttled at the limit, memory over the limit gets the container OOM-killed. The relationship between the two sets the QoS class: equal requests and limits everywhere means Guaranteed, requests below limits means Burstable, nothing set means BestEffort. QoS then decides eviction order under node pressure.

THINK OF IT LIKE

A request is the seat you booked; a limit is how far you may recline. Booking nothing gets you on the plane only if there is room, and you are the first one bumped.

WHY IT MATTERS IN PRODUCTION

Two symptoms fall directly out of this. Exit code 137 with OOMKilled means the memory limit was hit - the kernel did it, not Kubernetes, and raising the limit or fixing the leak are the only real options. An application that is slow while CPU usage looks low is being throttled against its CPU quota, visible in the throttled seconds metric. Being able to name which of the two you are looking at, from the metric, is the whole point of the question.

STEP BY STEP

1. Separate the roles: requests drive scheduling, limits drive runtime enforcement.
2. Explain the asymmetry - CPU throttles, memory kills.
3. Derive the three QoS classes and say what each means at eviction time.
4. Set requests from measured usage percentiles rather than guesses, and revisit them.

EVIDENCE YOU WOULD QUOTE

```
oc adm top pod -n <ns> --containers

container_cpu_cfs_throttled_seconds_total ; container_memory_working_set_bytes

oc get pod <pod> -o jsonpath='{.status.qosClass}{"\n"}'
```

RED FLAG - DO NOT SAY THIS

Do not say limits reserve capacity. That single misunderstanding leads to clusters that are either wildly overcommitted or half empty.

EXPECT THIS FOLLOW-UP

Would you set a CPU limit on a latency-sensitive service? Argue both sides.

Q50 FOUNDATION

What are taints and tolerations, and what do the three effects do?

SAY THIS

A taint is a property on a node that repels pods; a toleration on a pod says it accepts that taint. NoSchedule means new pods without the toleration will not be placed there. PreferNoSchedule is a soft version - the scheduler avoids it if it can. NoExecute is the strong one: it also evicts pods already running that do not tolerate it, optionally after a grace period set by tolerationSeconds.

THINK OF IT LIKE

A taint is a Staff Only sign. NoSchedule means new people do not go in. PreferNoSchedule means please avoid it. NoExecute means everyone without a badge leaves now.

WHY IT MATTERS IN PRODUCTION

Two operational uses dominate. First, dedicating nodes - GPU, infrastructure or licence-restricted nodes carry a taint so only workloads that explicitly opt in land there. Second, node failure handling: the node lifecycle controller applies a NoExecute not-ready taint, and the default toleration of around five minutes is why pods do not move instantly when a node dies. Tuning that number is a real trade-off between recovery speed and thrashing during transient network blips.

STEP BY STEP

1. Define taint and toleration as a repel-and-accept pair, not as a scheduling preference.
2. Walk the three effects and what each does to new versus existing pods.
3. Give the dedicated-node use case and the node-failure use case.
4. Mention that tolerating a taint does not attract a pod - you still need affinity or a selector for that.

EVIDENCE YOU WOULD QUOTE

```
oc get nodes -o custom-columns=NAME:.metadata.name,TAINTS:.spec.taints

oc adm taint nodes <node> workload=gpu:NoSchedule

oc get pod <pod> -o jsonpath='{.spec.tolerations}' | python3 -m json.tool
```

RED FLAG - DO NOT SAY THIS

Do not say a toleration makes a pod prefer that node. Tolerations permit; they never attract.

EXPECT THIS FOLLOW-UP

You taint GPU nodes and your monitoring agent disappears from them. Why, and what fixes it?

Q51 FOUNDATION

What is node affinity?

SAY THIS

Node affinity constrains which nodes a pod may run on based on node labels. The required form is a hard filter - if no node matches, the pod stays Pending. The preferred form is a scoring hint with a weight, so the scheduler tries to honour it but will place the pod elsewhere rather than leave it unscheduled. It is the modern, more expressive replacement for a plain nodeSelector.

THINK OF IT LIKE

Required affinity is "I must have a ground-floor room". Preferred affinity is "I would like a sea view". One of those can leave you sleeping in the lobby.

WHY IT MATTERS IN PRODUCTION

The choice between required and preferred is a genuine availability decision, and interviewers listen for whether you notice. Required affinity on a label that only three nodes carry means losing those three nodes takes your service down completely, even though the cluster has capacity. Preferred affinity degrades instead - the pod lands somewhere less ideal and keeps serving. Use required only when running elsewhere would be actually wrong, such as a licence or data residency constraint.

STEP BY STEP

1. Distinguish required from preferred and state the consequence of each.
2. Show the label-based matching and why labels must be applied consistently by automation.
3. Give the availability trade-off: hard constraints reduce the pool you can fail over into.
4. Mention that node affinity is evaluated at scheduling time only, not continuously.

EVIDENCE YOU WOULD QUOTE

```
oc get nodes --show-labels

oc get pod <pod> -o jsonpath='{.spec.affinity.nodeAffinity}' | python3 -m json.tool

oc get events -n <ns> --field-selector reason=FailedScheduling
```

RED FLAG - DO NOT SAY THIS

Do not use required node affinity for a preference. It converts a capacity shortage into an outage.

EXPECT THIS FOLLOW-UP

How would you place a workload on GPU nodes without hard-pinning it to three machines?

Q52 FOUNDATION

What are pod affinity and pod anti-affinity?

SAY THIS

They place a pod relative to other pods rather than relative to node labels. Pod affinity attracts - put this cache next to the application that uses it, within the same zone or node. Anti-affinity repels - keep replicas of the same service apart so a single node or zone failure cannot take them all. The topologyKey defines what apart means: hostname, zone, or any node label.

THINK OF IT LIKE

Affinity is seating a family together. Anti-affinity is not putting all the fire marshals on the same floor.

WHY IT MATTERS IN PRODUCTION

Anti-affinity is expensive to evaluate at scale, because the scheduler has to compare against existing pods across the topology domain, and required anti-affinity on a large deployment can slow scheduling noticeably or leave pods Pending when the domain runs out. That is exactly why topology spread constraints were introduced, and saying so is a good signal that you have hit the limit in practice rather than read about the feature.

STEP BY STEP

1. Define both, and stress that topologyKey is what gives the rule meaning.
2. Give the standard uses: co-locate for latency, separate for availability.
3. Note the scheduling cost and the Pending risk of required anti-affinity at scale.
4. Say when you would switch to topology spread constraints instead.

EVIDENCE YOU WOULD QUOTE

```
oc get pod <pod> -o jsonpath='{.spec.affinity.podAntiAffinity}' | python3 -m json.tool

oc get pods -o wide -n <ns> --sort-by=.spec.nodeName

oc get events -n <ns> --field-selector reason=FailedScheduling
```

RED FLAG - DO NOT SAY THIS

Do not apply required anti-affinity to a hundred-replica Deployment without checking node count. You will simply run out of places to put pods.

EXPECT THIS FOLLOW-UP

Why might topology spread constraints be a better fit than anti-affinity here?

Q53 FOUNDATION

What is a topology spread constraint?

SAY THIS

It tells the scheduler to distribute matching pods evenly across a topology domain - zones, nodes, racks - with a maxSkew that bounds how uneven the distribution may become. whenUnsatisfiable decides what happens if the constraint cannot be met: DoNotSchedule leaves the pod Pending, ScheduleAnyway places it and accepts the imbalance. It expresses "spread these out" more efficiently and more precisely than anti-affinity.

THINK OF IT LIKE

It is dealing cards evenly around the table rather than just insisting no two cards touch. The skew is how many more cards one player may hold before you stop dealing them.

WHY IT MATTERS IN PRODUCTION

This is the mechanism behind genuine zone resilience, and the setting people get wrong is whenUnsatisfiable. DoNotSchedule with maxSkew of one across three zones sounds strict and correct, right up until one zone is down for maintenance and new pods refuse to schedule because spreading them would breach the skew. For most applications ScheduleAnyway plus an alert on imbalance gives you the availability benefit without the self-inflicted outage.

STEP BY STEP

1. Define topologyKey, maxSkew and whenUnsatisfiable as the three knobs.
2. Explain the difference in behaviour between DoNotSchedule and ScheduleAnyway.
3. Recommend the safer default for most workloads and say why.
4. Pair it with a PDB and replica minimum, because spread alone does not guarantee capacity.

EVIDENCE YOU WOULD QUOTE

```
oc get pod <pod> -o jsonpath='{.spec.topologySpreadConstraints}' | python3 -m json.tool

oc get pods -o wide -n <ns> | awk '{print $7}' | sort | uniq -c

oc get nodes -L topology.kubernetes.io/zone
```

RED FLAG - DO NOT SAY THIS

Do not set DoNotSchedule with a tight skew on a critical service without thinking about zone maintenance. It will refuse to schedule exactly when you need it to.

EXPECT THIS FOLLOW-UP

One zone is down. Explain what happens to your spread constraints and your PDB.

Q54 FOUNDATION

What is a PriorityClass, and how does preemption work?

SAY THIS

A PriorityClass assigns a numeric priority to pods. When a high-priority pod cannot be scheduled, the scheduler may preempt - evict lower-priority pods on a node to make room, then schedule the high-priority pod there. Preempted pods are terminated gracefully and go back to Pending, where they compete for capacity again. Priority also influences eviction ordering under node pressure.

THINK OF IT LIKE

It is triage in an emergency department. The ambulance case takes the bay; the person waiting with a sprained wrist goes back to the waiting room, not out of the building.

WHY IT MATTERS IN PRODUCTION

The risk is that priority values get inflated until everything is critical and the mechanism stops meaning anything. A workable scheme is a small number of published classes with clear criteria - platform and system components highest, customer-facing services next, batch and development lowest - assigned by policy rather than by whoever writes the manifest. Without that governance, priority becomes a way for one team to evict another's pods.

STEP BY STEP

1. Define priority and preemption, and note that preemption is graceful, not a kill.
2. Explain that preempted pods return to Pending and may cause a cascade if capacity is tight.
3. Propose a small, governed set of classes with documented criteria.
4. Mention preemptionPolicy Never for high-priority pods that should wait rather than evict.

EVIDENCE YOU WOULD QUOTE

```
oc get priorityclass

oc get events -A --field-selector reason=Preempted --sort-by=.lastTimestamp

oc get pod <pod> -o jsonpath='{.spec.priorityClassName} {.spec.priority}{"\n"}'
```

RED FLAG - DO NOT SAY THIS

Do not let teams set their own priority values. Everything becomes the highest priority and you are back where you started, with extra churn.

EXPECT THIS FOLLOW-UP

Preemption is causing repeated evictions in one namespace. How do you stabilise it?

Q55 INTERMEDIATE

What is a PodDisruptionBudget, and what does it not protect against?

SAY THIS

A PDB limits voluntary disruptions - drains, evictions, MCO node updates - by declaring either minAvailable or maxUnavailable for pods matching a selector. The eviction API respects it and refuses evictions that would breach it. What it does not protect against is involuntary disruption: a node crashing, a kernel panic, a hypervisor failure or a zone outage ignore PDBs entirely, because nothing asked permission.

THINK OF IT LIKE

It is a rota rule saying at least two staff must be on the floor. It stops the manager scheduling everyone off at once; it does not stop three people catching flu.

WHY IT MATTERS IN PRODUCTION

PDBs are where availability meets maintainability, and they cut both ways. Too permissive and a node drain takes your service below capacity. Too strict - minAvailable equal to the replica count, or a PDB on a single-replica Deployment - and the drain can never succeed, which stalls MachineConfig rollout and therefore stalls cluster upgrades. That stall is one of the most common real-world upgrade blockers.

STEP BY STEP

1. Define voluntary versus involuntary disruption and say which one a PDB governs.
2. Explain how minAvailable and maxUnavailable interact with replica count.
3. Give the blocking failure mode - a PDB that can never be satisfied stalls drains and upgrades.
4. Pair PDBs with adequate replicas and spread, and test them by actually draining a node.

EVIDENCE YOU WOULD QUOTE

```
oc get pdb -A -o wide

oc adm drain <node> --ignore-daemonsets --delete-emptydir-data --dry-run=server

oc get events -A --field-selector reason=EvictionBlocked
```

RED FLAG - DO NOT SAY THIS

Do not claim a PDB protects against node failure. It is the classic senior trap and it is asked deliberately.

EXPECT THIS FOLLOW-UP

A drain has been blocked for thirty minutes by a PDB. What are your options, in order?

Q56 INTERMEDIATE

How do ResourceQuota and LimitRange work together?

SAY THIS

ResourceQuota caps the total a namespace may consume - aggregate CPU and memory requests and limits, storage, and object counts. LimitRange operates per object, setting defaults and minimum and maximum values for individual containers. They are complementary: LimitRange makes sure every pod has sensible requests, and ResourceQuota makes sure the namespace as a whole cannot exceed its share.

THINK OF IT LIKE

ResourceQuota is the household's monthly electricity budget. LimitRange is the rule that no single appliance may draw more than a certain wattage, and that unlabelled appliances get a default rating.

WHY IT MATTERS IN PRODUCTION

There is an interaction that surprises people: once a ResourceQuota that limits requests exists, every pod in the namespace must specify requests, or creation is rejected. Without a LimitRange supplying defaults, that breaks every existing manifest that omitted them. So the correct rollout order is LimitRange first, then quota - and doing it in the wrong order in production is a memorable way to learn the lesson.

STEP BY STEP

1. Separate the scopes: namespace aggregate versus per-container.
2. State the rejection behaviour when quota exists and requests are missing.
3. Roll out LimitRange first so defaults exist, then apply the quota.
4. Monitor quota utilisation and alert before it is exhausted, because exhaustion looks like a scheduling bug to the team.

EVIDENCE YOU WOULD QUOTE

```
oc describe quota -n <ns>

oc get limitrange -n <ns> -o yaml

kube_resourcequota{type="used"} / kube_resourcequota{type="hard"}
```

RED FLAG - DO NOT SAY THIS

Do not apply a quota to a live namespace without a LimitRange in place. Every deployment without explicit requests will start failing immediately.

EXPECT THIS FOLLOW-UP

A team says their deployments started failing after you enabled quota. What happened?

Q57 INTERMEDIATE

Why might a HorizontalPodAutoscaler fail to scale?

SAY THIS

Usually one of five things. The metric is unavailable - CPU-based HPA needs resource requests set, and custom metrics need an adapter. The metric never crosses the target. There is no cluster capacity, so new pods go Pending and the HPA looks stuck. Stabilisation windows and scaling policies are damping the change deliberately. Or something else is writing the replica count - a GitOps controller reconciling replicas will fight the HPA forever.

THINK OF IT LIKE

It is a thermostat that will not turn the heating up. Either it cannot read the temperature, the room is not actually cold, the boiler has no fuel, it is waiting out its cycle, or someone else keeps turning the dial back.

WHY IT MATTERS IN PRODUCTION

The GitOps conflict is the one that produces the most confusing incident. Argo CD sees replicas: 3 in Git, the HPA sets 8, Argo reverts to 3, the HPA scales up again, and the service oscillates under load while both systems believe they are correct. The fix is to remove replicas from the tracked manifest or tell the GitOps tool to ignore that field - and knowing that specific interaction is a strong practical signal.

STEP BY STEP

1. Check the HPA's own status conditions - they name the reason directly.
2. Verify metrics are available and that resource requests exist for CPU-based targets.
3. Look for Pending pods, which means the constraint is cluster capacity, not the HPA.
4. Check for a competing writer of replicas, especially a GitOps controller, and resolve ownership of the field.

EVIDENCE YOU WOULD QUOTE

```
oc describe hpa <name> | sed -n '/Conditions/,$p'

oc get --raw /apis/metrics.k8s.io/v1beta1/namespaces/<ns>/pods | head -c 400

oc get pods -n <ns> --field-selector status.phase=Pending
```

RED FLAG - DO NOT SAY THIS

Do not conclude "the HPA is broken" without reading its conditions. They usually contain the literal answer.

EXPECT THIS FOLLOW-UP

The HPA and Argo CD are fighting over replica count. How do you resolve it properly?

Q58 INTERMEDIATE

When do you use the ClusterAutoscaler versus an HPA?

SAY THIS

They solve different shortages and you usually need both. The HPA adds pod replicas when a workload metric rises. The ClusterAutoscaler adds nodes when pods are Pending because no node can fit them, and removes underutilised nodes when their pods can be placed elsewhere. The HPA is useless without capacity, and the ClusterAutoscaler is useless if nothing is asking for more pods.

THINK OF IT LIKE

The HPA hires more staff for the shift. The ClusterAutoscaler opens another floor of the building. Hiring twenty people into a room with ten desks helps nobody.

WHY IT MATTERS IN PRODUCTION

Scale-down is where the real operational detail lives. The autoscaler will not remove a node if it hosts pods that cannot be rescheduled - pods without a controller, pods with strict PDBs, pods using local storage, or pods in namespaces excluded by annotation. That is why clusters end up paying for near-empty nodes, and being able to name those blockers is a strong practical signal. Scale-up latency also matters: node provisioning takes minutes, so burst-sensitive services need headroom or over-provisioning pods rather than pure autoscaling.

STEP BY STEP

1. Separate the two axes: replicas versus nodes.
2. Explain the trigger for each - workload metric versus unschedulable pods.
3. Name the common scale-down blockers, because that is where money is wasted.

4 Address scale-up latency with headroom or low-priority placeholder pods for bursty workloads.

EVIDENCE YOU WOULD QUOTE

```
oc get clusterautoscaler,machineautoscaler -A

oc -n openshift-machine-api logs deploy/cluster-autoscaler-default --tail=100

oc get pods -A --field-selector status.phase=Pending -o wide
```

RED FLAG - DO NOT SAY THIS

Do not present autoscaling as a substitute for capacity planning. It changes how you buy capacity, not whether you need to think about it.

EXPECT THIS FOLLOW-UP

Nodes never scale down even at 20% utilisation. Give me three likely causes.

Q59 INTERMEDIATE

What problem does the descheduler solve?

SAY THIS

The scheduler places a pod once and never revisits it, so over time a cluster drifts: nodes added after a rollout stay empty, pods violate affinity rules that were introduced later, and load becomes lumpy. The descheduler periodically evicts pods that now sit badly according to configured strategies - duplicates on one node, low node utilisation, violated topology constraints - and lets the scheduler place them again.

THINK OF IT LIKE

It is rebalancing an investment portfolio. Nothing was wrong when you bought it; drift is just what time does to a fixed allocation.

WHY IT MATTERS IN PRODUCTION

The important caveat is that the descheduler evicts, it does not place. Eviction respects PDBs and priority, but if the cluster is genuinely full the evicted pod may come back to the same node or sit Pending - so an aggressive configuration on a tight cluster produces churn rather than balance. Start with conservative strategies, run it during quiet periods, and watch eviction counts before widening its remit.

STEP BY STEP

1 Explain the drift problem the scheduler cannot solve by design.
2 Name a couple of concrete strategies and what each one is for.
3 State clearly that it evicts rather than reschedules, and what that implies on a full cluster.
4 Introduce it conservatively, respect PDBs, and monitor eviction rate as a safety metric.

EVIDENCE YOU WOULD QUOTE

```
oc get kubedescheduler cluster -n openshift-kube-descheduler-operator -o yaml

oc get events -A --field-selector reason=Evicted --sort-by=.lastTimestamp | tail -20

oc get pods -o wide -A | awk '{print $8}' | sort | uniq -c | sort -rn | head
```

RED FLAG - DO NOT SAY THIS

Do not enable aggressive descheduling on a cluster that is already near capacity. You will trade imbalance for continuous churn.

EXPECT THIS FOLLOW-UP

After enabling the descheduler, eviction counts spiked. What would you change?

Q60 SENIOR

A pod has been Pending for ten minutes. Walk me through the diagnosis.

SAY THIS

I start with impact - is this one pod or a whole service - then read the FailedScheduling event, because it states the reason per node. From there it is a short decision tree: insufficient resources points at capacity or oversized requests; untolerated taints points at node dedication; node affinity or selector mismatch points at labels; volume node affinity conflict points at a PV bound to a different zone; and no events at all points at a scheduler or admission problem rather than a placement one.

THINK OF IT LIKE

The rejection letter tells you why. You do not need to guess whether it was your grades or your postcode - it is written at the bottom of the page.

WHY IT MATTERS IN PRODUCTION

The distinguishing senior move is checking whether the request is realistic before adding capacity. A pod asking for 16 CPUs on a cluster of 8-core nodes will be Pending forever and no amount of scaling fixes it. Equally, a PVC bound in zone A pins the pod to zone A, so a cluster with plenty of capacity in zones B and C still cannot place it. Both are common and both look like capacity problems until you read the event.

STEP BY STEP

1. State impact and scope first - one pod, one Deployment, or cluster-wide Pending.
2. Read the FailedScheduling event verbatim and translate each clause.
3. Sanity-check the request against the largest allocatable node before considering capacity.
4. Check zone-bound volumes and node labels, then take the smallest fix: adjust requests, relax a constraint, or add capacity deliberately.

EVIDENCE YOU WOULD QUOTE

```
oc describe pod <pod> | sed -n '/Events/,$p'

oc describe node <node> | sed -n '/Allocated resources/,$p'

oc get pv <pv> -o jsonpath='{.spec.nodeAffinity}' | python3 -m json.tool
```

RED FLAG - DO NOT SAY THIS

Do not add nodes as a reflex. If the request exceeds any single node's allocatable capacity, more nodes change nothing.

EXPECT THIS FOLLOW-UP

The event says 'volume node affinity conflict'. Explain what happened.

Q61 SENIOR

What are the considerations for the Vertical Pod Autoscaler?

SAY THIS

VPA recommends and can apply right-sized requests based on observed usage, which is genuinely useful because most requests are guesses that were never revisited. The catch is that changing a pod's resources requires recreating it, so applying mode causes restarts, and VPA in applying mode conflicts with an HPA on the same resource metric. In practice I run VPA in recommendation mode, feed the numbers into a right-sizing review, and change requests through Git.

THINK OF IT LIKE

It is a tailor measuring you accurately. Very useful - but you do not want the alterations happening while you are wearing the suit in a meeting.

WHY IT MATTERS IN PRODUCTION

Recommendation mode solves a real and expensive problem: clusters routinely run at low actual utilisation while appearing full, because requests are two or three times real usage. Feeding VPA recommendations into a quarterly right-sizing exercise typically reclaims meaningful capacity with no risk. Turning on applying mode without understanding the restart behaviour is how you get a surprise rolling restart of production at 2pm.

STEP BY STEP

1. Explain what VPA observes and what it produces.
2. State the restart consequence of applying mode and the HPA conflict on the same metric.
3. Recommend recommendation mode plus a human-reviewed change through Git.
4. Quantify the benefit - compare requested versus actually used resources cluster-wide.

EVIDENCE YOU WOULD QUOTE

```
oc get vpa -A -o
custom-columns=NS:.metadata.namespace,NAME:.metadata.name,MODE:.spec.updatePolicy.updateMode

oc describe vpa <name> | sed -n '/Recommendation/,$p'

sum(kube_pod_container_resource_requests{resource="cpu"}) vs
sum(rate(container_cpu_usage_seconds_total[1h]))
```

RED FLAG - DO NOT SAY THIS

Do not run VPA in applying mode alongside an HPA on CPU. They will fight, and the pods will restart while they do.

EXPECT THIS FOLLOW-UP

Cluster CPU requests are at 85% but actual usage is 25%. What do you do about it?

Q62 SENIOR

How do you do capacity planning for an OpenShift cluster?

SAY THIS

I plan against four numbers, not one. Allocatable capacity after system and platform reservations. Committed capacity, meaning the sum of requests. Actual utilisation. And failure headroom - the capacity that must stay free so that losing a node, or draining one for an upgrade, does not cause evictions. Then I project growth from real trend data and buy before the headroom is consumed, rather than reacting to the first Pending pod.

THINK OF IT LIKE

It is planning a car park. Total spaces, spaces already allocated to permit holders, spaces actually occupied at peak, and the row you keep clear for the fire engine.

WHY IT MATTERS IN PRODUCTION

The distinction between committed and used is where most clusters are mismanaged. A cluster can be 90% committed and 25% used, which means it refuses new work while sitting nearly idle - the answer there is right-sizing requests, not buying nodes. The reverse, high utilisation with low commitment, means requests are too low and you are one traffic spike away from evictions. Naming which of those you are in, with numbers, is the answer.

STEP BY STEP

1. Measure all four numbers per node pool, not as a cluster average.
2. Define failure headroom explicitly: N+1 node loss plus one node drained for maintenance.
3. Diagnose the gap between committed and used, and fix requests before buying capacity.
4. Forecast from trend, set a threshold that triggers procurement, and review it monthly.

EVIDENCE YOU WOULD QUOTE

```
oc describe node | grep -A6 'Allocated resources'

sum by (node) (kube_pod_container_resource_requests{resource="memory"})

oc adm top nodes ; oc get machineset -A -o wide
```

RED FLAG - DO NOT SAY THIS

Do not plan capacity from average utilisation alone. Averages hide both the peak that evicts you and the commitment that blocks scheduling.

EXPECT THIS FOLLOW-UP

Committed CPU is 90%, used is 25%. Do you buy nodes? Defend your answer.

Q63 ARCHITECT

How would you set overcommit policy for a shared cluster?

SAY THIS

I set it per workload class rather than cluster-wide. Latency-sensitive and regulated workloads get Guaranteed QoS on dedicated or lightly overcommitted node pools. General services run Burstable with requests set from measured p95 and limits at a defensible multiple. Batch and development run on heavily overcommitted pools with low priority, so they absorb the risk of the overcommit. The policy is published, enforced by LimitRange and admission, and reviewed against actual eviction and throttling data.

THINK OF IT LIKE

Airlines overbook economy, not the flight deck. The policy is deliberate, differentiated, and based on measured no-show rates rather than optimism.

WHY IT MATTERS IN PRODUCTION

The reason to make this explicit is that overcommit is happening whether or not you decided it. Without a policy, each team picks requests independently and the cluster's real overcommit ratio is an accident. With a policy, you can answer the two questions that matter to a business: what does this cluster cost per workload class, and what is the probability that a spike causes an eviction. Reviewing eviction and throttling metrics closes the loop.

STEP BY STEP

1. Define workload classes and the QoS and node pool each is entitled to.
2. Set requests from measured percentiles and document the limit multiple per class.
3. Enforce with LimitRange, quota, priority classes and node pool separation.
4. Review quarterly against eviction counts, throttling rates and cost per class, and adjust the ratios with evidence.

EVIDENCE YOU WOULD QUOTE

```
kube_pod_container_resource_requests vs container_memory_working_set_bytes by workload class

oc get events -A --field-selector reason=Evicted | wc -l

container_cpu_cfs_throttled_periods_total / container_cpu_cfs_periods_total
```

RED FLAG - DO NOT SAY THIS

Do not apply one overcommit ratio to the whole cluster. It will be too risky for the critical workloads and too wasteful for the batch ones.

EXPECT THIS FOLLOW-UP

Finance wants 30% cost reduction. Which lever do you pull first, and what is the risk?

PART 5 · Q64-Q79 · 16 QUESTIONS

Services, DNS and ingress

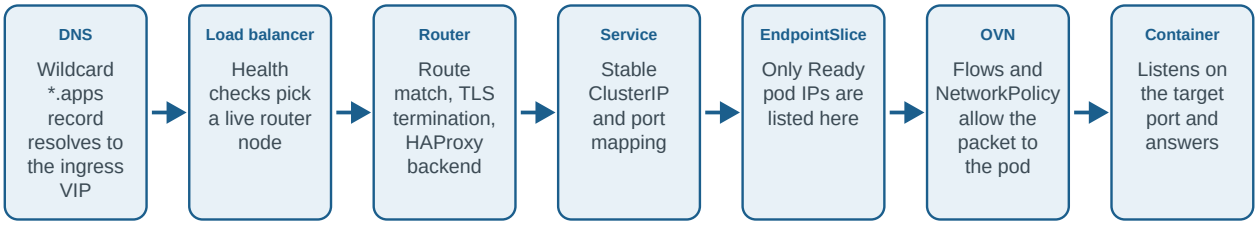
Following one request from the browser to the container, and back

Networking questions are the fastest way for an interviewer to find out whether you have actually operated a cluster. Anyone can define a Service. Far fewer people can say, without hesitating, which seven things stand between a browser and a container and what each one looks like when it breaks. That path - DNS, load balancer, router, Service, EndpointSlice, network policy, container - is the backbone of this part. Learn it in order and you will diagnose most outages faster than the person asking the question.

Level mix - Foundation: 7 · Intermediate: 5 · Senior: 3 · Architect: 1

One HTTPS request, seven places it can die

Learn this path in order. Almost every "the app is down" incident is one hop in this chain, and naming the hop is how you sound senior.



Q64 FOUNDATION

What is a Service, and what does ClusterIP give you?

SAY THIS

A Service is a stable virtual address in front of a changing set of pods. Pods come and go with new IPs on every restart, so the Service gives you one name and one ClusterIP that stays put, plus a label selector that decides which pods are behind it. The ClusterIP is virtual - nothing listens on it - and traffic sent to it is rewritten to a real pod IP by the node's datapath.

THINK OF IT LIKE

It is a company's main phone number. Staff change, desks move, extensions get reassigned - the number on the letterhead does not.

WHY IT MATTERS IN PRODUCTION

Two things follow that matter in incidents. First, because the ClusterIP is virtual, you cannot ping your way to a diagnosis - a Service with no matching pods answers nothing at all while looking perfectly healthy in oc get svc. Second, the selector is the whole relationship: change a pod label or a Service selector by one character and traffic stops, with no error anywhere except an empty EndpointSlice.

STEP BY STEP

1. Define the problem it solves: stable identity in front of ephemeral pods.
2. Explain the selector-to-endpoint relationship, because that is where failures live.
3. Note that the ClusterIP is virtual and implemented in the datapath, not by a process.
4. Give the first check when a Service seems dead: does its EndpointSlice have any addresses?

EVIDENCE YOU WOULD QUOTE

```
oc get svc <svc> -o jsonpath='{.spec.selector}{"\n"}'

oc get endpointslice -n <ns> -l kubernetes.io/service-name=<svc> -o yaml | head -40
```

```
oc get pods -n <ns> --show-labels
```

RED FLAG - DO NOT SAY THIS

Do not say a Service "load balances between pods" and stop there. The selector and readiness relationship is the part that fails.

EXPECT THIS FOLLOW-UP

A Service has no endpoints but the pods are Running. Give me two causes.

Q65 FOUNDATION

What are the Service types and when do you use each?

SAY THIS

ClusterIP is internal-only and the default. NodePort exposes the Service on a port on every node, which is mostly a building block rather than something you expose to users. LoadBalancer asks the infrastructure for an external load balancer and is how you expose non-HTTP traffic. ExternalName is just a DNS CNAME to something outside the cluster. On OpenShift, HTTP and HTTPS normally go through a Route rather than any of these.

THINK OF IT LIKE

ClusterIP is an internal extension, NodePort is a side door with a number on it, LoadBalancer is a public reception desk, and ExternalName is a sign that says the office you want is in another building.

WHY IT MATTERS IN PRODUCTION

The design point interviewers listen for is that LoadBalancer Services cost real money and real IP addresses - one per Service on most clouds - which is exactly why HTTP traffic is consolidated behind an ingress layer. Reaching for LoadBalancer per application is a common and expensive pattern in teams migrating from a per-service load balancer world.

STEP BY STEP

1. List the four types with one sentence of purpose each.
2. State the OpenShift default for HTTP: Route through the shared ingress layer.
3. Explain when LoadBalancer is genuinely right - non-HTTP protocols, or dedicated ingress.
4. Mention the cost and IP consumption trade-off, because that is the design conversation.

EVIDENCE YOU WOULD QUOTE

```
oc get svc -A -o custom-columns=NS:.metadata.namespace,NAME:.metadata.name,TYPE:.spec.type

oc get svc <svc> -o jsonpath='{.status.loadBalancer}' | python3 -m json.tool

oc get route -A | head
```

RED FLAG - DO NOT SAY THIS

Do not propose a LoadBalancer Service per application for HTTP. It is what the router layer exists to avoid.

EXPECT THIS FOLLOW-UP

A team needs to expose a TCP database port externally. What do you recommend?

Q66 FOUNDATION

What is an EndpointSlice and why did it replace Endpoints?

SAY THIS

An EndpointSlice lists the actual pod IPs and ports currently behind a Service, along with readiness and topology information. It replaced the older Endpoints object because a single Endpoints object had to contain every backend, so any change rewrote the whole thing and every node received the full update. Slices chunk that into pieces, which scales far better on large Services.

THINK OF IT LIKE

It is the difference between reprinting the entire phone book because one person moved, and reprinting only the page they were on.

WHY IT MATTERS IN PRODUCTION

For troubleshooting, EndpointSlice is the single most useful object in the request path. Only pods that pass their readiness probe appear as ready addresses, so an empty or all-unready slice immediately tells you whether the problem is routing or the application. That one check separates a router or DNS problem from a workload problem in about five seconds.

STEP BY STEP

1. Define it as the live backend list for a Service, including readiness.
2. Explain the scaling motivation versus the old Endpoints object.
3. Show the diagnostic use: empty slice means selector or readiness, not networking.
4. Mention topology hints, which allow zone-aware routing on supported setups.

EVIDENCE YOU WOULD QUOTE

```
oc get endpointslice -n <ns> -l kubernetes.io/service-name=<svc>

oc get endpointslice <slice> -o jsonpath='{.endpoints[*].conditions.ready}{"\n"}'

oc get pods -n <ns> -o wide --show-labels
```

RED FLAG - DO NOT SAY THIS

Do not troubleshoot a Service without looking at its EndpointSlice. You will spend an hour on the network for a readiness problem.

EXPECT THIS FOLLOW-UP

The slice lists three addresses, all with ready false. What does that tell you?

Q67 FOUNDATION

What is a Route, and how does it differ from an Ingress?

SAY THIS

A Route is OpenShift's native ingress object. It maps a hostname and path to a Service, and the OpenShift router - HAProxy, managed by the Ingress Operator - implements it. Ingress is the portable Kubernetes object that does a similar job; on OpenShift an Ingress is converted into a Route behind the scenes. Routes expose OpenShift-specific capabilities directly, such as re-encrypt termination and weighted backends for canary traffic.

THINK OF IT LIKE

Ingress is the international standard plug; a Route is the local socket that has been wired in for years and has a few extra pins.

WHY IT MATTERS IN PRODUCTION

In practice you use whichever fits the portability requirement. A team that deploys to both OpenShift and vanilla Kubernetes should write Ingress and accept the smaller feature set. A team that lives on OpenShift gets more from Routes - especially weighted backends, which give you canary releases without installing a service mesh. Knowing that Ingress is translated into Routes explains why you sometimes see Routes nobody created.

STEP BY STEP

1. Define Route as the OpenShift-native ingress object implemented by HAProxy.
2. State the translation: Ingress objects become Routes on OpenShift.
3. Name what Routes add - termination modes, weighted backends, per-route annotations.
4. Choose based on portability requirements and say so explicitly.

EVIDENCE YOU WOULD QUOTE

```
oc get route -n <ns> -o wide

oc get ingress -n <ns> ; oc get route -n <ns> -o
jsonpath='{.items[*].metadata.ownerReferences[*].kind}{"\n"}'

oc get route <route> -o jsonpath='{.status.ingress[*].conditions}' | python3 -m json.tool
```

RED FLAG - DO NOT SAY THIS

Do not say Routes and Ingress are interchangeable. The differences show up exactly when you need re-encrypt or weighting.

EXPECT THIS FOLLOW-UP

How would you do a 10% canary release using only Route features?

Q68 FOUNDATION

How does DNS work inside an OpenShift cluster?

SAY THIS

CoreDNS runs as the cluster DNS service, managed by the DNS Operator, and every pod's resolv.conf points at it. Services get records of the form service.namespace.svc.cluster.local, and the search path in resolv.conf lets pods use short names inside their own namespace. Anything not matching the cluster domain is forwarded upstream to the resolvers the cluster is configured with.

THINK OF IT LIKE

It is an internal switchboard with a local directory. Ask for a colleague by first name and it works; ask for an outside number and the call is forwarded to the public network.

WHY IT MATTERS IN PRODUCTION

DNS causes an unfair share of outages because everything depends on it and its failures are indirect - applications report connection timeouts, not resolution failures. The two classics are an upstream forwarder that is slow or unreachable, which makes every external lookup take five seconds, and ndots search-path behaviour turning one external lookup into several queries. Knowing to test resolution from inside a pod rather than from your laptop is the practical skill.

STEP BY STEP

1. Name the components: CoreDNS pods, the DNS Operator, per-pod resolv.conf.

2 Explain the record format and the search path behaviour for short names.
3 Describe upstream forwarding and why a slow forwarder looks like an application problem.
4 Always test from inside an affected pod, then from the node, to localise it.

EVIDENCE YOU WOULD QUOTE

```
oc exec <pod> -- cat /etc/resolv.conf

oc exec <pod> -- getent hosts <svc>.<ns>.svc.cluster.local

oc get dns.operator/default -o yaml | sed -n '/servers:/,$p'
```

RED FLAG - DO NOT SAY THIS

Do not test DNS from your workstation and conclude the cluster is fine. The only view that matters is from inside the pod.

EXPECT THIS FOLLOW-UP

External lookups take five seconds from every pod. What do you suspect?

Q69 FOUNDATION

What is an IngressController?

SAY THIS

An IngressController is the OpenShift object that defines a router deployment: which namespaces and routes it admits, its wildcard domain, replica count, node placement, TLS profile and how it is exposed. The Ingress Operator reconciles it into a router deployment and the corresponding Service. The default one serves the cluster's apps wildcard domain; you can create additional ones for isolation.

THINK OF IT LIKE

It is the specification for a reception desk: which visitors it accepts, which building entrance it sits at, how many staff it has, and what identification it checks.

WHY IT MATTERS IN PRODUCTION

The reason this matters is that everything about ingress behaviour is configured here rather than on individual Routes - TLS minimum version and cipher profile, HTTP/2, proxy protocol, logging, and the placement that decides which nodes handle external traffic. When someone asks how you would enforce TLS 1.2 minimum for all external traffic, this object is the answer, not a per-route annotation.

STEP BY STEP

1 Define it as the declarative spec for a router, reconciled by the Ingress Operator.
2 List the properties it controls - domain, admission scope, replicas, placement, TLS profile.
3 Explain the relationship to the router pods and the exposing Service.
4 Give the use case for additional controllers: separating internal from external traffic.

EVIDENCE YOU WOULD QUOTE

```
oc -n openshift-ingress-operator get ingresscontroller default -o yaml | head -40

oc -n openshift-ingress get pods -o wide

oc -n openshift-ingress-operator get ingresscontroller default -o
jsonpath='{.status.conditions}' | python3 -m json.tool
```

RED FLAG - DO NOT SAY THIS

Do not describe router configuration as something you set per Route. Most of it is controller-level and cluster-wide.

EXPECT THIS FOLLOW-UP

How would you enforce a minimum TLS version for all external traffic?

Q70 FOUNDATION

What is SNI and why does it matter for routing?

SAY THIS

Server Name Indication is a TLS extension where the client sends the hostname it wants during the handshake, before any encrypted data. That lets one IP address and one port serve many different certificates - the router reads the SNI value and presents the right one. Without it, a single listener could only serve one certificate, and shared ingress would not work.

THINK OF IT LIKE

It is telling reception who you are visiting before they let you through the door, rather than after. That is the only way one desk can serve fifty companies in the building.

WHY IT MATTERS IN PRODUCTION

SNI is why passthrough routes work at all: the router cannot decrypt the traffic, so the only thing it can route on is the hostname in the handshake. It is also the cause of a specific class of failure - an old client that does not send SNI gets the router's default certificate, and the user sees a certificate name mismatch that looks like a certificate problem but is actually a client capability problem.

STEP BY STEP

1. Define SNI as the hostname sent in the TLS ClientHello, before encryption.
2. Explain how it allows one IP and port to serve many certificates.
3. Connect it to passthrough routes, where SNI is the only routing information available.
4. Name the failure signature: no SNI means the default certificate and a name mismatch error.

EVIDENCE YOU WOULD QUOTE

```
openssl s_client -connect <host>:443 -servername <host> </dev/null 2>/dev/null | head -20

oc get route <route> -o jsonpath='{.spec.tls.termination}{"\n"}'

oc -n openshift-ingress logs deploy/router-default --tail=50
```

RED FLAG - DO NOT SAY THIS

Do not claim passthrough routes can do path-based routing. Without decryption there is no path to route on.

EXPECT THIS FOLLOW-UP

A legacy client gets a certificate mismatch that browsers do not. What is happening?

Q71 INTERMEDIATE

Explain edge, re-encrypt and passthrough termination.

SAY THIS

Edge terminates TLS at the router and talks plain HTTP to the pod - simplest, and the internal hop is unencrypted. Re-encrypt terminates at the router, then opens a new TLS connection to the pod, validating the backend certificate against a destination CA - so traffic is encrypted end to end and the router can still see and route on paths and headers. Passthrough does not terminate at all; the TLS connection goes straight to the pod, so the pod owns the certificate and the router routes only on SNI.

THINK OF IT LIKE

Edge is opening the envelope at the post room and hand-delivering the letter. Re-encrypt is opening it, reading the address, and sealing it in a new envelope. Passthrough is delivering the sealed envelope untouched.

WHY IT MATTERS IN PRODUCTION

In regulated environments re-encrypt is usually the default, because it satisfies encryption-in-transit requirements while keeping the operational benefits of a router that can see the request. Passthrough is chosen when the application must terminate TLS itself - mutual TLS or client certificate authentication - and the cost is that you lose path routing, header injection and centralised certificate management. Saying that trade-off out loud is what gets the mark.

STEP BY STEP

1. Describe where TLS terminates in each mode and what the internal hop looks like.
2. State what the router can and cannot do in each mode - paths, headers, SNI only.
3. Give the compliance angle: internal encryption requirements usually point at re-encrypt.
4. Name the certificate ownership consequence for passthrough - the app manages rotation.

EVIDENCE YOU WOULD QUOTE

```
oc get route <route> -o jsonpath='{.spec.tls}' | python3 -m json.tool

openssl s_client -connect <host>:443 -servername <host> </dev/null 2>/dev/null | openssl x509
-noout -subject -dates

oc get route <route> -o jsonpath='{.spec.tls.destinationCACertificate}' | head -c 100
```

RED FLAG - DO NOT SAY THIS

Do not default to edge in a regulated environment without saying that the router-to-pod hop is unencrypted. That is a finding waiting to happen.

EXPECT THIS FOLLOW-UP

An application needs client certificate authentication. Which mode, and what do you lose?

Q72 INTERMEDIATE

What does route admission policy control?

SAY THIS

It controls which Routes a given IngressController will serve. The two main levers are namespace selection - which namespaces this controller admits routes from - and the policy for claims across namespaces, which decides whether two namespaces may both claim the same hostname. The default is strict, so the first claim wins and later ones are rejected, which prevents one tenant from hijacking another's hostname.

THINK OF IT LIKE

It is the rule that stops a second shop putting up the same street address. First registration keeps the address; everyone else is told to pick another.

WHY IT MATTERS IN PRODUCTION

This is the mechanism behind a genuinely confusing symptom: a Route that exists, looks correct, and simply is not served. The status on the Route says it was not admitted and usually why - a hostname already claimed elsewhere, or a namespace this controller does not select. It is also how you build tenant isolation with separate internal and external routers, each admitting a different set of namespaces by label.

STEP BY STEP

1. Explain the two controls: namespace selection and cross-namespace hostname claims.
2. Describe the default strict behaviour and why it exists.
3. Show where the rejection is visible - the Route's status ingress conditions.
4. Give the design use: label-based routing of namespaces to internal or external controllers.

EVIDENCE YOU WOULD QUOTE

```
oc get route <route> -o jsonpath='{.status.ingress[*].conditions}' | python3 -m json.tool

oc -n openshift-ingress-operator get ingresscontroller <name> -o
jsonpath='{.spec.routeAdmission}{"\n"}'

oc get route -A -o custom-columns=NS:.metadata.namespace,HOST:.spec.host | sort -k2 | uniq -d
-f1
```

RED FLAG - DO NOT SAY THIS

Do not loosen the cross-namespace claim policy to fix a conflict. You are removing a tenant-isolation control to solve a naming problem.

EXPECT THIS FOLLOW-UP

A Route exists but is not being served. Walk me through it.

Q73 INTERMEDIATE

How does traffic actually get from a Service IP to a pod?

SAY THIS

The Service ClusterIP is virtual, so something on the node has to rewrite it. With OVN-Kubernetes, the endpoint list is programmed into OVN load balancers and implemented as OpenFlow rules in the node's integration bridge, so the destination is rewritten to a real pod IP and the reply is translated back through conntrack. Traffic to a pod on another node is then encapsulated with Geneve and sent over the node network.

THINK OF IT LIKE

It is a mailroom that rewrites the internal address on the envelope before it leaves the building, and rewrites the sender on the reply so the conversation still makes sense.

WHY IT MATTERS IN PRODUCTION

Knowing that the datapath is per-node explains a symptom that otherwise looks impossible: the Service works from most pods and fails from one node. That is a node-local datapath or OVS problem, not a Service problem, and it points you at ovnkube pods and OVS flows on that specific node rather than at the object definitions. It also explains why conntrack exhaustion on a busy node causes intermittent connection failures across unrelated services.

STEP BY STEP

1. State that the ClusterIP is virtual and translated in the node datapath.
2. Name the components: OVN load balancer entries, OpenFlow rules on br-int, conntrack.
3. Explain cross-node traffic and Geneve encapsulation, which is where MTU issues come from.
4. Use the per-node nature diagnostically: test from several nodes to localise the failure.

EVIDENCE YOU WOULD QUOTE

```
oc -n openshift-ovn-kubernetes get pods -o wide | grep <node>

oc debug node/<node> -- chroot /host ovs-vsctl show | head -20

oc exec <pod> -- curl -sS -m 3 http://<svc>.<ns>.svc:8080/healthz
```

RED FLAG - DO NOT SAY THIS

Do not say "kube-proxy iptables rules" for an OVN-Kubernetes cluster without qualifying it. The implementation matters when you are asked where to look.

EXPECT THIS FOLLOW-UP

The Service works from every node except one. What is your next step?

Q74 INTERMEDIATE

What is a NetworkPolicy, and how do you roll out default-deny safely?

SAY THIS

A NetworkPolicy is namespace-scoped, pod-selected, allow-only firewalling for pod traffic. The important rule is that a pod is unrestricted until some policy selects it, and from that moment only what policies explicitly allow is permitted. To roll out default-deny safely I first observe real traffic, write the allow rules from that evidence, apply the policies to a non-critical namespace, verify, then apply default-deny - never the other way around.

THINK OF IT LIKE

It is fitting door locks in an occupied building. You first find out who genuinely needs which door, hand out those keys, and only then change the locks.

WHY IT MATTERS IN PRODUCTION

The dependencies people forget are the ones that break everything: DNS to the cluster DNS service, the monitoring stack scraping metrics endpoints, the ingress router reaching application pods, and health probes. Blocking any of those turns a security improvement into an outage. Network Observability or flow logs give you the real dependency map instead of the one on the architecture diagram, which is usually out of date.

STEP BY STEP

1. Explain the selection model: unrestricted until selected, then allow-list only.
2. Gather evidence of real flows before writing rules - observability first, policy second.
3. Write explicit allows including DNS, monitoring, ingress and probes.
4. Apply to one namespace, verify service behaviour, then extend; keep a documented rollback.

EVIDENCE YOU WOULD QUOTE

```
oc get networkpolicy -n <ns> -o yaml

oc exec <pod> -- curl -sS -m 3 <target>:<port> # before and after

Network Observability flow data filtered by namespace and dropped verdict
```

RED FLAG - DO NOT SAY THIS

Do not apply default-deny across the cluster in one change. The DNS and monitoring breakages alone will make it memorable for the wrong reasons.

EXPECT THIS FOLLOW-UP

After applying default-deny, metrics stopped being collected. What rule is missing?

Q75 INTERMEDIATE

Why would you run multiple IngressControllers?

SAY THIS

To separate traffic that should not share a path. The common split is internal versus external: one controller on an internal load balancer serving internal namespaces, another facing the internet with stricter TLS settings and rate limiting. Other reasons are dedicating router capacity to a high-volume tenant, giving a business unit its own wildcard domain and certificate, or isolating a controller with different TLS or logging requirements.

THINK OF IT LIKE

It is having a staff entrance and a public entrance. Same building, different security checks, and a queue at one does not block the other.

WHY IT MATTERS IN PRODUCTION

The operational benefit that is easiest to defend is blast radius. With a single shared router, one tenant's traffic spike or a bad route annotation affects everyone, and router upgrades are a single change window for the whole cluster. With separate controllers, capacity and risk are partitioned, and you can roll out a TLS profile change to internal traffic first. The cost is more DNS, more certificates and more to keep consistent.

STEP BY STEP

1. Start from the requirement - isolation, capacity, different security posture, or domains.
2. Configure namespace selection and route admission so each controller owns a clear scope.

3 Give each its own domain, certificate and load balancer, and place router pods deliberately.
4 State the cost honestly: more moving parts, more certificates, more consistency to maintain.

EVIDENCE YOU WOULD QUOTE

```
oc -n openshift-ingress-operator get ingresscontroller

oc -n openshift-ingress get pods -l
ingresscontroller.operator.openshift.io/deployment-ingresscontroller=<name> -o wide

oc get route -A -o
custom-columns=NS:.metadata.namespace,HOST:.spec.host,STATUS:.status.ingress[0].routerName
```

RED FLAG - DO NOT SAY THIS

Do not create a separate controller per application. You are rebuilding the per-service load balancer problem with extra steps.

EXPECT THIS FOLLOW-UP

How do you make sure a namespace's routes land on the internal controller only?

Q76 SENIOR

A Route is returning 503. Walk me through it.

SAY THIS

503 from the router means it accepted the connection and had no healthy backend to send it to, so I work the path from the pod outwards. First, are there ready endpoints for the Service - if not, this is a readiness or selector problem and the network is irrelevant. If there are, I check that the Route points at the right Service and port, that it was admitted, and that the router can actually reach the pods, which is where NetworkPolicy and OVN come in. Then I check the router pods themselves.

THINK OF IT LIKE

The receptionist answered the phone and said nobody is available. That is not a phone fault - it is a staffing fault, and you go and look at the rota.

WHY IT MATTERS IN PRODUCTION

The reason to start at endpoints is that it is the highest-yield check by a wide margin. In most real 503 incidents the pods are running but not ready, because a probe is failing against a dependency. Starting at the router instead means restarting router pods, which briefly affects every application in the cluster and does not fix anything. Working inward to outward keeps blast radius at zero until you have evidence.

STEP BY STEP

1 State impact: one route, one namespace, or every route - that alone splits the diagnosis.
2 Check the EndpointSlice for ready addresses; if empty, investigate readiness and selectors.
3 Verify the Route's target Service and port, and that it was admitted by a controller.
4 Test connectivity from a router pod to a backend pod, check NetworkPolicy, and only then look at router pod health and logs.

EVIDENCE YOU WOULD QUOTE

```
oc get endpointslice -n <ns> -l kubernetes.io/service-name=<svc>

oc get route <route> -o jsonpath='{.spec.to} {.spec.port}{"\n"}'

oc -n openshift-ingress rsh deploy/router-default curl -sS -m 3 <podIP>:<port>/healthz
```

RED FLAG - DO NOT SAY THIS

Do not restart the router first. It affects every application in the cluster and almost never addresses the cause.

EXPECT THIS FOLLOW-UP

Every route in the cluster returns 503. How does that change your approach?

Q77 SENIOR

How do you troubleshoot cluster DNS?

SAY THIS

I establish scope first: is it one pod, one namespace, one node, or everything, and is it cluster names, external names or both. Then I test resolution from inside an affected pod, check the CoreDNS pods and the DNS Operator status, and look at whether a NetworkPolicy is blocking port 53 to the DNS service. If only external names fail, the upstream forwarders are the suspect. If only one node is affected, it is a node datapath problem, not a DNS problem.

THINK OF IT LIKE

It is a phone that cannot get through. Before blaming the network, you find out whether it is one handset, one floor, or every phone - and whether internal extensions still work.

WHY IT MATTERS IN PRODUCTION

Scoping first is what makes this fast, because each scope points at a different component. The most common real causes are a default-deny NetworkPolicy that forgot to allow DNS egress, a slow or unreachable upstream forwarder producing five-second timeouts, and CoreDNS being resource-starved on a busy node. Note also that a pod's DNS config is set at creation, so a DNS configuration change does not reach existing pods until they restart.

STEP BY STEP

1. Scope it: one pod, one node, one namespace, or cluster-wide; cluster names or external.
2. Test from inside the pod, then from the node, then directly against a CoreDNS pod IP.
3. Check DNS Operator and CoreDNS pod health, plus NetworkPolicy allowing UDP and TCP 53.
4. If only external resolution fails, test the upstream forwarders directly from a node.

EVIDENCE YOU WOULD QUOTE

```
oc exec <pod> -- getent hosts <name> ; oc exec <pod> -- cat /etc/resolv.conf

oc -n openshift-dns get pods -o wide ; oc get co dns

oc debug node/<node> -- chroot /host dig @<upstream> <external-name> +short
```

RED FLAG - DO NOT SAY THIS

Do not restart CoreDNS as a first move. If a NetworkPolicy is blocking port 53, restarting it proves nothing and delays the fix.

EXPECT THIS FOLLOW-UP

Only newly created pods resolve correctly. What does that tell you?

Q78 SENIOR

How do you tune the ingress layer for scale?

SAY THIS

I start by measuring where the limit actually is: router CPU, connection counts, TLS handshake rate, or backend latency being blamed on the router. Then the levers are router replica count and placement on dedicated infrastructure nodes, connection and timeout tuning, HTTP/2 and keepalive settings, and reducing the reload cost by limiting how often route objects change. For very large clusters, sharding routes across multiple IngressControllers is the structural answer.

THINK OF IT LIKE

A single reception desk can only process so many visitors. You add desks, put them in the right lobbies, and stop reprinting the visitor list every time one name changes.

WHY IT MATTERS IN PRODUCTION

The failure mode people miss is configuration churn. Every route change causes a router reload, and in a cluster where controllers create and delete routes frequently, the routers spend their time reloading rather than serving, producing latency spikes that look like backend problems. Placing routers on dedicated nodes also matters more than it sounds: routers competing with application workloads for CPU produce exactly the intermittent latency that is hardest to diagnose.

STEP BY STEP

1. Measure first - router CPU and memory, connection rate, TLS handshakes, reload frequency.
2. Scale replicas and place routers on dedicated infrastructure nodes with enough headroom.
3. Tune timeouts, keepalives and HTTP/2 deliberately, changing one thing at a time.
4. Reduce churn, and shard routes across controllers when a single router set is the limit.

EVIDENCE YOU WOULD QUOTE

```
haproxy_process_current_connections ; haproxy_backend_http_average_response_time_seconds

oc -n openshift-ingress adm top pod

oc -n openshift-ingress logs deploy/router-default | grep -c 'reload'
```

RED FLAG - DO NOT SAY THIS

Do not scale router replicas without measuring first. If the bottleneck is reload churn or backend latency, more routers change nothing.

EXPECT THIS FOLLOW-UP

Router latency spikes every few minutes with no traffic change. What do you suspect?

Q79 ARCHITECT

How would you design ingress for a multi-tenant cluster?

SAY THIS

I would separate by exposure and blast radius rather than by team. One internal controller and one external controller as the baseline, each with its own domain, certificate and load balancer, with namespace selection deciding which tenants land where. Tenants with genuinely different security requirements or very high volume get a dedicated controller on dedicated nodes. Hostname claims stay strict, certificates are issued automatically, and every route gets standard rate limiting and TLS policy from the controller rather than per route.

THINK OF IT LIKE

It is designing entrances for a mixed-use building. Residents, offices and the public do not share one door - but you also do not build one door per tenant.

WHY IT MATTERS IN PRODUCTION

The pressure in this design is always between isolation and operational load. Every extra controller means another certificate to rotate, another DNS entry, another thing to upgrade and to keep consistent. So the rule I use is that a new controller needs a stated reason - different exposure, different compliance boundary, or measured capacity contention - and anything else stays on the shared pair. That gives you a policy you can apply consistently instead of negotiating each request.

STEP BY STEP

1. Classify traffic by exposure and compliance requirement, not by organisational chart.
2. Baseline on an internal and an external controller with namespace-label admission.
3. Define the criteria that justify a dedicated controller, and hold the line on them.
4. Automate certificates, DNS and the standard TLS and rate-limit policy so consistency is free.

EVIDENCE YOU WOULD QUOTE

```
oc -n openshift-ingress-operator get ingresscontroller -o wide

oc get ns -L ingress-class ; oc get route -A --show-labels | head

Certificate inventory with expiry dates and issuing automation status
```

RED FLAG - DO NOT SAY THIS

Do not give every tenant their own IngressController by default. The consistency debt will outlive the isolation benefit.

EXPECT THIS FOLLOW-UP

A tenant demands a dedicated router for performance. How do you validate the claim?

PART 6 · Q80-Q94 · 15 QUESTIONS

OVN-Kubernetes, egress and secondary networks

The layer under the Service - where packets are actually forwarded, dropped or silently lost

This is where networking stops being objects and starts being packets. The questions here separate people who have read about the software-defined network from people who have opened it up during an incident at two in the morning. The recurring theme is that the SDN is not a black box: intent becomes logical flows, logical flows become OpenFlow rules, and OpenFlow rules become packets on a NIC. Every one of those layers is inspectable, and being able to say which layer you would inspect is worth more than memorising any single command.

Level mix - Foundation: 3 · Intermediate: 5 · Senior: 6 · Architect: 1

OVN-Kubernetes from the top down

Every layer is inspectable. When a candidate says "the SDN is broken", the follow-up is always "which layer, and what did you look at?"

Intent	NetworkPolicy, EgressFirewall, EgressIP, Service and Route objects in the API
ovnkube-master	Translates intent into logical switches, routers and ACLs in the OVN northbound DB
ovn-controller	Compiles logical flows into OpenFlow rules on each node
br-int + OVS	The datapath: actual packet forwarding, NAT and conntrack per node
Node NIC / MTU	Geneve encapsulation overhead; a wrong MTU silently drops large packets only
Secondary networks	Multus attaches extra interfaces: macvlan, bridge, SR-IOV VFs for HPC paths

Q80 FOUNDATION

What is a CNI plugin?

SAY THIS

CNI is the interface the runtime uses to attach a pod to a network. When a pod sandbox is created, the runtime calls the configured CNI plugin, which creates the interface, assigns an IP from the cluster's pod network, and sets up routes - then tears it all down when the pod goes away. On OpenShift the default is OVN-Kubernetes, and it is what gives every pod its own routable address inside the cluster.

THINK OF IT LIKE

It is the utility company connecting a new building to the grid. Standard socket, standard process, whichever supplier you have contracted.

WHY IT MATTERS IN PRODUCTION

The reason to know this precisely is that CNI failures have a specific signature: pods stick in ContainerCreating with a sandbox creation error mentioning the network plugin. That is not an image problem or a scheduling problem, and it usually means the CNI pods on that node are unhealthy or the node has run out of pod IPs. Recognising the message saves you from investigating the application.

STEP BY STEP

1. Define CNI as the contract between the runtime and the network plugin.
2. Describe what happens at pod creation - interface, address, routes - and at deletion.
3. Name OVN-Kubernetes as the OpenShift default and where its pods run.
4. Give the failure signature: ContainerCreating with a sandbox or network plugin error.

EVIDENCE YOU WOULD QUOTE

```
oc describe pod <pod> | grep -i -A5 sandbox

oc -n openshift-ovn-kubernetes get pods -o wide | grep <node>

oc debug node/<node> -- chroot /host ls /etc/cni/net.d
```

RED FLAG - DO NOT SAY THIS

Do not treat a sandbox creation failure as an application problem. The message names the network plugin for a reason.

EXPECT THIS FOLLOW-UP

Pods on one node are stuck in ContainerCreating. What do you check?

Q81 FOUNDATION

What is MTU and why does it matter so much in a cluster network?

SAY THIS

MTU is the largest packet a link will carry. The cluster network encapsulates pod traffic - Geneve in OVN-Kubernetes - which adds header overhead, so the pod MTU must be smaller than the underlying node MTU by that overhead. If it is not, large packets are dropped somewhere in the middle of the path while small ones sail through, which produces a failure that looks nothing like a network problem.

THINK OF IT LIKE

It is a low bridge on a delivery route. Vans get through all day; the one lorry that matters does not, and nobody connects the two events.

WHY IT MATTERS IN PRODUCTION

This is the classic silent failure. Health checks pass because they are tiny. DNS works because queries are small. Then a large HTTP response or a database result set hangs forever, and the application team reports intermittent timeouts. The giveaway is that small payloads always work and large ones always fail, and the test is a ping with the do-not-fragment flag at increasing sizes.

STEP BY STEP

1. Define MTU and the encapsulation overhead that makes pod MTU smaller than node MTU.
2. Describe the signature: small packets fine, large packets lost, no errors logged.
3. Test with do-not-fragment pings at increasing sizes to find the actual limit.
4. Fix it at the source - node or fabric MTU misconfiguration - rather than by lowering application payload sizes.

EVIDENCE YOU WOULD QUOTE

```
oc exec <pod> -- ping -M do -s 1400 -c 2 <target-pod-ip>

oc debug node/<node> -- chroot /host ip -d link show br-ex | head

oc get network.operator cluster -o
jsonpath='{.spec.defaultNetwork.ovnKubernetesConfig.mtu}{"\n"}'
```

RED FLAG - DO NOT SAY THIS

Do not describe this as an intermittent application bug. Size-dependent failure is a signature, not a coincidence.

EXPECT THIS FOLLOW-UP

Small requests work, large responses hang. Prove it is MTU in two commands.

Q82 FOUNDATION

What is a NetworkAttachmentDefinition?

SAY THIS

It is the custom resource that defines an additional network a pod can attach to - the CNI configuration for a secondary interface, expressed as a Kubernetes object. A pod requests it with an annotation, and Multus attaches the extra interface at creation time. It is how you give a pod a macvlan interface onto a VLAN, a bridge to a storage network, or an SR-IOV virtual function.

THINK OF IT LIKE

It is a second phone line specification for a desk. The main line is standard for everyone; this describes the dedicated line and who is allowed to have one.

WHY IT MATTERS IN PRODUCTION

Because it is a namespaced object, it is also an access control point: a team can only attach to networks defined in, or shared with, their namespace. That matters because a secondary interface typically bypasses the cluster's normal policy path - traffic on a macvlan interface is not subject to standard NetworkPolicy - so who may create and use these definitions is a security decision, not just a networking one.

STEP BY STEP

1. Define it as CNI configuration expressed as a namespaced custom resource.
2. Explain how a pod requests it and that Multus performs the attachment.
3. Name typical types: macvlan, bridge, ipvlan, SR-IOV.
4. Call out the governance angle - secondary networks sidestep default policy, so restrict who can define and use them.

EVIDENCE YOU WOULD QUOTE

```
oc get net-attach-def -A

oc get pod <pod> -o jsonpath='{.metadata.annotations.k8s\.v1\.cni\.cncf\.io/networks}{"\n"}'

oc exec <pod> -- ip -br addr
```

RED FLAG - DO NOT SAY THIS

Do not treat secondary networks as a purely technical detail. They can quietly bypass the network policy you spent months rolling out.

EXPECT THIS FOLLOW-UP

How do you stop a tenant attaching to a network they should not reach?

Q83 INTERMEDIATE

What is OVN-Kubernetes and how is it structured?

SAY THIS

OVN-Kubernetes is the default OpenShift network plugin. It translates Kubernetes intent - Services, NetworkPolicies, EgressFirewalls - into OVN logical switches, routers and access control lists held in a northbound database. ovn-controller on each node compiles those logical flows into OpenFlow rules in the node's integration bridge, and Open vSwitch forwards the actual packets, with Geneve encapsulation for traffic between nodes.

THINK OF IT LIKE

It is a rail network. The timetable is the logical intent, the signalling system compiles it into instructions for each junction, and the trains are the packets. A delay can come from the timetable, the signals or the track.

WHY IT MATTERS IN PRODUCTION

The structure is the diagnostic map. A policy that never takes effect is a problem between intent and the northbound database, so you look at ovnkube-master. A policy that exists everywhere but fails on one node is a compilation or datapath problem, so you look at ovn-controller and OVS flows on that node. Being able to split the problem that way is what keeps an OVN incident from becoming an all-hands guessing session.

STEP BY STEP

1. Describe the four layers: Kubernetes intent, OVN northbound, ovn-controller, OVS datapath.
2. Say where each component runs and which pods represent it.
3. Map symptoms to layers - cluster-wide policy failure versus single-node forwarding failure.
4. Note that Geneve encapsulation is why node MTU and the underlay fabric matter.

EVIDENCE YOU WOULD QUOTE

```
oc -n openshift-ovn-kubernetes get pods -o wide

oc -n openshift-ovn-kubernetes rsh <ovnkube-node-pod> ovn-nbctl show | head -30

oc debug node/<node> -- chroot /host ovs-ofctl dump-flows br-int | wc -l
```

RED FLAG - DO NOT SAY THIS

Do not call it "the SDN" and leave it there. The follow-up is always which layer you would inspect.

EXPECT THIS FOLLOW-UP

A NetworkPolicy works on every node except one. Which layer, and why?

Q84 INTERMEDIATE

What is Multus and when would you use it?

SAY THIS

Multus is a meta-plugin that lets a pod have more than one network interface. The default cluster network stays as eth0 for Services, DNS and normal traffic, and Multus attaches additional interfaces defined by NetworkAttachmentDefinitions. You use it when a workload genuinely needs a separate path - a VLAN carrying storage or telecom signalling traffic, a dedicated high-throughput interface, or an SR-IOV virtual function for latency-sensitive work.

THINK OF IT LIKE

It is a second network card in a server. The office LAN is still there for email; the extra card is for the thing that cannot share a queue.

WHY IT MATTERS IN PRODUCTION

The trap is assuming the secondary interface behaves like the primary one. Kubernetes Services do not front it, standard NetworkPolicy does not filter it, cluster DNS does not know about it, and IP address management is your responsibility. So it is the right tool for a real requirement and the wrong tool for "we want faster networking", where the honest answer is usually to measure first.

STEP BY STEP

1. Explain the model: default interface plus additional attachments.
2. Give real use cases - separated traffic planes, high throughput, SR-IOV.
3. State plainly what you lose: Services, standard policy, cluster DNS, automatic IPAM.
4. Insist on measuring the requirement before adding an interface, because complexity is permanent.

EVIDENCE YOU WOULD QUOTE

```
oc exec <pod> -- ip -br addr

oc get net-attach-def -n <ns> -o yaml | head -30

oc get pod <pod> -o jsonpath='{.metadata.annotations.k8s\.v1\.cni\.cncf\.io/network-status}' |
python3 -m json.tool
```

RED FLAG - DO NOT SAY THIS

Do not add a secondary network to solve a performance complaint you have not measured. You will inherit an IPAM problem and keep the latency.

EXPECT THIS FOLLOW-UP

Traffic on the secondary interface bypasses your NetworkPolicy. How do you control it?

Q85 INTERMEDIATE

What is an EgressFirewall?

SAY THIS

EgressFirewall is an OVN-Kubernetes object that controls which external destinations pods in a namespace may reach. Rules are ordered allow and deny entries matched on CIDR, DNS name or node selector, and the first match wins. It complements NetworkPolicy: NetworkPolicy is mostly about east-west traffic between pods, EgressFirewall is about north-south traffic leaving the cluster.

THINK OF IT LIKE

NetworkPolicy is the rules about which rooms staff may enter. EgressFirewall is the rule about which addresses the company car may drive to.

WHY IT MATTERS IN PRODUCTION

In regulated environments this is how you demonstrate that a workload cannot exfiltrate data to arbitrary internet destinations, so it appears in compliance conversations as often as technical ones. The operational subtlety is DNS-based rules: they resolve names periodically, so a destination behind a large CDN with rotating addresses can intermittently fail. Where reliability matters, CIDR rules or an egress proxy are the more predictable answer.

STEP BY STEP

1. Define it as ordered, first-match egress control at namespace scope.
2. Contrast it with NetworkPolicy: north-south versus east-west.
3. Warn about DNS-based rules and rotating addresses, and offer CIDR or a proxy instead.

4 Verify from inside a pod after applying, and keep a documented rollback.

EVIDENCE YOU WOULD QUOTE

```
oc get egressfirewall -A -o yaml | head -40

oc exec <pod> -- curl -sS -m 5 https://<external> # expected allow and expected deny

Network Observability flows filtered by namespace with a dropped verdict
```

RED FLAG - DO NOT SAY THIS

Do not rely on DNS-name egress rules for a critical integration behind a CDN. The intermittent failures will be blamed on the application for weeks.

EXPECT THIS FOLLOW-UP

A namespace intermittently loses access to an allowed external API. What do you suspect?

Q86 INTERMEDIATE

What is EgressIP, and what are its operational risks?

SAY THIS

EgressIP pins the source address that a namespace's outbound traffic appears to come from, so an external firewall or database can allow-list a stable IP. OVN assigns that address to an eligible node and source-NATs matching traffic through it. The risks are that the address is hosted on one node at a time, so node failure moves it and briefly interrupts connections, and that it depends on the underlying network allowing that address to move between nodes.

THINK OF IT LIKE

It is a company car with a recognised number plate that the security gate lets through. If the car is off the road, someone else has to take the plate - and the gate needs a moment to notice.

WHY IT MATTERS IN PRODUCTION

The reason this is asked at senior level is that EgressIP is usually adopted to satisfy an external team's firewall requirement, and it quietly creates a dependency between your node lifecycle and their access control. Node maintenance, autoscaling and address exhaustion all become externally visible events. Where the external system supports it, an egress proxy or identity-based access is a more robust answer, and saying that shows architectural judgement.

STEP BY STEP

1 Explain the mechanism: namespace selection, node assignment, source NAT.
2 Name the failure behaviour on node loss and what it does to long-lived connections.
3 Check that the underlay permits the address to move and that eligible nodes are labelled.
4 Offer the alternative - proxy or identity-based allow-listing - and state the trade-off.

EVIDENCE YOU WOULD QUOTE

```
oc get egressip ; oc get nodes -l k8s.ovn.org/egress-assignable

oc get egressip <name> -o jsonpath='{.status.items}' | python3 -m json.tool

oc exec <pod> -- curl -sS https://<echo-service> # confirm observed source address
```

RED FLAG - DO NOT SAY THIS

Do not present EgressIP as free. It couples your node lifecycle to somebody else's firewall policy.

EXPECT THIS FOLLOW-UP

You need to drain the node currently hosting an EgressIP. What do you tell the dependent team?

Q87 INTERMEDIATE

What is the device plugin pattern?

SAY THIS

A device plugin is a DaemonSet that advertises specialised hardware on a node as an extended resource - GPUs, SR-IOV virtual functions, FPGAs, QAT. It reports how many are available, and the scheduler then treats them like any other countable resource: a pod requests one, the scheduler only considers nodes with a free unit, and the plugin performs the device allocation and injects what the container needs to use it.

THINK OF IT LIKE

It is the hire desk for specialist equipment. It publishes how many units are on the shelf, and nobody is sent to a depot that has none left.

WHY IT MATTERS IN PRODUCTION

The important consequence is that these resources are integers and cannot be overcommitted - one virtual function is either yours or it is not. So a pod requesting a device that is fully allocated stays Pending regardless of CPU and memory availability, and the event says insufficient for a resource name most people do not recognise. Knowing to check the device plugin's health, and the node's advertised allocatable for that resource, is the fast path.

STEP BY STEP

1. Define the plugin as a DaemonSet advertising extended resources to the kubelet.
2. Explain that scheduling treats them as countable, non-overcommittable integers.
3. Give the Pending signature and where to read advertised versus allocated counts.
4. Check plugin pod health first - a crashed plugin makes the whole node's devices vanish.

EVIDENCE YOU WOULD QUOTE

```
oc get node <node> -o jsonpath='{.status.allocatable}' | python3 -m json.tool

oc get pods -n openshift-sriov-network-operator -o wide

oc describe pod <pod> | grep -i insufficient
```

RED FLAG - DO NOT SAY THIS

Do not treat extended resources as best-effort. They are exact counts, and the scheduler will never round down for you.

EXPECT THIS FOLLOW-UP

A GPU pod is Pending while nodes show plenty of CPU. What is your first check?

Q88 SENIOR

How do you diagnose an MTU black hole?

SAY THIS

I confirm the signature first: small payloads succeed, large ones hang or reset, and the failure is reproducible by size rather than by time. Then I use do-not-fragment pings at increasing sizes from a pod to find the actual maximum, repeat between nodes to see whether the limit is in the overlay or the underlay, and compare configured pod MTU against the node and fabric MTU. The fix belongs at whichever layer is inconsistent, usually the underlay.

THINK OF IT LIKE

You establish that vans get through and lorries do not, then you drive the route measuring bridge heights until you find the one that is too low.

WHY IT MATTERS IN PRODUCTION

These incidents are expensive because the symptom is reported as an application bug and everyone investigates the application. They often appear after an infrastructure change nobody told you about - a new switch, a VPN or tunnel inserted in the path, a migration to different hypervisor networking. Asking "what changed in the underlay" early is often faster than any packet capture.

STEP BY STEP

1. Establish the size-dependent signature and confirm it is reproducible.
2. Binary-search the working size with do-not-fragment pings, pod to pod and node to node.
3. Compare configured cluster MTU with node interface MTU and the documented fabric MTU.
4. Fix at the inconsistent layer, then re-test the original application path and record the expected values so it is caught by monitoring next time.

EVIDENCE YOU WOULD QUOTE

```
oc exec <pod> -- ping -M do -s 1472 -c 2 <peer-pod-ip>

oc debug node/<node> -- chroot /host ping -M do -s 8972 -c 2 <other-node-ip>

oc get network.operator cluster -o jsonpath='{.status.defaultNetwork}' | python3 -m json.tool
```

RED FLAG - DO NOT SAY THIS

Do not lower the application's payload size as the fix. You have hidden a fabric misconfiguration that will resurface elsewhere.

EXPECT THIS FOLLOW-UP

The cluster MTU is correct but node-to-node large pings fail. Where is the problem?

Q89 SENIOR

How do you isolate a node-specific OVN-Kubernetes failure?

SAY THIS

I prove it is node-specific first by running the same test from pods on several nodes, because that single fact eliminates most of the search space. Then I look at the ovnkube pods on the affected node, at ovn-controller's connection to the databases, and at whether OpenFlow rules were actually programmed in the integration bridge. If intent is correct cluster-wide but flows are missing on one node, the problem is compilation or the local datapath, not policy.

THINK OF IT LIKE

One junction on the rail network is not following the timetable. Everyone else is running fine, so you go to that signal box rather than reprinting the timetable.

WHY IT MATTERS IN PRODUCTION

The most common causes are an ovn-controller that lost its database connection and is serving stale flows, a node under such memory pressure that the OVS process is being starved, and a node that was cordoned or partially drained leaving inconsistent state. Restarting the ovnkube pod on that node often clears it, but doing that before you have captured flow state means you will never know why - and it will happen again.

STEP BY STEP

1. Prove node-specificity by testing the same path from pods on at least three nodes.
2. Check ovnkube pod health and ovn-controller connectivity to the OVN databases on that node.
3. Compare programmed flows on the affected node against a healthy one for the same policy.
4. Capture evidence, then restart the node's ovnkube pod as the smallest safe mitigation and verify.

EVIDENCE YOU WOULD QUOTE

```
oc -n openshift-ovn-kubernetes get pods -o wide | grep <node>

oc debug node/<node> -- chroot /host ovs-ofctl dump-flows br-int | grep <podIP>

oc -n openshift-ovn-kubernetes logs <ovnkube-node-pod> -c ovn-controller --tail=200
```

RED FLAG - DO NOT SAY THIS

Do not reboot the node as a first response. You will lose the evidence and inherit the same incident next week.

EXPECT THIS FOLLOW-UP

Flows are missing only for one namespace on one node. What does that narrow it to?

Q90 SENIOR

How do you troubleshoot an SR-IOV pod that will not schedule?

SAY THIS

I check the chain from hardware to request. Does the node advertise the extended resource, and how many are allocatable versus allocated? Is the SR-IOV operator healthy and has the policy actually configured virtual functions on that node's NIC? Does the pod request the right resource name and reference the right NetworkAttachmentDefinition? And is anything else on the node holding the remaining functions - because they cannot be shared.

THINK OF IT LIKE

It is checking whether the specialist tool exists in the depot, whether it is on the shelf, and whether the request form names the tool you actually meant.

WHY IT MATTERS IN PRODUCTION

Beyond simple exhaustion, the usual causes are a policy whose node selector does not match the nodes you expected, a NIC whose firmware or BIOS settings do not have virtualisation enabled, or a resource name mismatch between the policy and the pod spec. There is also a timing element: applying an SR-IOV policy triggers a MachineConfig rollout and node reboots, so the capacity genuinely does not exist until the pool has finished updating.

STEP BY STEP

1. Read the FailedScheduling event and note the exact resource name it reports as insufficient.
2. Compare node allocatable against allocated for that resource across candidate nodes.

3 Verify the SR-IOV policy applied - node selector, NIC selector, number of virtual functions, and that the pool finished rolling out.
4 Confirm the pod's resource name and network annotation match the policy exactly.

EVIDENCE YOU WOULD QUOTE

```
oc get sriovnetworknodestate -n openshift-sriov-network-operator -o yaml | head -60

oc get node <node> -o jsonpath='{.status.allocatable}' | python3 -m json.tool

oc get sriovnetworknodepolicy -A -o wide ; oc get mcp
```

RED FLAG - DO NOT SAY THIS

Do not conclude the hardware is faulty before checking that the MachineConfig rollout completed. The functions do not exist until the node has rebooted.

EXPECT THIS FOLLOW-UP

Half the nodes advertise the resource and half do not. Where do you look?

Q91 SENIOR

What does Kubernetes NMState provide, and how do you design node networking with it?

SAY THIS

NMState lets you declare node network configuration - bonds, VLANs, bridges, static addresses, routes - as NodeNetworkConfigurationPolicy objects, applied by an operator and reported back through NodeNetworkState. That turns node networking from something configured by hand or by an installer into something versioned, selectable by node label, and observable. For design I bond physical interfaces for redundancy, carry separate traffic classes on VLANs, and roll changes out to one node first.

THINK OF IT LIKE

It is building regulations for the wiring rather than trusting each electrician's habits. Same standard, applied per building type, inspected afterwards.

WHY IT MATTERS IN PRODUCTION

Node network changes are uniquely dangerous because a mistake can make the node unreachable, and you cannot fix it through the API you just cut off. That is why the practice matters more than the syntax: apply to a single labelled canary node, verify NodeNetworkState reports success and the node stays Ready, and only then widen the selector. Having out-of-band console access before you start is not optional.

STEP BY STEP

1 Declare the intended state as a policy selected by node label, not applied cluster-wide.
2 Design for redundancy - bonded uplinks, separated VLANs per traffic class, explicit MTU.
3 Roll out to one canary node, verify NodeNetworkState and node readiness, then widen.
4 Ensure out-of-band access exists before any change, and keep a documented revert policy.

EVIDENCE YOU WOULD QUOTE

```
oc get nncp ; oc get nnce

oc get nns <node> -o yaml | sed -n '/interfaces:/,/routes:/p' | head -40

oc get nodes -o wide # readiness after each wave
```

RED FLAG - DO NOT SAY THIS

Do not apply a network policy to every node at once. The failure mode is a cluster you cannot reach to fix.

EXPECT THIS FOLLOW-UP

You apply a bond change and the node goes NotReady. What is your recovery path?

Q92 SENIOR

What is the Network Observability Operator used for?

SAY THIS

It collects flow-level data from the cluster network using eBPF agents on each node, enriches it with Kubernetes identity - namespace, pod, service, node - and lets you query and visualise it. It answers questions that metrics and logs cannot: who is actually talking to whom, what is being dropped and by which rule, where cross-zone traffic is coming from, and what a namespace's real dependencies are before you apply a policy.

THINK OF IT LIKE

It is CCTV for the network with name badges attached. Not just that a packet moved, but which service sent it and which policy stopped it.

WHY IT MATTERS IN PRODUCTION

The highest-value use is de-risking NetworkPolicy rollout: instead of guessing dependencies from an architecture diagram, you observe two weeks of real flows and write allow rules from evidence. It is also how you find expensive cross-zone traffic and how you prove, during an incident, whether traffic is being dropped by policy or never arrived at all. The cost is real - flow collection consumes CPU and storage - so sampling and retention need deliberate tuning.

STEP BY STEP

1. Describe the mechanism: eBPF agents, flow records, Kubernetes enrichment.
2. Give the primary use cases - dependency discovery, drop attribution, cross-zone cost.
3. Tune sampling and retention against the cluster's traffic volume and storage budget.
4. Use it before and after a policy change as the evidence that the change was safe.

EVIDENCE YOU WOULD QUOTE

```
oc get flowcollector cluster -o yaml | head -40

oc -n netobserv get pods -o wide

Flow query filtered by namespace and drop reason around the incident window
```

RED FLAG - DO NOT SAY THIS

Do not enable full-fidelity flow collection cluster-wide without capacity planning. You can generate more data than the cluster you are observing.

EXPECT THIS FOLLOW-UP

How would you use flow data to write NetworkPolicy for an existing namespace?

Q93 SENIOR

What are the operational considerations for a dual-stack cluster?

SAY THIS

Dual-stack means Services and pods can have both IPv4 and IPv6 addresses, and the considerations are mostly about consistency across the whole path. Every layer needs to support both - load balancers, DNS records, firewalls, the application libraries - and Services need an explicit ipFamilyPolicy rather than relying on defaults. Address planning and monitoring have to cover both families, and troubleshooting means checking both, because a service can be reachable on one and broken on the other.

THINK OF IT LIKE

It is running a bilingual service. Every sign, every form and every member of staff has to handle both languages, or customers get a confusing half-service.

WHY IT MATTERS IN PRODUCTION

The realistic failure is partial support somewhere in the chain: an external firewall that only has IPv4 rules, a client library that prefers IPv6 and times out, a health check configured for one family only. The result is intermittent behaviour that depends on which address a client happened to resolve. Deciding the policy up front - which family is preferred, which services are single-stack - avoids most of it.

STEP BY STEP

1. Confirm end-to-end support across load balancers, DNS, firewalls and client libraries.
2. Set ipFamilyPolicy explicitly per Service rather than accepting defaults.
3. Plan addressing and NetworkPolicy for both families; policies must cover both or they leak.
4. Test and monitor both paths independently, and record which family clients actually use.

EVIDENCE YOU WOULD QUOTE

```
oc get svc <svc> -o jsonpath='{.spec.ipFamilies} {.spec.ipFamilyPolicy}{"\n"}'

oc get network.config cluster -o jsonpath='{.status.clusterNetwork}' | python3 -m json.tool

oc exec <pod> -- curl -sS -m 3 -6 http://<svc>:<port>/healthz
```

RED FLAG - DO NOT SAY THIS

Do not assume a NetworkPolicy written with IPv4 CIDRs also restricts IPv6. It does not, and that is an open path you did not intend.

EXPECT THIS FOLLOW-UP

A service is reachable over IPv4 and times out over IPv6. Where do you start?

Q94 ARCHITECT

How would you establish a network performance baseline for a new cluster?

SAY THIS

I measure before anyone depends on it, so that future complaints have something to compare against. That means pod-to-pod throughput and latency within a node, across nodes and across zones; DNS resolution latency; ingress request latency at a known concurrency; and packet loss under load. I record the numbers, the MTU, the topology and the date, publish them, and re-run the same tests after significant infrastructure changes.

THINK OF IT LIKE

It is weighing yourself the day you buy the scales. Without that first number, every later reading is an opinion.

WHY IT MATTERS IN PRODUCTION

The value shows up during arguments, not during the test. When an application team says the network is slow, a baseline lets you answer in minutes with data rather than defending the platform on instinct. It also catches infrastructure regressions that nobody announced - a firmware change, a new switch, a hypervisor migration - because the same test suddenly returns a different number.

STEP BY STEP

1. Define the test matrix: intra-node, inter-node, inter-zone, ingress path, DNS.
2. Run it with a repeatable tool and fixed parameters, and record MTU and topology alongside.
3. Publish the results with the date and the cluster configuration they describe.
4. Re-run after infrastructure changes and upgrades, and alert on regression beyond a threshold.

EVIDENCE YOU WOULD QUOTE

```
iperf3 between pods on different nodes and different zones, recorded

Ingress p50 and p99 latency at a fixed concurrency, from a fixed client location

DNS resolution latency histogram from a test pod, per zone
```

RED FLAG - DO NOT SAY THIS

Do not wait for a complaint to measure. Without a baseline every performance discussion becomes a matter of opinion.

EXPECT THIS FOLLOW-UP

A team claims the network got slower after an upgrade. How do you settle it?

PART 7 · Q95-Q110 · 16 QUESTIONS

Storage that survives the pod

Provisioning, attachment, snapshots and the failures that lose data

Storage is the part of the platform where mistakes are permanent. A bad network change causes an outage; a bad storage decision loses data that nobody can recreate. That is why storage questions tend to be scored on caution rather than cleverness - interviewers want to hear that you know which step in the provisioning chain is stuck, that you never assume a snapshot is a backup, and that you have restored something at least once rather than only configured backups. The chain from claim to mounted filesystem has five links, and almost every storage incident is one of them.

Level mix - Foundation: 6 · Intermediate: 4 · Senior: 4 · Architect: 2

From PVC to a mounted filesystem

"PVC is Pending" is not one failure, it is four. Knowing which arrow is stuck turns a guess into a diagnosis.

Q95 FOUNDATION

What are PersistentVolumes and PersistentVolumeClaims?

SAY THIS

A PersistentVolume is a piece of storage in the cluster - real capacity on a backend. A PersistentVolumeClaim is a workload's request for storage of a given size, access mode and class. The claim binds to a volume, and the pod references the claim rather than the volume, so applications ask for what they need without knowing anything about the backend. With dynamic provisioning the volume is created on demand by a CSI driver when the claim appears.

THINK OF IT LIKE

The claim is a hotel reservation for a room of a certain type; the volume is the actual room. The guest holds the reservation, not a key to a specific door number.

WHY IT MATTERS IN PRODUCTION

The separation is what makes manifests portable between clusters with completely different storage backends, and it is also where the most common confusion lives: the PVC is namespaced and belongs to the application, while the PV is cluster-scoped and belongs to the platform. Deleting a namespace deletes the claims, and what happens to the underlying data then depends entirely on the reclaim policy - which is why that setting deserves more attention than it usually gets.

STEP BY STEP

1. Define the claim as the request and the volume as the resource, and say which is namespaced.
2. Explain binding and dynamic provisioning through the StorageClass.
3. Point out that the pod references the claim, which is what makes manifests portable.
4. Flag that deletion behaviour depends on reclaim policy, not on intuition.

EVIDENCE YOU WOULD QUOTE

```
oc get pvc -n <ns> ; oc get pv | grep <ns>

oc get pvc <pvc> -o jsonpath='{.status.phase} {.spec.storageClassName}{"\n"}'
```

```
oc describe pvc <pvc> | sed -n '/Events/,$p'
```

RED FLAG - DO NOT SAY THIS

Do not say a PVC "is" storage. The binding relationship is exactly what fails when a claim stays Pending.

EXPECT THIS FOLLOW-UP

What happens to the data when the namespace holding the PVC is deleted?

Q96 FOUNDATION

What is a StorageClass?

SAY THIS

A StorageClass names a kind of storage the platform offers and tells the provisioner how to create it - which CSI driver, what parameters such as disk type or replication, the reclaim policy, whether volumes can be expanded, and the binding mode. A claim asks for a class by name and gets storage with those properties. One class is usually marked default, which is what claims get when they do not specify.

THINK OF IT LIKE

It is the room types on the booking page. Standard, sea view, accessible - each with its own price, size and cancellation policy, all bookable by name.

WHY IT MATTERS IN PRODUCTION

The default class is quietly one of the most consequential settings in a cluster: every team that omits storageClassName inherits it, including its reclaim policy and its performance tier. If the default is a Delete-policy class on expensive fast disks, you get surprising bills and surprising data loss. Setting the default deliberately - and naming classes for their guarantees rather than their backend - is a small decision with a long tail.

STEP BY STEP

1. Define it as a named storage offering plus the provisioner parameters behind it.
2. List what it controls: provisioner, parameters, reclaim policy, expansion, binding mode.
3. Explain the default class and why its choice matters more than people expect.
4. Recommend naming by guarantee - fast-retain, standard-delete - so intent is visible in the manifest.

EVIDENCE YOU WOULD QUOTE

```
oc get sc -o wide

oc get sc <class> -o jsonpath='{.reclaimPolicy} {.allowVolumeExpansion}
{.volumeBindingMode}{"\n"}'

oc get sc -o jsonpath='{.items[?(@.metadata.annotations.storageclass\.kubernetes\.io/is-default
-class=="true")].metadata.name}{"\n"}'
```

RED FLAG - DO NOT SAY THIS

Do not leave the default StorageClass unexamined. Every team that forgets to specify one inherits whatever it happens to be.

EXPECT THIS FOLLOW-UP

What would you name your storage classes, and why does the name matter?

Q97 FOUNDATION

What is a CSI driver?

SAY THIS

CSI is the standard interface between Kubernetes and a storage system, and a CSI driver implements it for one backend. It has two halves: a controller component that creates, deletes, attaches and snapshots volumes by talking to the storage API, and a node component running as a DaemonSet that stages and mounts volumes into pods. Splitting them that way is why some failures are cluster-wide and others affect only one node.

THINK OF IT LIKE

The controller is the warehouse office that allocates a pallet; the node component is the forklift at the loading bay that actually brings it to your door.

WHY IT MATTERS IN PRODUCTION

That split is the fastest diagnostic in storage. If new volumes are not being created at all, look at the controller and its credentials to the storage backend. If volumes provision fine but pods on one node cannot mount them, look at the node plugin on that node. Getting this right saves you from restarting the whole storage operator because of a single sick node.

STEP BY STEP

1. Define CSI as the standard interface and the driver as the backend-specific implementation.
2. Describe the controller and node split, and where each runs.
3. Map symptoms to halves: provisioning failures versus mount failures.
4. Check driver pod health and backend credentials before suspecting Kubernetes itself.

EVIDENCE YOU WOULD QUOTE

```
oc get csidrivers ; oc get csinode

oc -n openshift-cluster-csi-drivers get pods -o wide

oc get volumeattachment | grep <node>
```

RED FLAG - DO NOT SAY THIS

Do not treat all storage failures as one category. Provisioning and mounting are different components with different logs.

EXPECT THIS FOLLOW-UP

Volumes provision but will not mount on one node. Which component and which log?

Q98 FOUNDATION

Compare the access modes RWO, RWX and RWOP.

SAY THIS

ReadWriteOnce means the volume can be mounted read-write by pods on a single node - note node, not pod, so several pods on the same node can share it. ReadWriteMany allows read-write mounts from many nodes at once and requires a backend that supports shared access, typically file or object based rather than block. ReadWriteOncePod is the strict one: exactly one pod, cluster-wide, which is what you want for a database that must never have two writers.

THINK OF IT LIKE

RWO is a meeting room booked by one department - several of their people can be inside. RWX is a shared kitchen. RWOP is a single-occupancy office with one key that exists.

WHY IT MATTERS IN PRODUCTION

The subtlety that catches people is that RWO is per node, so it does not protect against two replicas of the same Deployment both writing, as long as they land on the same node. For data integrity that matters, RWOP is the correct mode. The other trap is assuming RWX is available: most block-based cloud storage cannot do it, so a workload designed around shared filesystems needs a different backend entirely, and that is an architecture decision rather than a manifest change.

STEP BY STEP

1. Define each mode precisely, and stress that RWO is node-scoped, not pod-scoped.
2. Say which backends can realistically offer RWX and which cannot.
3. Recommend RWOP where a second writer would corrupt data.
4. Check the driver's supported modes before designing around one.

EVIDENCE YOU WOULD QUOTE

```
oc get pvc <pvc> -o jsonpath='{.spec.accessModes}{"\n"}'

oc get pv <pv> -o jsonpath='{.spec.accessModes} {.spec.csi.driver}{"\n"}'

oc get csidriver <driver> -o yaml | head -30
```

RED FLAG - DO NOT SAY THIS

Do not assume RWO prevents two pods from writing. Two pods on the same node can both mount it.

EXPECT THIS FOLLOW-UP

A database has two replicas writing to one RWO volume. How did that happen?

Q99 FOUNDATION

What is volumeMode, and when would you use Block?

SAY THIS

volumeMode decides whether the volume is presented as a formatted filesystem mounted into the container, which is the default, or as a raw block device exposed at a device path. Filesystem is right for almost everything. Block is for workloads that manage their own storage layout and want to skip the filesystem layer - some databases and storage systems achieve better and more predictable performance that way.

THINK OF IT LIKE

Filesystem mode is a furnished flat. Block mode is an empty shell handed over with the keys - more work, but you control every wall.

WHY IT MATTERS IN PRODUCTION

The practical consequences of Block are worth stating because they surprise people: there is no filesystem for the platform to resize or inspect, standard file-level backup tools cannot read it, and the application is entirely responsible for what is on the device. So choosing Block is also choosing a different backup strategy - snapshots or application-native dumps rather than file-level copies.

STEP BY STEP

1. Define the two modes and what the container actually sees in each.
2. Give the legitimate reasons to pick Block - performance and applications that manage layout.

3 State the consequences: no filesystem-level tooling, different backup approach, application owns the format.
4 Confirm the driver supports Block before designing around it.

EVIDENCE YOU WOULD QUOTE

```
oc get pvc <pvc> -o jsonpath='{.spec.volumeMode}{"\n"}'

oc get pod <pod> -o jsonpath='{.spec.containers[0].volumeDevices}' | python3 -m json.tool

oc get sc <class> -o yaml | grep -i volumemode
```

RED FLAG - DO NOT SAY THIS

Do not choose Block for general workloads. You are giving up tooling for a benefit you probably cannot measure.

EXPECT THIS FOLLOW-UP

How would you back up a Block-mode volume?

Q100 FOUNDATION

What does reclaimPolicy control?

SAY THIS

It decides what happens to the underlying storage when the claim is deleted. Delete removes the backend volume and the data with it. Retain keeps the volume and its data, leaving the PV in Released state for an administrator to handle deliberately. Dynamic provisioning usually defaults to Delete, which is convenient for ephemeral environments and dangerous for anything you would miss.

THINK OF IT LIKE

It is what the storage company does when you cancel the unit. Delete means they empty it immediately; Retain means they keep the contents until someone signs for them.

WHY IT MATTERS IN PRODUCTION

This is one of the few settings where the wrong choice loses data with no recovery path. The realistic scenario is a namespace cleanup script, or a GitOps prune, removing PVCs whose class is Delete - and the volumes are gone before anyone notices. Retain on production classes, plus a documented process for reclaiming Released volumes, converts that from a data-loss event into an administrative chore.

STEP BY STEP

1 Define the two policies and what each does to the backend volume.
2 State the typical default for dynamic provisioning and why that is risky in production.
3 Recommend Retain for production data classes, with a documented reclaim process.
4 Note that the policy is set on the class at provisioning time, and check it before you rely on it.

EVIDENCE YOU WOULD QUOTE

```
oc get pv -o custom-columns=NAME:.metadata.name,POLICY:.spec.persistentVolumeReclaimPolicy,CLAI
M:.spec.claimRef.name

oc get sc <class> -o jsonpath='{.reclaimPolicy}{"\n"}'

oc get pv --field-selector status.phase=Released
```

RED FLAG - DO NOT SAY THIS

Do not use a Delete-policy class for production data because it is the default. That is how irreversible data loss happens quietly.

EXPECT THIS FOLLOW-UP

Someone deleted a production PVC an hour ago. What are your options?

Q101 INTERMEDIATE

What problem does WaitForFirstConsumer solve?

SAY THIS

With immediate binding, the volume is provisioned as soon as the claim is created - before anyone knows where the pod will run. In a zoned cluster that volume might land in zone A while the scheduler wanted to place the pod in zone B, and the pod then cannot be scheduled at all. WaitForFirstConsumer delays provisioning until a pod is scheduled, so the volume is created in the right topology.

THINK OF IT LIKE

It is waiting until you know which office someone will sit in before installing their desk. Otherwise you end up with a desk on the wrong floor and a person who cannot use it.

WHY IT MATTERS IN PRODUCTION

The failure this prevents shows up as a FailedScheduling event mentioning volume node affinity conflict, and it is genuinely confusing the first time because the cluster has plenty of capacity - just not in the zone the volume was born in. The side effect to expect is that PVCs now sit Pending until a pod references them, which looks broken to someone who does not know the binding mode. Saying that out loud avoids a false alarm.

STEP BY STEP

1. Explain the ordering problem between provisioning and scheduling in a zoned cluster.
2. Describe how delayed binding fixes it by letting the scheduler decide first.
3. Name the symptom it prevents: volume node affinity conflict.
4. Warn that a Pending PVC with no consumer is expected behaviour, not a fault.

EVIDENCE YOU WOULD QUOTE

```
oc get sc <class> -o jsonpath='{.volumeBindingMode}{"\n"}'

oc describe pvc <pvc> | grep -i 'waiting for first consumer'

oc get pv <pv> -o jsonpath='{.spec.nodeAffinity}' | python3 -m json.tool
```

RED FLAG - DO NOT SAY THIS

Do not use immediate binding on a multi-zone cluster. You are creating scheduling deadlocks for no benefit.

EXPECT THIS FOLLOW-UP

A pod is Pending with 'volume node affinity conflict'. Explain the sequence that caused it.

Q102 INTERMEDIATE

How do CSI volume snapshots work, and are they a backup?

SAY THIS

A VolumeSnapshot asks the CSI driver to take a point-in-time snapshot of a volume, governed by a VolumeSnapshotClass, and you can then create a new PVC from it. They are fast and cheap because they are usually copy-on-write on the same storage system. That is exactly why they are not a backup: if the array fails or is destroyed, the snapshots go with it, and a crash-consistent snapshot of a running database may not restore cleanly.

THINK OF IT LIKE

A snapshot is a bookmark in the book you are holding. A backup is a photocopy stored in another building. Losing the book loses every bookmark in it.

WHY IT MATTERS IN PRODUCTION

Snapshots are excellent for what they actually are: a fast rollback before a risky change, a way to clone production data into a test namespace, the storage half of a backup tool's workflow. The interview point is being explicit about the two independent limitations - same failure domain, and crash consistency - and then describing how you get application consistency by quiescing the application or using its native dump before snapshotting.

STEP BY STEP

1. Describe the objects: VolumeSnapshotClass, VolumeSnapshot, VolumeSnapshotContent, and restore by creating a PVC from the snapshot.
2. State the two limits clearly: same failure domain, and crash-consistent by default.
3. Explain how to reach application consistency - quiesce, hook, or native dump.
4. Give the legitimate uses and pair them with a real off-array backup for durability.

EVIDENCE YOU WOULD QUOTE

```
oc get volumesnapshotclass ; oc get volumesnapshot -n <ns>

oc get volumesnapshot <snap> -o jsonpath='{.status.readyToUse}{"\n"}'

oc get volumesnapshotcontent | head
```

RED FLAG - DO NOT SAY THIS

Do not call snapshots a backup strategy. It is the answer that ends senior interviews early.

EXPECT THIS FOLLOW-UP

How would you take an application-consistent snapshot of a running database?

Q103 INTERMEDIATE

How do you expand a persistent volume?

SAY THIS

If the StorageClass allows expansion, you edit the claim's requested size upward and the CSI driver grows the backend volume. Whether the filesystem is grown online or needs the pod to restart depends on the driver. Shrinking is not supported at all - the only route to a smaller volume is to create a new one and copy the data. So the operational rule is to plan for growth and monitor usage rather than treating expansion as a routine dial.

THINK OF IT LIKE

You can knock through into the next room, but you cannot un-knock it. Extensions are one way.

WHY IT MATTERS IN PRODUCTION

Expansion becomes an incident when nobody is watching usage, because a full volume usually means the application has already failed - a database that cannot write, or a log volume that filled and took the pod with it. The right posture is alerting on percentage used with enough lead time to expand calmly, plus knowing in advance whether your driver needs a restart, because that changes expansion from a background task into a change window.

STEP BY STEP

1. Verify allowVolumeExpansion on the class before promising anything.
2. Patch the claim's requested size and watch the PVC conditions for resize progress.
3. Confirm whether the filesystem resize is online or requires a pod restart for your driver.
4. Alert on volume usage with enough headroom that expansion is planned, not emergency work.

EVIDENCE YOU WOULD QUOTE

```
oc get sc <class> -o jsonpath='{.allowVolumeExpansion}{"\n"}'

oc patch pvc <pvc> -p '{"spec":{"resources":{"requests":{"storage":"200Gi"}}}}'

oc get pvc <pvc> -o jsonpath='{.status.conditions}' | python3 -m json.tool
```

RED FLAG - DO NOT SAY THIS

Do not promise you can shrink a volume later. You cannot, and someone will plan around your answer.

EXPECT THIS FOLLOW-UP

A volume is 95% full and expansion needs a restart. How do you sequence it?

Q104 INTERMEDIATE

When is ephemeral or node-local storage the right choice?

SAY THIS

emptyDir is right for scratch space that dies with the pod - caches, temporary files, a shared directory between containers in the same pod. Local persistent volumes are right when the workload needs the performance of a directly attached disk and handles its own replication, such as a distributed database. hostPath is almost never right for applications; it is for node-level system components and it breaks isolation.

THINK OF IT LIKE

emptyDir is a notepad on the desk that gets binned each evening. A local volume is a safe bolted to that specific floor - fast to reach, useless if you move offices.

WHY IT MATTERS IN PRODUCTION

The trade-off with local volumes is that the data is pinned to one node, so the pod is pinned too. Losing that node means losing that replica's data, which is fine for a distributed database that replicates across nodes and unacceptable for a single-instance one. It also blocks the cluster autoscaler from removing the node and complicates upgrades, because draining is no longer free.

STEP BY STEP

1. Distinguish emptyDir, local persistent volumes and hostPath by purpose and lifetime.
2. State the pinning consequence of local storage for scheduling, drains and autoscaling.
3. Require application-level replication before accepting local storage for stateful work.

4 Set size limits on emptyDir so a runaway process cannot fill the node and trigger DiskPressure.

EVIDENCE YOU WOULD QUOTE

```
oc get pod <pod> -o jsonpath='{.spec.volumes}' | python3 -m json.tool

oc get pv -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeAffinity.required.nodeSelectorTe
rms[0].matchExpressions[0].values[0]

oc describe node <node> | grep -i diskpressure
```

RED FLAG - DO NOT SAY THIS

Do not use hostPath for application storage. It bypasses isolation and ties the workload to a machine in a way nothing else will respect.

EXPECT THIS FOLLOW-UP

A local-volume workload blocks node drain during upgrades. What do you do?

Q105 SENIOR

A PVC has been Pending for twenty minutes. Walk me through it.

SAY THIS

I read the PVC's events first, because they usually name the stage that is stuck. Then I work the chain: does the requested StorageClass exist and is it spelled correctly, is the binding mode WaitForFirstConsumer with no pod yet - which is normal - is the CSI controller healthy and authenticated to the backend, does the backend have capacity and quota, and does the requested access mode exist on this driver at all.

THINK OF IT LIKE

It is a delivery that has not arrived. You check the order was placed, the address was valid, the warehouse has stock, and the courier is actually operating - in that order.

WHY IT MATTERS IN PRODUCTION

The two answers that surprise people are the harmless one and the invisible one. Harmless: with delayed binding, a claim with no consuming pod is supposed to stay Pending, so the "problem" is a missing or unschedulable pod. Invisible: the backend is out of capacity or the driver's credentials expired, and the only place that is stated is the CSI controller log, not the Kubernetes objects.

STEP BY STEP

1 Read PVC events and conditions and take the message literally.
2 Confirm the StorageClass name resolves, and check the binding mode before assuming failure.
3 Check CSI controller pod health and its logs for backend errors, quota or authentication.
4 Validate the requested access mode and size against what the driver and backend actually support.

EVIDENCE YOU WOULD QUOTE

```
oc describe pvc <pvc> | sed -n '/Events/,$p'

oc -n openshift-cluster-csi-drivers logs deploy/<driver>-controller -c csi-provisioner
--tail=100

oc get sc ; oc get pvc <pvc> -o jsonpath='{.spec.storageClassName}{"\n"}'
```

RED FLAG - DO NOT SAY THIS

Do not delete and recreate the PVC as a first move. If provisioning succeeded partially you may orphan backend capacity nobody is tracking.

EXPECT THIS FOLLOW-UP

The events say nothing at all. Where do you look next?

Q106 SENIOR

A volume is stuck attached to a failed node. What do you do?

SAY THIS

This is a safety problem before it is an availability problem. The volume is attached to a node that is unreachable, and the replacement pod cannot start until it is detached. The critical question is whether the old node is genuinely dead or merely unreachable - if it is alive and still writing, force-detaching an RWO volume and mounting it elsewhere risks corruption. So I confirm the node is fenced or powered off, then let the controller detach, and force it only with that confirmation.

THINK OF IT LIKE

It is a safe deposit box still signed out to someone you cannot reach. You do not drill it open until you are certain they are not in the vault.

WHY IT MATTERS IN PRODUCTION

This is exactly what MachineHealthCheck automates when it is configured with proper remediation: an unhealthy node is deleted and, on infrastructure that supports it, the machine is destroyed, which guarantees it is not writing. Without that, a network partition produces a node that looks dead from the cluster and is happily running - the classic split-brain that turns an outage into data corruption.

STEP BY STEP

1. Establish whether the node is dead or partitioned - those need different actions.
2. Confirm fencing: the machine is deleted, powered off, or otherwise proven not to be writing.
3. Let the attach-detach controller complete the detach; delete the VolumeAttachment only after fencing is confirmed.
4. Verify the pod starts and the filesystem is clean, then fix the underlying cause and consider MachineHealthCheck for automated fencing.

EVIDENCE YOU WOULD QUOTE

```
oc get volumeattachment | grep <node>

oc get node <node> -o jsonpath='{.status.conditions}' | python3 -m json.tool

oc get machine -A -o wide | grep <node>
```

RED FLAG - DO NOT SAY THIS

Do not force-delete the VolumeAttachment to speed things up. If the old node is alive you have just created two writers.

EXPECT THIS FOLLOW-UP

How does MachineHealthCheck change this scenario, and what does it require?

Q107 SENIOR

How do you achieve application-consistent backup?

SAY THIS

Crash-consistent means the copy looks like the power was pulled; the application may recover or may not. Application-consistent means the application was told to quiesce - flush buffers, freeze writes - at the moment of the copy. In practice I use the backup tool's pre and post hooks to run the application's own quiesce or dump command around the snapshot, and then I prove it by restoring into a scratch namespace and starting the application.

THINK OF IT LIKE

Crash-consistent is photographing a kitchen mid-service. Application-consistent is asking the chef to put the knives down for three seconds first.

WHY IT MATTERS IN PRODUCTION

The proof step is the part that separates a real answer from a plausible one. Plenty of teams have hooks configured that silently fail - the command name changed, the container does not have the binary, the hook timed out - and nobody notices until a restore is needed. A scheduled restore test into a scratch namespace, with the application actually started and queried, is the only evidence that counts.

STEP BY STEP

1. Define the difference and say which one your current setup actually produces.
2. Use pre and post hooks to quiesce the application around the snapshot or copy.
3. Prefer the application's native backup mechanism where one exists, and back that up.
4. Restore into a scratch namespace on a schedule, start the application, and record the result as evidence.

EVIDENCE YOU WOULD QUOTE

```
oc get backup,restore -n openshift-adp

oc get backup <name> -n openshift-adp -o jsonpath='{.status.phase} {.status.errors}{"\n"}'

Documented restore test: date, dataset, measured restore time, verification query
```

RED FLAG - DO NOT SAY THIS

Do not claim backups work because the job reports success. Until you have restored one, you have a backup job, not a backup.

EXPECT THIS FOLLOW-UP

Your restore test fails on one of five applications. What do you do about the other four?

Q108 SENIOR

How do you troubleshoot storage performance complaints?

SAY THIS

I turn the complaint into numbers before doing anything else: which operation is slow, what latency is being observed, at what IOPS and block size, and compared with what baseline. Then I separate the layers - application, filesystem, node, and backend - by measuring at each. Very often the limit is the class of storage that was requested rather than a fault, and that is a capacity conversation, not a troubleshooting one.

THINK OF IT LIKE

"The tap is slow" could be the tap, the pipe, or the water pressure into the street. You measure at each point rather than replacing the tap and hoping.

WHY IT MATTERS IN PRODUCTION

The most common real causes are noisy neighbours on shared backend capacity, a workload using a general-purpose class when it needs a provisioned-IOPS one, and small random IO against storage optimised for sequential throughput. Node-level contention matters too - a log-heavy pod saturating the same path. Having a baseline from cluster build makes this a ten-minute comparison instead of a week of theories.

STEP BY STEP

1. Quantify the complaint: operation, latency, IOPS, block size, and when it started.
2. Measure at each layer - inside the pod, on the node, and from the backend's own metrics.
3. Compare against the class's expected performance and your recorded baseline.
4. Decide between a fix, a class change, or a capacity purchase, and record the new expected numbers.

EVIDENCE YOU WOULD QUOTE

```
oc exec <pod> -- dd if=/dev/zero of=/data/test bs=1M count=1024 oflag=direct

node_disk_io_time_seconds_total ; node_disk_read_time_seconds_total by device

Backend array metrics: queue depth, latency, throughput for the volume in question
```

RED FLAG - DO NOT SAY THIS

Do not accept "storage is slow" without numbers. Half of these tickets end with the workload being on the class it asked for.

EXPECT THIS FOLLOW-UP

The backend reports 2ms latency and the application sees 40ms. Where is the time going?

Q109 ARCHITECT

How would you design the storage class catalogue for a platform?

SAY THIS

I would offer a small number of classes that describe guarantees rather than products - something like standard, fast, shared and retained-critical - each with a documented performance envelope, reclaim policy, expansion support, snapshot support and cost. The default would be the safe, general-purpose one. Anything outside the catalogue needs a conversation, because every extra class is another thing to test, monitor and migrate later.

THINK OF IT LIKE

It is a menu, not a warehouse inventory. Four dishes people understand beat forty they have to ask about.

WHY IT MATTERS IN PRODUCTION

A small catalogue is an operational decision as much as a user-experience one. Each class carries a testing burden - expansion, snapshot, restore, failure behaviour - and a migration burden when the backend changes. Teams also make better choices when the names describe guarantees, because a developer can reason about "fast-retained" without knowing anything about the array underneath. Publishing cost per gigabyte alongside is what actually changes behaviour.

STEP BY STEP

1. Gather the real requirements: latency, throughput, sharing, durability, retention, cost sensitivity.
2. Define a small set of classes by guarantee, and document the envelope and cost of each.

3 Set a safe default and make retention policy explicit in the name where it matters.
4 Test each class for expansion, snapshot and restore before publishing it, and review the catalogue as usage data arrives.

EVIDENCE YOU WOULD QUOTE

```
oc get sc -o wide ; usage and cost per class per namespace

Documented performance envelope and test results per class

oc get pvc -A -o custom-columns=NS:.metadata.namespace,CLASS:.spec.storageClassName,SIZE:.spec.
resources.requests.storage | sort -k2
```

RED FLAG - DO NOT SAY THIS

Do not expose one class per backend feature. You will end up supporting a dozen combinations nobody chose deliberately.

EXPECT THIS FOLLOW-UP

A team wants a class with guaranteed IOPS. What do you need to know before saying yes?

Q110 ARCHITECT

How would you plan a migration from one storage backend to another?

SAY THIS

I treat it as a data migration project rather than a storage change. First an inventory - every claim, its size, class, access mode, owning application and criticality. Then a per-application method: snapshot-and-restore, a backup tool's migration workflow, or application-native replication for databases, which is usually the safest. Then a pilot on something low-risk, measured cutover windows, and a documented rollback while the old backend still holds the data.

THINK OF IT LIKE

It is moving house one room at a time, with the old keys still in your pocket until the last box is unpacked and checked.

WHY IT MATTERS IN PRODUCTION

The two things that derail these projects are discovered late: applications that cannot tolerate the downtime the chosen method requires, and access modes that do not exist on the new backend. Both are found in the inventory phase if you actually do it. Keeping the old backend live until every application has been verified is what makes rollback real rather than theoretical, and that has a cost you should surface early.

STEP BY STEP

1 Inventory every volume with owner, size, class, access mode, criticality and downtime tolerance.
2 Choose a migration method per application, preferring native replication for databases.
3 Pilot on low-risk workloads, measure the actual cutover time, and refine the runbook.
4 Migrate in waves with verification after each, keeping the old backend as rollback until sign-off.

EVIDENCE YOU WOULD QUOTE

```
oc get pvc -A -o wide # full inventory with class and size

Per-application migration runbook with measured cutover and verification steps

Post-migration verification: application checks, performance comparison against baseline
```

RED FLAG - DO NOT SAY THIS

Do not decommission the old backend before every application has been verified. Rollback that depends on data you deleted is not rollback.

EXPECT THIS FOLLOW-UP

One application cannot tolerate any downtime. What is your approach for that one?

PART 8 · Q111-Q128 · 18 QUESTIONS

Security, identity and compliance

Six independent gates, and why collapsing any two of them costs you the offer

Security is the topic where confident vagueness is punished hardest. Interviewers ask about RBAC and SCC in the same breath specifically to see whether you know they answer different questions - one is about API calls, the other is about what a pod may be on the node. The same applies to secrets versus encryption, certificates versus trust, and policy versus enforcement. This part keeps those gates separate, and it consistently prefers the answer that fixes the workload over the answer that grants a privilege, because that preference is exactly what is being measured.

Level mix - Foundation: 5 · Intermediate: 6 · Senior: 5 · Architect: 2

Defence in depth: six independent gates

Each gate answers a different question. Candidates lose points by collapsing them - RBAC and SCC are not two names for the same control.

Identity	Who are you? OAuth, IdP, ServiceAccount and bound tokens
RBAC	What API calls may you make? Verbs on resources in a scope
SCC	What may your pod be? UID, capabilities, host access, volume types
NetworkPolicy	Who may talk to you? Default-deny plus explicit allow, east-west
Runtime	What may the process do on the node? SELinux MCS, seccomp, dropped capabilities
Supply chain	Is the image trustworthy? Digest pinning, signing, SBOM, scanning, allowed registries

Q111 FOUNDATION

How do users authenticate to OpenShift?

SAY THIS

OpenShift runs an integrated OAuth server that issues tokens after delegating the actual identity check to a configured identity provider - LDAP, OIDC, an enterprise SSO, or htpasswd for bootstrap. The oc client presents that token on every request, and the API server maps it to a user and their groups. Group membership from the provider is what should drive RBAC, so that access follows the corporate directory rather than a local list.

THINK OF IT LIKE

OAuth is the security desk that issues a visitor badge. It does not decide who you are - it checks with your employer - and the badge is what every door reads afterwards.

WHY IT MATTERS IN PRODUCTION

The operational point is that authentication and authorisation are separate, and both can fail independently. A user who can log in but sees nothing has an RBAC problem; a user who cannot log in at all has an identity provider problem, and the authentication Cluster Operator will usually say so. The other thing to say is that the kubeadmin bootstrap credential should be removed once a real provider is configured, because it is an unattributable cluster-admin account.

STEP BY STEP

1 Separate the two steps: the identity provider proves who you are, OAuth issues the token.

2 Bind RBAC to groups from the provider so joiners and leavers are handled by the directory.
3 Read the authentication Cluster Operator's conditions when logins fail - it names the cause.
4 Remove the bootstrap kubeadmin account once a real provider is live, and audit who has cluster-admin.

EVIDENCE YOU WOULD QUOTE

```
oc get oauth cluster -o yaml | sed -n '/identityProviders/,$p'

oc get co authentication -o jsonpath='{.status.conditions}' | python3 -m json.tool

oc get users ; oc get identities ; oc get clusterrolebinding | grep cluster-admin
```

RED FLAG - DO NOT SAY THIS

Do not leave kubeadmin in place on a production cluster. It is an anonymous administrator and it will be found in the first audit.

EXPECT THIS FOLLOW-UP

A user can log in but sees no projects. Where do you look?

Q112 FOUNDATION

Explain RBAC: Role versus ClusterRole, RoleBinding versus ClusterRoleBinding.

SAY THIS

A Role is a namespaced set of permissions - verbs on resources within one namespace. A ClusterRole is the same idea but cluster-scoped, and it is also the only way to grant access to cluster-scoped resources such as nodes or PersistentVolumes. A RoleBinding grants a role within one namespace, and importantly it can reference a ClusterRole, which is how you reuse a standard permission set namespace by namespace. A ClusterRoleBinding grants it everywhere.

THINK OF IT LIKE

A ClusterRole is a job description; a binding is the employment contract that says where you do that job. The same job description can apply to one branch or to the whole company.

WHY IT MATTERS IN PRODUCTION

The mistake that shows up in real clusters is using a ClusterRoleBinding when a RoleBinding would do. Someone needs edit access in one namespace, gets a ClusterRoleBinding to edit, and now has write access to every namespace in the cluster including platform ones. Because nothing breaks, nobody notices until an audit. Reviewing ClusterRoleBindings periodically is one of the highest-value security chores available.

STEP BY STEP

1 Define the four objects and which of them are namespaced.
2 Highlight the useful pattern: RoleBinding referencing a ClusterRole for reusable permission sets.
3 Explain why cluster-scoped resources require a ClusterRole.
4 Audit ClusterRoleBindings regularly and bind to groups rather than individual users.

EVIDENCE YOU WOULD QUOTE

```
oc auth can-i --list --as=<user> -n <ns>

oc get clusterrolebinding -o wide | grep -v 'system:'

oc describe clusterrole <role> | head -30
```

RED FLAG - DO NOT SAY THIS

Do not grant cluster-admin to solve a namespace-scoped permission problem. It is fast, it works, and it is the finding that ends up in the report.

EXPECT THIS FOLLOW-UP

Someone needs to read pods in every namespace. Which objects do you create?

Q113 FOUNDATION

How do RBAC and Security Context Constraints solve different problems?

SAY THIS

RBAC answers "what API calls may this identity make?" - can you create a pod, read a secret, delete a deployment. SCC answers "what may this pod actually be on the node?" - which UID it runs as, whether it can escalate privileges, which capabilities and volume types it may use, whether it can touch host namespaces. You can have permission to create a pod and still have that pod rejected because it asked to run as root.

THINK OF IT LIKE

RBAC is whether you are allowed to book the meeting room. SCC is what you are allowed to do once you are inside it - and no amount of booking permission lets you drill into the wall.

WHY IT MATTERS IN PRODUCTION

This distinction is asked in almost every OpenShift interview, and the tell is a candidate who tries to solve an SCC rejection with RBAC or vice versa. When a pod is rejected because it requests a UID or privilege the SCC does not allow, the right response is to fix the workload so it runs under restricted-v2 - and only if that is truly impossible, create a narrowly scoped custom SCC bound to one ServiceAccount, never a broad grant of privileged.

STEP BY STEP

1. State the two questions each control answers, in one sentence each.
2. Give the concrete case: permission to create a pod, rejection because of its security context.
3. Explain how SCC selection works - it is matched against the pod's ServiceAccount.
4. Give the escalation ladder: fix the image, then a narrow custom SCC, never blanket privileged.

EVIDENCE YOU WOULD QUOTE

```
oc get scc ; oc describe scc restricted-v2 | head -30

oc get pod <pod> -o jsonpath='{.metadata.annotations.openshift\.io/scc}{"\n"}'

oc adm policy who-can use scc privileged
```

RED FLAG - DO NOT SAY THIS

Do not describe SCC as "RBAC for pods". They are orthogonal, and the interviewer is listening for exactly that word.

EXPECT THIS FOLLOW-UP

A pod is rejected with a UID range error. Walk me through your options in order.

Q114 FOUNDATION

What is a Secret, and how is it different from a ConfigMap?

SAY THIS

Both hold key-value configuration data and both can be mounted as files or exposed as environment variables. A Secret is intended for sensitive data: it is base64-encoded in the API, can be encrypted at rest in etcd, is not shown by default in some tooling, and has its own RBAC surface so you can grant access to config without granting access to credentials. Base64 is encoding, not encryption, and that distinction matters.

THINK OF IT LIKE

A ConfigMap is a note on the noticeboard. A Secret is a note in a locked drawer - but the drawer is only locked if someone actually turned the key.

WHY IT MATTERS IN PRODUCTION

The practical guidance is about handling rather than the object. Secrets mounted as files are safer than environment variables, because environment variables leak into crash dumps, process listings and logs. Secrets should never be committed to Git, which is why external secret operators and sealed secrets exist. And etcd encryption at rest needs to be enabled deliberately - it is not on by default.

STEP BY STEP

1. State what is genuinely different: RBAC surface, encryption-at-rest option, tooling treatment.
2. Be explicit that base64 is encoding and provides no protection.
3. Prefer file mounts over environment variables, and explain why.
4. Describe how secrets get into the cluster safely - external secret manager or sealed secrets, never plaintext in Git.

EVIDENCE YOU WOULD QUOTE

```
oc get secret <name> -o jsonpath='{.data}' | python3 -m json.tool

oc get apiserver cluster -o jsonpath='{.spec.encryption.type}{"\n"}'

oc get rolebinding -n <ns> -o wide | grep secret
```

RED FLAG - DO NOT SAY THIS

Do not say Secrets are encrypted. They are base64-encoded, and encryption at rest is a separate setting you must enable.

EXPECT THIS FOLLOW-UP

How do secrets reach the cluster in your GitOps workflow?

Q115 FOUNDATION

What does the restricted-v2 SCC enforce?

SAY THIS

restricted-v2 is the default SCC for most workloads and it enforces the baseline that a well-behaved container should already meet: run as a non-root, arbitrarily assigned UID from the namespace's range, no privilege escalation, all Linux capabilities dropped, the default seccomp profile applied, no host namespaces, host network or host paths, and only safe volume types. It is deliberately strict so that the safe option is the automatic one.

THINK OF IT LIKE

It is the standard building fire code. Nobody argues about it per room - the constraints are the same for everyone, and exceptions require a documented case.

WHY IT MATTERS IN PRODUCTION

The reason this matters in interviews is that it is where most vendor images fail, and the response distinguishes candidates. An image that hardcodes a UID or writes to a directory owned by a fixed user will be rejected, and the correct sequence is to make the image arbitrary-UID friendly - group-writable paths owned by the root group - rather than to escalate the pod's privileges. Every escalation is permanent in practice, because nobody ever comes back to remove it.

STEP BY STEP

1. List what it enforces, grouping into identity, privilege, capabilities and host access.
2. Explain the arbitrary UID model and why images must be written for it.
3. Give the correct remediation order when a workload fails admission.
4. Note that exceptions should be narrow, bound to one ServiceAccount, documented and reviewed.

EVIDENCE YOU WOULD QUOTE

```
oc describe scc restricted-v2

oc get ns <ns> -o jsonpath='{.metadata.annotations.openshift\.io/sa\.scc\.uid-range}{"\n"}'

oc get pod <pod> -o jsonpath='{.spec.securityContext}' | python3 -m json.tool
```

RED FLAG - DO NOT SAY THIS

Do not reach for anyuid as the standard fix. It is a five-minute shortcut that becomes a permanent exception.

EXPECT THIS FOLLOW-UP

A vendor image writes to /var/lib/app as UID 1001. What do you do?

Q116 INTERMEDIATE

What do allowPrivilegeEscalation: false and dropping capabilities actually achieve?

SAY THIS

allowPrivilegeEscalation false sets the no-new-privileges flag on the process, so it cannot gain more privileges than it started with - setuid binaries and file capabilities stop working as an escalation path. Dropping capabilities removes specific kernel privileges from the process, such as changing network configuration or loading modules. Together they mean that even a compromised process inside the container has very little to work with.

THINK OF IT LIKE

It is confiscating the master key and the ladder before letting a contractor into the building. They can still do their job; they just cannot reach the roof.

WHY IT MATTERS IN PRODUCTION

The practical value is in reducing what an exploited application can do next. Most container escapes in the wild depend on either a capability that was never needed or a setuid path that was never removed. Dropping ALL and adding back only what is demonstrably required is the standard, and the exercise of finding out what is required usually reveals that the answer is nothing.

STEP BY STEP

1. Explain no-new-privileges and what escalation paths it closes.
2. Explain capabilities as fine-grained slices of root, and the drop-ALL-then-add-back practice.

3 Give a legitimate example that needs a capability, such as binding a low port, and the alternative.
4 Enforce it through the SCC rather than trusting each manifest to get it right.

EVIDENCE YOU WOULD QUOTE

```
oc get pod <pod> -o jsonpath='{.spec.containers[0].securityContext}' | python3 -m json.tool

oc describe scc restricted-v2 | grep -i -A3 capabilities

oc debug node/<node> -- chroot /host grep CapEff /proc/<pid>/status
```

RED FLAG - DO NOT SAY THIS

Do not add NET_ADMIN or SYS_ADMIN because an application "needs networking". Almost nothing legitimately needs those.

EXPECT THIS FOLLOW-UP

An application needs to bind port 80. What are your options besides a capability?

Q117 INTERMEDIATE

What is the seccomp RuntimeDefault profile?

SAY THIS

seccomp filters which system calls a process may make. RuntimeDefault applies the container runtime's curated profile, which blocks the syscalls that normal application workloads never use but that exploits frequently rely on. It is a cheap, broad reduction in kernel attack surface, it is part of the restricted-v2 baseline, and it very rarely breaks anything because the blocked calls are genuinely unusual.

THINK OF IT LIKE

It is a menu rather than an open kitchen. You can order anything a normal customer would want; you cannot walk in and use the gas line.

WHY IT MATTERS IN PRODUCTION

When seccomp does break something the failure is distinctive - the process dies with a signal or gets EPERM on an unusual syscall, and the node's audit log records the blocked call. The correct response is a custom profile that permits that specific syscall, not Unconfined, which removes the entire filter. Being able to describe that debugging path is what makes this more than a definition.

STEP BY STEP

1 Define seccomp as syscall filtering and RuntimeDefault as the runtime's curated profile.
2 Say why it is low risk - the blocked calls are rare in application code.
3 Describe the failure signature and how to identify the blocked syscall from audit logs.
4 Fix with a narrow custom profile; never fall back to Unconfined as the default.

EVIDENCE YOU WOULD QUOTE

```
oc get pod <pod> -o jsonpath='{.spec.securityContext.seccompProfile}{"\n"}'

oc debug node/<node> -- chroot /host ausearch -m seccomp -ts recent

oc describe scc restricted-v2 | grep -i seccomp
```

RED FLAG - DO NOT SAY THIS

Do not set seccompProfile to Unconfined to make a problem go away. You have removed the control rather than adjusted it.

EXPECT THIS FOLLOW-UP

A workload dies with a signal after enabling RuntimeDefault. How do you find out why?

Q118 INTERMEDIATE

What is etcd encryption at rest, and what does it protect?

SAY THIS

It encrypts sensitive API resources - Secrets, ConfigMaps and a few others - before they are written to etcd, so the data on disk and in etcd backups is not readable in plaintext. It protects against someone obtaining the disk, a snapshot or a backup file. It does not protect against anyone with API access and RBAC permission to read the Secret, because the API server decrypts on read.

THINK OF IT LIKE

It is a safe in the records room. It stops someone who steals the filing cabinet; it does nothing about the person who is legitimately allowed to open it.

WHY IT MATTERS IN PRODUCTION

Being precise about that boundary is the whole point of the question. Teams sometimes enable encryption at rest and consider secret management solved, when the far more likely exposure is over-broad RBAC or a secret printed into application logs. It is worth enabling - it is a simple, supported setting and it materially improves backup safety - but it belongs alongside least-privilege RBAC and external secret management, not instead of them.

STEP BY STEP

1. Explain what is encrypted and at what point in the write path.
2. State the threat model it addresses: stolen disks, snapshots and backups.
3. State what it does not address: authorised API reads and leaked secrets in logs.
4. Enable it, then verify the migration completed and re-check backup handling procedures.

EVIDENCE YOU WOULD QUOTE

```
oc get apiserver cluster -o jsonpath='{.spec.encryption.type}{"\n"}'

oc get co kube-apiserver -o jsonpath='{.status.conditions}' | python3 -m json.tool

oc get secret -A | wc -l # scope of the migration
```

RED FLAG - DO NOT SAY THIS

Do not claim encryption at rest protects secrets from users. It protects them from disks.

EXPECT THIS FOLLOW-UP

Where would you store the credentials that the cluster itself needs to bootstrap?

Q119 INTERMEDIATE

How do ServiceAccount tokens work securely?

SAY THIS

Modern tokens are projected into the pod by the kubelet: they are short-lived, bound to a specific audience, and bound to the pod's lifetime, so they are automatically rotated and become useless once the pod is gone. That replaces the old model of a permanent token stored in a Secret, which never expired and which anyone with read access to that Secret could use indefinitely.

THINK OF IT LIKE

It is a day pass that expires at 5pm and only opens the doors on your floor, instead of a master key that works forever and can be copied.

WHY IT MATTERS IN PRODUCTION

Two operational habits follow. First, disable automounting for workloads that never call the API - most applications do not need a token at all, and mounting one is free attack surface. Second, when integrating an external system, use a dedicated ServiceAccount with a narrow role and a bound token rather than creating a long-lived token Secret, because those are exactly what turns up in a repository six months later.

STEP BY STEP

1. Describe the projected token model: short-lived, audience-bound, rotated automatically.
2. Contrast it with legacy long-lived token Secrets and the risk they carry.
3. Set automountServiceAccountToken false for workloads that do not call the API.
4. For external integrations, use a dedicated ServiceAccount with least privilege and audit its use.

EVIDENCE YOU WOULD QUOTE

```
oc get pod <pod> -o jsonpath='{.spec.volumes[?(@.projected)]}' | python3 -m json.tool

oc get sa <sa> -o jsonpath='{.secrets}{"\n"}' # legacy tokens if any remain

oc auth can-i --list --as=system:serviceaccount:<ns>:<sa>
```

RED FLAG - DO NOT SAY THIS

Do not create long-lived ServiceAccount token Secrets for convenience. They are permanent credentials with no expiry and no rotation.

EXPECT THIS FOLLOW-UP

An external CI system needs to deploy into one namespace. How do you set that up?

Q120 INTERMEDIATE

How do service serving certificates work?

SAY THIS

You annotate a Service, and the service CA operator issues a certificate for that Service's internal DNS name and puts it in a Secret, which the workload mounts. Clients trust it by mounting the service CA bundle, which is injected into a ConfigMap by another annotation. The operator rotates the certificates automatically. It is the simple way to get internal TLS between cluster services without running your own PKI.

THINK OF IT LIKE

It is an in-house pass office. Staff get their own passes issued and renewed automatically, and everyone in the building already trusts the office that issues them.

WHY IT MATTERS IN PRODUCTION

The value is that it removes the most common excuse for unencrypted internal traffic, which is that certificate management is painful. The thing to know operationally is that these certificates are only trusted inside the cluster - they are not for external clients, who need a certificate from a public or corporate CA. Mixing those two up produces trust errors that look like application bugs.

STEP BY STEP

1. Describe the annotation on the Service and the Secret it produces.
2. Describe the CA bundle injection annotation on the client side.
3. Note that rotation is automatic, and that workloads must reload certificates or be restarted.
4. State the boundary: internal trust only; external clients need a corporate or public CA.

EVIDENCE YOU WOULD QUOTE

```
oc annotate svc <svc> service.beta.openshift.io/serving-cert-secret-name=<secret>

oc get secret <secret> -o jsonpath='{.data.tls\.crt}' | base64 -d | openssl x509 -noout -dates

oc get cm <bundle-cm> -o jsonpath='{.data.service-ca\.crt}' | head -5
```

RED FLAG - DO NOT SAY THIS

Do not offer service serving certificates to external clients. Nothing outside the cluster trusts that CA.

EXPECT THIS FOLLOW-UP

An internal client gets a certificate trust error after rotation. What happened?

Q121 INTERMEDIATE

What are admission webhooks and how do they affect reliability?

SAY THIS

Admission webhooks are external services the API server calls during admission - mutating ones can modify an object, validating ones can reject it. They are how policy engines, service meshes and many operators enforce rules. The reliability catch is failurePolicy: with Fail, if the webhook is unavailable the API server rejects the affected requests, so an unhealthy webhook can block object creation cluster-wide.

THINK OF IT LIKE

It is a compulsory inspection step on the production line. If the inspector does not turn up and the rule says nothing ships uninspected, the whole line stops.

WHY IT MATTERS IN PRODUCTION

This produces one of the most alarming failure modes in Kubernetes: a webhook whose pods are down, with failurePolicy Fail and a broad rule scope, means nobody can create pods - including the pods that would restore the webhook. Mitigations are namespace exclusions for system namespaces, tight timeouts, narrow scoping so the webhook only sees what it needs, and running the webhook itself with high availability.

STEP BY STEP

1. Define mutating and validating webhooks and where they sit in the admission chain.
2. Explain failurePolicy and the deadlock it can create when set to Fail.
3. Scope narrowly by namespace, resource and operation, and set short timeouts.
4. Exclude platform namespaces so a broken webhook cannot block cluster recovery.

EVIDENCE YOU WOULD QUOTE

```
oc get validatingwebhookconfiguration,mutatingwebhookconfiguration

oc get validatingwebhookconfiguration <name> -o jsonpath='{.webhooks[*].failurePolicy}{"\n"}'

oc get events -A --field-selector reason=FailedCreate | grep -i webhook
```

RED FLAG - DO NOT SAY THIS

Do not deploy a cluster-wide webhook with failurePolicy Fail and no namespace exclusions. You have built a single point of failure into object creation.

EXPECT THIS FOLLOW-UP

No pods can be created cluster-wide and a webhook is suspected. What do you do?

Q122 SENIOR

A vendor asks for the privileged SCC. How do you respond?

SAY THIS

I ask what specifically fails, because "needs privileged" is almost always a guess. Then I work up the ladder: fix the image so it runs as an arbitrary UID, then grant only the specific capability, host path or volume type it genuinely needs through a narrow custom SCC bound to one ServiceAccount in one namespace. Privileged is a last resort with a documented risk acceptance, an expiry date and compensating controls such as a dedicated node pool.

THINK OF IT LIKE

Somebody asking for the master key to every door usually needs one cupboard opened. You find out which cupboard first.

WHY IT MATTERS IN PRODUCTION

This question is really a test of how you handle pressure from a delivery deadline. The answer that scores is the one that offers a path forward rather than a flat refusal - a temporary, narrowly scoped exception on a dedicated node pool, with a ticket to fix the image, keeps the project moving without permanently weakening the cluster. Refusing outright and granting immediately are both wrong for the same reason: neither engages with the actual requirement.

STEP BY STEP

1. Get the evidence: the exact admission error, the syscall, the path or the port involved.
2. Try the image fix first - arbitrary UID, group-writable paths, no root requirement.
3. If a genuine privilege is needed, write a custom SCC granting only that, bound to one ServiceAccount.
4. If privileged is unavoidable, isolate on dedicated nodes, document risk acceptance with an owner and expiry, and monitor its use.

EVIDENCE YOU WOULD QUOTE

```
oc get events -n <ns> | grep -i 'unable to validate against any security context constraint'

oc adm policy who-can use scc privileged

oc get pod <pod> -o jsonpath='{.metadata.annotations.openshift\.io/scc}{"\n"}'
```

RED FLAG - DO NOT SAY THIS

Do not grant privileged to unblock a deadline. It never gets removed, and it will be attributed to you in the audit.

EXPECT THIS FOLLOW-UP

They say the vendor will not change the image and go live is Friday. Now what?

Q123 SENIOR

How do you manage and rotate secrets across the platform?

SAY THIS

I keep the source of truth outside the cluster in a secret manager, and sync into Kubernetes Secrets through an operator so nothing sensitive is ever in Git. Rotation is driven by the manager on a schedule, and workloads either reload the mounted file or are restarted deliberately by the sync controller. Access is least-privilege per namespace, etcd encryption is on, and I audit who reads secrets rather than assuming RBAC is correct.

THINK OF IT LIKE

It is a key cabinet with a sign-out log, not a bowl of keys by the door. Keys are reissued on a schedule, and you can tell who took which one.

WHY IT MATTERS IN PRODUCTION

Rotation is where most designs quietly fail, because rotating the secret is easy and getting the application to pick it up is not. Applications that read a secret once at startup keep using the old value until they restart, so rotation without a restart strategy produces a false sense of security - and rotation with an unplanned restart produces an outage. Deciding that behaviour per workload, in advance, is the real work.

STEP BY STEP

1. Keep the authoritative secret outside the cluster; sync in, never commit to Git.
2. Define rotation cadence per secret class and the propagation mechanism for each workload.
3. Decide per application whether it reloads or needs a controlled restart, and automate that.
4. Enable encryption at rest, restrict read access per namespace, and audit secret reads.

EVIDENCE YOU WOULD QUOTE

```
oc get externalsecret,secretstore -A # or the equivalent operator objects

oc get secret <name> -o jsonpath='{.metadata.annotations}' | python3 -m json.tool

Audit log query for get and list on secrets, grouped by identity
```

RED FLAG - DO NOT SAY THIS

Do not put secrets in Git, even encrypted, without a clear key management story. "It is encrypted" is not an answer if the key is in the same repository.

EXPECT THIS FOLLOW-UP

You rotate a database password. Walk me through what happens to the running pods.

Q124 SENIOR

How do you manage certificates, and how do you troubleshoot a certificate problem?

SAY THIS

I separate the three families: internal platform certificates that OpenShift rotates itself, service serving certificates issued by the service CA operator, and external certificates for routes and APIs that come from a corporate or public CA. For troubleshooting I check what the server actually presents, its dates and chain, whether the client trusts the issuer, and whether the name matches - and I always test from the same network position as the failing client.

THINK OF IT LIKE

A certificate problem is a passport problem: it is either expired, issued by an authority the officer does not recognise, or it has the wrong name on it. Three checks, in that order.

WHY IT MATTERS IN PRODUCTION

Expiry is the one that causes real incidents, because it is silent until the moment it is not, and it tends to hit at the worst time - a cluster that has been powered off past its certificate rotation window, or a route certificate nobody owned after a team reorganised. The prevention is boring and effective: an inventory of every externally managed certificate with owner and expiry, plus alerts at thirty and seven days.

STEP BY STEP

1. Classify the certificate: platform-managed, service CA, or externally issued.
2. Inspect what the server presents - subject, SAN, issuer, validity dates - from the client's position.
3. Check the client's trust store and whether the full chain is being served.
4. Renew through the owning mechanism, then verify; maintain an inventory with expiry alerts for anything external.

EVIDENCE YOU WOULD QUOTE

```
openssl s_client -connect <host>:443 -servername <host> </dev/null 2>/dev/null | openssl x509
-noout -subject -issuer -dates

oc get csr | grep -i pending # node certificates awaiting approval

oc get secret -A -o json | jq -r
'.items[]|select(.type=="kubernetes.io/tls")|.metadata.namespace+"/"+.metadata.name'
```

RED FLAG - DO NOT SAY THIS

Do not tell clients to skip verification to get past a certificate error. You have turned a known problem into an invisible one.

EXPECT THIS FOLLOW-UP

The cluster was powered off for six weeks. What certificate problems do you expect?

Q125 SENIOR

How do you secure the container image supply chain?

SAY THIS

End to end: build from approved base images in a controlled pipeline, scan for vulnerabilities and fail the build on policy, generate an SBOM, sign the image, push to a trusted registry, and enforce at admission that only signed images from allowed registries may run, referenced by digest. Then promote the same digest through environments rather than rebuilding, so what was tested is what runs.

THINK OF IT LIKE

It is food safety in a restaurant: approved suppliers, inspection on arrival, a label on every batch, and a rule that nothing unlabelled goes on a plate.

WHY IT MATTERS IN PRODUCTION

The step teams skip is enforcement. Scanning and signing produce reports and signatures that nobody checks at runtime, so an unsigned image from a random registry still runs. Admission policy is what turns the pipeline's work into an actual control. The other frequently missing piece is the response process - a scan result is only useful if there is an agreed path from finding to rebuild to redeploy, with a deadline.

STEP BY STEP

1. Control the inputs: approved, pinned base images and a controlled build environment.
2. Scan and generate an SBOM in the pipeline, with policy that fails the build.
3. Sign the image and promote the identical digest across environments.
4. Enforce at admission - allowed registries, signature required, digest references - and define the remediation process with deadlines.

EVIDENCE YOU WOULD QUOTE

```
oc get image.config.openshift.io cluster -o jsonpath='{.spec.registrySources}' | python3 -m
json.tool

cosign verify <registry>/<image>@sha256:<digest>

oc get pods -A -o jsonpath='{range .items[*]}{.spec.containers[*].image}{"\n"}{end}' | grep -v
'@sha256' | head
```

RED FLAG - DO NOT SAY THIS

Do not describe scanning as your supply chain security. Without admission enforcement it is a report, not a control.

EXPECT THIS FOLLOW-UP

A critical CVE lands in your base image. Walk me through the next four hours.

Q126 SENIOR

How do audit logs support security and operations?

SAY THIS

The API server audit log records who did what, to which object, when, and what the outcome was. It is how you answer questions that nothing else can: who deleted this namespace, who read this secret, which ServiceAccount is generating the request storm. The policy is tunable, because logging every request at full fidelity is expensive, and logs should be forwarded off-cluster so they survive the incident they describe.

THINK OF IT LIKE

It is the building's access log. Nobody reads it on a normal day, and it is the only thing that answers the question after something goes missing.

WHY IT MATTERS IN PRODUCTION

The operational uses are broader than security. Audit logs are how you find the client hammering the API server, how you attribute a surprise configuration change during an incident, and how you prove a control was in force during an audit. The two design decisions that matter are the audit profile - default, write-request bodies, or all-request bodies - and forwarding, because logs stored only on the control plane are lost in exactly the scenarios you care about.

STEP BY STEP

1. Explain what an audit event contains and the levels available.
2. Set an audit profile that balances fidelity against volume, and document the choice.
3. Forward off-cluster with retention that matches your compliance requirement.
4. Practise the queries you will need under pressure: who deleted X, who read secret Y, which identity is generating this load.

EVIDENCE YOU WOULD QUOTE

```
oc get apiserver cluster -o jsonpath='{.spec.audit.profile}{"\n"}'

oc adm node-logs <master> --path=kube-apiserver/audit.log | tail -5

Query: verb=delete, resource=namespaces, sorted by timestamp with user attribution
```

RED FLAG - DO NOT SAY THIS

Do not rely on audit logs that are only stored on the control plane. If you lose the cluster you lose the evidence.

EXPECT THIS FOLLOW-UP

A production namespace vanished overnight. How do you find out who and when?

Q127 ARCHITECT

How does the Compliance Operator fit into a governance programme?

SAY THIS

It runs benchmark-based scans against the cluster and its nodes - CIS, PCI and similar profiles - and reports each rule as pass, fail or not applicable, with remediation content for many findings. The value is that compliance becomes continuous and evidence-based rather than an annual spreadsheet exercise. The judgement it requires is deciding which findings to remediate, which to accept with justification, and which are false positives for your architecture.

THINK OF IT LIKE

It is a standing health inspection rather than a once-a-year visit. The point is not the certificate on the wall; it is that problems are found in days rather than months.

WHY IT MATTERS IN PRODUCTION

The trap is auto-applying every remediation. Some benchmark rules conflict with how a platform is legitimately operated, and blind remediation can break authentication, monitoring or node configuration. The mature pattern is to scan continuously, triage into remediate, accept-with-justification and not-applicable, apply remediations through the normal change process, and track the exception register as a living document with owners and review dates.

STEP BY STEP

1. Select the profiles that match the actual regulatory obligation, not every available one.
2. Scan on a schedule and triage findings into remediate, accept or not-applicable.
3. Apply remediations through change control, in a non-production cluster first.
4. Maintain an exception register with owner, justification and review date, and report trend rather than raw counts.

EVIDENCE YOU WOULD QUOTE

```
oc get compliancescan,compliancecheckresult -n openshift-compliance | head -20

oc get compliancecheckresult -n openshift-compliance -l
compliance.openshift.io/check-status=FAIL
```

```
Exception register with owner, justification and next review date
```

RED FLAG - DO NOT SAY THIS

Do not auto-remediate everything a scanner reports. Some rules will break your cluster, and the scanner does not know your architecture.

EXPECT THIS FOLLOW-UP

A benchmark rule conflicts with how your monitoring works. How do you handle it?

Q128 ARCHITECT

How would you design the security model for a multi-tenant cluster?

SAY THIS

In layers, with each one independently defensible. Identity from the corporate directory with RBAC bound to groups. Namespaces as the tenancy unit with quota, LimitRange and default-deny NetworkPolicy applied automatically. restricted-v2 as the baseline with exceptions narrow, owned and time-boxed. Supply chain enforcement at admission. Egress control and audit logging forwarded off-cluster. And a clear, written line where soft tenancy stops and a separate cluster starts.

THINK OF IT LIKE

It is a shared office building with real security: one directory of who works here, locks on each floor, rules about what tenants may install, deliveries checked at the door, and cameras that record.

WHY IT MATTERS IN PRODUCTION

The architectural judgement being tested is knowing the limit of the model. Namespaces plus policy is genuinely strong for teams within one organisation who are not adversarial. It is not sufficient for untrusted code or for tenants with incompatible regulatory obligations, and pretending otherwise is how organisations end up with a security incident that was predictable from the architecture diagram. Saying where the line is - and what you would do on the other side of it - is the answer.

STEP BY STEP

1. Layer the controls: identity, authorisation, workload constraints, network, supply chain, audit.
2. Automate the per-namespace baseline so no tenant can exist without it.
3. Define exception handling: narrow scope, named owner, expiry, compensating control.
4. State the escalation criteria to dedicated nodes or a separate cluster, and price both options honestly.

EVIDENCE YOU WOULD QUOTE

```
Namespace conformance report: quota, LimitRange, NetworkPolicy, SCC exceptions

oc get clusterrolebinding -o wide | grep -v 'system:' # standing privilege review

Exception register and audit log forwarding status
```

RED FLAG - DO NOT SAY THIS

Do not claim namespaces give you hard multi-tenancy. The follow-up question is always about untrusted workloads, and there is only one honest answer.

EXPECT THIS FOLLOW-UP

A tenant runs untrusted customer-supplied code. Does your model still hold?

PART 9 · Q129-Q140 · 12 QUESTIONS

Operators and lifecycle management

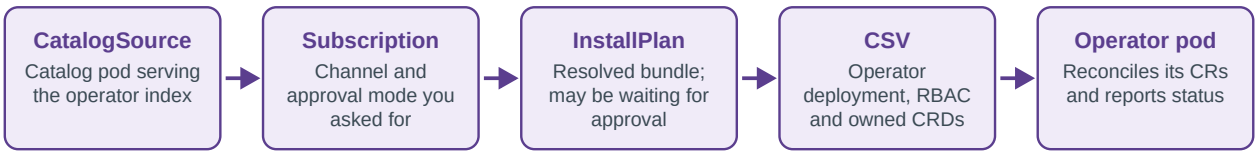
The chain from catalogue to running controller, and what to do when it stops halfway

Operators are how OpenShift ships almost everything optional, so being fluent in the OLM object chain is a practical skill rather than trivia. The chain is short - CatalogSource, Subscription, InstallPlan, ClusterServiceVersion, operator pod - and almost every installation problem is diagnosed by walking it forwards and stopping at the first object that has no child. The other half of this topic is judgement: which operators you should adopt at all, and what you take on when you put someone else's controller in your cluster with cluster-wide permissions.

Level mix - Foundation: 4 · Intermediate: 4 · Senior: 3 · Architect: 1

How an operator actually gets installed

When an operator will not install, walk this chain forwards and stop at the first object that has no child. That object's status holds your answer.



Q129 FOUNDATION

What is an Operator?

SAY THIS

An operator is a controller that encodes the operational knowledge for a specific application. It watches custom resources and reconciles the real world toward them - installing, configuring, upgrading, backing up, failing over. The idea is that the instructions that used to live in a runbook and a person's head become code that runs continuously, so the system repairs and manages itself the same way Kubernetes manages pods.

THINK OF IT LIKE

It is hiring a specialist who never sleeps. Instead of a runbook that says how to add a database replica, you have someone watching who does it whenever the spec says so.

WHY IT MATTERS IN PRODUCTION

The reason this matters beyond definitions is that installing an operator is a trust decision. You are placing a controller with broad permissions into your cluster, and it will act on its own schedule - including upgrading itself if you chose automatic approval. That is enormously valuable when the operator is mature and enormously risky when it is not, which is why maturity level and permission scope are the questions to ask before installation, not after.

STEP BY STEP

1. Define it as a controller plus custom resources encoding operational knowledge.
2. Give the capability spectrum - install, configure, upgrade, backup, autopilot behaviours.
3. Point out the trust implication: broad permissions and autonomous action.
4. State what you check before adopting one: maturity, permissions, update channel, support status.

EVIDENCE YOU WOULD QUOTE

```
oc get csv -A -o custom-columns=NS:.metadata.namespace,NAME:.metadata.name,PHASE:.status.phase
```

```
oc get csv <csv> -n <ns> -o jsonpath='{.spec.install.spec.clusterPermissions}' | python3 -m
json.tool

oc get crd | grep <operator-domain>
```

RED FLAG - DO NOT SAY THIS

Do not describe an operator as "an installer". Installation is the least interesting thing a good operator does.

EXPECT THIS FOLLOW-UP

What do you check before installing a third-party operator in production?

Q130 FOUNDATION

What is a CatalogSource?

SAY THIS

A CatalogSource is a pod serving an index of available operator bundles, and it is where OLM looks to find operators you can install. OpenShift ships default sources - the Red Hat operators catalogue, certified operators, community operators - and you can add your own, which is exactly what you do in a disconnected environment where the catalogue is mirrored into an internal registry.

THINK OF IT LIKE

It is the shop's catalogue. Nothing can be ordered that is not in a catalogue the shop actually has on the shelf.

WHY IT MATTERS IN PRODUCTION

The reason to know this precisely is that a broken CatalogSource makes operators disappear from the console with no obvious explanation, and it is the first link in the install chain. In disconnected clusters it is also the most common failure point: the catalogue pod cannot pull its index image because of a proxy, a missing trust bundle, or a mirror that was never refreshed after an upgrade.

STEP BY STEP

1. Define it as an index-serving pod that OLM queries.
2. Name the default sources and the disconnected use case for custom ones.
3. Explain the symptom of failure: operators missing from the catalogue entirely.
4. Check the catalog pod's status and logs first - image pull, proxy, and trust are the usual causes.

EVIDENCE YOU WOULD QUOTE

```
oc get catalogsource -A ; oc -n openshift-marketplace get pods

oc get catalogsource <name> -n openshift-marketplace -o jsonpath='{.status}' | python3 -m
json.tool

oc -n openshift-marketplace logs <catalog-pod> --tail=50
```

RED FLAG - DO NOT SAY THIS

Do not conclude an operator "does not exist" because it is not in the console. Check whether its catalogue is healthy first.

EXPECT THIS FOLLOW-UP

In a disconnected cluster, no operators are listed at all. What do you check?

Q131 FOUNDATION

What is a Subscription?

SAY THIS

A Subscription is your declaration that you want a particular operator, from a particular catalogue, following a particular channel, with either automatic or manual approval for updates. OLM reads it and resolves an InstallPlan. The channel is the important part - it determines which stream of versions you follow, and choosing a channel that is not compatible with your cluster version is a common way to get stuck.

THINK OF IT LIKE

It is a magazine subscription. You choose the title, the edition and whether new issues arrive automatically or wait for you to say yes.

WHY IT MATTERS IN PRODUCTION

The automatic versus manual choice is a real operational decision rather than a preference. Automatic keeps you current with security fixes and is right for well-tested platform operators; manual gives you a change window and is right for anything that touches data or that you want to test first. Whichever you choose, the Subscription's status is where OLM reports resolution failures, and that message is usually specific enough to act on.

STEP BY STEP

1. Define the four inputs: operator name, catalogue, channel, approval mode.
2. Explain channel selection and its relationship to the cluster version.
3. Contrast automatic and manual approval and say when each is appropriate.
4. Read the Subscription's status conditions when nothing installs - it names the resolution failure.

EVIDENCE YOU WOULD QUOTE

```
oc get subscription -A -o custom-columns=NS:.metadata.namespace,NAME:.metadata.name,CHANNEL:.spec.channel,APPROVAL:.spec.installPlanApproval

oc get subscription <name> -n <ns> -o jsonpath='{.status.conditions}' | python3 -m json.tool

oc get packagemanifest <name> -o jsonpath='{.status.channels[*].name}{"\n"}'
```

RED FLAG - DO NOT SAY THIS

Do not set every operator to automatic approval without thinking. Some of them will upgrade themselves during your busiest week.

EXPECT THIS FOLLOW-UP

How do you decide between automatic and manual approval for a data-plane operator?

Q132 FOUNDATION

What is a ClusterServiceVersion?

SAY THIS

The CSV is the manifest for a specific operator version: the deployment that runs the operator, the RBAC it requires, the CRDs it owns, its dependencies, and metadata such as install modes and upgrade edges. When you look at whether an operator is actually installed and healthy, the CSV's phase is the authoritative answer - Succeeded means it installed, anything else has a reason attached.

THINK OF IT LIKE

It is the appointment letter for a specific employee: their job title, their access rights, their responsibilities and who they report to.

WHY IT MATTERS IN PRODUCTION

The CSV is where you go to answer the security question that should always be asked about a third-party operator: what permissions does it want? The clusterPermissions section lists the RBAC it will be granted, and it is not unusual to find operators requesting far more than they need. Reading it before installation takes two minutes and occasionally changes the decision entirely.

STEP BY STEP

1. Define the CSV as the versioned operator manifest with deployment, RBAC and owned CRDs.
2. Use its phase and conditions as the source of truth for install health.
3. Read clusterPermissions before adopting a third-party operator.
4. Note install modes, which determine whether it can watch one namespace or must be cluster-wide.

EVIDENCE YOU WOULD QUOTE

```
oc get csv -n <ns> -o
custom-columns=NAME:.metadata.name,PHASE:.status.phase,MESSAGE:.status.message

oc get csv <csv> -n <ns> -o jsonpath='{.spec.installModes}' | python3 -m json.tool

oc describe csv <csv> -n <ns> | sed -n '/Conditions/,$p'
```

RED FLAG - DO NOT SAY THIS

Do not install a third-party operator without reading the permissions in its CSV. You are granting them, whether or not you looked.

EXPECT THIS FOLLOW-UP

An operator requests cluster-wide secret read access. What do you do?

Q133 INTERMEDIATE

What is an InstallPlan and how does approval work?

SAY THIS

An InstallPlan is the concrete set of resources OLM has resolved for a Subscription - the CSV to install, its dependencies, and the CRDs to create. With automatic approval it is approved and executed immediately. With manual approval it sits in a pending state until someone approves it, which is what gives you a change window. An InstallPlan that stays pending forever is usually a manual approval nobody knew about.

THINK OF IT LIKE

It is the quotation before the work starts. Automatic approval means the builders begin immediately; manual means it waits on your desk for a signature.

WHY IT MATTERS IN PRODUCTION

The failure this produces is quietly common: an operator is on manual approval, a new version becomes available, an InstallPlan is created and never approved, and months later someone notices the operator has not been updated since installation. Alerting on pending InstallPlans older than a threshold turns that from an audit finding into a routine ticket.

STEP BY STEP

1. Define the InstallPlan as the resolved set of resources for a Subscription.

2 Explain the two approval modes and where the approval is recorded.
3 Describe the stuck-pending symptom and how long it typically goes unnoticed.
4 Alert on pending InstallPlans and review them as part of routine maintenance.

EVIDENCE YOU WOULD QUOTE

```
oc get installplan -A -o custom-columns=NS:.metadata.namespace,NAME:.metadata.name,APPROVED:.spec.approved,PHASE:.status.phase

oc patch installplan <name> -n <ns> --type merge -p '{"spec":{"approved":true}}'

oc get installplan -A -o json | jq '.items[]|select(.spec.approved==false)|.metadata.name'
```

RED FLAG - DO NOT SAY THIS

Do not use manual approval without an alert on pending plans. You have created a process with no reminder attached.

EXPECT THIS FOLLOW-UP

An operator has not updated in eight months. Give me the two most likely causes.

Q134 INTERMEDIATE

What is an OperatorGroup?

SAY THIS

An OperatorGroup declares which namespaces the operators in its namespace are allowed to watch. It gives the operator the target namespace list and the RBAC scope to match. If it is missing, misconfigured, or if two OperatorGroups exist in one namespace, operator installation fails or the operator sits doing nothing, because it has no valid watch scope.

THINK OF IT LIKE

It is the territory assigned to a regional manager. Without a defined territory they turn up for work and have no idea which branches are theirs.

WHY IT MATTERS IN PRODUCTION

This is one of the most confusing OLM failures because the symptom - a CSV stuck in a pending or failed phase with a message about install modes - does not obviously point at the OperatorGroup. The usual cause is a mismatch between the operator's supported install modes and what the OperatorGroup asks for: an operator that only supports AllNamespaces placed in an OperatorGroup scoped to a single namespace, or two groups in the same namespace.

STEP BY STEP

1 Define it as the watch-scope declaration for operators in a namespace.
2 Connect it to the CSV's supported install modes - they must be compatible.
3 Name the failure signatures: no valid OperatorGroup, multiple groups, unsupported mode.
4 Check it early when a CSV will not reach Succeeded and the message mentions install modes.

EVIDENCE YOU WOULD QUOTE

```
oc get operatorgroup -A

oc get csv <csv> -n <ns> -o jsonpath='{.status.message}{"\n"}'

oc get operatorgroup <og> -n <ns> -o jsonpath='{.spec.targetNamespaces}{"\n"}'
```

RED FLAG - DO NOT SAY THIS

Do not create a second OperatorGroup in a namespace to fix a scope problem. Two groups is itself an error state.

EXPECT THIS FOLLOW-UP

A CSV is stuck with an install mode message. Walk me through it.

Q135 INTERMEDIATE

How do operator reconciliation loops work in practice?

SAY THIS

The operator watches its custom resources and the objects it owns. On any change - or on a periodic resync - it computes the difference between the spec and the observed world and takes action, then writes what it did and what it is blocked on into the custom resource's status conditions. Well-behaved operators are idempotent and level-triggered, meaning they act on current state rather than on the event that woke them.

THINK OF IT LIKE

It is a gardener who walks the whole garden each morning rather than remembering every instruction they were given. The state of the garden is the instruction.

WHY IT MATTERS IN PRODUCTION

The operational consequence is that the custom resource's status is your primary diagnostic, and the operator's logs are your secondary one. It also explains a specific frustration: manual changes to objects an operator owns are reverted, sometimes within seconds. The right response is to change the custom resource, not the managed object - and if the custom resource does not expose what you need, that is a conversation with the operator's maintainers, not a reason to fight the loop.

STEP BY STEP

1. Describe the loop: watch, diff, act, write status.
2. Explain level-triggered versus edge-triggered and why idempotency matters.
3. Use the CR's status conditions as the first diagnostic, operator logs as the second.
4. State the rule: change the CR, never the managed object, because reconciliation reverts it.

EVIDENCE YOU WOULD QUOTE

```
oc get <cr-kind> <name> -o jsonpath='{.status.conditions}' | python3 -m json.tool

oc -n <operator-ns> logs deploy/<operator> --tail=200

oc get events -n <ns> --sort-by=.lastTimestamp | tail -20
```

RED FLAG - DO NOT SAY THIS

Do not edit a resource an operator owns and expect it to stick. The loop will undo it and you will have learned nothing.

EXPECT THIS FOLLOW-UP

Your change to an operator-managed Deployment keeps disappearing. What do you do?

Q136 INTERMEDIATE

How do you sequence operator updates around a cluster upgrade?

SAY THIS

Before upgrading the cluster I check every installed operator's channel against the target OpenShift version, because an operator on a channel that does not support the target can block the upgrade or break afterwards. Where a newer channel is required, I move the operator first, in a maintenance window, and validate. Then I upgrade the cluster, then I revisit operators that have newer channels aligned with the new version.

THINK OF IT LIKE

It is checking that your plugins support the new version of the application before upgrading the application, not after it fails to start.

WHY IT MATTERS IN PRODUCTION

This is one of the most common causes of an upgrade that technically succeeded but left something broken. Operators declare compatibility, and the cluster's Upgradeable condition can reflect it, but not every operator is well-behaved about signalling. Building an explicit pre-upgrade inventory - operator, current version, channel, supported cluster versions - turns this from a surprise into a checklist item.

STEP BY STEP

1. Inventory installed operators with current version, channel and stated compatibility.
2. Move operators to a compatible channel first, one at a time, with validation.
3. Check the cluster's Upgradeable condition and any operator-reported blockers.
4. Upgrade the cluster, validate, then align operators with the new version's channels.

EVIDENCE YOU WOULD QUOTE

```
oc get subscription -A -o custom-columns=NS:.metadata.namespace,NAME:.metadata.name,CHANNEL:.sp
ec.channel,CSV:.status.installedCSV

oc adm upgrade # Upgradeable condition and any blocking messages

oc get clusterversion version -o jsonpath='{.status.conditions}' | python3 -m json.tool
```

RED FLAG - DO NOT SAY THIS

Do not upgrade the cluster and deal with operators afterwards. Some of them will not start on the new version and you will be debugging two changes at once.

EXPECT THIS FOLLOW-UP

An operator's channel does not support your target version. What are your options?

Q137 SENIOR

An operator will not install. Walk me through it.

SAY THIS

I walk the chain forwards and stop at the first object with no child. Is the CatalogSource pod running and serving? Does the Subscription resolve, or does its status name a resolution failure? Was an InstallPlan created, and is it waiting for approval? Did the CSV install, and if it is failed, what does its message say? Is there a valid OperatorGroup with a compatible install mode? Only then do I look at the operator pod itself.

THINK OF IT LIKE

It is following a paper trail through an office. You do not search the whole building - you find the desk where the form stopped moving.

WHY IT MATTERS IN PRODUCTION

In practice most failures cluster into four causes: a catalogue that cannot pull its index image, especially behind a proxy or in a disconnected cluster; a pending InstallPlan on manual approval; an OperatorGroup mismatch; and dependency resolution failing because another operator's version conflicts. Each has a distinct message, so the discipline of reading the status rather than guessing pays off immediately.

STEP BY STEP

1. Check CatalogSource pod health and whether the package appears in the package manifests.
2. Read the Subscription's status conditions for a resolution failure message.
3. Check whether an InstallPlan exists and whether it is approved.
4. Read the CSV phase and message, verify the OperatorGroup and install mode, then inspect the operator pod's logs.

EVIDENCE YOU WOULD QUOTE

```
oc get catalogsource,subscription,installplan,csv -n <ns>

oc get subscription <name> -n <ns> -o jsonpath='{.status.conditions}' | python3 -m json.tool

oc -n openshift-operator-lifecycle-manager logs deploy/catalog-operator --tail=100
```

RED FLAG - DO NOT SAY THIS

Do not delete and reinstall the Subscription as a first response. You lose the status message that was about to tell you the answer.

EXPECT THIS FOLLOW-UP

The Subscription reports a constraints-not-satisfiable error. What does that mean?

Q138 SENIOR

How do you run operators in a disconnected environment?

SAY THIS

Everything has to be mirrored and the cluster has to be told where to find it. I use oc-mirror to pull the release payload, the operator catalogues and their bundle images into an internal registry, then configure the cluster with ImageDigestMirrorSet and ImageTagMirrorSet so image references are redirected, add the registry's CA to the cluster trust bundle, and replace the default CatalogSources with the mirrored ones. Then I verify before anyone depends on it.

THINK OF IT LIKE

It is stocking a remote site's warehouse before winter. Nothing can be ordered later, so the inventory list has to be right the first time.

WHY IT MATTERS IN PRODUCTION

The failures here are almost always trust and redirection rather than the mirroring itself. A catalogue pod that cannot pull its index because the registry CA is not trusted, an image reference that was not covered by the mirror set, or a proxy configuration that intercepts registry traffic. It is also a recurring obligation: every cluster upgrade needs its payload and catalogues re-mirrored first, and forgetting that is what leaves a disconnected cluster unable to upgrade.

STEP BY STEP

1. Mirror the release payload and the required operator catalogues with oc-mirror, using a pinned image set configuration.
2. Apply ImageDigestMirrorSet and ImageTagMirrorSet, and add the registry CA to the cluster trust bundle.
3. Disable the default sources and create CatalogSources pointing at the mirrored index.
4. Verify by installing a test operator, and re-mirror as a standing step before every upgrade.

EVIDENCE YOU WOULD QUOTE

```
oc get imagedigestmirrorset,imagetagmirrorset

oc get catalogsource -n openshift-marketplace -o wide

oc -n openshift-marketplace logs <catalog-pod> | grep -i -E 'x509|refused|denied'
```

RED FLAG - DO NOT SAY THIS

Do not forget the trust bundle. A perfectly mirrored registry is useless if the cluster does not trust its certificate.

EXPECT THIS FOLLOW-UP

A mirrored catalogue pod crashes with an x509 error. What is missing?

Q139 SENIOR

How do you evaluate whether to adopt a third-party operator?

SAY THIS

I look at four things. Support status - is it Red Hat supported, certified, or community, and who do I call at 3am. Permission scope - what does the CSV request, and is it proportionate. Failure behaviour - what happens to running workloads if the operator is down, upgraded badly, or removed. And exit cost - can I uninstall it cleanly, or does it leave finalizers and CRDs behind that make the cluster hard to clean up.

THINK OF IT LIKE

It is hiring a contractor with keys to the building. References, scope of access, what happens if they do not turn up, and how hard it is to end the arrangement.

WHY IT MATTERS IN PRODUCTION

The exit cost question is the one people skip and later regret. Operators that own CRDs with finalizers can make namespaces impossible to delete once the operator is gone, and uninstalling in the wrong order - operator first, custom resources second - is exactly how you get stuck. Testing the uninstall path in a non-production cluster before adopting is cheap insurance.

STEP BY STEP

1. Establish support status and the escalation path for production incidents.
2. Read the requested RBAC in the CSV and challenge anything disproportionate.
3. Test failure behaviour: stop the operator and confirm running workloads survive.
4. Test the uninstall path, including custom resource removal order, before adopting.

EVIDENCE YOU WOULD QUOTE

```
oc get csv <csv> -o jsonpath='{.spec.install.spec.clusterPermissions}' | python3 -m json.tool

oc get packagemanifest <name> -o jsonpath='{.status.catalogSource}{"\n"}'

Documented uninstall test result from a non-production cluster
```

RED FLAG - DO NOT SAY THIS

Do not adopt an operator without testing what happens when it is not running. Some of them take the workload with them.

EXPECT THIS FOLLOW-UP

An operator you removed left a namespace stuck Terminating. How do you recover?

Q140 ARCHITECT

When would you write your own operator rather than use Helm or GitOps?

SAY THIS

Only when the thing you need is genuinely continuous and stateful - ongoing reconciliation, failover, backup orchestration, complex upgrade sequencing that depends on runtime state. If the requirement is templating and deploying manifests, Helm plus GitOps does it with far less to maintain. An operator is a long-lived piece of software with its own upgrade, security and on-call burden, and that cost is easy to underestimate at the point of writing it.

THINK OF IT LIKE

You do not build a robot to hang your washing out twice a week. You build one when the job has to be done continuously, precisely, and while nobody is watching.

WHY IT MATTERS IN PRODUCTION

The pattern that usually wins in enterprises is to reserve operators for the two or three genuinely stateful platform capabilities and use GitOps for everything else. The teams that write operators for every internal service end up with a dozen controllers that all need maintaining, none of which anyone remembers how to debug. If you do write one, the operator SDK and a clear conditions contract matter more than clever logic.

STEP BY STEP

1. Ask whether the work is continuous and state-dependent, or a deployment of static manifests.
2. Prefer GitOps plus Helm or Kustomize for anything declarative and stateless.
3. If an operator is justified, scope it narrowly and define its status conditions contract first.
4. Budget for its lifecycle - upgrades, CVEs, on-call ownership - before committing.

EVIDENCE YOU WOULD QUOTE

```
Inventory of in-house controllers with owner, last release and open issue count

oc get crd -l app.kubernetes.io/managed-by=<team>

Comparison of maintenance effort: GitOps application versus custom controller
```

RED FLAG - DO NOT SAY THIS

Do not write an operator to deploy static YAML. You have replaced a template with a codebase and an on-call rota.

EXPECT THIS FOLLOW-UP

A team wants an operator for their microservice deployment. How do you respond?

PART 10 · Q141-Q156 · 16 QUESTIONS

Installation, machines and updates

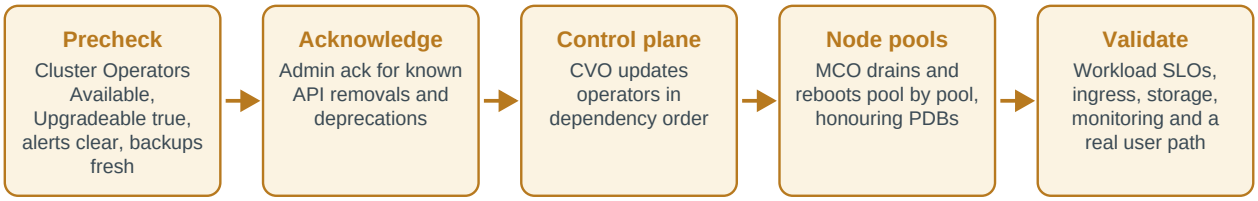
Bringing a cluster into existence, keeping its nodes honest, and moving versions without drama

Cluster lifecycle is where platform engineering earns its name. Anyone can install a cluster once with the defaults; the job is doing it repeatably, keeping every node identical to its declared configuration, and moving through versions on a schedule without turning each upgrade into an event. Interviewers probe this area hard because it is where the platform's own reliability lives - and because the words a candidate uses about upgrades, particularly around rollback, reveal immediately whether they have done one in production.

Level mix - Foundation: 4 · Intermediate: 5 · Senior: 5 · Architect: 2

A supported OpenShift upgrade, in order

Upgrades are the highest-risk routine change you own. The order is not negotiable and "rollback" is not a step that exists.



Q141 FOUNDATION

How do IPI and UPI installations differ?

SAY THIS

With installer-provisioned infrastructure the installer creates the infrastructure itself - networks, load balancers, machines, DNS on supported platforms - and the cluster then manages those machines through the Machine API, so scaling and replacement are native. With user-provisioned infrastructure you build the infrastructure and the installer only bootstraps the cluster onto it, which gives you full control and means node lifecycle is your responsibility.

THINK OF IT LIKE

IPI is buying a house from a developer who also lays the road and connects the utilities. UPI is being given the plans and doing the groundwork yourself.

WHY IT MATTERS IN PRODUCTION

The choice has a long tail well beyond installation day. With IPI you get MachineSets, autoscaling and automated node replacement for free. With UPI, adding a worker means provisioning a machine, booting it with the right Ignition config, and approving its certificate signing requests - so every capacity change is a runbook rather than a scale command. Many enterprises still choose UPI because their network and security teams own the infrastructure, and that is a legitimate reason as long as everyone understands the operational cost.

STEP BY STEP

1. State who creates the infrastructure in each model.
2. Explain the day-two consequence: Machine API integration versus manual node lifecycle.
3. Name the legitimate reasons to choose UPI - existing infrastructure ownership, unsupported platforms, strict network control.
4. Note the hybrid possibility: UPI control plane with machine-managed workers where the platform supports it.

EVIDENCE YOU WOULD QUOTE

```
oc get machineset -A ; oc get machine -A -o wide
```

```
oc get infrastructure cluster -o jsonpath='{.status.platform}{"\n"}'

oc get csr | grep -i pending # a UPI signature during node addition
```

RED FLAG - DO NOT SAY THIS

Do not claim UPI is simply harder. It is a deliberate trade of automation for control, and plenty of organisations make it for good reasons.

EXPECT THIS FOLLOW-UP

On a UPI cluster, how do you add a worker node? Walk me through it.

Q142 FOUNDATION

Which DNS records are critical for an OpenShift cluster?

SAY THIS

Three groups. The API endpoint, api.<cluster>.<domain>, which clients and nodes use to reach the control plane. The internal API endpoint, api-int, which the nodes themselves use and which must resolve inside the cluster network. And the applications wildcard, *.apps.<cluster>.<domain>, which resolves to the ingress load balancer and is how every Route is reachable. On UPI you also need forward and reverse records for the nodes.

THINK OF IT LIKE

They are the building's street address, the internal extension list, and the sign on the public entrance. Lose any one and a different set of people cannot find you.

WHY IT MATTERS IN PRODUCTION

DNS is the single most common cause of a failed installation and of a cluster that comes back wrong after a restart. The distinction between api and api-int matters more than it looks: they can resolve to different addresses, and a node that cannot resolve api-int cannot join or stay healthy even though administrators using api see a working cluster. Wildcard problems are equally distinctive - the console and every application fail while oc works perfectly.

STEP BY STEP

1. List the three records and who depends on each.
2. Explain why api and api-int can differ and what breaks when api-int is wrong.
3. Describe the wildcard symptom: oc works, every application and the console do not.
4. Verify resolution from a node, not from your workstation, because that is the view that matters.

EVIDENCE YOU WOULD QUOTE

```
oc debug node/<node> -- chroot /host dig +short api-int.<cluster>.<domain>

dig +short console-openshift-console.apps.<cluster>.<domain>

oc get co dns ingress authentication
```

RED FLAG - DO NOT SAY THIS

Do not verify DNS only from your laptop. The record that breaks clusters is the one the nodes resolve, not the one you do.

EXPECT THIS FOLLOW-UP

oc works fine but the console is unreachable. Which record do you check first?

Q143 FOUNDATION

What load-balancer functions does an OpenShift cluster require?

SAY THIS

Two distinct roles. An API load balancer in front of the control plane nodes, handling port 6443 for the Kubernetes API and 22623 for the machine config server, which must be reachable by nodes but not exposed externally. And an ingress load balancer in front of the router nodes for ports 80 and 443, which is what the applications wildcard points at. Both need health checks so that a failed backend is removed rather than continuing to receive traffic.

THINK OF IT LIKE

One reception desk for staff and deliveries, another for the public. Same building, different queues, and each needs to notice when a lift is out of service.

WHY IT MATTERS IN PRODUCTION

The port that gets forgotten is 22623, the machine config server, and forgetting it produces a very specific failure: existing nodes work, but new nodes cannot bootstrap because they cannot fetch their Ignition configuration. The other classic is a health check configured as a plain TCP connect rather than a real endpoint check, so the load balancer happily keeps sending traffic to a control plane node whose API server is failing.

STEP BY STEP

1. Separate the API and ingress roles and list the ports each carries.
2. Stress that 22623 is required for node bootstrap and must not be publicly exposed.
3. Insist on real health checks rather than TCP connect, so failures are detected.
4. Verify from a node and from outside, because the two paths can differ.

EVIDENCE YOU WOULD QUOTE

```
oc debug node/<node> -- chroot /host curl -sk https://api-int.<cluster>.<domain>:22623/healthz
-o /dev/null -w '%{http_code}\n'

curl -sk https://api.<cluster>.<domain>:6443/healthz

oc get co ingress -o jsonpath='{.status.conditions}' | python3 -m json.tool
```

RED FLAG - DO NOT SAY THIS

Do not expose port 22623 to the internet. It serves Ignition configuration, and that is not a public document.

EXPECT THIS FOLLOW-UP

New nodes fail to bootstrap while existing ones are healthy. What do you suspect?

Q144 FOUNDATION

What are Machines and MachineSets?

SAY THIS

A Machine represents one host that the cluster manages - a cloud instance or a bare-metal server - and its lifecycle is owned by the Machine API. A MachineSet is the template and replica count for a group of identical Machines, so scaling workers means changing a MachineSet's replica count and the controller provisions or removes machines accordingly. It is the same declarative pattern as a ReplicaSet, applied to infrastructure.

THINK OF IT LIKE

A MachineSet is a standing order for staff of a given role: three of these, with this specification. Individual Machines are the people who arrive to fill it.

WHY IT MATTERS IN PRODUCTION

Because MachineSets are per failure domain on most platforms, you typically have one per zone, and that is what makes zone-aware scaling and autoscaling work. It is also why scaling should always be done by editing the MachineSet rather than by touching the infrastructure directly - a cloud instance created outside the Machine API is invisible to the cluster's lifecycle management and becomes a snowflake immediately.

STEP BY STEP

1. Define both objects and the ReplicaSet analogy for infrastructure.
2. Explain the per-zone MachineSet pattern and why it enables balanced scaling.
3. State the rule: scale by editing the MachineSet, never by creating instances manually.
4. Mention MachineAutoscaler as the object that lets the cluster autoscaler drive them.

EVIDENCE YOU WOULD QUOTE

```
oc get machineset -n openshift-machine-api -o wide

oc scale machineset <name> -n openshift-machine-api --replicas=5

oc get machine -n openshift-machine-api -o wide | grep -v Running
```

RED FLAG - DO NOT SAY THIS

Do not create instances outside the Machine API on an IPI cluster. They will not be managed, replaced or upgraded with everything else.

EXPECT THIS FOLLOW-UP

A Machine is stuck in Provisioning. Where do you look?

Q145 INTERMEDIATE

Explain the OpenShift bootstrap process.

SAY THIS

A temporary bootstrap machine starts first and runs a minimal control plane. The permanent control plane nodes boot, fetch their Ignition configuration from the bootstrap machine, form the etcd cluster and start the real control plane. Once the permanent control plane is serving and the bootstrap machine's job is done, it is removed and the cluster continues on its own, with the Cluster Version Operator installing the rest of the payload.

THINK OF IT LIKE

It is the scaffolding on a building. Essential while the structure goes up, deliberately temporary, and its removal is a milestone rather than a loss.

WHY IT MATTERS IN PRODUCTION

Understanding this explains most installation failures. If the control plane nodes cannot reach the bootstrap machine on port 22623, they never get their configuration and the install hangs with nothing obviously wrong. If DNS for api-int is incorrect, etcd members cannot find each other. The bootstrap machine's journal is the single most useful artefact when an installation fails, and gathering it before tearing the environment down is the difference between a diagnosis and a repeat.

STEP BY STEP

1. Describe the three phases: bootstrap control plane, permanent control plane forms, bootstrap removed.
2. Name the dependencies at each phase - DNS, load balancer, port 22623, image access.
3. Explain how to gather bootstrap logs before the environment is destroyed.
4. Note that the CVO takes over once the permanent control plane is serving.

EVIDENCE YOU WOULD QUOTE

```
openshift-install wait-for bootstrap-complete --log-level=debug

openshift-install gather bootstrap --bootstrap <ip> --master <ip>

oc get co ; oc get clusterversion
```

RED FLAG - DO NOT SAY THIS

Do not destroy a failed installation before gathering the bootstrap logs. You are throwing away the only evidence of why it failed.

EXPECT THIS FOLLOW-UP

The install hangs waiting for the bootstrap to complete. What are your first checks?

Q146 INTERMEDIATE

What is a CSR, and what do you do about a node waiting for approval?

SAY THIS

A CertificateSigningRequest is how a node asks the cluster to issue it a certificate - first to join, then again for its serving certificate, and periodically on renewal. On IPI clusters the machine approver handles them automatically for machines it recognises. On UPI clusters, or when the approver cannot correlate the request to a known machine, they sit Pending and the node stays NotReady until an administrator approves them.

THINK OF IT LIKE

It is a new starter waiting at reception for their pass to be printed. They are on the system, they just cannot get through the barrier yet.

WHY IT MATTERS IN PRODUCTION

The important discipline is verifying before approving. A pending CSR is a request for a cluster credential, so approving everything in sight is a genuine security risk - you should know which node you are expecting and that the request matches it. The other thing to watch is a cluster that has been powered off long enough for certificates to expire: on restart there can be a burst of CSRs, and nodes will not become Ready until they are handled.

STEP BY STEP

1. Explain the two CSR types - client for joining, serving for the kubelet endpoint.

2 Describe automatic approval on IPI and why UPI often needs manual handling.
3 Verify the requesting identity and the expected node before approving anything.
4 After approval, confirm the node reaches Ready and check for a second wave of serving CSRs.

EVIDENCE YOU WOULD QUOTE

```
oc get csr -o wide | grep -i pending

oc adm certificate approve <csr>

oc get nodes -o wide ; oc describe csr <csr> | head -20
```

RED FLAG - DO NOT SAY THIS

Do not blanket-approve every pending CSR without checking. You are issuing cluster credentials to whoever asked.

EXPECT THIS FOLLOW-UP

After a long shutdown, dozens of CSRs are pending. What is your sequence?

Q147 INTERMEDIATE

What does MachineHealthCheck do?

SAY THIS

A MachineHealthCheck watches nodes matching a selector and, when a node has been unhealthy for longer than a configured timeout, deletes its Machine so the MachineSet provisions a replacement. On infrastructure that supports it, destroying the machine also provides fencing, which is what makes it safe to reattach storage elsewhere. A maxUnhealthy threshold stops it acting during a large-scale event.

THINK OF IT LIKE

It is a rule that replaces a broken machine on the production line automatically - but stops itself if half the factory is down, because that is a different problem.

WHY IT MATTERS IN PRODUCTION

The maxUnhealthy setting is the part that matters most and the part people forget. Without it, a network partition that makes twenty nodes appear unhealthy results in twenty machines being destroyed and recreated, turning a transient event into a genuine outage. Set it conservatively, and set the unhealthy timeout longer than your longest expected transient - a node that is briefly busy is not a node that should be destroyed.

STEP BY STEP

1 Define the watch, the timeout and the remediation - delete the Machine, let the set replace it.
2 Explain the fencing benefit for storage reattachment.
3 Set maxUnhealthy so a mass event does not trigger mass remediation.
4 Tune the unhealthy timeout above realistic transient conditions and monitor remediation events.

EVIDENCE YOU WOULD QUOTE

```
oc get machinehealthcheck -n openshift-machine-api -o wide

oc get machinehealthcheck <name> -n openshift-machine-api -o
jsonpath='{.spec.maxUnhealthy}{"\n"}'

oc get events -n openshift-machine-api --field-selector reason=MachineDeleted
```

RED FLAG - DO NOT SAY THIS

Do not deploy MachineHealthCheck without maxUnhealthy. A network blip becomes a fleet rebuild.

EXPECT THIS FOLLOW-UP

A network partition makes ten nodes look unhealthy. What should happen, and why?

Q148 INTERMEDIATE

How do you apply a custom kernel argument or kubelet setting to a subset of nodes?

SAY THIS

By creating a custom MachineConfigPool selected by a node label, then applying a MachineConfig or KubeletConfig targeted at that pool. The MCO renders the merged configuration for the pool and the Machine Config Daemon rolls it out node by node, draining and rebooting each one. That gives you a declarative, auditable change that survives reboots and applies automatically to any new node that joins the pool.

THINK OF IT LIKE

It is writing a building specification for one floor rather than sending an electrician round to rewire each office individually.

WHY IT MATTERS IN PRODUCTION

Creating a separate pool is the part that makes this safe. Applying a kernel argument to the default worker pool means every worker reboots, whereas a dedicated pool limits the blast radius to the nodes that actually need the change and lets you validate on one node first. It also matters for upgrades, because pools update independently and you can pause one while investigating a problem without stalling the rest.

STEP BY STEP

1. Label the target nodes and create a MachineConfigPool selecting that label.
2. Apply the MachineConfig or KubeletConfig targeted at the new pool.
3. Watch the pool roll out with a low maxUnavailable, validating the first node before continuing.
4. Confirm the node's current config annotation matches the new rendered config, and record the change.

EVIDENCE YOU WOULD QUOTE

```
oc get mcp -o wide ; oc get mc | tail

oc get node <node> -o
jsonpath='{.metadata.annotations.machineconfiguration\.openshift\.io/currentConfig}{"\n"}'

oc debug node/<node> -- chroot /host cat /proc/cmdline
```

RED FLAG - DO NOT SAY THIS

Do not apply node changes to the default worker pool for a subset requirement. You will reboot every worker in the cluster to configure four of them.

EXPECT THIS FOLLOW-UP

The pool is Degraded after your change. What do you do first?

Q149 INTERMEDIATE

What do you check before starting an OpenShift update?

SAY THIS

Every Cluster Operator Available and none Degraded, the ClusterVersion reporting Upgradeable true with any admin acknowledgements handled, no critical alerts firing, MachineConfigPools all updated and none paused, a fresh and verified etcd backup, adequate capacity to drain nodes without evictions, PDBs that will actually permit drains, and operator channels compatible with the target version. Then a communicated window and a written validation plan.

THINK OF IT LIKE

It is a pre-flight checklist. Nothing on it is clever, and skipping items is exactly how routine flights become incidents.

WHY IT MATTERS IN PRODUCTION

The two items that most often cause a stall are unrelated to the platform itself: a PDB that cannot be satisfied, so a node will not drain, and insufficient capacity, so evicted pods cannot be placed and the drain hangs. Both are discoverable in advance with a dry-run drain, and both are far cheaper to fix the day before than mid-upgrade with a change window burning.

STEP BY STEP

1. Verify cluster health: Cluster Operators, ClusterVersion Upgradeable, alerts, pool status.
2. Verify recoverability: a recent etcd backup you have actually validated.
3. Verify capacity and drainability with a server-side dry-run drain on a representative node.
4. Verify compatibility - operator channels and any deprecated API usage - then communicate the window and the validation plan.

EVIDENCE YOU WOULD QUOTE

```
oc adm upgrade ; oc get co | grep -v 'True.*False.*False'

oc adm drain <node> --dry-run=server --ignore-daemonsets --delete-emptydir-data

oc get mcp ; oc get pdb -A -o wide
```

RED FLAG - DO NOT SAY THIS

Do not start an upgrade with a Degraded Cluster Operator. You are adding a change to a system that is already telling you something is wrong.

EXPECT THIS FOLLOW-UP

Upgradeable is false with a message about a deprecated API. What do you do?

Q150 SENIOR

An update has stalled. How do you investigate?

SAY THIS

I read the ClusterVersion status, which names the operator the CVO is waiting on, then read that operator's conditions, which name the reason. From there it is usually one of a small set: a node that will not drain because of a PDB or a pod with no controller, a MachineConfigPool that is Degraded, a Cluster Operator blocked on an unavailable dependency such as storage or DNS, or an image that cannot be pulled in a disconnected cluster.

THINK OF IT LIKE

The departure board says which flight is delayed and why. You go to that gate rather than walking the whole terminal.

WHY IT MATTERS IN PRODUCTION

The failure of nerve here is skipping ahead to drastic actions - force-deleting pods, pausing pools at random, restarting operators. The upgrade is a controlled process and it will resume once the blocker is cleared, so the job is to identify and clear that one blocker. It is also worth saying out loud that an upgrade being slow is not the same as an upgrade being stuck; MCO rollout across a large pool takes hours by design.

STEP BY STEP

1. Read ClusterVersion conditions to find which component the CVO is waiting on.
2. Read that component's conditions and take the message literally.
3. If it is node-related, find the node and the specific pod or PDB blocking the drain.
4. Clear the single blocker with the smallest safe action, confirm progress resumes, and capture must-gather if support may be needed.

EVIDENCE YOU WOULD QUOTE

```
oc get clusterversion version -o jsonpath='{.status.conditions}' | python3 -m json.tool

oc get co | grep -v 'True.*False.*False' ; oc get mcp -o wide

oc get pods -A --field-selector spec.nodeName=<node> --sort-by=.metadata.name
```

RED FLAG - DO NOT SAY THIS

Do not force-delete pods to speed up a drain without understanding what they are. That is how a stalled upgrade becomes a data-loss incident.

EXPECT THIS FOLLOW-UP

The MachineConfigPool says Degraded with one node stuck. Walk me through it.

Q151 SENIOR

How do you describe rollback of an OpenShift update?

SAY THIS

I do not describe it as rollback, because a general downgrade is not supported. Once the control plane has moved, CRDs and stored object versions have moved with it, and going backwards is not a supported path. In limited cases a partially updated cluster can be returned to the previous minor version under guidance, but the real answer is that recovery is forward: fix the blocker and complete the update, or restore from backup if the cluster is genuinely unrecoverable.

THINK OF IT LIKE

It is a bridge you drive across, not a lift you can send back down. The plan has to be about not getting stranded halfway, because reversing is not on the menu.

WHY IT MATTERS IN PRODUCTION

This is a deliberate trap question, and answering "I would roll back" is a strong signal that you have never done a production OpenShift upgrade. The credible answer talks about the things that make forward recovery reliable: staged upgrades through non-production first, a verified etcd backup, capacity headroom, and a validation plan that catches problems while the control plane is still on the old version.

STEP BY STEP

1. State plainly that downgrade is not a supported general operation, and say why.

2 Describe forward recovery: identify the blocker, fix it, complete the update.
3 Describe the catastrophic path: restore from a verified etcd backup, with its data-loss implications.
4 Emphasise prevention - staged environments, backups, capacity, and a validation plan.

EVIDENCE YOU WOULD QUOTE

```
oc get clusterversion version -o jsonpath='{.status.history}' | python3 -m json.tool

oc adm upgrade # available targets from the current version

Verified etcd backup timestamp and location
```

RED FLAG - DO NOT SAY THIS

Do not say you would roll back the cluster. It is the fastest way to reveal you have not run an upgrade in anger.

EXPECT THIS FOLLOW-UP

Halfway through an upgrade a critical application breaks. What are your options?

Q152 SENIOR

How do you replace a failed control-plane node safely?

SAY THIS

Carefully and in order, because quorum is at stake. I confirm the remaining members are healthy and that quorum exists, remove the failed member from the etcd cluster so it is not counted, remove its secrets and the Machine object, then provision a replacement and let it join. Throughout, I make sure I am never taking a second member out, and I verify etcd health at each step rather than at the end.

THINK OF IT LIKE

It is replacing one leg of a three-legged stool while someone is sitting on it. The order matters enormously, and you never lift two legs to save time.

WHY IT MATTERS IN PRODUCTION

The reason this is a senior question is that the tempting shortcuts are the dangerous ones. Provisioning the replacement before removing the failed member leaves a stale member counted in quorum arithmetic. Doing maintenance on another control plane node at the same time takes you below quorum. And skipping the etcd health verification between steps means you discover a problem two steps later, when the recovery is much harder.

STEP BY STEP

1 Verify quorum and the health of the remaining etcd members before touching anything.
2 Remove the failed member from etcd and delete its associated secrets.
3 Delete the Machine object so a replacement is provisioned, or provision manually on UPI.
4 Watch the new member join, verify etcd health and Cluster Operator status, and only then consider other maintenance.

EVIDENCE YOU WOULD QUOTE

```
oc -n openshift-etcd rsh <etcd-pod> etcdctl member list -w table

oc -n openshift-etcd rsh <etcd-pod> etcdctl endpoint health --cluster

oc get machine -n openshift-machine-api -l machine.openshift.io/cluster-api-machine-role=master
```

RED FLAG - DO NOT SAY THIS

Do not add the replacement before removing the failed member. You will be reasoning about quorum with a member that no longer exists.

EXPECT THIS FOLLOW-UP

Two of three control plane nodes are down. Does your procedure still apply?

Q153 SENIOR

When would you pause a MachineConfigPool?

SAY THIS

To hold node changes for a specific, time-boxed reason - a business freeze, an investigation into a suspected bad configuration, or coordinating a large change across pools. It is a temporary control, not a configuration choice. While a pool is paused, configuration changes queue rather than apply, so certificate rotation and security fixes for those nodes are also delayed, and an upgrade will not complete.

THINK OF IT LIKE

It is holding the lift on a floor. Fine for thirty seconds while you load something; not fine as a way of running the building.

WHY IT MATTERS IN PRODUCTION

The specific danger is forgetting. A pool paused during an incident and never unpaused quietly accumulates pending changes, and the first sign is often an upgrade that will not finish or nodes whose certificates are about to expire. Treat pausing like a change: it needs a ticket, an owner, an expected duration and an alert if it stays paused beyond that.

STEP BY STEP

1. Give a specific reason and an expected duration before pausing anything.
2. Record it as a change with an owner, and alert on pools paused beyond the expected window.
3. Understand what is being delayed - configuration, certificate rotation, upgrade progress.
4. Unpause deliberately and watch the queued changes roll out with a low maxUnavailable.

EVIDENCE YOU WOULD QUOTE

```
oc get mcp -o custom-columns=NAME:.metadata.name,PAUSED:.spec.paused,UPDATED:.status.conditions
[?(@.type=="Updated")].status

oc patch mcp <pool> --type merge -p '{"spec":{"paused":false}}'

oc get mcp <pool> -o jsonpath='{.status.conditions}' | python3 -m json.tool
```

RED FLAG - DO NOT SAY THIS

Do not leave a pool paused indefinitely. You are silently deferring security updates and certificate rotation.

EXPECT THIS FOLLOW-UP

You find a pool that has been paused for three months. What are the risks?

Q154 SENIOR

How do you detect and remediate node configuration drift?

SAY THIS

The MCO already detects it: the Machine Config Daemon compares the node's actual configuration against the rendered config it should have and marks the node and pool Degraded when they differ. So detection is mostly about noticing - alerting on Degraded pools and on nodes whose current config annotation does not match desired. Remediation is to remove whatever made the manual change, let the MCO reapply, and if the node cannot be reconciled, replace it.

THINK OF IT LIKE

It is a building inspector who checks each floor against the plans. Finding the discrepancy is automatic; the work is in stopping whoever keeps moving the walls.

WHY IT MATTERS IN PRODUCTION

Drift almost always has a human cause: someone SSHed to a node during an incident and changed something to get through the night. The technical remediation is easy; the durable fix is making the legitimate version of that change available as a MachineConfig so nobody needs to do it by hand next time, and removing standing SSH access so it is not the path of least resistance.

STEP BY STEP

1. Alert on Degraded MachineConfigPools and on config annotation mismatches per node.
2. Identify what changed and why - the MCD logs usually name the file or unit.
3. Reconcile by reapplying the rendered config, or replace the node if it will not converge.
4. Close the loop: make the legitimate change declarative and reduce the access that allowed the drift.

EVIDENCE YOU WOULD QUOTE

```
oc get mcp -o wide ; oc get nodes -o custom-columns=NAME:.metadata.name,CURRENT:.metadata.annot
ations.machineconfiguration\.openshift\.io/currentConfig,DESIRED:.metadata.annotations.machinec
onfiguration\.openshift\.io/desiredConfig

oc -n openshift-machine-config-operator logs ds/machine-config-daemon -c machine-config-daemon
--tail=200

oc debug node/<node> -- chroot /host rpm-ostree status
```

RED FLAG - DO NOT SAY THIS

Do not treat drift as a purely technical fix. If the process that caused it stays the same, the drift comes back.

EXPECT THIS FOLLOW-UP

A node is Degraded because someone edited a file during an incident. What now?

Q155 ARCHITECT

How would you design a version lifecycle strategy for your clusters?

SAY THIS

I would pick a supported cadence and hold it, rather than upgrading reactively. That means deciding whether the estate follows the regular stream or Extended Update Support releases, defining an environment order - development, then test, then a canary production cluster, then the rest - with a soak period at each stage, and setting a rule that no cluster falls more than a stated number of versions behind. Then publishing the calendar so application teams can plan around it.

THINK OF IT LIKE

It is a service schedule for a fleet of vehicles. Predictable, staggered, and the point is that nothing is ever so far behind that servicing becomes a rebuild.

WHY IT MATTERS IN PRODUCTION

The failure mode this prevents is the death spiral: upgrades feel risky, so they are deferred, so the eventual upgrade spans several versions and genuinely is risky, which confirms the fear. EUS releases exist precisely for organisations that cannot absorb frequent change, and choosing them deliberately is far better than drifting into being years behind. The publishing part matters too - teams cannot plan around a calendar they have never seen.

STEP BY STEP

1. Choose the stream - regular or EUS - based on the organisation's real change appetite.
2. Define the environment order and soak period, and a maximum acceptable version lag.
3. Automate the pre-upgrade checks so each cycle costs less than the last.
4. Publish the calendar, review after each cycle, and track version lag as a standing metric.

EVIDENCE YOU WOULD QUOTE

```
Version inventory across the estate with age and target version

oc get clusterversion -o jsonpath='{.status.desired.version}{"\n"}' per cluster

Upgrade cycle metrics: duration, incidents, rollback-free completion rate
```

RED FLAG - DO NOT SAY THIS

Do not let clusters drift several versions behind because upgrades feel risky. The risk grows faster than the delay.

EXPECT THIS FOLLOW-UP

Half the estate is two versions behind. What is your recovery plan?

Q156 ARCHITECT

How would you design node pools for a production cluster?

SAY THIS

I separate by role and by failure domain. Control plane on its own dedicated nodes, sized for etcd's disk latency needs. Infrastructure nodes for routers, monitoring, logging and the registry, so platform components do not compete with applications and so licensing is predictable. Then application pools per workload class - general purpose, memory-heavy, GPU or latency-sensitive - each as its own MachineSet per zone with taints and labels that make placement explicit.

THINK OF IT LIKE

It is zoning a building: plant rooms, offices and the loading bay each get space suited to them, rather than everyone sharing one open floor and competing for the lift.

WHY IT MATTERS IN PRODUCTION

Infrastructure nodes are the design decision with the clearest payoff. Routers and the monitoring stack are latency-sensitive and resource-hungry, and leaving them to compete with application pods produces exactly the intermittent problems that are hardest to diagnose. Separating pools also gives you independent MachineConfigPools, so a kernel change or an upgrade can be validated on one class of node without touching the rest.

STEP BY STEP

1. Separate control plane, infrastructure and application roles onto distinct pools.
2. Create one MachineSet per pool per failure domain so scaling stays balanced.
3. Use taints, tolerations and labels so placement is explicit rather than accidental.
4. Size each pool for its workload profile and validate with real utilisation data, revisiting quarterly.

EVIDENCE YOU WOULD QUOTE

```
oc get nodes -L node-role.kubernetes.io/infra -L topology.kubernetes.io/zone

oc get machineset -n openshift-machine-api -o wide

oc get mcp ; oc describe node <infra-node> | sed -n '/Taints/,/Capacity/p'
```

RED FLAG - DO NOT SAY THIS

Do not run routers and monitoring on the same nodes as applications in a production cluster. The contention shows up as latency nobody can attribute.

EXPECT THIS FOLLOW-UP

How would you move the router and monitoring workloads onto infra nodes with no downtime?

PART 11 · Q157-Q172 · 16 QUESTIONS

Observability and reliability engineering

Knowing what is happening, proving it, and being woken up only when it matters

Observability questions separate people who install monitoring from people who use it. The first group can name the components; the second can tell you which signal answers which question, why their alert count went down after they improved reliability, and what they deleted from the dashboard because nobody ever acted on it. Interviewers are listening for a connection between technical measurement and user impact - if every answer stops at CPU graphs, the conversation never reaches the senior half of the scorecard.

Level mix - Foundation: 5 · Intermediate: 5 · Senior: 4 · Architect: 2

Which signal answers which question

Senior answers move between signals deliberately. Junior answers stare at CPU graphs.

Signal	Answers	Where it lives	Failure mode
Metrics	Is it happening now, and how much?	Prometheus, ServiceMonitor, PodMonitor	Cardinality explosion kills the stack that was meant to warn you
Logs	What exactly happened in this request?	Cluster Logging, forwarded to an external store	Unbounded retention on cluster storage; no correlation ID
Events	What did the platform decide, and when?	oc get events, namespaced and short-lived	Default retention is about three hours - capture them early
Traces	Where did the latency go across services?	Distributed tracing, sampled	Sampling too low to catch the rare slow path
Conditions	What does the owning controller think?	status.conditions on every object	Ignored, because pod status looked green

Q157 FOUNDATION

Explain the OpenShift monitoring architecture.

SAY THIS

The platform monitoring stack is managed by the Cluster Monitoring Operator. Prometheus instances scrape metrics from the control plane, nodes through node-exporter, and cluster components; kube-state-metrics turns API objects into metrics; Alertmanager handles routing, grouping and silencing of alerts; and Thanos provides query aggregation and, where configured, longer retention. There is a separate stack for user workload monitoring so application metrics do not interfere with platform ones.

THINK OF IT LIKE

It is a building management system. Sensors everywhere, one place that collects readings, another that decides who gets phoned, and a records room for the history.

WHY IT MATTERS IN PRODUCTION

The separation between platform and user-workload monitoring is the detail worth knowing, because it is the answer to a very common question: how do application teams get their own metrics and alerts without being given access to the platform stack. It also explains why the platform Prometheus is not configurable in arbitrary ways - it is operator-managed, and changes go through the CMO's configuration rather than by editing its objects.

STEP BY STEP

1. Name the components and what each is responsible for.
2. Explain the platform versus user-workload split and why it exists.
3. Say that the stack is operator-managed, so configuration goes through the CMO ConfigMap.
4. Point out retention as the design decision: local Prometheus retention is short by default.

EVIDENCE YOU WOULD QUOTE

```
oc -n openshift-monitoring get pods

oc -n openshift-monitoring get cm cluster-monitoring-config -o yaml

oc get co monitoring -o jsonpath='{.status.conditions}' | python3 -m json.tool
```

RED FLAG - DO NOT SAY THIS

Do not offer to edit the Prometheus object directly. It is operator-managed and your change will be reverted.

EXPECT THIS FOLLOW-UP

How do application teams get their own alerts without access to platform monitoring?

Q158 FOUNDATION

What is a ServiceMonitor, and when would you use a PodMonitor instead?

SAY THIS

A ServiceMonitor tells Prometheus to scrape the endpoints behind a Service, selected by labels, on a named port and path. It is the normal choice because it follows the Service abstraction and picks up new pods automatically. A PodMonitor scrapes pods directly by label, which you need when the pods are not behind a Service at all, or when you need to scrape a port the Service does not expose.

THINK OF IT LIKE

A ServiceMonitor is delivering to the department; a PodMonitor is delivering to named individuals. The first is usually right, and occasionally the person is not in a department.

WHY IT MATTERS IN PRODUCTION

The overwhelmingly common failure with both is that nothing gets scraped and no error appears anywhere. The causes are always the same short list: label selector does not match, the port is referenced by the wrong name, the namespace is not in the monitor's scope, or user-workload monitoring is not enabled. Checking Prometheus's target list directly is the fastest way to see which of those it is.

STEP BY STEP

1. Define both and state the default preference for ServiceMonitor.
2. Give the cases that require a PodMonitor - no Service, or a port not exposed by one.
3. Name the four usual causes of silent scrape failure.
4. Verify in Prometheus's targets view rather than assuming the object is enough.

EVIDENCE YOU WOULD QUOTE

```
oc get servicemonitor,podmonitor -A

oc -n openshift-user-workload-monitoring get pods

Prometheus UI: Status, Targets - filter by job and read the last scrape error
```

RED FLAG - DO NOT SAY THIS

Do not assume creating a ServiceMonitor means metrics are being collected. Check the target list; silent failure is the norm.

EXPECT THIS FOLLOW-UP

Your ServiceMonitor exists and no metrics appear. What are your four checks?

Q159 FOUNDATION

What is a PrometheusRule?

SAY THIS

A PrometheusRule holds two kinds of rules. Alerting rules define a PromQL expression, a duration it must hold true for, labels such as severity, and annotations carrying the summary and runbook link. Recording rules precompute expensive expressions into new time series. Both are namespaced objects, so application teams can manage their own alerts declaratively in Git alongside the workload.

THINK OF IT LIKE

It is the standing instruction to the night porter: if this condition holds for this long, phone this person, and here is what to tell them.

WHY IT MATTERS IN PRODUCTION

The annotations are the part that determines whether the alert is useful at 3am. An alert that says "HighMemoryUsage" with no summary, no impact statement and no runbook link produces a phone call and then twenty minutes of orientation. The same alert with a one-line description of user impact and a link to the runbook produces action. Treating annotations as required fields rather than optional decoration is a small policy with a large effect.

STEP BY STEP

1. Define alerting and recording rules and the fields each needs.
2. Insist on severity, a summary describing user impact, and a runbook link as mandatory.
3. Explain the for duration as the noise filter it is.
4. Keep rules in Git with the workload so they are reviewed like code.

EVIDENCE YOU WOULD QUOTE

```
oc get prometheusrule -A

oc get prometheusrule <name> -n <ns> -o yaml | head -40

Prometheus UI: Alerts - check for rules firing without annotations
```

RED FLAG - DO NOT SAY THIS

Do not ship alerts without a runbook link. You are scheduling somebody else's confusion for the middle of the night.

EXPECT THIS FOLLOW-UP

What fields would you make mandatory on every alert in your organisation?

Q160 FOUNDATION

What does Alertmanager do?

SAY THIS

Alertmanager takes alerts that Prometheus has fired and decides what happens to them: grouping related alerts into one notification, routing by label to the right team and channel, inhibiting lower-priority alerts when a higher-priority one covers the same problem, silencing during maintenance, and deduplicating across replicas. Prometheus decides what is true; Alertmanager decides who finds out and how.

THINK OF IT LIKE

It is the switchboard. The smoke detector decides there is smoke; the switchboard decides whether that means one phone call to the duty manager or fifty calls to everybody.

WHY IT MATTERS IN PRODUCTION

Inhibition and grouping are what make a large cluster survivable during an incident. When a node fails, dozens of alerts fire - pods down, endpoints missing, probes failing - and without inhibition rules the on-call engineer gets buried in symptoms while the one alert that names the cause scrolls past. Configuring inhibition so a node-level alert suppresses its downstream symptoms is a high-value, low-effort improvement.

STEP BY STEP

1. Separate the roles: Prometheus evaluates, Alertmanager routes.
2. Name the five behaviours - group, route, inhibit, silence, deduplicate.
3. Explain inhibition with a concrete cause-and-symptom example.
4. Route by label to owning teams, and use silences for planned maintenance rather than disabling rules.

EVIDENCE YOU WOULD QUOTE

```
oc -n openshift-monitoring get secret alertmanager-main -o jsonpath='{.data.alertmanager\.yaml}'
| base64 -d | head -40

oc -n openshift-monitoring get pods -l alertmanager=main

Alertmanager UI: active alerts, grouped, with silences listed
```

RED FLAG - DO NOT SAY THIS

Do not disable an alert rule to stop noise during maintenance. Use a silence, so the rule comes back automatically.

EXPECT THIS FOLLOW-UP

A node fails and forty alerts fire. How do you make that one notification?

Q161 FOUNDATION

What are the four golden signals, and what does saturation mean?

SAY THIS

Latency, traffic, errors and saturation. Latency is how long requests take, and it should be measured at percentiles rather than averages. Traffic is demand - requests per second. Errors is the failure rate, including requests that succeed but too slowly to be useful. Saturation is how full the constrained resource is - the queue depth, the connection pool, the disk throughput - which is what tells you how close to falling over you are.

THINK OF IT LIKE

It is a motorway. Journey time, number of cars, crashes, and how close to capacity the lanes are. The last one is the only one that predicts the others.

WHY IT MATTERS IN PRODUCTION

Saturation is the signal most teams neglect and the only leading indicator among the four. Latency, traffic and errors tell you what is happening now; saturation tells you what will happen in twenty minutes. It is also the hardest to instrument because the constrained resource is application-specific - a thread pool, a connection limit, an IO queue - and generic CPU dashboards do not capture it.

STEP BY STEP

1. Name the four signals and what each answers.
2. Insist on percentiles for latency and explain why averages hide the problem.
3. Define saturation as the fill level of the actual constraint, and call it the leading indicator.
4. Identify the real constraint per service rather than defaulting to CPU.

EVIDENCE YOU WOULD QUOTE

```
histogram_quantile(0.99, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))

sum(rate(http_requests_total{code=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))

Queue depth, connection pool utilisation or IO wait for the specific service
```

RED FLAG - DO NOT SAY THIS

Do not present average latency as your reliability metric. The average is fine while the worst 1% of users are having an outage.

EXPECT THIS FOLLOW-UP

What is the saturation signal for a service backed by a connection pool?

Q162 INTERMEDIATE

How do you set up monitoring for application workloads?

SAY THIS

I enable user-workload monitoring in the cluster monitoring configuration, which starts a separate Prometheus stack for application namespaces. Teams then create ServiceMonitors and PrometheusRules in their own namespaces, and they can query and alert on their own metrics without any access to platform monitoring. The platform team provides the dashboards, the alert routing and the guardrails on cardinality and retention.

THINK OF IT LIKE

It is giving each department its own filing cabinet in a shared records room. Their documents, their keys, the building's fire rules.

WHY IT MATTERS IN PRODUCTION

The guardrails matter as much as the enablement. Without limits, one team's high-cardinality metric can consume the memory of the shared user-workload Prometheus and take monitoring down for everyone. So the platform offer should include a documented cardinality budget, retention limits, and a review of new metrics as part of the golden path - not just a feature flag and good luck.

STEP BY STEP

1. Enable user-workload monitoring through the cluster monitoring ConfigMap.
2. Publish how teams create ServiceMonitors and PrometheusRules in their namespaces.

3 Set resource limits, retention and a documented cardinality budget for the stack.
4 Provide default dashboards and alert routing so teams start from something that works.

EVIDENCE YOU WOULD QUOTE

```
oc -n openshift-monitoring get cm cluster-monitoring-config -o jsonpath='{.data.config\.yaml}'

oc -n openshift-user-workload-monitoring get pods

prometheus_tsdb_head_series # in the user-workload stack
```

RED FLAG - DO NOT SAY THIS

Do not enable user-workload monitoring without limits. One team's label choice can take down monitoring for the whole cluster.

EXPECT THIS FOLLOW-UP

One namespace's metrics are consuming most of the stack's memory. What do you do?

Q163 INTERMEDIATE

Why is metrics cardinality dangerous?

SAY THIS

Every unique combination of label values creates a separate time series, and Prometheus holds those series in memory. Put a high-cardinality value in a label - a user ID, a request ID, a full URL path, a pod name on a workload that restarts constantly - and one metric becomes millions of series. The monitoring stack then consumes enormous memory, queries slow to a crawl, and eventually the thing that was supposed to warn you is the thing that is down.

THINK OF IT LIKE

It is filing one document per customer interaction instead of one per customer. The cabinet is technically correct and nobody can find anything, including the fire inspector.

WHY IT MATTERS IN PRODUCTION

The bitter irony is the failure mode: monitoring dies during the incident it was meant to observe, because the incident generated the churn that exploded the cardinality. Prevention is a review habit - challenge every new label, aggregate high-cardinality dimensions into buckets, and watch the head series count as a first-class metric with an alert on its growth rate rather than just its absolute value.

STEP BY STEP

1 Explain the series-per-label-combination model and where the memory goes.
2 Name the usual offenders: IDs, full paths, timestamps, unbounded user-supplied values.
3 Monitor head series count and alert on growth rate, not just a threshold.
4 Review new metrics before they ship, and bucket high-cardinality dimensions instead of labelling them.

EVIDENCE YOU WOULD QUOTE

```
prometheus_tsdb_head_series ; topk(10, count by (__name__)({__name__=~".+"}))

oc -n openshift-monitoring adm top pod

Prometheus UI: Status, TSDB Status - top series by metric name
```

RED FLAG - DO NOT SAY THIS

Do not put a request ID or a raw URL in a metric label. It is the single most reliable way to destroy a monitoring stack.

EXPECT THIS FOLLOW-UP

Head series doubled overnight. How do you find the cause in five minutes?

Q164 INTERMEDIATE

What is a recording rule and when do you need one?

SAY THIS

A recording rule evaluates an expression on a schedule and stores the result as a new time series. You need one when an expression is expensive and used repeatedly - a dashboard panel that aggregates across thousands of series, or an alert that would otherwise recompute a heavy query every evaluation cycle. It trades a small amount of storage for a large reduction in query cost and much more predictable dashboard load times.

THINK OF IT LIKE

It is doing the monthly totals once and writing them down, rather than adding up every receipt each time somebody asks.

WHY IT MATTERS IN PRODUCTION

The practical trigger is dashboards that take ten seconds to load or alerts that time out during evaluation, which is exactly when you need them most. Naming matters too - the convention of level:metric:operation makes recorded series discoverable, whereas arbitrarily named rules become a set of series nobody understands and nobody dares delete.

STEP BY STEP

1. Define the mechanism: scheduled evaluation stored as a new series.
2. Identify candidates - expensive expressions used by multiple dashboards or alerts.
3. Follow a naming convention so recorded series are discoverable.
4. Measure the improvement in query duration, and remove rules nothing consumes.

EVIDENCE YOU WOULD QUOTE

```
prometheus_rule_evaluation_duration_seconds by rule group

oc get prometheusrule -A -o yaml | grep -c 'record:'

Dashboard load time before and after
```

RED FLAG - DO NOT SAY THIS

Do not create recording rules for every query. Each one is a permanent cost, and most queries are not run often enough to justify it.

EXPECT THIS FOLLOW-UP

An alert times out during evaluation. Is a recording rule the right fix?

Q165 INTERMEDIATE

How do you design cluster logging?

SAY THIS

I start from the questions logs need to answer and the retention the organisation is legally required to keep, then work backwards. Collection is a per-node agent gathering container and node logs; forwarding sends them to an external store, because keeping long retention on cluster storage is expensive and couples log availability to cluster health. Applications should log structured JSON to stdout with a correlation ID, and never write logs to a persistent volume.

THINK OF IT LIKE

It is CCTV footage. Recording it is easy, storing years of it is the expensive part, and the value depends entirely on whether you can find the right three minutes.

WHY IT MATTERS IN PRODUCTION

The design decision with the biggest cost impact is retention tiering: short retention hot and searchable, longer retention in cheap object storage. The decision with the biggest usefulness impact is structure - unstructured multi-line stack traces are almost unsearchable, whereas structured logs with a correlation ID let you follow one request across services. Both are conventions you set in the golden path rather than negotiating per team.

STEP BY STEP

1. Define the questions and the compliance retention requirement first.
2. Collect at the node, forward off-cluster, and tier retention by cost and search need.
3. Standardise structured JSON to stdout with correlation IDs in the golden path.
4. Budget for volume, alert on ingestion rate, and review what is actually being queried.

EVIDENCE YOU WOULD QUOTE

```
oc get clusterlogforwarder,clusterlogging -n openshift-logging

oc -n openshift-logging get pods -o wide

Log ingestion rate and storage growth per namespace
```

RED FLAG - DO NOT SAY THIS

Do not keep long log retention on cluster storage. It is expensive and it disappears in exactly the incident where you need it.

EXPECT THIS FOLLOW-UP

Log volume tripled after a release. How do you find out which service and why?

Q166 INTERMEDIATE

What makes an alert actionable?

SAY THIS

It fires on user-visible impact or on a genuine leading indicator, not on a raw resource number. It has a clear owner and routes to them. It says what is broken and for whom in the summary. It links to a runbook with steps that work. And there is something the responder can actually do at the moment it fires. If an alert fails any of those, it is noise, and noise trains people to ignore the alerts that matter.

THINK OF IT LIKE

A smoke alarm that goes off when you make toast is not a safety device any more. It is the thing people take the battery out of.

WHY IT MATTERS IN PRODUCTION

The measurable version of this is alert-to-action ratio: what fraction of pages resulted in somebody doing something. Most organisations that measure it for the first time find numbers well under half, and the fix is deletion rather than tuning - remove the alerts nobody acts on, and convert the informational ones into dashboards or tickets. That single exercise usually improves on-call more than any tooling change.

STEP BY STEP

1. Alert on symptoms and impact; leave causes to dashboards unless they are leading indicators.
2. Require owner, severity, impact summary and runbook link on every alert.
3. Measure alert-to-action ratio and delete alerts that nobody acts on.
4. Review firing alerts after every incident and after every quiet week, in both directions.

EVIDENCE YOU WOULD QUOTE

```
ALERTS{alertstate="firing"} counts by alertname over 30 days

Paging history with disposition - actioned, acknowledged only, or auto-resolved

Alert definitions missing runbook_url annotations
```

RED FLAG - DO NOT SAY THIS

Do not defend an alert that has never been acted on because it "might matter one day". It is actively degrading the alerts that do.

EXPECT THIS FOLLOW-UP

Your team gets 200 pages a month and acts on 30. Where do you start?

Q167 SENIOR

Explain SLOs and error budgets.

SAY THIS

An SLI is a measurement of user experience - the proportion of requests served successfully and fast enough. An SLO is the target for that measurement over a window, say 99.9% over thirty days. The error budget is what is left over: 0.1% of requests may fail without breaking the promise. That budget turns reliability from an argument into arithmetic - if it is being consumed quickly, you pause risky changes; if it is barely touched, you can afford to move faster.

THINK OF IT LIKE

It is a monthly allowance for failure. Spend it slowly and you can take risks at the end of the month; blow it in week one and you are on a strict budget until it resets.

WHY IT MATTERS IN PRODUCTION

The reason this appears in senior interviews is that it changes conversations rather than dashboards. Without an SLO, "is this reliable enough?" is a matter of opinion and the loudest voice wins. With one, it is a number both sides can look at. The failure mode is setting SLOs from aspiration rather than measurement - a 99.99% target on a service whose dependencies cannot support it produces a permanently exhausted budget that everyone learns to ignore.

STEP BY STEP

1. Define SLI, SLO and error budget precisely, with a concrete example.

2 Derive the target from measured behaviour and user need, not from a round number.
3 Agree the policy in advance: what happens when the budget burns fast, and who decides.
4 Alert on burn rate rather than on instantaneous breaches, and review targets quarterly.

EVIDENCE YOU WOULD QUOTE

```
SLI query: successful and fast requests over total requests, over the SLO window

Error budget remaining and burn rate over 1h and 6h windows

Change freeze policy tied to budget exhaustion, agreed with the product owner
```

RED FLAG - DO NOT SAY THIS

Do not propose 99.99% because it sounds good. If the dependencies cannot deliver it, the SLO is decoration.

EXPECT THIS FOLLOW-UP

Your error budget is exhausted in week one. What actually changes?

Q168 SENIOR

How do you handle monitoring stack storage pressure?

SAY THIS

First I find out whether the growth is legitimate or a cardinality accident, because the responses are completely different. If a metric exploded, I fix the label at source. If growth is genuine, the levers are retention, scrape interval, dropping metrics nobody queries through relabelling, and moving long-term data into object storage through Thanos rather than expanding local volumes indefinitely.

THINK OF IT LIKE

The archive room is full. You could rent more space, or you could notice that half of it is duplicate paperwork nobody has ever asked for.

WHY IT MATTERS IN PRODUCTION

The urgent part is that Prometheus running out of disk means losing observability, and it usually happens during a period of high churn - which is to say, during an incident. So the real answer includes prevention: alerting on storage growth rate with enough lead time, and reviewing the top metrics by series count on a schedule so a bad label is caught in days rather than at capacity.

STEP BY STEP

1 Determine whether growth is cardinality-driven or volume-driven before acting.
2 Fix bad labels at source; use relabelling to drop metrics nothing queries.
3 Tune retention and scrape interval deliberately, and document the trade-off.
4 Move long-term retention to object storage, and alert on growth rate with lead time.

EVIDENCE YOU WOULD QUOTE

```
prometheus_tsdb_storage_blocks_bytes ; rate(prometheus_tsdb_head_series[1d])

topk(20, count by (__name__)({__name__=~".+"}))

oc -n openshift-monitoring get pvc
```

RED FLAG - DO NOT SAY THIS

Do not just expand the volume. If a bad label caused it, you have bought a week and the same incident.

EXPECT THIS FOLLOW-UP

Prometheus has two days of disk left. What do you do in the next hour?

Q169 SENIOR

When does distributed tracing earn its cost?

SAY THIS

When a request crosses enough services that latency cannot be attributed from metrics alone. Metrics tell you the service is slow; traces tell you which of the eleven downstream calls consumed the time and whether it was one slow dependency or a fan-out of many. In a monolith or a two-service system the cost of instrumenting and sampling is rarely worth it; past five or six services in a request path it becomes the only practical answer.

THINK OF IT LIKE

Metrics tell you the parcel arrived late. Tracing is the scan history showing it sat in one depot for nine hours.

WHY IT MATTERS IN PRODUCTION

The practical constraints are sampling and propagation. Sample too low and the rare slow request - the one you actually care about - is never captured, which is why tail-based sampling that keeps slow and errored traces is worth the extra complexity. Propagation is the harder organisational problem: a single service that does not forward trace headers breaks the chain for everyone downstream, so it has to be a platform-wide convention rather than a per-team choice.

STEP BY STEP

1. State the threshold: request paths deep enough that attribution from metrics fails.
2. Standardise context propagation across every service, as a platform convention.
3. Choose a sampling strategy that retains slow and errored traces rather than a flat percentage.
4. Connect traces to metrics and logs through shared identifiers so you can move between them.

EVIDENCE YOU WOULD QUOTE

```
Trace showing per-span duration across the request path

Sampling configuration and retained trace volume

Correlation ID present in logs, metrics exemplars and traces for the same request
```

RED FLAG - DO NOT SAY THIS

Do not sample at a flat 1% and expect to debug rare latency. The slow requests are exactly the ones you will miss.

EXPECT THIS FOLLOW-UP

A service is slow at p99 only. How do traces help where metrics did not?

Q170 SENIOR

How do you build an on-call practice people can sustain?

SAY THIS

Page only on user impact, and route everything else to tickets or dashboards. Give every page a runbook that has been tested by someone who did not write it. Track pages per shift and treat a rising number as a defect, not as a fact of life. Run blameless reviews for anything that woke someone, and make the follow-up action a real backlog item with an owner. And rotate fairly, with enough people that nobody is permanently on call.

THINK OF IT LIKE

It is a night shift rota. Sustainable if the workload is real and predictable; a resignation letter if it is constant false alarms.

WHY IT MATTERS IN PRODUCTION

The metric that changes behaviour is pages per shift, because it is visible and uncomfortable. Once it is tracked, alert quality improvements get prioritised the same way features do. The other habit that matters is treating the runbook as a product: after every incident, the responder updates the runbook while it is fresh, which is what stops the same twenty minutes of orientation happening every time.

STEP BY STEP

1. Define the paging bar - user impact or imminent user impact only.
2. Require a tested runbook per page, and update it immediately after each use.
3. Track pages per shift and time-to-acknowledge as reliability metrics of the alerting system.
4. Review every page in a blameless forum, and convert findings into owned backlog items.

EVIDENCE YOU WOULD QUOTE

```
Pages per shift, trended over months

Percentage of alerts with a tested runbook link

Follow-up action completion rate from incident reviews
```

RED FLAG - DO NOT SAY THIS

Do not treat alert fatigue as an individual resilience problem. It is a defect in the alerting system and it should be tracked as one.

EXPECT THIS FOLLOW-UP

On-call is burning people out. What do you change in the first month?

Q171 ARCHITECT

How would you design observability across a fleet of clusters?

SAY THIS

Local Prometheus in every cluster for short retention and fast local queries, with metrics shipped to a central long-term store for cross-cluster querying and history. Alerting stays local so a network partition does not blind a cluster, but alerts route to central handling. Dashboards are templated per cluster from one source. And I would decide the metric allow-list deliberately, because shipping everything from fifty clusters is a cost problem before it is a technical one.

THINK OF IT LIKE

It is regional offices with local records and a central archive. Each office works if the line to head office drops, and head office can still see the whole picture.

WHY IT MATTERS IN PRODUCTION

Keeping alert evaluation local is the design decision that gets tested during a real incident. Central evaluation means a network partition produces silence rather than alerts, which is the worst possible failure mode for a monitoring system. The cost decision is equally consequential: an unfiltered federation of fifty clusters can easily cost more than the clusters, and an allow-list of metrics that dashboards and alerts actually use is usually a fraction of what gets scraped.

STEP BY STEP

1. Keep scraping and alert evaluation local so a partition degrades gracefully.
2. Ship a deliberately filtered metric set to central long-term storage.
3. Template dashboards and alert rules from one source, deployed by GitOps to every cluster.
4. Track observability cost per cluster and review the allow-list against what is actually queried.

EVIDENCE YOU WOULD QUOTE

```
Central store ingestion rate and cost per cluster

oc get prometheusrule -A per cluster, compared against the templated source

Query patterns: which metrics are actually used by dashboards and alerts
```

RED FLAG - DO NOT SAY THIS

Do not centralise alert evaluation. A network problem then produces silence instead of alarms.

EXPECT THIS FOLLOW-UP

Observability cost is now 20% of the platform budget. What do you cut first?

Q172 ARCHITECT

How do you measure whether the platform is actually getting more reliable?

SAY THIS

With a small set of outcome metrics rather than activity metrics. SLO attainment per critical service. Incident count and severity trend. Time to detect and time to restore. Change failure rate. Percentage of incidents with a completed follow-up action. And a leading indicator or two, such as the proportion of workloads meeting the production readiness standard. Then I review them monthly and let them drive where engineering effort goes.

THINK OF IT LIKE

It is a patient's vital signs rather than a list of treatments administered. What matters is whether they are getting better, not how busy the ward was.

WHY IT MATTERS IN PRODUCTION

The trap is measuring activity - alerts configured, dashboards built, tickets closed - which rewards motion rather than outcomes. Time to detect and time to restore are the pair that most reliably expose whether observability and runbooks are improving, and change failure rate is the one that keeps delivery honest. Publishing the numbers, including the ones going the wrong way, is what makes the exercise credible internally.

STEP BY STEP

1. Choose outcome metrics: SLO attainment, incident trend, detect and restore times, change failure rate.
2. Add one or two leading indicators tied to the production readiness standard.

3 Review monthly with the teams, and let the numbers set the improvement backlog.
4 Publish the trend openly, including regressions, and revisit the metric set annually.

EVIDENCE YOU WOULD QUOTE

```
SLO attainment per service over rolling 90 days

MTTD and MTTR distributions, not averages, per severity

Change failure rate and follow-up action completion rate
```

RED FLAG - DO NOT SAY THIS

Do not report activity metrics as reliability progress. Nobody cares how many dashboards exist if restore time is getting worse.

EXPECT THIS FOLLOW-UP

Incident count is flat but restore time halved. Is that success? Argue it.

PART 12 · Q173-Q188 · 16 QUESTIONS

Troubleshooting and incident response

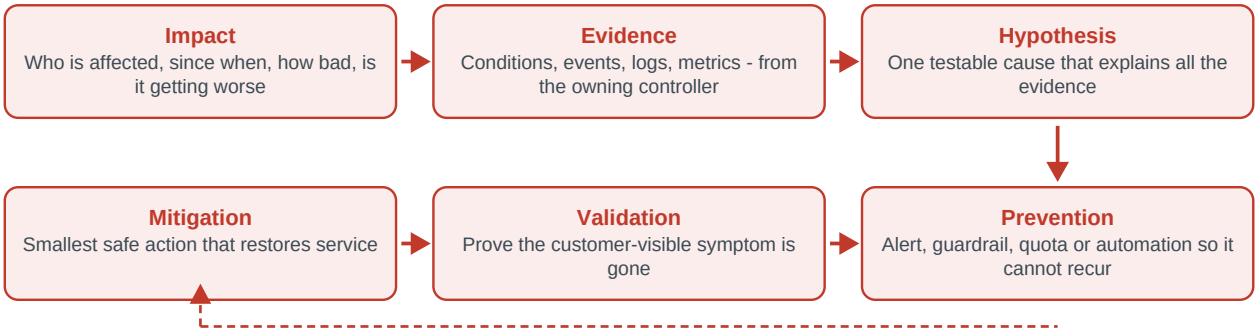
Working from evidence under pressure, and leaving the system better than you found it

This is the part of the interview where the scoring gets brutal, because troubleshooting answers expose how you actually think. Two candidates can name the same command and score completely differently: one says "I would restart the pod", the other says "I would state the impact, read the events, form one hypothesis, take the smallest safe action, validate against the user-visible symptom, and then make it impossible to happen again". The second answer is not more knowledgeable. It is more disciplined, and discipline is what an interviewer can actually assess in forty minutes.

Level mix - Foundation: 4 · Intermediate: 5 · Senior: 5 · Architect: 2

The evidence-first loop - use this for every troubleshooting question

Say these six words out loud before you say anything technical. Candidates who jump straight to "I would restart the pod" lose the point even when restarting works.



Q173 FOUNDATION

How do you approach a troubleshooting question?

SAY THIS

With the same six steps every time. Impact - who is affected, since when, is it getting worse. Evidence - conditions, events, logs and metrics from the owning controller, not just the symptom. Hypothesis - one testable explanation that accounts for all the evidence. Mitigation - the smallest safe action that restores service. Validation - prove the user-visible symptom is gone. Prevention - the alert, guardrail or automation that stops it recurring.

THINK OF IT LIKE

It is triage in an emergency department. Assess, examine, form a diagnosis, treat conservatively, confirm improvement, then discuss how to avoid the next admission.

WHY IT MATTERS IN PRODUCTION

Having a structure matters most when you do not know the answer. If a question catches you cold, the loop gives you something correct to say while you think, and it demonstrates the behaviour being assessed even on an unfamiliar technology. It also protects you in real incidents from the two most expensive mistakes: acting before understanding, and stopping once service is restored without fixing the cause.

STEP BY STEP

1. State impact and scope before touching anything - it also determines urgency.
2. Gather evidence from the owning controller's conditions, then events, logs and metrics.
3. Form one hypothesis that explains all the evidence, and say how you would test it.
4. Apply the smallest safe mitigation, validate against the user symptom, then define prevention.

EVIDENCE YOU WOULD QUOTE

```
oc get events -A --sort-by=.lastTimestamp | tail -30

oc get co ; oc get nodes ; oc get pods -A --field-selector status.phase!=Running

oc describe <kind> <name> | sed -n '/Conditions/,$p'
```

RED FLAG - DO NOT SAY THIS

Do not open with "I would restart it". Even when restarting works, it is the answer that scores lowest.

EXPECT THIS FOLLOW-UP

Which of those six steps do people skip most often, and what does it cost?

Q174 FOUNDATION

How do you troubleshoot CrashLoopBackOff?

SAY THIS

CrashLoopBackOff means the container starts, exits, and Kubernetes is backing off before retrying - so the container is failing, not the platform. I read the previous container's logs first, because the current one may not have got far enough to log anything. Then the exit code: 137 is a memory limit kill, 1 or 2 is usually application error or misconfiguration, 126 or 127 means the command could not be executed. Then I check whether a liveness probe is killing it before it becomes healthy.

THINK OF IT LIKE

It is an engine that keeps stalling on ignition. You do not investigate the road - you look at what the engine says as it dies.

WHY IT MATTERS IN PRODUCTION

The exit code shortcut saves a lot of time. Exit 137 points at memory and you go straight to limits and working set. Exit 0 in a crash loop means the process completed and exited, which usually means the container is running the wrong command or a script that ends. And if the pod becomes healthy for a few seconds before dying, suspect a liveness probe whose initial delay is shorter than the application's real startup time - which is what startup probes exist to solve.

STEP BY STEP

1. Read logs from the previous instance, not the current one.
2. Read the exit code and last state, and use it to narrow the class of failure.
3. Check probe configuration and restart count timing to rule out probe-induced kills.
4. Verify configuration and dependencies - missing ConfigMap, Secret, or an unreachable dependency at startup.

EVIDENCE YOU WOULD QUOTE

```
oc logs <pod> --previous --tail=100

oc describe pod <pod> | grep -A6 'Last State'

oc get pod <pod> -o jsonpath='{.status.containerStatuses[0].lastState}' | python3 -m json.tool
```

RED FLAG - DO NOT SAY THIS

Do not delete the pod before reading the previous logs. You have just destroyed the only record of why it died.

EXPECT THIS FOLLOW-UP

Exit code 137 but the node has plenty of free memory. Explain that.

Q175 FOUNDATION

How do you troubleshoot ImagePullBackOff?

SAY THIS

The kubelet could not pull the image, and the event message almost always says why. The usual causes are a wrong image name or tag, a private registry with no pull secret attached to the ServiceAccount, an expired or wrong credential, the registry being unreachable because of a proxy or network policy, a certificate the node does not trust, or rate limiting from a public registry.

THINK OF IT LIKE

It is a delivery that failed. The card through the door says whether the address was wrong, nobody was in, or the courier could not get through the gate.

WHY IT MATTERS IN PRODUCTION

The one that surprises people is when it works on some nodes and not others - that is almost always a node-level trust or proxy difference rather than anything to do with the pod. Public registry rate limiting is the other modern classic, particularly in clusters that pull unauthenticated: everything works for weeks and then a busy afternoon produces pull failures across unrelated namespaces.

STEP BY STEP

1. Read the exact event message - it distinguishes not-found from unauthorised from unreachable.
2. Verify the image reference and confirm it exists in the registry, ideally by digest.
3. Check the pull secret is attached to the pod's ServiceAccount and that the credential is valid.
4. If node-specific, check proxy configuration, registry trust and mirror configuration on that node.

EVIDENCE YOU WOULD QUOTE

```
oc describe pod <pod> | sed -n '/Events/,$p'

oc get sa <sa> -n <ns> -o jsonpath='{.imagePullSecrets}{"\n"}'

oc debug node/<node> -- chroot /host crictl pull <image>
```

RED FLAG - DO NOT SAY THIS

Do not add a pull secret without reading the error first. If the tag does not exist, the credential was never the problem.

EXPECT THIS FOLLOW-UP

The image pulls on three nodes and fails on the fourth. What is different?

Q176 FOUNDATION

Events, conditions, logs - which do you read first and why?

SAY THIS

Conditions first, because they are the owning controller's own statement of what it thinks and what is blocking it. Events second, because they are the timeline of what the platform decided and when - though they expire after a few hours, so capture them early. Logs third, because they are the most detailed and the least structured, and you want to know where to look before you start reading them.

THINK OF IT LIKE

Conditions are the diagnosis on the chart, events are the nursing notes, logs are the raw monitor trace. You read them in that order for a reason.

WHY IT MATTERS IN PRODUCTION

The default retention on events is short - roughly three hours - which is exactly long enough to be gone by the time an incident review starts. So part of the discipline is capturing events into the incident record at the beginning rather than assuming they will still be there. Conditions, by contrast, persist on the object, which is why they are the reliable starting point.

STEP BY STEP

1. Read status conditions on the object and on its owning controller.
2. Pull events for the namespace sorted by time, and save them to the incident record immediately.
3. Then go to logs, targeted by what the conditions and events pointed at.
4. Correlate all three against metrics for the same window before forming a hypothesis.

EVIDENCE YOU WOULD QUOTE

```
oc get <kind> <name> -o jsonpath='{.status.conditions}' | python3 -m json.tool

oc get events -n <ns> --sort-by=.lastTimestamp > incident-events.txt

oc logs <pod> --since=30m --tail=200
```

RED FLAG - DO NOT SAY THIS

Do not start with logs. Without direction you will read the wrong ten thousand lines.

EXPECT THIS FOLLOW-UP

The events you need have already expired. What do you do?

Q177 INTERMEDIATE

How do you troubleshoot a node that is NotReady?

SAY THIS

I read the node conditions first, because the kubelet usually states the reason - MemoryPressure, DiskPressure, PIDPressure, or a plain unreachable status meaning the kubelet is not reporting at all. Unreachable points at the kubelet or the network path to the API server; pressure conditions point at node resources. Then I get onto the node and check kubelet and CRI-O logs, disk usage, and kernel messages before deciding anything.

THINK OF IT LIKE

It is a member of staff who has stopped answering. Either they are unwell, or the phone line is down - and those need very different responses.

WHY IT MATTERS IN PRODUCTION

The distinction that matters most is that a NotReady node is often still running its workloads and serving traffic. The kubelet has lost contact; the containers have not stopped. So rebooting immediately can turn a management-plane problem into a genuine outage. The other thing to know is the eviction timer: after the toleration period, pods will be marked for eviction and rescheduled, so you have a window in which to decide deliberately.

STEP BY STEP

1. Read node conditions to classify the failure: pressure versus unreachable.
2. Establish whether workloads on the node are still serving before considering a reboot.
3. Get on the node and check kubelet and CRI-O logs, disk, memory and kernel messages.
4. Fix the identified cause, or cordon and drain deliberately if the node must be replaced.

EVIDENCE YOU WOULD QUOTE

```
oc get node <node> -o jsonpath='{.status.conditions}' | python3 -m json.tool

oc debug node/<node> -- chroot /host sh -c 'df -h /var; journalctl -u kubelet -n 100 --no-pager'

oc get pods -A -o wide --field-selector spec.nodeName=<node>
```

RED FLAG - DO NOT SAY THIS

Do not reboot the node first. Its pods may still be serving customers, and you will lose the evidence either way.

EXPECT THIS FOLLOW-UP

The node is unreachable but its pods still answer traffic. What does that mean?

Q178 INTERMEDIATE

A namespace is stuck in Terminating. What do you do?

SAY THIS

Something is blocking deletion, and it is either a resource that will not delete or a finalizer with no controller left to clear it. I list every resource still present in the namespace, look at what has finalizers, and identify which controller owns them. If the controller was uninstalled, the correct fix is to bring it back so it can clean up properly. Removing finalizers by hand works, but it silently leaks whatever they were protecting, so it is a documented last resort.

THINK OF IT LIKE

It is a house sale stuck at completion. Someone has not signed, and forging the signature gets you the keys and an unresolved liability.

WHY IT MATTERS IN PRODUCTION

The usual sequence that creates this is an operator being removed before its custom resources were deleted. The custom resources still carry finalizers, the controller that would clear them no longer exists, and the namespace waits forever. Reinstalling the operator briefly, letting it clean up, then removing it in the right order is slower and much safer than patching finalizers out.

STEP BY STEP

1. List every remaining resource in the namespace across all API groups.
2. Identify which resources carry finalizers and which controller owns each one.
3. Restore the missing controller so cleanup can complete properly.
4. Only if that is impossible, remove the finalizer, document what was leaked, and clean it up at the backend.

EVIDENCE YOU WOULD QUOTE

```
oc get ns <ns> -o jsonpath='{.status.conditions}' | python3 -m json.tool

oc api-resources --verbs=list --namespaced -o name | xargs -n1 oc get -n <ns> --ignore-not-found
2>/dev/null

oc get <kind> <name> -n <ns> -o jsonpath='{.metadata.finalizers}{"\n"}'
```

RED FLAG - DO NOT SAY THIS

Do not patch finalizers out as your first move. You are guaranteeing an orphaned cloud resource nobody will ever find.

EXPECT THIS FOLLOW-UP

The finalizer belongs to an operator that was uninstalled last month. Now what?

Q179 INTERMEDIATE

A pod is Running but never becomes Ready. How do you diagnose it?

SAY THIS

Running but not Ready means the container process started and the readiness probe is failing, so the pod is deliberately kept out of the EndpointSlice. I look at the probe definition - path, port, scheme, timing - then test that exact endpoint from inside the pod, then from another pod. If it works from inside and not from outside, it is network policy or a port mismatch. If it fails from inside, the application is genuinely not ready and its logs will say why.

THINK OF IT LIKE

The shop is lit and staffed but the Open sign is off. Either the staff know something you do not, or the sign is wired wrong.

WHY IT MATTERS IN PRODUCTION

The two causes are roughly equally common and pull in opposite directions. A genuinely unready application - waiting on a database, still loading a cache - is doing exactly the right thing and the fix is upstream. A misconfigured probe - wrong port, HTTPS versus HTTP, a path that requires authentication, a timeout shorter than the endpoint's real response time under load - is a self-inflicted outage. Testing from inside the pod distinguishes them in one command.

STEP BY STEP

1. Read the readiness probe definition and the describe output showing the probe failure.
2. Curl the exact probe path and port from inside the container.
3. If that succeeds, test from another pod to check policy and port mapping.
4. If it fails, read the application logs for what it is waiting on, and fix upstream rather than loosening the probe.

EVIDENCE YOU WOULD QUOTE

```
oc describe pod <pod> | grep -A4 Readiness

oc exec <pod> -- curl -sS -m 3 -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8080/healthz

oc get endpointslice -n <ns> -l kubernetes.io/service-name=<svc>
```

RED FLAG - DO NOT SAY THIS

Do not relax the readiness probe to make the pod Ready. You have just told the router to send users to a broken pod.

EXPECT THIS FOLLOW-UP

The probe passes from inside the pod and fails from the kubelet. What is going on?

Q180 INTERMEDIATE

How do you investigate an OOMKilled container?

SAY THIS

The kernel killed the process because the container's cgroup exceeded its memory limit. I confirm it from the last state and exit code 137, then look at the working set over time to see whether this was a steady climb, which suggests a leak, or a spike, which suggests a particular request or job. Then I decide between raising the limit, fixing the application, or both - and I check whether the limit was ever based on measurement in the first place.

THINK OF IT LIKE

It is a circuit breaker tripping. The breaker is not the fault; either the appliance is faulty or the circuit was rated for less than you actually plug in.

WHY IT MATTERS IN PRODUCTION

The detail that catches people is that the node can have plenty of free memory and the container still be killed, because the limit is per-cgroup rather than per-node. The other one is the JVM and similar runtimes: a heap size configured larger than the container limit guarantees an eventual kill, and the fix is to make the runtime container-aware rather than to keep raising the limit.

STEP BY STEP

1. Confirm OOMKilled from the container's last state and exit code.
2. Chart working set over time to distinguish a leak from a spike.
3. Check whether the runtime is aware of the container limit, especially for JVM workloads.
4. Set the limit from measured p99 usage with headroom, and fix the leak if that is the pattern.

EVIDENCE YOU WOULD QUOTE

```
oc describe pod <pod> | grep -A5 'Last State'

container_memory_working_set_bytes{pod="<pod>"} over 24h

oc debug node/<node> -- chroot /host dmesg -T | grep -i 'killed process'
```

RED FLAG - DO NOT SAY THIS

Do not just double the memory limit. If it is a leak you have bought a few hours and learned nothing.

EXPECT THIS FOLLOW-UP

The graph shows a steady climb over four days. What is your recommendation?

Q181 INTERMEDIATE

How do you investigate intermittent 5xx errors you cannot reproduce?

SAY THIS

I start by making the intermittent measurable: what percentage of requests, on which paths, at what times, from which clients, and does it correlate with deploys, traffic peaks or a specific backend pod. Then I look for a pattern that turns "random" into "conditional" - one unhealthy pod still in rotation, a node with a datapath problem, a dependency timing out under load, or connection churn during rollouts.

THINK OF IT LIKE

It is a rattle that only happens above forty miles an hour. It feels random until you notice the condition, and then it is completely reproducible.

WHY IT MATTERS IN PRODUCTION

The single most common cause in practice is one bad backend among many: a pod that passes its readiness probe but fails real requests, so a fraction of traffic hits it and the error rate looks random. Per-pod metrics collapse that immediately. The second most common is deploy-time connection handling - errors that cluster tightly around rollout windows and trace back to shutdown behaviour rather than to anything in the request path.

STEP BY STEP

1. Quantify it: error rate, affected paths, time distribution, and correlation with events.
2. Break metrics down per pod, per node and per zone to find a non-uniform distribution.
3. Check whether errors cluster around deployments, scaling events or peak traffic.
4. Reproduce under load if possible, and use traces or flow data to attribute the failing hop.

EVIDENCE YOU WOULD QUOTE

```
sum by (pod) (rate(http_requests_total{code=~"5.."}[5m]))

oc get events -n <ns> --sort-by=.lastTimestamp | grep -i -E 'unhealthy|killing|scaled'

Router or ingress logs filtered to 5xx, grouped by backend
```

RED FLAG - DO NOT SAY THIS

Do not call it random. Errors that appear random are almost always conditional on something you have not measured yet.

EXPECT THIS FOLLOW-UP

Errors cluster tightly around every deployment. What is your first hypothesis?

Q182 SENIOR

Somebody says "the application is slow". How do you triage that?

SAY THIS

I turn it into a measurable statement first: which operation, how slow compared with what, for which users, since when, and is it all requests or a percentile tail. Then I walk the request path and measure at each hop - ingress, service, application, dependency, database, storage - to find where the time is actually spent. Most "slow application" reports resolve to one dependency, one saturated resource, or a change that landed recently.

THINK OF IT LIKE

"The journey took ages" is not a diagnosis. You need to know which leg of it, compared with what, and whether it was the traffic or the train.

WHY IT MATTERS IN PRODUCTION

Two habits make this fast. First, always ask what changed - a deployment, a configuration change, a data volume increase, an infrastructure event - because the answer is in that list far more often than not. Second, check whether it is the tail or the whole distribution: a p99 regression with a flat p50 points at a specific slow path, contention or garbage collection, whereas everything shifting points at a shared resource or a dependency.

STEP BY STEP

1. Convert the complaint into numbers: operation, percentile, baseline, scope, start time.
2. Ask what changed - deploys, config, data volume, infrastructure - and check the timeline.
3. Measure at each hop of the request path rather than assuming where the time goes.
4. Fix the identified bottleneck, then record the new baseline and add an alert on the relevant percentile.

EVIDENCE YOU WOULD QUOTE

```
histogram_quantile(0.99, ...) and (0.50, ...) for the same service, over 7 days

oc rollout history deploy/<name> ; recent config and image changes

Dependency latency: database, cache, downstream service, storage
```

RED FLAG - DO NOT SAY THIS

Do not start tuning before measuring. Tuning without a baseline is how people spend a week making something 3% faster.

EXPECT THIS FOLLOW-UP

p99 doubled but p50 is unchanged. What does that pattern suggest?

Q183 SENIOR

Multiple unrelated things are failing at once. How do you triage?

SAY THIS

Simultaneous unrelated symptoms almost always mean one shared dependency, so I stop looking at the symptoms and look for the common factor. The usual candidates are the API server or etcd, DNS, the ingress layer, storage, identity, or a whole failure domain such as a zone. I establish scope and blast radius first, communicate early, and work down the shared dependency list rather than investigating each symptom in parallel.

THINK OF IT LIKE

When every appliance in the house stops at once, you check the fuse box - you do not open up the fridge, the washing machine and the boiler simultaneously.

WHY IT MATTERS IN PRODUCTION

The organisational failure in these incidents is as important as the technical one. Under pressure, several people start investigating different symptoms and nobody is looking at the shared layer, while stakeholders receive three contradictory updates. Naming an incident commander, declaring the working hypothesis explicitly, and communicating on a fixed cadence is what turns a scramble into a controlled response.

STEP BY STEP

1. Establish scope: which services, which namespaces, which zones, and since when.
2. Look for the shared dependency instead of investigating each symptom independently.

3 Assign a commander and communicate early with a stated hypothesis and next update time.
4 Mitigate at the shared layer, validate broadly, and capture evidence for the review.

EVIDENCE YOU WOULD QUOTE

```
oc get co ; oc get nodes ; oc get clusterversion

oc get events -A --sort-by=.lastTimestamp | tail -50

Zone-level view: pods, nodes and dependencies grouped by topology label
```

RED FLAG - DO NOT SAY THIS

Do not let three people investigate three symptoms in parallel. You will get three partial answers and no diagnosis.

EXPECT THIS FOLLOW-UP

Every namespace reports DNS failures. Where do you look, in order?

Q184 SENIOR

How do you preserve evidence while restoring service?

SAY THIS

By capturing before mitigating, deliberately and quickly. Events expire, pod logs vanish when the pod is deleted, and node state changes on reboot - so the first two minutes should capture events, describe output, previous logs and a must-gather if the cluster is involved. Where possible I isolate rather than destroy: cordon a node instead of rebooting it, scale a bad replica to zero instead of deleting it, so the artefact still exists.

THINK OF IT LIKE

It is photographing the scene before the road is reopened. The traffic has to move, but the crash investigation still needs to happen.

WHY IT MATTERS IN PRODUCTION

This is the difference between an incident you learn from and one you repeat. The classic loss is deleting a crashing pod to force a fresh one, which removes the only copy of the logs that explained the crash. Building capture into the first step of every runbook - with the commands already written down - means it happens under pressure instead of being remembered afterwards.

STEP BY STEP

1 Capture first: events to a file, describe output, previous container logs, relevant metrics screenshots or queries.
2 Start a must-gather in the background if the platform is implicated.
3 Prefer isolation over destruction - cordon, scale to zero, or move traffic away.
4 Record a timeline as you go, with timestamps, so the review does not rely on memory.

EVIDENCE YOU WOULD QUOTE

```
oc adm must-gather --dest-dir=./mg-$(date +%s)

oc get events -A --sort-by=.lastTimestamp > events-$(date +%s).txt

oc logs <pod> --previous > pod-previous.log ; oc describe pod <pod> > pod-describe.txt
```

RED FLAG - DO NOT SAY THIS

Do not delete the failing pod to get a fresh one before capturing its logs. That evidence does not come back.

EXPECT THIS FOLLOW-UP

You have five minutes before you must restore service. What do you capture?

Q185 SENIOR

When and how do you escalate to vendor support?

SAY THIS

When the problem is in the platform rather than the workload, when a supported procedure is not behaving as documented, or when resolution needs a code fix or product guidance. I escalate early with a complete package: must-gather, a clear problem statement with impact and business severity, a timeline of what changed, what I have already tried and ruled out, and the specific question I need answered. Vague cases with no data sit in queues.

THINK OF IT LIKE

It is a referral to a specialist. Sending the notes, the scans and a clear question gets an answer; sending "patient feels unwell" gets a waiting list.

WHY IT MATTERS IN PRODUCTION

Two practical points matter. Escalate in parallel with your own investigation rather than after exhausting it - the case takes time to move regardless, and you can always close it. And gather must-gather while the problem is present, because a bundle collected after the cluster recovered often contains nothing useful. Setting severity honestly matters too: inflating it burns credibility you will need later.

STEP BY STEP

1. Decide it is a platform problem rather than a workload one, and say why.
2. Collect must-gather while the symptom is present, plus targeted component gathers.
3. Write the case as impact, timeline, what changed, what you ruled out, and the specific question.
4. Set severity honestly, escalate in parallel with your own work, and keep the case updated as you learn more.

EVIDENCE YOU WOULD QUOTE

```
oc adm must-gather ; oc adm must-gather -- /usr/bin/gather_<component>

oc adm inspect ns/<namespace> --dest-dir=./inspect

oc get clusterversion -o jsonpath='{.status.desired.version}{"\n"}' ; oc get co
```

RED FLAG - DO NOT SAY THIS

Do not open a case without must-gather. The first reply will ask for it, and you will have lost a day.

EXPECT THIS FOLLOW-UP

The cluster recovered before you collected must-gather. What do you send instead?

Q186 SENIOR

What does good incident communication look like?

SAY THIS

One person owns communication, updates go out on a fixed cadence even when there is nothing new, and each update says the same four things: what is affected in user terms, what we currently believe, what we are doing next, and when the next update will be. Uncertainty is stated plainly rather than smoothed over. Technical detail goes in the engineering channel; stakeholders get impact and expectations.

THINK OF IT LIKE

It is an airline announcing a delay. "We are waiting on a part, next update in twenty minutes" keeps people calm. Silence does not, and neither does a technical explanation of the part.

WHY IT MATTERS IN PRODUCTION

The cadence is what buys you room to work. Without it, stakeholders interrupt constantly for status and the responders lose focus, which is a real and measurable cost during an incident. Separating the commander from the person communicating and from the person typing commands is not bureaucracy - it is what stops the one person who understands the problem from spending the incident writing updates.

STEP BY STEP

1. Assign roles explicitly: commander, communicator, and the people doing the work.
2. Send updates on a fixed cadence with impact, belief, next action and next update time.
3. State uncertainty honestly and avoid predicting resolution times you cannot support.
4. Separate stakeholder communication from the technical channel, and log the timeline as you go.

EVIDENCE YOU WOULD QUOTE

```
Incident timeline with timestamped updates and decisions

Stakeholder update log showing cadence adherence

Post-incident survey: did stakeholders feel informed
```

RED FLAG - DO NOT SAY THIS

Do not go quiet while you investigate. Silence is read as loss of control and generates more interruptions than any update would.

EXPECT THIS FOLLOW-UP

A stakeholder demands an ETA you cannot give. What do you say?

Q187 ARCHITECT

What makes a useful blameless postmortem?

SAY THIS

A factual timeline, an honest account of what people believed at each point and why those beliefs were reasonable, contributing factors rather than a single root cause, and follow-up actions with named owners and dates. Blameless means focusing on why the system made the mistake easy rather than on who made it - because in a blame culture people stop reporting near misses, and you lose the information that prevents the next incident.

THINK OF IT LIKE

It is an air accident investigation. Nobody blames the pilot for the design of a confusing switch - they redesign the switch, and every other aircraft benefits.

WHY IT MATTERS IN PRODUCTION

The measure of whether postmortems are working is not how well written they are but whether the actions get done. Most organisations produce good documents and complete a minority of the follow-ups, which means the same incident recurs and people quietly conclude the process is theatre. Tracking action completion rate, and reviewing repeat incidents against previous postmortems, is what makes it real.

STEP BY STEP

1. Build a factual timeline including detection, decisions and communication, with timestamps.
2. Describe contributing factors and what made the wrong action easy or the right one hard.
3. Write actions that are specific, owned and dated - and reject vague ones like "be more careful".
4. Track completion rate and check repeat incidents against earlier postmortems.

EVIDENCE YOU WOULD QUOTE

```
Postmortem documents with completed action items and dates

Action completion rate over the last twelve months

Repeat incident analysis: which recurrences had a prior postmortem
```

RED FLAG - DO NOT SAY THIS

Do not produce postmortems whose actions are never completed. It is worse than not writing them, because it teaches people the process is decoration.

EXPECT THIS FOLLOW-UP

The same incident recurred despite a postmortem. What went wrong with the process?

Q188 ARCHITECT

How do you systematically reduce recurring incidents?

SAY THIS

By treating incidents as data rather than as individual events. I categorise them by contributing factor over a quarter, look for the themes - configuration drift, capacity, a particular dependency, change process - and attack the theme rather than the instance. That usually means a platform change that removes a whole class: a guardrail in admission policy, a default in the golden path, an automated check in the pipeline, or better capacity headroom.

THINK OF IT LIKE

It is public health rather than medicine. Treating each patient works; fixing the water supply stops the queue forming.

WHY IT MATTERS IN PRODUCTION

The distinction that matters is between incident management, which is about restoring service, and problem management, which is about eliminating causes. Organisations that only do the first get very good at recovering from the same failure repeatedly. Making the quarterly theme review a standing commitment - with engineering time reserved for it - is what converts firefighting capability into reliability.

STEP BY STEP

1. Categorise incidents by contributing factor and review the distribution quarterly.
2. Pick the top one or two themes and design a platform-level control that removes the class.
3. Reserve engineering capacity for that work explicitly, not as spare-time effort.

4 Measure the effect - incident rate within that category before and after - and publish it.

EVIDENCE YOU WOULD QUOTE

```
Incident categorisation over the last four quarters, by contributing factor

Guardrails added: admission policies, golden path defaults, pipeline checks

Incident rate per category before and after each intervention
```

RED FLAG - DO NOT SAY THIS

Do not measure success by incidents resolved. Getting faster at fixing the same thing is not reliability improvement.

EXPECT THIS FOLLOW-UP

Your top theme is configuration drift. What platform change removes that class?

PART 13 · Q189-Q200 · 12 QUESTIONS

High availability, backup and disaster recovery

Three different problems that people keep answering as if they were one

The fastest way to fail this section is to treat availability, backup and disaster recovery as synonyms. They answer different questions: availability is about surviving a component failure without anyone noticing, backup is about recovering data somebody destroyed, and disaster recovery is about continuing to operate when a whole site is gone. Different mechanisms, different costs, different tests. The other thing being assessed here is honesty - specifically whether you have ever actually restored something, or only ever configured a backup job and assumed.

Level mix - Foundation: 3 · Intermediate: 4 · Senior: 3 · Architect: 2

What each recovery mechanism actually protects

The classic senior trap: "we take etcd backups, so we are covered". Say which failure each control answers and you separate yourself instantly.

Mechanism	Protects against	Does not protect against	Proof it works
Multiple replicas + spread	Single pod, node or zone loss	Bad config or bad image rolled everywhere	Drain a node in business hours and watch SLOs
etcd backup	Loss of cluster state or quorum	Application data loss; PVs are not in etcd	Documented, rehearsed restore on a lab cluster
OADP / Velero	Namespace, object and PV loss; migration	Cluster-level corruption or a lost control plane	Restore into a scratch namespace and run the app
CSI snapshots	Fast rollback of a volume	Site loss if snapshots live on the same array	Clone the snapshot and mount it read-write
Second cluster / ACM DR	Site or region loss	Human error replicated by GitOps within seconds	Timed failover test against the stated RTO

Q189 FOUNDATION

What does an etcd backup protect, and how do you take one?

SAY THIS

An etcd backup is a snapshot of the cluster's own state - every API object, including Secrets, RBAC, custom resources and workload definitions. OpenShift provides a supported backup script that runs on a control plane node and produces a snapshot plus the static pod resources needed to restore. It protects against cluster state loss or corruption. It does not contain application data, because that lives on persistent volumes.

THINK OF IT LIKE

It is a photograph of the building's blueprints and tenancy register. Invaluable if the records room burns down; useless for replacing the contents of the flats.

WHY IT MATTERS IN PRODUCTION

The operational details that matter are cadence, storage location and validation. A backup sitting on the control plane node it came from protects against very little - it needs to be off-cluster and, ideally, off-site. And an untested backup is a hypothesis: the only evidence that it works is having restored it onto a lab cluster and watched the cluster come up.

STEP BY STEP

1. Describe what is captured: the etcd snapshot plus the static pod resources.
2. Run backups on a schedule from a control plane node, and move them off-cluster immediately.
3. Encrypt them, because they contain every Secret in the cluster.

4 Validate by restoring onto a lab cluster periodically and recording the result.

EVIDENCE YOU WOULD QUOTE

```
oc debug node/<master> -- chroot /host /usr/local/bin/cluster-backup.sh /home/core/backup

ls -lh /home/core/backup # snapshot and static pod resources

Documented restore test date, target cluster and outcome
```

RED FLAG - DO NOT SAY THIS

Do not leave etcd backups on the cluster they came from, unencrypted. They contain every secret you have.

EXPECT THIS FOLLOW-UP

How would you prove your etcd backup actually works?

Q190 FOUNDATION

How do application backup and etcd backup differ?

SAY THIS

They protect different things and neither substitutes for the other. An etcd backup restores the cluster's definition of the world - namespaces, deployments, secrets, custom resources - to a point in time. An application backup captures the application's data: persistent volume contents, database dumps, object storage. If you lose a database's data, an etcd restore gives you a perfectly reconstructed empty database.

THINK OF IT LIKE

One is the recipe book, the other is the contents of the fridge. Losing either one leaves you unable to cook dinner, and they are stored in completely different places.

WHY IT MATTERS IN PRODUCTION

This is the single most common senior-level trap in the whole DR topic, because "we back up etcd" sounds like a complete answer. The follow-up is always about application data, and the credible response is a matrix: for each critical workload, what protects its definition, what protects its data, where those copies live, and when each was last restored successfully.

STEP BY STEP

1 State the scope of each clearly - cluster state versus application data.
2 Point out that persistent volume contents are not in etcd at all.
3 Describe the combination needed for a real recovery of a stateful workload.
4 Maintain a per-application protection matrix with last successful restore dates.

EVIDENCE YOU WOULD QUOTE

```
oc get backup -n openshift-adp -o wide # application backups

Documented etcd backup schedule and location

Protection matrix: workload, state backup, data backup, last restore test
```

RED FLAG - DO NOT SAY THIS

Do not answer "we take etcd backups" to a question about application data. It is the exact trap the question was built around.

EXPECT THIS FOLLOW-UP

A team deletes their production database's PVC. Does your etcd backup help?

Q191 FOUNDATION

What are RPO and RTO?

SAY THIS

Recovery Point Objective is how much data you can afford to lose, measured in time - an RPO of one hour means you accept losing up to an hour of transactions. Recovery Time Objective is how long you can afford to be down before service is restored. They are business decisions with technical costs: tighter numbers require more replication, more frequent copies and more standby capacity, and the price rises steeply near zero.

THINK OF IT LIKE

RPO is how many pages of the manuscript you can bear to rewrite. RTO is how long the publisher will wait. Both have a price, and the price of zero is enormous.

WHY IT MATTERS IN PRODUCTION

The value of these two numbers is that they turn a vague conversation about "we need high availability" into a design constraint. An RPO of fifteen minutes and an RTO of four hours points at a very different architecture from an RPO of zero and an RTO of five minutes, and the cost difference is often an order of magnitude. Getting the business to state them, and then demonstrating what they actually cost, is the conversation this question is testing.

STEP BY STEP

1. Define both precisely and note that they are business decisions, not technical ones.
2. Set them per service tier rather than one number for everything.
3. Design the mechanism to meet them - backup frequency, replication, standby capacity.
4. Test and measure the actual achieved numbers, and report the gap honestly.

EVIDENCE YOU WOULD QUOTE

```
Documented RPO and RTO per service tier, signed off by the business

Measured recovery time from the last DR test, against the target

Backup frequency and replication lag metrics per critical dataset
```

RED FLAG - DO NOT SAY THIS

Do not accept "zero downtime and zero data loss" as a requirement without pricing it. It is a wish until somebody has seen the invoice.

EXPECT THIS FOLLOW-UP

The business wants RPO zero for everything. How do you run that conversation?

Q192 INTERMEDIATE

What does OADP protect, and what does it not?

SAY THIS

OADP - the OpenShift API for Data Protection, built on Velero - backs up namespaced resources and their persistent volume data to object storage, and restores them into the same or a different cluster. That makes it the right tool for namespace-level recovery, accidental deletion and migration between clusters. It is not a cluster recovery tool: it does not restore a lost control plane, and it does not replace etcd backups.

THINK OF IT LIKE

It is a removals company for a flat. Excellent at packing one household and delivering it elsewhere; not the people you call when the building has collapsed.

WHY IT MATTERS IN PRODUCTION

Two mechanisms matter in practice. CSI snapshots are fast but stay on the same storage system, so for real durability you want file-system backup or a snapshot that is copied to object storage. And consistency is your responsibility: without hooks to quiesce the application, you get a crash-consistent copy that may or may not restore cleanly. Both details are where a confident-sounding backup strategy quietly fails.

STEP BY STEP

1. Define the scope: namespaced resources plus volume data, to and from object storage.
2. Choose the volume method deliberately - snapshot for speed, file-system backup for portability and durability.
3. Configure hooks for application consistency on stateful workloads.
4. Test restores into a scratch namespace regularly, and record the measured restore time.

EVIDENCE YOU WOULD QUOTE

```
oc get dataprotectionapplication,backupstoragelocation -n openshift-adp

oc get backup <name> -n openshift-adp -o jsonpath='{.status.phase} {.status.progress}{"\n"}'

oc get restore -n openshift-adp -o wide
```

RED FLAG - DO NOT SAY THIS

Do not present OADP as your disaster recovery plan for the cluster itself. It restores namespaces, not control planes.

EXPECT THIS FOLLOW-UP

Your OADP backups use CSI snapshots only. What risk have you accepted?

Q193 INTERMEDIATE

How do you design application high availability inside one cluster?

SAY THIS

Multiple replicas as the baseline, spread across failure domains with topology spread constraints so a node or zone loss cannot take them all. A PodDisruptionBudget that permits maintenance while protecting a minimum. Honest readiness probes so traffic never reaches a broken replica, and graceful shutdown so rollouts and drains do not drop connections. Then enough spare capacity that losing a node does not cause evictions elsewhere.

THINK OF IT LIKE

It is not enough to have three fire exits if they all open onto the same corridor. Spread matters as much as count.

WHY IT MATTERS IN PRODUCTION

The part teams miss is capacity headroom. Three replicas spread across three zones is genuinely resilient only if the cluster can still run all three when one zone is gone - otherwise the failover produces Pending pods and you have availability on paper only. The other missing piece is testing: draining a node during business hours, deliberately, is the cheapest resilience test available and almost nobody does it.

STEP BY STEP

1. Set replica minimums and spread across real failure domains, not just across nodes.
2. Add a PDB that permits maintenance while protecting a meaningful minimum.
3. Get probes and graceful shutdown right, because they govern behaviour during every disruption.
4. Maintain capacity headroom for the loss of a domain, and prove it by draining a node deliberately.

EVIDENCE YOU WOULD QUOTE

```
oc get deploy <name> -o jsonpath='{.spec.replicas}
{.spec.template.spec.topologySpreadConstraints}{"\n"}'

oc get pods -o wide -n <ns> | awk '{print $7}' | sort | uniq -c

oc adm drain <node> --dry-run=server --ignore-daemonsets
```

RED FLAG - DO NOT SAY THIS

Do not claim high availability from replica count alone. Three pods on one node is one node's worth of availability.

EXPECT THIS FOLLOW-UP

One zone goes down. Walk me through what happens to your three-replica service.

Q194 INTERMEDIATE

What is the etcd restore procedure and what does it cost you?

SAY THIS

It is a disruptive, last-resort operation. You stop the existing control plane static pods, restore the snapshot onto one control plane node using the supported restore script, bring that node up as the single member, then rejoin the others so they resynchronise. Once it completes, the cluster is at the snapshot's point in time - everything created since is gone, and certificates and tokens issued since may need attention.

THINK OF IT LIKE

It is restoring a building's records from last month's photocopy. Everything filed since then simply never happened, as far as the records are concerned.

WHY IT MATTERS IN PRODUCTION

The consequences are what make this a senior question. Objects created after the snapshot vanish, but the things they created in the outside world - cloud load balancers, volumes, DNS entries - still exist and are now orphaned. Nodes that joined after the snapshot are unknown to the restored cluster and may need to be removed and re-added. That is why an etcd restore is the option you reach for when there is genuinely nothing else.

STEP BY STEP

1. Confirm there is no less disruptive option - this is not a routine recovery method.
2. Follow the documented restore procedure on one control plane node, then rejoin the others.
3. Reconcile the aftermath: orphaned external resources, unknown nodes, certificate and token state.
4. Validate cluster operators, workloads and a real user path before declaring recovery.

EVIDENCE YOU WOULD QUOTE

```
oc debug node/<master> -- chroot /host /usr/local/bin/cluster-restore.sh <backup-dir>

oc get co ; oc get nodes ; oc get clusterversion

oc get csr | grep -i pending # nodes re-establishing identity after restore
```

RED FLAG - DO NOT SAY THIS

Do not describe etcd restore as a routine recovery step. It is disruptive, lossy and reserved for genuine cluster loss.

EXPECT THIS FOLLOW-UP

After a restore, three nodes are unknown to the cluster. What happened and what do you do?

Q195 INTERMEDIATE

How do you back up and restore a stateful application properly?

SAY THIS

I prefer the application's own mechanism where one exists - a database's native dump or continuous archiving is consistent by design and understood by the people who run it. That output is then captured and shipped off-cluster. Where the application has no native mechanism, I use volume backup with pre and post hooks that quiesce it. Either way, the backup includes the namespace's objects too, because data without its configuration is not a recovery.

THINK OF IT LIKE

You would not back up a bank by photographing the safe. You export the ledger, in a format the bank can read back.

WHY IT MATTERS IN PRODUCTION

Restore rehearsal is what makes this real, and it exposes things nobody predicted: the restore takes six hours rather than the assumed one, the application needs a secret that was never backed up, or the restored data is consistent but the schema version does not match the deployed image. All three are common, and all three are only ever discovered by actually doing it.

STEP BY STEP

1. Prefer the application's native backup mechanism and ship its output off-cluster.
2. Where none exists, use volume backup with quiesce hooks, and verify the hooks actually ran.
3. Include the namespace's objects and secrets, because data alone will not start the application.
4. Rehearse the full restore on a schedule and record the measured time and any gaps found.

EVIDENCE YOU WOULD QUOTE

```
oc get backup <name> -n openshift-adp -o jsonpath='{.status.hooks}' | python3 -m json.tool

Restore test log: start time, completion time, verification query result

oc get pvc,secret,cm -n <restored-ns> # completeness check after restore
```

RED FLAG - DO NOT SAY THIS

Do not back up a database by snapshotting its volume with no quiesce and call it done. It may restore, and you will not know until it matters.

EXPECT THIS FOLLOW-UP

Your restore works but takes six hours. Your RTO is two. What now?

Q196 SENIOR

How do you decide between an application restore and an etcd restore?

SAY THIS

By scope. If the damage is confined to one namespace or one application - deleted objects, corrupted data, a bad release - an application restore is targeted, fast and low risk. An etcd restore is only appropriate when the cluster's own state is lost or corrupted cluster-wide, because it rewinds everything and everyone. So my first question is always how wide the damage is, and my strong default is the narrowest recovery that fixes it.

THINK OF IT LIKE

One flat has flooded. You do not restore the whole building from last month's plans to fix one bathroom.

WHY IT MATTERS IN PRODUCTION

The pressure in a real incident pushes the other way, because an etcd restore feels decisive. Resisting that is the judgement being tested. It is also worth saying that the two are not mutually exclusive in a genuine disaster: you may restore the cluster state and then restore application data on top, and the order matters - cluster first, then data, then validation of a real user path.

STEP BY STEP

1. Establish the blast radius before choosing a mechanism.
2. Default to the narrowest recovery that addresses the damage.
3. Reserve etcd restore for cluster-wide state loss, and state its side effects out loud.
4. In a genuine disaster, sequence cluster state first, then application data, then end-to-end validation.

EVIDENCE YOU WOULD QUOTE

```
Scope assessment: affected namespaces, objects and data

oc get backup,restore -n openshift-adp

Recovery decision log with the reasoning recorded at the time
```

RED FLAG - DO NOT SAY THIS

Do not reach for etcd restore to fix a single namespace. You will turn one team's incident into everybody's.

EXPECT THIS FOLLOW-UP

A GitOps prune deleted twelve namespaces. Which mechanism, and why?

Q197 SENIOR

How do you test disaster recovery credibly?

SAY THIS

By running it, on a schedule, with timing. A credible test restores into a real target - a lab cluster or an isolated namespace - starts the application, runs a functional check, and measures the elapsed time against the stated RTO. It is run by someone other than the author of the runbook, because that is what exposes the missing steps. And the result, including failures, is recorded and reported.

THINK OF IT LIKE

It is a fire drill rather than a fire policy. Reading the evacuation plan proves nothing about whether the door at the end of the corridor is locked.

WHY IT MATTERS IN PRODUCTION

The specific value of having someone else run it is that runbooks are always written with assumed knowledge - a credential the author has, a step they do automatically, a system they know the address of. A fresh person hits every one of those in twenty minutes. The other value is the measured time, because stated RTOs are usually optimistic by a factor of two or three until someone has actually timed a restore.

STEP BY STEP

1. Schedule tests per critical service, with a defined scenario and success criteria.
2. Restore into a real target and run a functional verification, not just a status check.
3. Have someone other than the runbook author execute it, and capture every gap they hit.
4. Record measured RTO and RPO against target, report the gap, and fix the runbook immediately.

EVIDENCE YOU WOULD QUOTE

```
DR test register: date, scenario, executor, measured RTO, gaps found

Restore logs with start and completion timestamps

Runbook change history following each test
```

RED FLAG - DO NOT SAY THIS

Do not count a successful backup job as a DR test. Nothing has been proven until something has been restored and used.

EXPECT THIS FOLLOW-UP

Your measured RTO is three times the target. What do you tell the business?

Q198 SENIOR

How would you recover from complete loss of a cluster?

SAY THIS

By rebuilding rather than repairing. The realistic sequence is: provision a new cluster from infrastructure as code, restore platform configuration from Git through GitOps, restore application namespaces and data from OADP backups held off-site, repoint DNS and load balancers, then validate a real user journey before declaring recovery. This only works if the cluster was reproducible in the first place, which is the actual prerequisite.

THINK OF IT LIKE

It is rebuilding from the architectural plans rather than sifting the rubble. The plans have to exist before the fire.

WHY IT MATTERS IN PRODUCTION

That prerequisite is the point of the question. A cluster built by hand, with configuration that lives only in the cluster, cannot be recovered this way at all - and that is the moment people discover their GitOps coverage was partial. The other frequently missing piece is anything held outside Git and backups: image registry contents, secrets in an external vault, DNS records, load balancer configuration and certificates.

STEP BY STEP

1. Provision a replacement cluster from versioned infrastructure as code.
2. Restore platform configuration through GitOps, and confirm operators reach a healthy state.
3. Restore application namespaces and data from off-site backups, in dependency order.
4. Repoint DNS and load balancers, validate a real user journey, then review what was missing from the automation.

EVIDENCE YOU WOULD QUOTE

```
Infrastructure as code repository and last successful apply

GitOps repository coverage: what percentage of cluster configuration is in Git

Off-site backup inventory with restore test dates
```

RED FLAG - DO NOT SAY THIS

Do not assume the cluster can be rebuilt from Git without checking. Most estates have configuration that exists nowhere else.

EXPECT THIS FOLLOW-UP

What in your current cluster exists only inside the cluster? How would you find out?

Q199 ARCHITECT

How would you design multi-site disaster recovery for a critical service?

SAY THIS

From the RPO and RTO backwards. A warm standby cluster in a second site, with platform and application configuration delivered to both by GitOps so they never drift, and data replicated by the application's own mechanism where possible because it understands consistency better than storage replication does. Then DNS or global load balancing to shift traffic, a written and rehearsed failover decision process, and an explicit answer for how you fail back.

THINK OF IT LIKE

It is a second kitchen in another building, kept stocked and staffed. The point is not that it exists - it is that you have cooked a full service in it recently.

WHY IT MATTERS IN PRODUCTION

Two things reliably derail these designs. The first is failback, which is almost always harder than failover because data has diverged, and it is routinely left undesigned. The second is the human decision: who declares a disaster, on what evidence, and how quickly - because a technically perfect standby that nobody is authorised to activate at 2am does not meet any RTO. Both belong in the design, not in a later document.

STEP BY STEP

1. Derive the topology from RPO and RTO, and price each option honestly.
2. Deliver configuration to both sites from one Git source so they cannot drift.
3. Replicate data with the application's own mechanism where it exists, and measure the lag.
4. Write and rehearse the failover decision process, including authority, and design failback explicitly.

EVIDENCE YOU WOULD QUOTE

```
Replication lag metrics per dataset, against the stated RPO

GitOps sync status for both clusters from the same source repository

Failover rehearsal record with measured time and the decision timeline
```

RED FLAG - DO NOT SAY THIS

Do not design failover without designing failback. Getting back is usually the harder half and it is the half nobody rehearses.

EXPECT THIS FOLLOW-UP

You failed over successfully. Walk me through how you get back.

Q200 ARCHITECT

How do you agree and defend RPO and RTO targets with the business?

SAY THIS

By pricing the options rather than debating the numbers. I present two or three concrete architectures with their achievable RPO and RTO, their cost, and their operational burden, and let the business choose with the trade-off visible. Then I write the chosen numbers down as a commitment, test against them, and report the measured result - including when we miss - so the target stays honest rather than aspirational.

THINK OF IT LIKE

It is insurance. Nobody argues about the premium in the abstract; they choose a policy once they can see what each level of cover costs and excludes.

WHY IT MATTERS IN PRODUCTION

The failure mode is agreeing to an aspirational number that nobody funds. It survives until the first real incident, at which point the gap between the commitment and reality becomes an accountability problem. Reporting measured recovery times against target on a regular cadence keeps everyone honest, and it is also the most effective way to fund the improvements you actually need.

STEP BY STEP

1. Classify services into tiers rather than negotiating each one individually.
2. Present costed architecture options with achievable numbers for each.
3. Record the decision as a commitment with an owner, and design to it.
4. Test against the target and publish measured results, including misses, on a fixed cadence.

EVIDENCE YOU WOULD QUOTE

```
Service tier definitions with agreed RPO and RTO and named business owners

Costed options document with achievable numbers per option

Measured recovery times from tests, trended against target
```

RED FLAG - DO NOT SAY THIS

Do not agree to a target you cannot demonstrate. The first real incident will convert that into a much harder conversation.

EXPECT THIS FOLLOW-UP

The business will not fund the architecture their stated RTO requires. What do you do?

PART 14 · Q201-Q212 · 12 QUESTIONS

HPC and performance engineering

Latency-sensitive workloads, and why alignment beats tuning every time

High-performance workloads on OpenShift are less about making things faster and more about removing sources of variability. A trading engine, a real-time telecom function or a GPU training job does not need extra resources so much as it needs the same resources, on the same NUMA node, without being interrupted. That is why almost every answer in this part comes back to alignment: CPUs, memory, huge pages and devices all sitting on the same socket, with the scheduler refusing the pod rather than quietly placing it badly. Interviewers here reward measurement and caution far more than they reward knowing tuning parameters.

Level mix - Foundation: 3 · Intermediate: 4 · Senior: 3 · Architect: 2

Why an HPC pod is fast or slow: alignment, not tuning

Latency-sensitive workloads do not need more resources, they need the same NUMA node. Every layer here has to agree or you pay for a cross-socket hop on every packet.

Guaranteed QoS	Integer CPU requests equal to limits - the entry ticket for exclusive CPUs
CPU Manager	Static policy pins the container to dedicated cores; no noisy-neighbour throttling
Memory + huge pages	Pre-allocated huge pages on the same NUMA node reduce TLB misses
Device alignment	SR-IOV VF or GPU must sit on the same NUMA node as the CPUs and memory
Topology Manager	single-numa-node policy refuses the pod rather than silently misaligning it
PerformanceProfile	Isolated and reserved core split, kernel args and tuned profile, applied via MCO

Q201 FOUNDATION

What is NUMA, and why does it matter to a container?

SAY THIS

Non-Uniform Memory Access means a multi-socket server's memory is divided between sockets, and a CPU reaches its own socket's memory much faster than the other socket's. For most workloads the difference is invisible. For latency-sensitive ones it is not: a process pinned to socket zero reading memory attached to socket one pays a cross-socket penalty on every access, and the same applies to devices such as NICs and GPUs.

THINK OF IT LIKE

It is a library with two floors. Books on your floor take seconds; books on the other floor mean a trip up the stairs each time. Fine occasionally, disastrous as your main workflow.

WHY IT MATTERS IN PRODUCTION

The practical consequence is that performance can vary run to run for reasons invisible in any dashboard: the same pod, the same node, different NUMA placement, measurably different latency. That non-determinism is what Topology Manager exists to eliminate. For interview purposes, the sign of experience is talking about NUMA as a consistency problem rather than a raw speed one.

STEP BY STEP

1. Define NUMA as per-socket memory with asymmetric access cost.
2. Extend it to devices - NICs and accelerators are also attached to a specific socket.

3 Explain the symptom: run-to-run variability rather than uniform slowness.
4 Point forward to CPU Manager and Topology Manager as the mechanisms that make placement deterministic.

EVIDENCE YOU WOULD QUOTE

```
oc debug node/<node> -- chroot /host lscpu | grep -i numa

oc debug node/<node> -- chroot /host numactl --hardware

oc debug node/<node> -- chroot /host cat /sys/class/net/<nic>/device/numa_node
```

RED FLAG - DO NOT SAY THIS

Do not dismiss NUMA as irrelevant on modern hardware. For latency-sensitive workloads it is usually the largest single factor.

EXPECT THIS FOLLOW-UP

The same pod is 30% slower on some nodes than others. How does NUMA explain that?

Q202 FOUNDATION

What are huge pages and when do they help?

SAY THIS

Normal memory pages are four kilobytes, so a process using many gigabytes needs an enormous number of page-table entries and the translation lookaside buffer misses constantly. Huge pages - typically two megabytes or one gigabyte - reduce the number of entries dramatically, cutting TLB misses and page-table walk overhead. They help memory-intensive workloads with large working sets: databases, in-memory analytics, packet processing.

THINK OF IT LIKE

It is moving house with a few large crates instead of hundreds of small boxes. Fewer trips, less time spent checking labels.

WHY IT MATTERS IN PRODUCTION

The operational catch is that huge pages are pre-allocated on the node and reserved from general memory, so they are unavailable to everything else whether or not they are used. They are also NUMA-specific, which is why they belong in the same conversation as CPU pinning. And a pod must request them explicitly as a resource - a workload that would benefit but does not request them simply does not get them.

STEP BY STEP

1 Explain the TLB and page-table cost that huge pages reduce.
2 Name the workloads that benefit and those that do not.
3 Describe the pre-allocation trade-off: reserved memory is unavailable to other workloads.
4 Configure them per NUMA node through a PerformanceProfile, and have pods request them explicitly.

EVIDENCE YOU WOULD QUOTE

```
oc get node <node> -o jsonpath='{.status.allocatable}' | python3 -m json.tool | grep -i
hugepages

oc debug node/<node> -- chroot /host cat /proc/meminfo | grep -i huge

oc get pod <pod> -o jsonpath='{.spec.containers[0].resources}' | python3 -m json.tool
```

RED FLAG - DO NOT SAY THIS

Do not enable huge pages cluster-wide by default. You are reserving memory that most workloads cannot use.

EXPECT THIS FOLLOW-UP

You allocate huge pages and general workloads start getting evicted. Why?

Q203 FOUNDATION

What does the CPU Manager static policy do?

SAY THIS

With the static policy, containers in Guaranteed QoS pods that request whole CPUs get exclusive access to specific cores - they are pinned, and no other container is scheduled onto those cores. That removes CPU contention and the context switching that comes with sharing, which is what latency-sensitive workloads actually need. The requirements are strict: Guaranteed QoS, and integer CPU requests equal to limits.

THINK OF IT LIKE

It is a reserved parking space rather than a permit for the shared car park. Nobody else can be in it, so you never circle looking for a spot.

WHY IT MATTERS IN PRODUCTION

The requirement people trip over is the integer one - a request of 1.5 CPUs is not eligible for pinning, so a workload configured with fractional CPUs silently gets shared cores and the tuning appears to do nothing. The other consideration is capacity: pinned cores are removed from the shared pool, so a node running several pinned workloads has meaningfully less capacity for everything else, and that needs to be planned rather than discovered.

STEP BY STEP

1. State the eligibility rules: Guaranteed QoS and integer CPU request equal to limit.
2. Explain what exclusivity buys - no contention, no context switching, predictable latency.
3. Note the capacity cost: pinned cores leave the shared pool.
4. Enable it through a KubeletConfig or PerformanceProfile on a dedicated pool, and validate with measurements.

EVIDENCE YOU WOULD QUOTE

```
oc get kubeletconfig -o yaml | grep -i cpumanager

oc debug node/<node> -- chroot /host cat /var/lib/kubelet/cpu_manager_state

oc get pod <pod> -o jsonpath='{.status.qosClass}{"\n"}'
```

RED FLAG - DO NOT SAY THIS

Do not promise CPU pinning for a pod with fractional CPU requests. It will not be pinned and nobody will notice until performance testing.

EXPECT THIS FOLLOW-UP

A pod requests 1500m CPU and is not being pinned. Explain why.

Q204 INTERMEDIATE

What do Topology Manager policies do?

SAY THIS

Topology Manager coordinates the hints from CPU Manager, memory manager and device plugins so that a pod's CPUs, memory and devices come from the same NUMA node. The policies escalate in strictness: none does nothing, best-effort tries and continues either way, restricted rejects the pod if the preferred alignment cannot be met for a container, and single-numa-node requires everything on one node or the pod is not admitted.

THINK OF IT LIKE

It is seating a team together for a workshop. Best-effort means they will try; single-numa-node means if they cannot all sit together, the workshop does not happen.

WHY IT MATTERS IN PRODUCTION

For genuinely latency-sensitive workloads, single-numa-node is usually right, and the reason is counter-intuitive: you want the pod to fail admission rather than run misaligned. A pod that runs with cross-socket access produces intermittent latency that nobody can attribute, whereas a pod that refuses to schedule produces a clear event and a capacity conversation. Choosing predictable failure over silent degradation is the senior instinct here.

STEP BY STEP

1. Explain the hint-coordination role across CPU, memory and devices.
2. Walk the four policies and what each does when alignment is impossible.
3. Argue for single-numa-node on latency-critical pools, and say why failure is preferable to misalignment.
4. Expect and plan for admission failures as nodes fill unevenly, and monitor for them.

EVIDENCE YOU WOULD QUOTE

```
oc get kubeletconfig -o yaml | grep -i topologyManagerPolicy

oc describe pod <pod> | grep -i topology

oc get events -n <ns> --field-selector reason=TopologyAffinityError
```

RED FLAG - DO NOT SAY THIS

Do not choose best-effort for a workload with a hard latency requirement. Silent misalignment is worse than a clear rejection.

EXPECT THIS FOLLOW-UP

Pods start failing admission with a topology error. Is that a bug or working as intended?

Q205 INTERMEDIATE

What is a PerformanceProfile?

SAY THIS

A PerformanceProfile is a single declarative object, handled by the Node Tuning Operator, that configures a node pool for low-latency work: which cores are isolated for workloads and which are reserved for the system, huge page allocation per NUMA node, kernel arguments and optionally a real-time kernel, the CPU and Topology Manager policies, and the associated tuned profile. It renders into MachineConfigs, so applying it triggers a rolling reboot of the pool.

THINK OF IT LIKE

It is a single specification sheet for a class of machine, rather than a folder of individual work orders that might not agree with each other.

WHY IT MATTERS IN PRODUCTION

The two things to say about it operationally are that it applies per MachineConfigPool - so you create a dedicated pool for HPC nodes rather than applying it to all workers - and that it reboots nodes. That makes it a change-controlled operation with a validation step, not something to iterate on interactively. Reserved core sizing is the setting most often wrong: too few reserved cores and the kubelet and system daemons contend with the workload they were meant to serve.

STEP BY STEP

1. Describe what it configures and that it renders into MachineConfigs.
2. Apply it to a dedicated MachineConfigPool, never to all workers.
3. Size reserved versus isolated cores deliberately, leaving enough for system daemons.
4. Roll out to one node first, measure against a baseline, then widen.

EVIDENCE YOU WOULD QUOTE

```
oc get performanceprofile -o yaml | head -40

oc get mcp ; oc debug node/<node> -- chroot /host cat /proc/cmdline

oc debug node/<node> -- chroot /host tuned-adm active
```

RED FLAG - DO NOT SAY THIS

Do not apply a PerformanceProfile to the default worker pool. Every worker in the cluster will reboot to tune the four nodes that needed it.

EXPECT THIS FOLLOW-UP

How many cores would you reserve for the system, and how would you decide?

Q206 INTERMEDIATE

What does the Node Tuning Operator do, and when would you use a real-time kernel?

SAY THIS

The Node Tuning Operator manages tuned profiles across nodes, so kernel and sysctl settings are applied declaratively per node pool rather than by hand. It is also what implements PerformanceProfile. A real-time kernel goes further, trading throughput for determinism - it bounds worst-case scheduling latency, which matters for workloads with a hard deadline such as telecom signalling or industrial control.

THINK OF IT LIKE

A standard kernel is a commuter train optimised for total passengers carried. A real-time kernel is a courier that guarantees delivery within the hour, even if it carries less.

WHY IT MATTERS IN PRODUCTION

The decision hinges on whether the requirement is average performance or worst-case bounded latency. Most workloads described as latency-sensitive actually want good average latency, and a real-time kernel makes them slower overall for no benefit. Real-time is right when missing a deadline is a functional failure rather than a slow response, and that distinction is worth drawing out explicitly before agreeing to it.

STEP BY STEP

1. Describe tuned profile management and the relationship to PerformanceProfile.
2. Distinguish average latency requirements from hard deadline requirements.

3 Choose real-time only for bounded worst-case needs, and expect lower total throughput.
4 Apply to a dedicated pool, measure worst-case latency, and validate the workload actually benefits.

EVIDENCE YOU WOULD QUOTE

```
oc get tuned -A ; oc debug node/<node> -- chroot /host tuned-adm active

oc get performanceprofile <name> -o jsonpath='{.spec.realTimeKernel}{"\n"}'

Worst-case latency measurement, for example cyclictest results, before and after
```

RED FLAG - DO NOT SAY THIS

Do not deploy a real-time kernel because a workload is described as latency-sensitive. Ask whether a missed deadline is a failure or just a slow response.

EXPECT THIS FOLLOW-UP

How would you prove a workload actually needs a real-time kernel?

Q207 INTERMEDIATE

How do SR-IOV and RDMA help latency-sensitive workloads?

SAY THIS

SR-IOV lets a physical NIC present virtual functions that are assigned directly to a pod, so packets bypass the software networking stack entirely - lower latency, less jitter, less CPU spent on packet processing. RDMA goes further and lets one machine read or write another's memory without involving the remote CPU, which is what makes tightly coupled HPC and distributed training workloads scale.

THINK OF IT LIKE

SR-IOV is a private lane onto the motorway instead of queueing at the roundabout. RDMA is a conveyor belt directly into the other warehouse.

WHY IT MATTERS IN PRODUCTION

What you give up is worth stating, because it is significant. A pod on an SR-IOV interface is outside the normal cluster network for that traffic: no Service abstraction, no standard NetworkPolicy, IP addressing you manage, and a hard dependency on specific hardware and firmware. It also has to be NUMA-aligned with the CPUs to deliver the benefit, which brings you straight back to Topology Manager.

STEP BY STEP

1 Explain the mechanism: direct device assignment bypassing the software datapath.
2 State the requirements - supported NIC, BIOS and firmware settings, the SR-IOV operator.
3 Name what you lose: Services, standard policy, portability, simple IPAM.
4 Insist on NUMA alignment between the device and the pinned CPUs, and measure the gain.

EVIDENCE YOU WOULD QUOTE

```
oc get sriovnetworknodestate -n openshift-sriov-network-operator -o yaml | head -40

oc exec <pod> -- ip -br addr

Latency percentiles before and after, measured with a repeatable tool
```

RED FLAG - DO NOT SAY THIS

Do not adopt SR-IOV without measuring the current latency first. You may be paying a large complexity cost for a gain nobody can detect.

EXPECT THIS FOLLOW-UP

Which policy layer protects an SR-IOV interface, given NetworkPolicy does not?

Q208 SENIOR

How do you make performance-related node changes safely?

SAY THIS

By treating them like any other high-risk change, because a PerformanceProfile change reboots nodes. I measure a baseline first with a repeatable benchmark, apply the change to a dedicated pool with a single canary node, measure again with the same tool, and only widen when the improvement is real and nothing else regressed. Every change is one variable at a time, recorded, with a documented revert.

THINK OF IT LIKE

It is engine tuning on a dynamometer. Measure, change one thing, measure again. Change three things at once and you have learned nothing except that something happened.

WHY IT MATTERS IN PRODUCTION

The failure mode is a well-intentioned tuning session that applies several kernel parameters, isolates cores and enables huge pages in one change, after which the workload is faster and nobody knows which part mattered - or, worse, it is slower and nobody knows what to revert. Because each iteration costs a reboot cycle, the discipline of one variable at a time is not pedantry; it is the only way to converge.

STEP BY STEP

1. Establish a baseline with a repeatable benchmark and record the exact conditions.
2. Apply one change, to a dedicated pool, starting with a single canary node.
3. Re-measure with the identical method and compare, including the metrics you did not expect to move.
4. Record the result and the revert path, then widen the rollout or roll back deliberately.

EVIDENCE YOU WOULD QUOTE

```
Benchmark results before and after, same tool and parameters

oc get mcp -o wide during rollout ; node reboot timeline

oc debug node/<node> -- chroot /host cat /proc/cmdline # confirm the change applied
```

RED FLAG - DO NOT SAY THIS

Do not change several tuning parameters in one rollout. You will not be able to attribute the result in either direction.

EXPECT THIS FOLLOW-UP

Your change improved latency and increased CPU usage 20%. Ship it or not?

Q209 SENIOR

A latency-sensitive workload is missing its target. How do you investigate?

SAY THIS

I check alignment before I check tuning. Is the pod actually Guaranteed and actually pinned? Are its CPUs, memory and device on the same NUMA node? Is it sharing cores with system daemons because reserved cores were undersized? Then I look for interference - another workload on the node, interrupts landing on the isolated cores, throttling - and only then at the application itself and its dependencies.

THINK OF IT LIKE

Before adjusting the engine you check the handbrake is off. Most disappointing performance results are something not being applied at all.

WHY IT MATTERS IN PRODUCTION

In practice the most common finding is that the configuration people believed was in effect is not: fractional CPU requests so no pinning, a device on a different NUMA node than the pinned cores, or a PerformanceProfile that never finished rolling out because the pool is Degraded. Verifying what is actually true on the node - rather than what the manifest says - is the step that resolves most of these.

STEP BY STEP

1. Verify QoS class, pinning and actual CPU assignment on the node.
2. Verify NUMA alignment of CPUs, memory, huge pages and the device.
3. Look for interference: co-tenants, interrupts on isolated cores, throttling metrics.
4. Only then investigate the application and its dependencies, with per-hop measurement.

EVIDENCE YOU WOULD QUOTE

```
oc debug node/<node> -- chroot /host cat /var/lib/kubelet/cpu_manager_state

container_cpu_cfs_throttled_seconds_total{pod="<pod>"}

oc debug node/<node> -- chroot /host cat /proc/interrupts | head -20
```

RED FLAG - DO NOT SAY THIS

Do not start tuning kernel parameters before verifying the pod is pinned at all. That is how a week disappears.

EXPECT THIS FOLLOW-UP

The pod is Guaranteed but not pinned. Give me two possible reasons.

Q210 SENIOR

How do you validate that a performance change actually worked?

SAY THIS

By comparing against a recorded baseline using the same benchmark, the same load pattern and the same measurement points, and by looking at the distribution rather than an average. Percentiles matter most - p99 and p99.9 - because latency-sensitive workloads are judged on their tail. I also check what got worse: CPU cost, utilisation, another workload's behaviour, or the cluster's capacity for everything else.

THINK OF IT LIKE

It is a before-and-after photo taken from the same spot in the same light. Anything else is an impression, not a comparison.

WHY IT MATTERS IN PRODUCTION

The discipline of looking for regressions is what distinguishes this from marketing. A change that improves p99 latency by 15% while raising CPU consumption by 40% may be a good trade or a terrible one depending on the cluster's capacity, and the only way to have that conversation is to have measured both. Recording the result - including the conditions - also means the next person does not repeat the experiment.

STEP BY STEP

1. Use the identical benchmark, load and measurement points as the baseline.
2. Compare percentiles and the full distribution, not averages.
3. Explicitly check for regressions in CPU, memory, capacity and neighbouring workloads.
4. Record the result with conditions and conclusions, and keep it with the change record.

EVIDENCE YOU WOULD QUOTE

```
Baseline and post-change percentile latency, same tool and parameters

Node CPU and memory utilisation before and after

Neighbouring workload SLI over the same window
```

RED FLAG - DO NOT SAY THIS

Do not report an average improvement for a latency-sensitive workload. The tail is the thing users experience.

EXPECT THIS FOLLOW-UP

p99 improved but p50 got worse. What would you conclude?

Q211 ARCHITECT

How would you design a node pool for HPC or AI workloads?

SAY THIS

As a dedicated pool with its own MachineConfigPool, taints so only opted-in workloads land there, and a PerformanceProfile setting isolated and reserved cores, huge pages per NUMA node and the topology policy. Hardware chosen for the workload - accelerators and NICs on the sockets the workload will use - with the device plugins and SR-IOV configuration to expose them. Then a validation suite that runs after every change so the pool's behaviour is a known quantity.

THINK OF IT LIKE

It is a specialist workshop rather than a corner of the general factory floor. Different tools, different rules, different people allowed in.

WHY IT MATTERS IN PRODUCTION

The dedicated pool is not primarily about performance, it is about change isolation: kernel arguments, real-time settings and reboot cycles that are appropriate for HPC nodes would be reckless applied to the whole worker fleet. It also lets you upgrade and validate the pool independently, which matters because performance-sensitive nodes are exactly where you want the longest soak time before rolling a change further.

STEP BY STEP

1. Separate into a dedicated MachineConfigPool with taints and labels for explicit opt-in.
2. Specify hardware with NUMA topology in mind, including accelerator and NIC placement.
3. Configure PerformanceProfile, device plugins and SR-IOV, and document the intended topology.
4. Build a validation suite - latency, throughput, alignment checks - and run it after every change.

EVIDENCE YOU WOULD QUOTE

```
oc get mcp ; oc get nodes -l node-role.kubernetes.io/hpc -o wide

oc get performanceprofile,sriovnetworknodepolicy -o wide

Validation suite results per node after the last change
```

RED FLAG - DO NOT SAY THIS

Do not mix HPC and general workloads on the same pool. The tuning that helps one actively harms the other.

EXPECT THIS FOLLOW-UP

How do you stop general workloads landing on your expensive GPU nodes?

Q212 ARCHITECT

How do you balance performance isolation against cluster utilisation?

SAY THIS

By being explicit that isolation costs utilisation and deciding where that trade is worth it. Pinned cores, reserved huge pages and dedicated pools all reduce how much of the hardware is usable by anything else, and for a genuinely latency-critical service that is a reasonable price. For everything else it is waste. So I tier workloads, apply isolation only to the tier that needs it, and report utilisation per pool so the cost is visible rather than assumed.

THINK OF IT LIKE

It is reserved seating in a restaurant. Worth it for the regular who books every Friday; ruinous if you reserve every table for people who might turn up.

WHY IT MATTERS IN PRODUCTION

The conversation this enables is the useful one: a workload owner asking for dedicated nodes can be shown what that costs in utilisation terms and asked to justify it with measured latency requirements. Often the honest answer is that the workload has never been measured and the request is precautionary - and measuring first resolves it without spending anything.

STEP BY STEP

1. Tier workloads by measured latency requirement, not by team preference.
2. Apply isolation only to the top tier, and document what it costs in usable capacity.
3. Report utilisation per pool so the cost of isolation is visible to the people requesting it.
4. Require measurement before granting isolation, and review allocations as workloads change.

EVIDENCE YOU WOULD QUOTE

```
Utilisation per node pool: allocatable, requested, used

Latency measurements justifying each isolated workload

Cost per pool, compared against the general worker pool
```

RED FLAG - DO NOT SAY THIS

Do not grant dedicated nodes on request without measurement. You will end up with an expensive, half-idle cluster and no way to argue it back.

EXPECT THIS FOLLOW-UP

A team wants dedicated nodes but has never measured their latency. What do you say?

PART 15 · Q213-Q226 · 14 QUESTIONS

Automation, GitOps and change control

Making change repeatable, reviewable and reversible - and knowing what not to automate

Automation questions are really change-management questions. The interviewer is not checking whether you can write a playbook; they are checking whether changes to your platform have an author, a reviewer, a timestamp and a way back. That is why GitOps comes up so often, and why the best answers talk about drift detection, emergency changes and secret handling rather than about tools. The other thing being assessed is judgement about scope: automating the wrong thing, or automating something nobody understands manually yet, creates a machine for producing incidents at speed.

Level mix - Foundation: 4 · Intermediate: 5 · Senior: 3 · Architect: 2

Change control that survives an audit

The point of GitOps is not the tool, it is that every cluster change has an author, a reviewer, a timestamp and a revert path.

Q213 FOUNDATION

What is idempotency and why does it matter in automation?

SAY THIS

An idempotent operation produces the same result whether you run it once or fifty times. In practice it means describing the desired state - this package installed, this file with this content, this object present - rather than the actions to get there. It matters because automation gets re-run: after a partial failure, during a retry, on a schedule. Non-idempotent automation turns a re-run into a new incident.

THINK OF IT LIKE

"Make sure there are three chairs in the room" is idempotent. "Add three chairs" is not, and after four runs the room is unusable.

WHY IT MATTERS IN PRODUCTION

This is also why declarative tools are preferred for infrastructure: Kubernetes objects, Ansible modules that describe state, Terraform resources. The classic failure is a shell script that appends a line to a configuration file - run it three times and the file has three copies, and the failure appears days later when something parses it. Testing automation by running it twice in a row is a trivially cheap check that catches most of these.

STEP BY STEP

1. Define idempotency and connect it to describing state rather than actions.
2. Give the appending-to-a-file example as the canonical failure.
3. Prefer declarative modules and objects over imperative commands.
4. Test by running twice and asserting that the second run reports no change.

EVIDENCE YOU WOULD QUOTE

```
ansible-playbook site.yml --check --diff # second run should show no changes

oc apply -f manifests/ --dry-run=server

CI job that applies the same change twice and asserts a no-op
```

RED FLAG - DO NOT SAY THIS

Do not present a script that only works on a clean system as automation. Real systems are never clean and re-runs always happen.

EXPECT THIS FOLLOW-UP

How would you make a config-file-appending script idempotent?

Q214 FOUNDATION

What is GitOps and what problem does it solve?

SAY THIS

GitOps means the desired state of the cluster lives in Git, and a controller in the cluster continuously reconciles reality toward it. The problem it solves is not deployment - it is accountability and recoverability. Every change has an author, a review, a timestamp and a revert. Drift is detected because the controller keeps comparing. And rebuilding a cluster becomes pointing the controller at the repository rather than remembering what was done.

THINK OF IT LIKE

It is the difference between a building maintained by whoever was on shift and one maintained to a documented specification that is checked weekly.

WHY IT MATTERS IN PRODUCTION

The audit and recovery benefits are what make this land with senior interviewers. "Who changed this and why" becomes a Git question rather than an investigation. "Can we rebuild this cluster" becomes a test rather than a hope. The caveat worth adding is coverage: GitOps only helps for what is actually in Git, and most estates have configuration that lives only in the cluster until somebody checks.

STEP BY STEP

1. Define the model: Git as desired state, an in-cluster controller reconciling continuously.
2. Name the real benefits - attribution, review, revert, drift detection, reproducibility.
3. Note that manual changes are reverted, which is the feature and the friction.
4. Measure coverage: what fraction of cluster configuration is actually in Git.

EVIDENCE YOU WOULD QUOTE

```
oc get application -n openshift-gitops -o wide

oc get application <app> -n openshift-gitops -o jsonpath='{.status.sync.status}
{.status.health.status}{"\n"}'

Git history for the manifest in question
```

RED FLAG - DO NOT SAY THIS

Do not describe GitOps as just a deployment method. The value is in change control, and that is what the question is about.

EXPECT THIS FOLLOW-UP

What in your cluster is not in Git today, and how would you find out?

Q215 FOUNDATION

What is Ansible check mode and when do you use it?

SAY THIS

Check mode runs a playbook without making changes and reports what would have changed, especially useful combined with diff output. You use it to preview a change before a maintenance window, to validate that a playbook is genuinely idempotent - a second run in check mode should report nothing - and as a drift detector, since a check-mode run against production that reports changes means something has drifted.

THINK OF IT LIKE

It is a dress rehearsal with no audience. You find out what is missing without anyone seeing it go wrong.

WHY IT MATTERS IN PRODUCTION

The limitation to state honestly is that check mode is not a perfect simulation: modules that depend on the results of earlier tasks, or custom modules that do not implement check mode, can report inaccurately. So it is a strong signal rather than a guarantee, and it does not replace testing in a non-production environment. Using it as a scheduled drift detector is the underused application.

STEP BY STEP

1. Explain what check mode does and pair it with diff for readable output.
2. Use it as a pre-change preview and as an idempotency test.
3. Run it on a schedule against production as a drift detector.
4. State the limitation - dependent tasks and modules without check support can mislead.

EVIDENCE YOU WOULD QUOTE

```
ansible-playbook site.yml --check --diff --limit <host>

Scheduled check-mode run output, compared over time

Molecule or equivalent test results for the role
```

RED FLAG - DO NOT SAY THIS

Do not treat a clean check-mode run as proof it is safe. It is evidence, not a guarantee.

EXPECT THIS FOLLOW-UP

Check mode reports changes on a host you have not touched. What does that mean?

Q216 FOUNDATION

What is an Ansible execution environment?

SAY THIS

It is a container image containing the Ansible runtime, the collections and the Python dependencies your automation needs. Instead of every control node and laptop having a slightly different set of versions, the automation runs inside a versioned image that is identical everywhere. It makes runs reproducible, makes dependency upgrades a deliberate change, and removes the entire class of "it works on my machine" failures.

THINK OF IT LIKE

It is a toolbox that travels with the job, rather than hoping the site has the right spanner.

WHY IT MATTERS IN PRODUCTION

The operational benefit is that dependency changes become reviewable. Upgrading a collection is a new image with a new tag, tested in a pipeline, promoted deliberately - rather than someone running a package upgrade on the control node and discovering three playbooks now behave differently. In disconnected environments it is close to mandatory, because it removes the need to fetch collections at runtime.

STEP BY STEP

1. Define it as a versioned container image carrying runtime, collections and dependencies.
2. Explain reproducibility across control nodes, CI and developer machines.
3. Build and promote images through a pipeline, pinned by digest.
4. Note the disconnected benefit: no runtime dependency fetching.

EVIDENCE YOU WOULD QUOTE

```
ansible-builder build -t ee-platform:1.4

ansible-navigator run site.yml --execution-environment-image ee-platform:1.4

Image inventory: which execution environment each job template uses
```

RED FLAG - DO NOT SAY THIS

Do not run production automation from a laptop's local Ansible install. Nobody can reproduce what you ran.

EXPECT THIS FOLLOW-UP

Two engineers get different results from the same playbook. How does this fix it?

Q217 INTERMEDIATE

What is drift, and how does GitOps handle it?

SAY THIS

Drift is the difference between what Git says should exist and what actually exists, usually created by someone making a manual change. A GitOps controller detects it continuously by comparing, and reports the application as OutOfSync. Whether it corrects it depends on configuration: with automated self-healing it reverts the change; without it, it reports and waits for a human decision.

THINK OF IT LIKE

It is a stocktake that runs every few minutes. Whether it puts things back on the shelf or just tells you they moved is your policy choice.

WHY IT MATTERS IN PRODUCTION

The self-healing choice is a genuine trade-off. Automatic reversion enforces the model strictly and is right for platform configuration, but it will fight anyone making an emergency change and it will fight an HPA over replica count unless that field is excluded. Reporting only preserves human control at the cost of drift persisting quietly. Most mature setups use self-healing for platform repositories and reporting for application ones, and are explicit about which is which.

STEP BY STEP

1. Define drift and how continuous comparison detects it.
2. Explain self-healing versus report-only and where each is appropriate.
3. Configure field-level exclusions for things legitimately managed elsewhere, such as HPA replica counts.
4. Alert on sustained OutOfSync so drift is noticed rather than accumulating.

EVIDENCE YOU WOULD QUOTE

```
oc get application -A -o
custom-columns=NAME:.metadata.name,SYNC:.status.sync.status,HEALTH:.status.health.status

oc get application <app> -n openshift-gitops -o jsonpath='{.spec.syncPolicy}' | python3 -m
json.tool

Argo CD diff view for the OutOfSync resource
```

RED FLAG - DO NOT SAY THIS

Do not enable self-healing on a repository whose replica counts are managed by an HPA. The two will fight continuously.

EXPECT THIS FOLLOW-UP

Argo CD and the HPA are fighting over replicas. How do you fix it properly?

Q218 INTERMEDIATE

What are sync waves and why do you need ordering?

SAY THIS

Sync waves let you order the application of resources so dependencies exist before the things that need them - namespaces and CRDs before the custom resources that use them, secrets and config before the workloads that mount them, database migrations before the application that expects the new schema. Without ordering, a first-time apply fails on resources whose prerequisites have not been created yet.

THINK OF IT LIKE

It is a construction schedule. Foundations, then walls, then roof. The materials list does not tell you the order, and the order is what makes it stand up.

WHY IT MATTERS IN PRODUCTION

The tell-tale symptom of missing ordering is an apply that fails the first time and succeeds on retry, because by then the prerequisites exist. That is a genuine bug even though it looks harmless, because it means a fresh cluster cannot be built from the repository in one pass - which is exactly the property you rely on during a recovery. Hooks are the related tool for one-off tasks such as migrations.

STEP BY STEP

1. Explain waves as explicit ordering annotations applied to resources.
2. Give concrete dependency examples: CRDs, namespaces, secrets, migrations.
3. Name the symptom of missing ordering: fails once, succeeds on retry.
4. Test by applying to an empty cluster in a single pass, which is the recovery scenario.

EVIDENCE YOU WOULD QUOTE

```
grep -r 'argocd.argoproj.io/sync-wave' manifests/ | head

oc get application <app> -o jsonpath='{.status.operationState.phase}{"\n"}'

Fresh-cluster bootstrap test result
```

RED FLAG - DO NOT SAY THIS

Do not accept "it works on the second sync". It means you cannot rebuild from scratch, which is the whole point.

EXPECT THIS FOLLOW-UP

Your bootstrap fails on the first apply and succeeds on the second. Why does that matter?

Q219 INTERMEDIATE

How would you structure an enterprise Ansible automation framework?

SAY THIS

Roles and collections for reusable logic, kept small and single-purpose; inventories and group variables that separate environment data from logic; versioned execution environments; secrets from a vault rather than from files; and job templates in a controller with role-based access, surveys for parameters, and scheduling. Everything is in Git, tested in CI with linting and Molecule, and promoted through environments in the same way application code is.

THINK OF IT LIKE

It is a professional workshop: standard parts, labelled drawers, a jobs board and a sign-out sheet - not a shed where each person keeps their own tools.

WHY IT MATTERS IN PRODUCTION

The organisational benefit is bigger than the technical one. A controller with job templates means a service desk or an application team can run a vetted operation without having cluster credentials, which removes a large category of privilege that would otherwise be granted permanently. It also produces an audit trail of who ran what, which is usually the thing that makes the compliance team stop asking questions.

STEP BY STEP

1. Separate logic from data: roles and collections versus inventories and group variables.
2. Version execution environments and promote them like any other artefact.
3. Source secrets from a vault at runtime, never from the repository.
4. Publish operations as controller job templates with RBAC, surveys and an audit trail.

EVIDENCE YOU WOULD QUOTE

```
Repository structure: collections, roles, inventories, molecule scenarios

CI results: ansible-lint, yamllint, molecule converge and idempotence

Controller job history with initiating user and outcome
```

RED FLAG - DO NOT SAY THIS

Do not keep environment-specific values inside roles. It guarantees a role that only works in the environment it was written for.

EXPECT THIS FOLLOW-UP

How would you let an application team restart a service without giving them cluster access?

Q220 INTERMEDIATE

How do you write safe shell automation for cluster operations?

SAY THIS

Fail fast and loudly - set -euo pipefail, quote everything, and check that required tools and context exist before acting. Confirm which cluster you are pointed at, because the worst shell incidents are correct scripts run against the wrong context. Support a dry-run mode, act on a narrow selector rather than everything, log what is being done, and make it idempotent so a re-run after failure is safe.

THINK OF IT LIKE

It is a power tool with a guard and a trigger lock. The tool is fine; the missing safety features are what remove fingers.

WHY IT MATTERS IN PRODUCTION

The single highest-value safety feature is the context check. A script that deletes resources matching a label is completely reasonable in a development cluster and catastrophic in production, and the difference is one environment variable nobody looked at. Printing the cluster name and requiring confirmation for destructive actions costs three lines and prevents the incident that ends careers.

STEP BY STEP

1. Start with strict mode, quoting and explicit dependency checks.
2. Verify and print the target cluster context, and require confirmation for destructive operations.
3. Provide a dry-run mode and act on narrow, explicit selectors.
4. Log actions with timestamps and make the script safe to re-run after a partial failure.

EVIDENCE YOU WOULD QUOTE

```
set -euo pipefail ; oc whoami --show-server

shellcheck script.sh

Script log output showing dry-run and confirmed-run modes
```

RED FLAG - DO NOT SAY THIS

Do not write a destructive script without a cluster context check. Everyone who has skipped it has a story about it.

EXPECT THIS FOLLOW-UP

What is the single most valuable safety line in a cluster automation script?

Q221 INTERMEDIATE

When and how would you use Python for OpenShift automation?

SAY THIS

When the task needs real logic - conditional branching, data transformation, calling several APIs and correlating the results, or generating reports. I use the official Kubernetes client, authenticate through the in-cluster ServiceAccount when it runs as a Job, handle errors and retries explicitly with backoff, and treat it like production code: version control, tests, linting and a pinned dependency set in a container image.

THINK OF IT LIKE

Shell is a screwdriver and Ansible is a power tool. Python is the workshop you build when the job needs measuring, cutting and fitting rather than turning one screw.

WHY IT MATTERS IN PRODUCTION

The boundary worth stating is that Python should not be a way to reimplement declarative operations. Applying manifests is better done by GitOps; configuring nodes is better done by MachineConfig. Python earns its place for reporting, reconciliation checks, cross-system integration and one-off analyses - and the moment it starts continuously reconciling state, you have written an operator and should treat it as one.

STEP BY STEP

1. Choose Python when logic, correlation or data transformation is genuinely needed.
2. Use the official client and in-cluster ServiceAccount authentication with least privilege.
3. Handle API errors, rate limits and retries explicitly with backoff.
4. Package it as a container image with pinned dependencies, tests and a CI pipeline.

EVIDENCE YOU WOULD QUOTE

```
Repository with tests, lint configuration and pinned requirements

oc get cronjob <automation-job> -o yaml # runs in-cluster with a scoped ServiceAccount

oc auth can-i --list --as=system:serviceaccount:<ns>:<sa>
```

RED FLAG - DO NOT SAY THIS

Do not write Python that applies manifests in a loop. You have rebuilt GitOps without the audit trail.

EXPECT THIS FOLLOW-UP

At what point does your Python script become an operator?

Q222 SENIOR

An emergency change needs to be made and GitOps keeps reverting it. What do you do?

SAY THIS

I stop fighting the controller. The clean options are to make the change in Git and sync it immediately - which is often quicker than people assume - or, if Git is unavailable or the change genuinely cannot wait, to suspend automated sync for that application, apply the change with a documented reason, and open the corresponding pull request straight away so the emergency state has an expiry.

THINK OF IT LIKE

It is breaking the glass on a fire alarm. Legitimate, loud, and it leaves visible evidence that someone has to account for afterwards.

WHY IT MATTERS IN PRODUCTION

The real risk is the forgotten suspension. An application left with sync disabled after an incident drifts silently for weeks, and nobody discovers it until a rebuild fails or a config nobody remembers turns out to be load-bearing. Alerting on applications with sync disabled, and reviewing them in the incident follow-up, is what makes the emergency path safe to use.

STEP BY STEP

1. Try the fast path first: commit and sync, which is often quicker than the workaround.
2. If not possible, suspend sync for that specific application only, with a recorded reason.
3. Apply the change, and open the pull request immediately so the state is temporary.
4. Alert on suspended applications and clear them as part of the incident follow-up.

EVIDENCE YOU WOULD QUOTE

```
oc get application -A -o json | jq
'.items[]|select(.spec.syncPolicy.automated==null)|.metadata.name'

Incident record with the emergency change and its reason

Pull request linking back to the incident
```

RED FLAG - DO NOT SAY THIS

Do not disable sync cluster-wide to get past one change. You have turned off change control for everything to fix one thing.

EXPECT THIS FOLLOW-UP

Two weeks later you find an application still has sync disabled. What now?

Q223 SENIOR

How do you handle secrets in a GitOps workflow?

SAY THIS

Secrets do not go in Git in plaintext, and I prefer them not to go in Git at all. The cleanest pattern is an external secret manager with an operator that pulls values into Kubernetes Secrets at runtime, so Git holds only a reference. Where an external manager is not available, sealed secrets encrypted to a cluster-held key are an acceptable second choice - as long as the key management story is written down and the key itself is backed up somewhere other than the cluster.

THINK OF IT LIKE

The recipe book can say "add the house spice mix". It should not print the formula, and it definitely should not print it in a code that everyone on the team can read.

WHY IT MATTERS IN PRODUCTION

The reason to prefer external references is rotation. With a sealed secret, rotating a credential means re-encrypting and committing, so rotation is a code change and therefore rarely happens. With an external manager, rotation happens in the manager on a schedule and the cluster picks it up, which is the behaviour you actually want. The gap that remains in both cases is whether the workload reloads or needs a restart.

STEP BY STEP

1. Keep plaintext secrets out of Git entirely; store references instead.
2. Prefer an external manager with an operator syncing into Kubernetes Secrets.
3. If using sealed secrets, document key management and back the key up off-cluster.
4. Define the rotation path per workload, including whether a restart is required.

EVIDENCE YOU WOULD QUOTE

```
oc get externalsecret,secretstore -A

git log --all -p | grep -i -c 'BEGIN PRIVATE KEY' # should be zero

Rotation schedule and last-rotated timestamps per secret class
```

RED FLAG - DO NOT SAY THIS

Do not commit encrypted secrets without a key management plan. If the key lives only in the cluster you cannot recover, you have encrypted nothing useful.

EXPECT THIS FOLLOW-UP

Your sealing key is lost with the cluster. What can you recover?

Q224 SENIOR

How do you test automation before it touches production?

SAY THIS

In layers. Static checks first - linting, schema validation, policy checks in CI. Then functional tests against an ephemeral or non-production cluster, including running the automation twice to prove idempotency. Then a canary: apply to one namespace, one node or one cluster and validate before widening. And for destructive operations, a dry-run mode that is exercised in the pipeline rather than only documented.

THINK OF IT LIKE

It is testing a new medicine: laboratory, then trial, then limited release, then general availability. Nobody skips to the last step because they are confident.

WHY IT MATTERS IN PRODUCTION

The step most often skipped is the idempotency run, and it is the cheapest of the lot. Running the same automation twice and asserting no changes on the second pass catches a large fraction of real bugs before they reach anything important. The canary step is the one that saves you from the bugs testing cannot catch, because production always has something the test environment does not.

STEP BY STEP

1. Run static analysis and policy checks on every commit.
2. Test functionally against a non-production cluster, and run twice to assert idempotency.
3. Canary to a limited scope and validate before widening.
4. Exercise dry-run paths in CI so they are known to work when someone needs them.

EVIDENCE YOU WOULD QUOTE

```
CI pipeline stages and their pass rates

Ephemeral cluster test results including the idempotency run

Canary scope definition and validation criteria per automation
```

RED FLAG - DO NOT SAY THIS

Do not rely on review alone for automation that changes many things at once. Reviews catch intent errors, not behaviour.

EXPECT THIS FOLLOW-UP

Your automation passed every test and broke production. What layer was missing?

Q225 ARCHITECT

How do you decide what to automate and what to leave manual?

SAY THIS

I automate work that is frequent, well understood and low variance - node scaling, certificate renewal, namespace creation, routine checks. I leave manual, for now, anything rare, poorly understood, or where the cost of an automated mistake is very high and recovery is slow. And I never automate a procedure nobody can perform by hand, because when the automation fails - and it will - somebody needs to be able to finish the job.

THINK OF IT LIKE

You automate the assembly line, not the emergency surgery. And you keep surgeons who can still operate when the machine is down.

WHY IT MATTERS IN PRODUCTION

The frequency-times-risk framing is what makes this defensible in a design review. High frequency and low risk is obvious automation. Low frequency and high risk usually deserves a good runbook and a rehearsal instead, because automation that runs twice a year is never tested and rots quietly. The interesting middle is high frequency and high risk, where the answer is usually to automate with strong guardrails and a human approval gate.

STEP BY STEP

1. Score candidate tasks by frequency, risk and variance.
2. Automate high-frequency low-risk work first; it pays back fastest and builds confidence.
3. For high-risk work, automate with guardrails and an approval gate rather than fully autonomously.
4. Keep runbooks current for the manual path, and rehearse them so the skill survives.

EVIDENCE YOU WOULD QUOTE

```
Task inventory with frequency, duration and risk rating

Toil measurement: hours per week spent on repeated manual work

Runbook currency: last reviewed and last rehearsed dates
```

RED FLAG - DO NOT SAY THIS

Do not automate a rare, high-risk procedure that nobody has performed manually. You have built something untested that runs when you are least able to check it.

EXPECT THIS FOLLOW-UP

Which of your current manual tasks would you automate first, and why that one?

Q226 ARCHITECT

How would you design change control for the platform end to end?

SAY THIS

Everything that defines the platform lives in Git. Changes arrive as pull requests with automated validation - lint, policy, dry-run - and human review proportional to risk. Promotion runs through environments in a fixed order with a soak period. Deployment is by GitOps so the applied state is always the reviewed state. Emergency changes have a defined break-glass path that is visible and expires. And the evidence for an auditor is the Git history plus the controller's sync history.

THINK OF IT LIKE

It is planning permission for a building. Proposal, review proportional to the size of the change, inspection, and a record anyone can look up years later.

WHY IT MATTERS IN PRODUCTION

The design decision that determines whether people follow this is proportionality. If a one-line label change requires the same approval as a network redesign, engineers route around the process and the audit trail becomes fiction. Tiering changes by blast radius - auto-merge for low-risk, peer review for medium, change board for high - keeps the process credible and keeps the evidence real.

STEP BY STEP

1. Put all platform configuration in Git and define the repository structure and ownership.
2. Automate validation in CI and tier human review by blast radius.
3. Promote through environments in a fixed order with a defined soak period.
4. Define and monitor the break-glass path, and use Git plus sync history as the audit evidence.

EVIDENCE YOU WOULD QUOTE

```
Pull request history with review evidence and CI results

GitOps sync history per environment with timestamps

Break-glass usage log with reasons and follow-up pull requests
```

RED FLAG - DO NOT SAY THIS

Do not apply the same heavyweight approval to every change. People will bypass it, and then you have no change control at all.

EXPECT THIS FOLLOW-UP

How would you prove to an auditor that no unreviewed change reached production?

PART 16 · Q227-Q238 · 12 QUESTIONS

Fleet operations with Advanced Cluster Management

Doing everything you have just learned, fifty times, without doing it fifty times

Once an organisation has more than a handful of clusters, the interesting questions change. It is no longer "how do I configure this" but "how do I know every cluster is configured this way, and what do I do about the three that are not". That shift - from cluster administration to fleet operations - is what this part is about, and it is where interviews for senior platform roles usually end up. The test is simple: if your answer only works when you log into a cluster, it does not scale to the estate you are being hired to run.

Level mix - Foundation: 3 · Intermediate: 4 · Senior: 3 · Architect: 2

ACM: one hub, many clusters, four jobs

Fleet questions are really about repeatability. If your answer only works when you log into a cluster, it does not scale to fifty of them.

Hub cluster	Runs the multicluster engine, policy controllers, observability and Argo CD
Klusterlet	Agent on each managed cluster; pulls work, reports status back to the hub
Lifecycle	Create, import, upgrade, hibernate and detach clusters from one place
Governance	Policies in inform mode report drift; enforce mode remediates it
Placement	Label-driven targeting: which clusters get this policy, app or upgrade wave
Observability	Metrics and alerts aggregated centrally with per-cluster drill-down

Q227 FOUNDATION

What is Advanced Cluster Management and what does the hub do?

SAY THIS

ACM is Red Hat's fleet management layer. One hub cluster runs the management components, and every other cluster is imported or created as a managed cluster. From the hub you get cluster lifecycle - create, import, upgrade, hibernate, detach - policy-based governance, application placement and delivery, and aggregated observability. The point is to make operations declarative and label-driven rather than repeated per cluster.

THINK OF IT LIKE

It is head office for a chain of shops. Each shop runs itself day to day; head office sets the standards, sees the numbers, and rolls out changes.

WHY IT MATTERS IN PRODUCTION

The design consequence people forget is that the hub is itself a critical cluster. If it is down, managed clusters keep running - workloads are unaffected - but you lose central governance, observability and the ability to roll out changes. So the hub needs the same availability treatment as anything else critical, and it is worth being explicit about which operations degrade when it is unavailable.

STEP BY STEP

1 Describe the hub and spoke model and the four capability areas.

2 State the failure behaviour: managed clusters keep serving, central operations stop.
3 Treat the hub as a critical cluster with its own HA and DR plan.
4 Emphasise label-driven targeting as the mechanism that makes it scale.

EVIDENCE YOU WOULD QUOTE

```
oc get managedcluster ; oc get multiclusterhub -A

oc get managedcluster -o custom-columns=NAME:.metadata.name,AVAILABLE:.status.conditions[?(@.ty
pe=="ManagedClusterConditionAvailable")].status

oc -n open-cluster-management get pods | head
```

RED FLAG - DO NOT SAY THIS

Do not describe ACM as a dashboard. It is a control plane for the fleet, and the interesting questions are about what it enforces.

EXPECT THIS FOLLOW-UP

The hub is down for four hours. What actually stops working?

Q228 FOUNDATION

What is the klusterlet and how does a cluster become managed?

SAY THIS

The klusterlet is the agent installed on each managed cluster. It registers with the hub, pulls work - policies, application manifests, upgrade instructions - and reports status back. The connection is initiated outbound from the managed cluster, which matters for network design because the hub does not need inbound access to every cluster. Importing a cluster is essentially installing and registering that agent.

THINK OF IT LIKE

It is the branch manager who phones head office for instructions and reports the numbers. Head office never has to phone the branch.

WHY IT MATTERS IN PRODUCTION

The outbound-only model is worth knowing because it makes ACM viable in network topologies where the hub could never reach the managed clusters directly - clusters behind NAT, in customer environments, or at edge sites on intermittent links. It also means that when a cluster appears unavailable in the hub, the first thing to check is the klusterlet's connectivity outbound, not the hub's ability to reach it.

STEP BY STEP

1 Define the klusterlet as the pull-based agent on each managed cluster.
2 Explain the outbound-only connection model and why it matters for network design.
3 Describe import: install the agent, register, accept on the hub.
4 For an unavailable cluster, check klusterlet pods and outbound connectivity first.

EVIDENCE YOU WOULD QUOTE

```
oc -n open-cluster-management-agent get pods # on the managed cluster

oc get managedcluster <name> -o jsonpath='{.status.conditions}' | python3 -m json.tool

oc -n open-cluster-management-agent logs deploy/klusterlet --tail=100
```

RED FLAG - DO NOT SAY THIS

Do not assume the hub connects into managed clusters. The direction of the connection is often the reason the architecture works at all.

EXPECT THIS FOLLOW-UP

A managed cluster shows Unknown on the hub but is serving traffic fine. Where do you look?

Q229 FOUNDATION

What is a ManagedClusterSet?

SAY THIS

A ManagedClusterSet is a grouping of managed clusters that acts as an RBAC and placement boundary. You put clusters into a set - production, development, a particular business unit, a region - and then bind that set to namespaces on the hub. Teams with access to those namespaces can place workloads and policies onto the clusters in their set and nothing else.

THINK OF IT LIKE

It is a region assigned to a regional manager. They can act across their region, and they cannot touch anyone else's shops.

WHY IT MATTERS IN PRODUCTION

This is how you delegate fleet operations without giving everyone the whole estate. Without cluster sets, the practical choice is between doing everything centrally - which does not scale - and giving broad hub access, which is unacceptable. With them, an application team can manage placement across their own clusters while the platform team retains cluster-wide governance.

STEP BY STEP

1. Define it as a grouping that serves as both an RBAC and a placement boundary.
2. Explain binding to hub namespaces and what that grants.
3. Give the delegation use case, which is the reason it exists.
4. Design the set structure to match your real organisational boundaries, not the infrastructure layout.

EVIDENCE YOU WOULD QUOTE

```
oc get managedclusterset ; oc get managedclustersetbinding -A

oc get managedcluster --show-labels | head

oc auth can-i --list -n <hub-ns> --as=<user>
```

RED FLAG - DO NOT SAY THIS

Do not give teams hub-wide access instead of using cluster sets. It is the difference between delegation and handing over the estate.

EXPECT THIS FOLLOW-UP

How would you let an application team deploy to their clusters but not to production?

Q230 INTERMEDIATE

How does Placement work?

SAY THIS

Placement selects which managed clusters a policy, application or upgrade applies to, based on labels and cluster claims rather than on a hard-coded list. You express intent - all production clusters in Europe, all clusters with GPU nodes, any three clusters from this set - and the result is a PlacementDecision naming the matching clusters. New clusters that match the criteria are picked up automatically.

THINK OF IT LIKE

It is a mailing list defined by a rule rather than by names. New joiners who meet the criteria start receiving the newsletter without anyone editing a list.

WHY IT MATTERS IN PRODUCTION

Label discipline is what determines whether this works. If clusters are labelled inconsistently - some with env=prod, some with environment=production - placement silently misses clusters, and a policy you believe is enforced everywhere is enforced on two thirds of the estate. Defining and enforcing a cluster labelling standard is unglamorous and it is the single highest-value thing you can do for fleet operations.

STEP BY STEP

1. Explain label and claim based selection producing a PlacementDecision.
2. Stress that new matching clusters are included automatically, which is the point.
3. Insist on a documented cluster labelling standard, applied at import.
4. Verify by reading PlacementDecisions rather than assuming the selector matched.

EVIDENCE YOU WOULD QUOTE

```
oc get placement,placementdecision -A

oc get placementdecision <name> -o jsonpath='{.status.decisions}' | python3 -m json.tool

oc get managedcluster --show-labels
```

RED FLAG - DO NOT SAY THIS

Do not hard-code cluster names in placement. The whole value is that the fleet can grow without editing every policy.

EXPECT THIS FOLLOW-UP

A policy is not applying to two clusters. How do you find out why?

Q231 INTERMEDIATE

How do inform and enforce governance modes differ?

SAY THIS

Inform detects and reports non-compliance without changing anything - you get visibility and a compliance status per cluster. Enforce actively remediates: it applies the desired configuration and keeps applying it. Inform is where you start, because it tells you what the estate actually looks like before you change anything, and because enforcing a policy against unknown drift can break workloads you did not know depended on it.

THINK OF IT LIKE

Inform is an inspection that leaves a report. Enforce is an inspector who fixes the wiring while they are there, whether or not you were using it.

WHY IT MATTERS IN PRODUCTION

The progression matters as much as the definition. Start in inform, look at the non-compliance report, understand every deviation - some will be legitimate exceptions you did not know about - then move to enforce on a subset, validate, and widen. Going straight to enforce across a fleet is how a well-intentioned security policy causes a multi-cluster outage.

STEP BY STEP

1. Define both modes and what each does on detecting non-compliance.
2. Always start in inform and read the resulting compliance report properly.
3. Investigate every deviation before enforcing, because some are legitimate.
4. Move to enforce progressively by placement, validating at each stage.

EVIDENCE YOU WOULD QUOTE

```
oc get policy -A -o custom-columns=NAME:.metadata.name,REMEDIATION:.spec.remediationAction,COMP
LIANCE:.status.compliant

oc get policy <name> -n <ns> -o jsonpath='{.status.status}' | python3 -m json.tool

Compliance report per cluster before and after enforcement
```

RED FLAG - DO NOT SAY THIS

Do not deploy a new policy in enforce mode across the fleet. You are making an untested change to every cluster simultaneously.

EXPECT THIS FOLLOW-UP

Inform mode shows 12 of 40 clusters non-compliant. What do you do next?

Q232 INTERMEDIATE

What is a PolicySet?

SAY THIS

A PolicySet groups related policies so they can be placed and reported on together - a security baseline, a compliance profile, a set of operational standards. Instead of attaching placement to twenty individual policies and reading twenty compliance statuses, you place the set once and get a single rolled-up view alongside the per-policy detail.

THINK OF IT LIKE

It is a building regulations package rather than a hundred separate rules. Adopt the package, get one certificate, and the details are still there if you need them.

WHY IT MATTERS IN PRODUCTION

The practical benefit is that it makes governance legible to people outside the platform team. "Cluster X is 94% compliant with the security baseline" is a sentence an auditor or a manager can act on, whereas twenty individual policy statuses is not. It also makes onboarding a new cluster a single placement change rather than twenty.

STEP BY STEP

1. Define it as a grouping of policies with shared placement and rolled-up status.
2. Organise sets around meaningful bundles - security baseline, compliance profile, operational standards.
3. Use the rolled-up view for reporting and the per-policy detail for remediation.
4. Onboard new clusters by adding them to the placement, not by attaching policies individually.

EVIDENCE YOU WOULD QUOTE

```
oc get policyset -A ; oc get policy -A | wc -l

oc get policyset <name> -o jsonpath='{.status}' | python3 -m json.tool

Compliance dashboard grouped by policy set and cluster
```

RED FLAG - DO NOT SAY THIS

Do not manage placement on dozens of individual policies. It becomes unmaintainable at exactly the scale where you need it.

EXPECT THIS FOLLOW-UP

How would you onboard a new cluster into your full governance baseline?

Q233 INTERMEDIATE

How does multi-cluster observability work in ACM?

SAY THIS

An observability add-on is deployed to managed clusters, which forwards a selected set of metrics to a central store on the hub backed by object storage. You then get cross-cluster dashboards and the ability to query the fleet, with drill-down into individual clusters. Alerting still evaluates locally on each cluster, so a cluster that loses connectivity to the hub keeps alerting rather than going silent.

THINK OF IT LIKE

It is regional sales figures sent to head office nightly. Head office sees the trend across the chain; each shop still has its own fire alarm.

WHY IT MATTERS IN PRODUCTION

The cost decision is the metric allow-list. Forwarding everything from fifty clusters produces a central store that is expensive to run and slow to query, and most of the series are never looked at. Curating the forwarded set to what fleet-level dashboards and reports actually use typically cuts volume dramatically with no loss of usefulness, and it is worth revisiting as dashboards change.

STEP BY STEP

1. Describe the add-on, forwarding, and central storage backed by object storage.
2. Note that alert evaluation stays local so partitions degrade gracefully.
3. Curate the forwarded metric list against what fleet dashboards actually query.
4. Size and monitor the object storage, and review cost against value periodically.

EVIDENCE YOU WOULD QUOTE

```
oc get multiclusterobservability -A -o yaml | head -30

oc -n open-cluster-management-observability get pods

Central store ingestion rate and storage growth per cluster
```

RED FLAG - DO NOT SAY THIS

Do not forward every metric from every cluster. The cost curve is steeper than anyone expects and most of it is never queried.

EXPECT THIS FOLLOW-UP

Central observability storage is growing 40% a month. What do you look at first?

Q234 SENIOR

How would you upgrade a fleet of clusters through ACM?

SAY THIS

In waves defined by placement, not all at once. Development clusters first, then a canary production cluster, then production in groups, with a soak period and validation criteria between waves. Each wave has explicit gates - cluster operators healthy, application SLOs steady, no new alerts - and a documented decision to proceed. ACM handles the mechanics; the discipline is in the wave structure and the gates.

THINK OF IT LIKE

It is rolling out a new aircraft procedure across a fleet. One route first, observed, then a region, then everywhere - never all at once regardless of how confident you are.

WHY IT MATTERS IN PRODUCTION

The gate that matters most is the soak period, because the problems that only appear at scale or over time - a memory leak, a certificate that rotates weekly, a batch job that runs monthly - are invisible in the first hour. Defining the soak duration and the specific signals you will watch turns "we upgraded and it seemed fine" into a repeatable process with evidence.

STEP BY STEP

1. Define waves by placement labels, starting with non-production and a production canary.
2. Set explicit gate criteria and a soak period between waves.
3. Verify operator channel compatibility across the fleet before starting.
4. Track progress per cluster from the hub, and hold the wave if any gate fails.

EVIDENCE YOU WOULD QUOTE

```
oc get managedcluster -o custom-columns=NAME:.metadata.name,VERSION:.status.version.kubernetes

oc get clustercurator -A # upgrade orchestration state

Wave gate evidence: operator health, SLO stability, alert delta per cluster
```

RED FLAG - DO NOT SAY THIS

Do not upgrade the whole fleet in one operation because ACM makes it possible. Capability is not a reason.

EXPECT THIS FOLLOW-UP

Wave two shows a regression in one application. What do you do about waves three to six?

Q235 SENIOR

How do ApplicationSets support fleet GitOps?

SAY THIS

An ApplicationSet generates Argo CD Applications from a template plus a generator - a cluster list, a placement decision, a directory structure in Git. So one definition produces a deployment for every matching cluster, with per-cluster values substituted. Add a new cluster with the right labels and it gets the application automatically; remove it and the application goes away.

THINK OF IT LIKE

It is a mail merge for deployments. One letter, one list, and the list can change without rewriting the letter.

WHY IT MATTERS IN PRODUCTION

This is what makes fleet GitOps sustainable, because the alternative - one Application per cluster per workload - grows multiplicatively and nobody keeps it in sync. The care needed is around the generator and the prune behaviour: a mistake in the generator can remove applications from clusters that should have kept them, so changes to ApplicationSets deserve more review than changes to a single application.

STEP BY STEP

1. Explain generators and templating producing per-cluster Applications.
2. Connect it to ACM placement so the fleet and GitOps share one source of truth about targeting.
3. Handle per-cluster differences through values rather than forked manifests.
4. Review generator changes carefully and understand prune behaviour before applying.

EVIDENCE YOU WOULD QUOTE

```
oc get applicationset -A ; oc get application -A | wc -l

oc get applicationset <name> -o jsonpath='{.spec.generators}' | python3 -m json.tool

Argo CD application list grouped by destination cluster
```

RED FLAG - DO NOT SAY THIS

Do not maintain one Application per cluster by hand. It works for three clusters and collapses at thirty.

EXPECT THIS FOLLOW-UP

A generator change removes applications from five clusters. How do you prevent that?

Q236 SENIOR

When would you use Submariner?

SAY THIS

When workloads in different clusters need direct pod-to-pod or service-to-service connectivity across the cluster boundary - a stateful system replicating between clusters, or a service in one cluster consuming another's internal service without going out through ingress. Submariner builds the cross-cluster network and provides service discovery across them. If the requirement is just north-south traffic through public endpoints, you do not need it.

THINK OF IT LIKE

It is a private tunnel between two office buildings. Worth building if people cross constantly; unnecessary if they occasionally send an email.

WHY IT MATTERS IN PRODUCTION

The prerequisites are the part that decides feasibility: non-overlapping pod and service CIDRs across the clusters, and network paths that permit the tunnels. Overlapping CIDRs is extremely common in estates that grew organically, and discovering it late turns a connectivity project into a re-addressing project. Ask about CIDR planning before agreeing to the design.

STEP BY STEP

1. Establish the requirement: genuine east-west cross-cluster traffic, not north-south.
2. Check the prerequisites - non-overlapping CIDRs and permitted network paths - before committing.
3. Consider simpler alternatives such as exposed endpoints or a service mesh gateway.
4. Plan for the operational cost: another network layer to monitor, secure and upgrade.

EVIDENCE YOU WOULD QUOTE

```
oc get submariner -A ; subctl show all

oc get network.config cluster -o jsonpath='{.status.clusterNetwork}' per cluster

Cross-cluster connectivity test results
```

RED FLAG - DO NOT SAY THIS

Do not propose Submariner without checking CIDR overlap first. It is the constraint that most often kills the design.

EXPECT THIS FOLLOW-UP

Two clusters have overlapping pod CIDRs. What are your options?

Q237 ARCHITECT

How would you decide how many clusters an organisation should have?

SAY THIS

From boundaries that genuinely need separating, then from operational capacity. Real boundaries are regulatory or data residency requirements, blast radius for critical services, environment separation, network or latency constraints, and untrusted tenancy. Everything else is better served by namespaces in a shared cluster, because every cluster carries a fixed cost in control plane, upgrades, monitoring and human attention.

THINK OF IT LIKE

It is deciding how many offices to open. Each one needs a lease, a manager and a fire certificate, so you open one where the business genuinely needs presence - not because a department asked.

WHY IT MATTERS IN PRODUCTION

Both failure modes are expensive. Too few clusters means one incident affects everyone and upgrades become impossible to schedule because there is no window that suits every tenant. Too many means the platform team spends its time on cluster maintenance rather than capability, and consistency degrades because nobody can keep forty snowflakes aligned. The honest answer names both and states the criteria you would use.

STEP BY STEP

1. List the boundaries that genuinely require separation and test each request against them.
2. Estimate the fixed cost per cluster - control plane, upgrades, monitoring, attention.
3. Default to namespaces within a shared cluster unless a real boundary applies.
4. Ensure whatever number you choose is manageable by fleet automation, not by people.

EVIDENCE YOU WOULD QUOTE

```
Cluster inventory with purpose, tenant, criticality and justification

Platform team effort per cluster: upgrade hours, incident hours, maintenance hours

Utilisation per cluster - a fleet of half-empty clusters is a design smell
```

RED FLAG - DO NOT SAY THIS

Do not give every team their own cluster. You have distributed the platform team's attention until none of the clusters get enough of it.

EXPECT THIS FOLLOW-UP

A team demands their own cluster for isolation. How do you evaluate the request?

Q238 ARCHITECT

How do you keep configuration consistent across a fleet while allowing exceptions?

SAY THIS

One source of truth in Git, delivered by placement so targeting is label-driven, with policies in inform mode giving continuous compliance reporting. Exceptions exist - they always do - but they are declared, not discovered: a documented deviation with an owner, a justification and a review date, expressed as a label that changes placement rather than as a manual change on the cluster.

THINK OF IT LIKE

It is a building standard with a register of approved variations. The variation is fine; the undocumented variation is what fails the inspection.

WHY IT MATTERS IN PRODUCTION

The reason to make exceptions first-class is that suppressing them does not work. Teams have genuine needs, and if the process offers no legitimate route they make the change manually and nobody records it. A visible exception register with expiry dates turns invisible drift into a managed backlog, and it gives you a number to report - how many exceptions exist and how old they are - which is what drives them down over time.

STEP BY STEP

1. Define the baseline in Git and deliver it by label-driven placement.
2. Run compliance policies in inform mode continuously to see reality, not intent.
3. Make exceptions declared objects with owner, justification and expiry.
4. Report exception count and age as a standing metric, and drive it down deliberately.

EVIDENCE YOU WOULD QUOTE

```
Compliance percentage per policy set across the fleet

Exception register with owner, justification and review date

oc get managedcluster --show-labels # exception labels visible in placement
```

RED FLAG - DO NOT SAY THIS

Do not pretend exceptions do not exist. Undeclared exceptions are just drift with better manners.

EXPECT THIS FOLLOW-UP

You have 30 exceptions, half of them over a year old. What is your plan?

PART 17 · Q239-Q250 · 12 QUESTIONS

Architecture, influence and the behavioural round

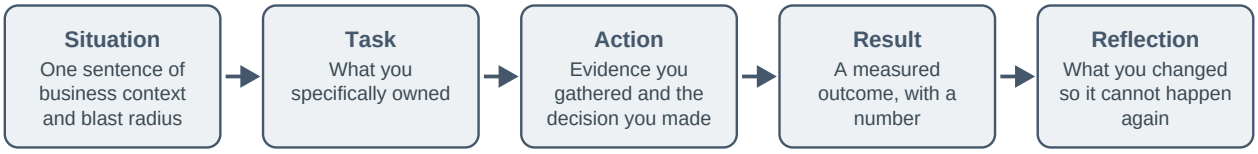
The half of the interview that is not about OpenShift, and often decides the outcome

Technical rounds decide whether you could do the job. This part decides whether people want to work with you while you do it. Senior platform roles are unusual in that most of the work is influence: persuading a team to adopt a standard, telling an executive that the date is at risk, explaining a trade-off to someone who will pay for it, and running an incident without making anyone defensive. Interviewers assess this through stories, so the practical skill is having three or four real ones ready, structured, honest, and with a number in them.

Level mix - Foundation: 2 · Intermediate: 3 · Senior: 4 · Architect: 3

STAR-R: how to tell a production story in 90 seconds

Behavioural questions are scored on structure. Without the final R most candidates ramble; with it they sound like someone who has owned a platform.



Q239 FOUNDATION

How do you structure an answer to a behavioural question?

SAY THIS

Situation, Task, Action, Result, Reflection. One sentence of context and blast radius. What I specifically owned - not what the team did. What I actually did, including the evidence I gathered and the decision I made. A measured result with a number. And what I changed afterwards so it could not happen again. Ninety seconds, no rambling, and the reflection is the part that makes it sound senior.

THINK OF IT LIKE

It is a news report rather than a diary entry. What happened, what was done, what the outcome was, what it means - in that order, and finite.

WHY IT MATTERS IN PRODUCTION

Most candidates lose points here not through weak stories but through unstructured ones that take four minutes and never reach the outcome. Having three or four stories prepared and rehearsed - an incident, a difficult trade-off, a disagreement handled well, a failure you learned from - covers the majority of behavioural questions, because most of them are variations on those themes.

STEP BY STEP

1. Set the scene in one sentence, including who was affected.
2. State your specific ownership, using I rather than we.
3. Describe the actions and the evidence behind the key decision.
4. Give a measured result and finish with the change you made so it could not recur.

EVIDENCE YOU WOULD QUOTE

```
Three or four prepared stories, each timed at 90 seconds

A concrete metric in each: minutes saved, incidents avoided, percentage improvement

The prevention action for each, stated as something that actually shipped
```

RED FLAG - DO NOT SAY THIS

Do not tell a story with no measured outcome. "It went well" is not a result and the interviewer will not ask twice.

EXPECT THIS FOLLOW-UP

Tell me about a time your fix did not work. What did you do next?

Q240 FOUNDATION

What would you ask the interviewer about the environment?

SAY THIS

Which OpenShift version and infrastructure provider, and how many clusters. Who owns upgrades and how often they happen. What the current incident volume looks like and the biggest recurring cause. How much is in Git today. What the on-call rota is and how often it fires. And what the biggest platform problem is that they would want solved in the first six months. Those answers tell you what the job actually is.

THINK OF IT LIKE

It is asking to see the kitchen before accepting the head chef job. The menu tells you the ambition; the kitchen tells you the reality.

WHY IT MATTERS IN PRODUCTION

Asking good questions is genuinely part of the assessment, because it shows what you consider important. Questions about upgrade cadence, GitOps coverage and incident themes signal that you think about the operational reality rather than the technology stack. They also protect you: a role where nothing is in Git, upgrades have not happened in eighteen months and on-call fires nightly is a specific kind of job, and you should know that before you accept it.

STEP BY STEP

1. Ask about scale and versions to calibrate everything else.
2. Ask about change and upgrade practice - cadence, ownership, automation coverage.
3. Ask about incident volume, themes and the on-call experience.
4. Ask what success looks like in six months, and listen for whether they have an answer.

EVIDENCE YOU WOULD QUOTE

```
Their answers on cluster count, version spread and upgrade cadence

Their answer on GitOps and automation coverage

Their answer on the biggest current platform problem
```

RED FLAG - DO NOT SAY THIS

Do not say you have no questions. It reads as either disinterest or a lack of experience in judging environments.

EXPECT THIS FOLLOW-UP

Which of those answers would most change how you approached the first month?

Q241 INTERMEDIATE

How do you conduct an architecture review?

SAY THIS

I start with requirements and constraints rather than the proposed solution, because half the disagreements in a review come from unstated assumptions. Then I look for the failure modes - what happens when each dependency is unavailable - the operational cost of running it, the security and data boundaries, and the recovery story. I ask what alternatives were considered and why they were rejected, and I make sure the decision and its reasoning are written down.

THINK OF IT LIKE

It is a building inspection before the concrete is poured. Cheap now, extremely expensive after the fact, and the questions are always the same ones.

WHY IT MATTERS IN PRODUCTION

The most valuable question in the room is usually "what happens when this dependency is down?", because it exposes the difference between a design that has been thought through and one that only describes the happy path. The second most valuable is "who operates this at 3am and what do they do?", which surfaces operational cost that architecture diagrams hide entirely.

STEP BY STEP

1. Establish requirements, constraints and non-functional targets before discussing the solution.
2. Walk the failure modes dependency by dependency, including partial failures.
3. Assess operational cost, security boundaries and the recovery story explicitly.
4. Ask what alternatives were rejected and why, and record the decision as an ADR.

EVIDENCE YOU WOULD QUOTE

```
Architecture decision records with alternatives and rejection reasoning

Failure mode analysis per dependency

Operational cost assessment: who runs it, what they do when it breaks
```

RED FLAG - DO NOT SAY THIS

Do not review a design by critiquing technology choices. The questions that matter are about failure, cost and recovery.

EXPECT THIS FOLLOW-UP

A design has no answer for what happens when its database is unavailable. What do you do?

Q242 INTERMEDIATE

What is an architecture decision record and why keep them?

SAY THIS

An ADR is a short document capturing one decision: the context and constraints at the time, the options considered, the decision taken, and the consequences accepted. They are kept because eighteen months later somebody will ask why the platform works this way, and without an ADR the answer is either speculation or an argument. They also make it possible to revisit a decision honestly when the constraints change.

THINK OF IT LIKE

It is the minutes of the meeting where the decision was made. Nobody reads them until somebody disputes what was agreed, and then they are worth everything.

WHY IT MATTERS IN PRODUCTION

The property that makes ADRs useful is that they are immutable and superseded rather than edited. A decision that was correct given 2023's constraints is not wrong just because the constraints changed, and recording it that way removes the blame from revisiting it. Kept in the same repository as the code they describe, they also get reviewed like code.

STEP BY STEP

1. Keep them short - context, options, decision, consequences - and dated.
2. Store them in the repository they relate to, reviewed through pull requests.
3. Supersede rather than edit, so the history of thinking is preserved.
4. Reference them in design reviews and revisit when constraints materially change.

EVIDENCE YOU WOULD QUOTE

```
ADR directory with numbered, dated records

Superseded records linked to their replacements

Design review references to existing ADRs
```

RED FLAG - DO NOT SAY THIS

Do not edit an old ADR to match current thinking. You have destroyed the record of why the original decision made sense.

EXPECT THIS FOLLOW-UP

A decision from two years ago is now wrong. How do you handle it?

Q243 INTERMEDIATE

How do you work with application teams to improve reliability?

SAY THIS

By making the reliable option the easy one rather than by writing standards and hoping. That means templates that already contain probes, requests, PDBs and spread constraints; dashboards and alerts provided by default; and shared review of their SLOs and incidents so the conversation is about their service rather than about platform rules. Where a team resists, I show them their own incident data rather than arguing from principle.

THINK OF IT LIKE

It is designing the path where people already walk. Fencing the grass makes enemies; paving the desire line makes converts.

WHY IT MATTERS IN PRODUCTION

The dynamic to avoid is the platform team as compliance function, which produces adversarial relationships and workarounds. Teams adopt standards when the standard obviously saves them work or prevents an incident they remember. That is why incident data is the most persuasive artefact available - a team that was paged three times last quarter for something a readiness probe would have prevented needs no further argument.

STEP BY STEP

1. Provide working defaults in templates rather than requirements in documents.
2. Give teams their own dashboards, alerts and SLOs so reliability is visible to them.

3 Review incidents jointly and let the data make the argument.
4 Track adoption and off-path incident rate to prioritise where to help next.

EVIDENCE YOU WOULD QUOTE

```
Golden path adoption rate across services

Incidents traced to missing platform practices, per team

Joint incident reviews held and actions completed
```

RED FLAG - DO NOT SAY THIS

Do not position the platform team as the team that says no. You will be routed around, and then you will own the consequences anyway.

EXPECT THIS FOLLOW-UP

A team refuses to add probes because they are 'confident in their code'. What do you do?

Q244 SENIOR

Tell me about a production incident you handled.

SAY THIS

Use the structure and be specific. One sentence on impact and scale. What you owned. The evidence you gathered and how you narrowed it down - naming the signal that turned the corner. The mitigation you chose and why it was the smallest safe one. The measured outcome: how long, how many users, what recovered. And the prevention that shipped afterwards, with evidence it worked.

THINK OF IT LIKE

It is a flight incident report, not a war story. Facts, decisions, outcome, and what changed in the procedure afterwards.

WHY IT MATTERS IN PRODUCTION

What distinguishes a strong answer is the moment of narrowing - the specific piece of evidence that took you from three hypotheses to one. Candidates who tell this well sound like they were there; candidates who describe a sequence of actions without saying what each one ruled out sound like they are reciting a runbook. Mentioning something you got wrong along the way makes the whole story more credible, not less.

STEP BY STEP

1 Open with impact and scale in one sentence, including duration.
2 Describe the evidence path and name the signal that narrowed it to one cause.
3 Explain the mitigation choice in terms of blast radius, and the validation you did.
4 Close with the measured outcome and the prevention that shipped afterwards.

EVIDENCE YOU WOULD QUOTE

```
Incident timeline with timestamps and decisions

The specific metric, log line or condition that narrowed the diagnosis

Post-incident action and the measured effect since
```

RED FLAG - DO NOT SAY THIS

Do not tell an incident story where everything you tried worked first time. It is not believable and it wastes the chance to show judgement.

EXPECT THIS FOLLOW-UP

What would you do differently if the same incident happened tomorrow?

Q245 SENIOR

How do you handle disagreeing with a senior stakeholder?

SAY THIS

I separate the decision from the evidence. I make sure I understand their constraint - often the disagreement is about a requirement I did not know existed - then present the risk in terms of impact rather than technology, propose an alternative that meets their actual constraint, and if the decision still goes the other way, I document the risk, accept the decision and support it properly. Escalating past someone should be rare and reserved for genuine safety or compliance issues.

THINK OF IT LIKE

It is a second opinion, not a mutiny. You give your professional assessment clearly, in writing, and then you help with the plan that was chosen.

WHY IT MATTERS IN PRODUCTION

The behaviour interviewers are checking for is whether you can disagree without becoming a problem. Someone who cannot let a decision go is expensive to work with; someone who never pushes back is not doing the job. The written record is the mature middle - it protects the organisation, gives the decision-maker the information, and lets everyone move on without the disagreement resurfacing every week.

STEP BY STEP

1. Understand their constraint before arguing the technical position.
2. Frame the risk in business impact terms, with a probability and a consequence.
3. Offer an alternative that satisfies their constraint, not just your preference.
4. Document the accepted risk, support the decision, and revisit it when evidence changes.

EVIDENCE YOU WOULD QUOTE

```
Written risk assessment with impact, likelihood and mitigation options

Decision record noting the accepted risk and its owner

Follow-up when evidence changed the picture
```

RED FLAG - DO NOT SAY THIS

Do not describe going over someone's head as your normal approach. It answers a different question than the one being asked.

EXPECT THIS FOLLOW-UP

They accepted the risk and it materialised. How do you handle that conversation?

Q246 SENIOR

How do you explain technical risk to a non-technical audience?

SAY THIS

In their terms, not mine. What could happen, to whom, how likely it is, what it would cost, and what it would take to reduce it - with numbers where I have them and honest uncertainty where I do not. I avoid component names entirely and talk about service impact, customers affected and recovery time. Then I offer options with costs so the decision is theirs to make rather than mine to justify.

THINK OF IT LIKE

It is a doctor explaining a procedure. Not the biochemistry - the odds, the recovery time, and what happens if you do nothing.

WHY IT MATTERS IN PRODUCTION

The mistake that undermines otherwise strong engineers is explaining the mechanism instead of the consequence. An executive does not need to know what etcd quorum is; they need to know that a particular design means a data centre failure takes the service down for four hours, and that a different design costs a certain amount and reduces that to ten minutes. Presented that way, the decision is straightforward and it is theirs.

STEP BY STEP

1. Describe impact in customer and business terms, with duration and scope.
2. Give likelihood honestly, including your uncertainty about it.
3. Present two or three costed options rather than a single recommendation.
4. Record the decision and who made it, and revisit it when circumstances change.

EVIDENCE YOU WOULD QUOTE

```
Risk register entries written in business impact language

Costed options presented, with the decision recorded

Post-decision review when the situation changed
```

RED FLAG - DO NOT SAY THIS

Do not explain the technology to justify the risk. The audience will remember confusion, not concern.

EXPECT THIS FOLLOW-UP

They ask for a percentage likelihood you do not have. What do you say?

Q247 SENIOR

How do you grow the capability of a platform team?

SAY THIS

By spreading knowledge deliberately rather than hoping it diffuses. That means pairing on incidents so the same person is not always the one who knows, runbooks written by whoever learned the thing most recently, game days where someone unfamiliar runs the recovery, and rotating ownership of areas so there is no single point of human failure. I also make space for people to do the boring reliability work, because that is where most learning actually happens.

THINK OF IT LIKE

It is cross-training a kitchen brigade. If only one person can work the grill, the restaurant closes when they are ill - and they never get a holiday.

WHY IT MATTERS IN PRODUCTION

The measurable version is bus factor per critical system, and it is uncomfortable to look at honestly in most teams. Game days are the most effective intervention because they surface the gap immediately: the person who did not write the runbook discovers what is missing from it in twenty minutes, and the runbook improves in a way that no review process achieves.

STEP BY STEP

1. Measure bus factor per critical system and make it visible to the team.
2. Pair on incidents and rotate ownership so knowledge does not concentrate.
3. Run game days where an unfamiliar person executes the recovery, and fix what they hit.
4. Protect time for reliability work, and treat runbook improvement as real output.

EVIDENCE YOU WOULD QUOTE

```
Bus factor assessment per critical system

Game day schedule, participants and gaps found

Runbook update frequency and who authored the changes
```

RED FLAG - DO NOT SAY THIS

Do not let one person own the recovery procedure for a critical system. It is a risk to the business and unfair to them.

EXPECT THIS FOLLOW-UP

One engineer is the only person who understands your storage layer. What do you do?

Q248 ARCHITECT

How would you approach modernising a legacy platform?

SAY THIS

Incrementally, and starting with the thing that hurts most. First an honest assessment - what runs where, what the actual pain is, and what constraints are real versus assumed. Then pick a target state and a migration path that delivers value in stages, with a reference workload proving each stage before it becomes policy. Big-bang migrations fail because they defer all the learning to the end, when the cost of being wrong is highest.

THINK OF IT LIKE

It is renovating an occupied building. One floor at a time, tenants stay housed, and you learn how the plumbing actually works before you commit to the whole block.

WHY IT MATTERS IN PRODUCTION

The organisational half matters as much as the technical. Modernisation projects stall when application teams are asked to change without getting anything in return, so sequencing matters: deliver a paved path that is genuinely better - faster builds, better visibility, less on-call - and migration becomes something teams want rather than something imposed. Measuring and publishing progress keeps sponsorship alive.

STEP BY STEP

1. Assess honestly: inventory, pain points, real constraints, and current cost.
2. Define a target state and a staged path where each stage delivers standalone value.
3. Prove each stage with a reference workload before making it policy.
4. Make the new path clearly better for teams, and publish migration progress and outcomes.

EVIDENCE YOU WOULD QUOTE

```
Application inventory with migration complexity and business criticality

Reference workload migrated end to end, with measured before and after

Migration progress and benefit metrics published regularly
```

RED FLAG - DO NOT SAY THIS

Do not propose a big-bang migration. It concentrates all the risk at the point where you understand the least.

EXPECT THIS FOLLOW-UP

Six months in, three teams have not started. How do you unblock it?

Q249 ARCHITECT

What would your first 90 days look like?

SAY THIS

First thirty days, learn: architecture, versions, ownership, incident history, change process, and who the platform's customers actually are - while delivering a couple of small visible wins so I am useful rather than just observing. By sixty days, baseline: cluster health, risk register, capacity, upgrade posture, backup and recovery status, tested. By ninety, a prioritised roadmap with measurable outcomes, agreed with the people who will have to live with it.

THINK OF IT LIKE

It is a new head chef learning the kitchen before changing the menu - but still cooking service every night while they learn.

WHY IT MATTERS IN PRODUCTION

The balance being assessed is between listening and delivering. Someone who proposes a rewrite in week two has not understood the constraints; someone who delivers nothing for three months has not built any credibility to spend later. Small early wins - a recurring alert silenced properly, a manual task automated, a stale runbook fixed - buy the goodwill you need for the larger changes.

STEP BY STEP

1. Days 1-30: learn the environment and the people, and ship two or three small visible wins.
2. Days 31-60: baseline health, risk, capacity and recovery, and actually test the recovery.
3. Days 61-90: propose a prioritised roadmap with measurable outcomes and named owners.
4. Throughout: write down what you learn, because the newcomer's view is only available once.

EVIDENCE YOU WOULD QUOTE

```
Stakeholder map and platform inventory produced in the first month

Risk register and tested recovery results by day sixty

Roadmap with measurable outcomes agreed by day ninety
```

RED FLAG - DO NOT SAY THIS

Do not promise a platform rewrite before understanding the constraints. It signals confidence rather than judgement, and interviewers can tell the difference.

EXPECT THIS FOLLOW-UP

What would be your first small win, based on what you know about this role?

Q250 ARCHITECT

How do you prioritise platform work against competing demands?

SAY THIS

Against a small number of stated outcomes rather than by who asks loudest. Reliability risk, security exposure, toil reduction and enablement value are the four lenses I use, with anything that is currently causing incidents taking precedence. I keep a visible backlog with the reasoning attached, reserve capacity for unplanned work and for reducing toil, and review priorities with stakeholders on a fixed cadence so trade-offs are explicit.

THINK OF IT LIKE

It is a hospital waiting list. Triage by severity, not by who is most insistent - and keep some capacity free for emergencies, because there will be emergencies.

WHY IT MATTERS IN PRODUCTION

The reserved capacity is the part that keeps platform teams from being permanently reactive. A team that plans to be one hundred percent allocated has no room for the incident that will certainly arrive, so every incident pushes planned work later and the toil that caused the incident never gets fixed. Explicitly reserving a proportion for unplanned work and improvement breaks that cycle, and it is defensible with data once you measure where time actually goes.

STEP BY STEP

1. Publish the prioritisation lenses and apply them consistently and visibly.
2. Give anything currently causing incidents precedence over new capability.
3. Reserve explicit capacity for unplanned work and for toil reduction.
4. Review with stakeholders on a fixed cadence, showing what was traded off and why.

EVIDENCE YOU WOULD QUOTE

```
Visible backlog with priority reasoning per item

Time allocation: planned, unplanned, toil reduction, measured over quarters

Stakeholder review cadence and recorded trade-off decisions
```

RED FLAG - DO NOT SAY THIS

Do not plan for full allocation. Incidents are not exceptional events and a plan that assumes none is a plan that always slips.

EXPECT THIS FOLLOW-UP

Two directors both want their work first and neither will yield. What do you do?

The last hour

You will not learn anything new in the last hour, so use it to protect what you already know. Say these six sentences out loud until they sound unrehearsed:

1. "Let me start with impact and scope, then the evidence I would collect."
2. "The owning controller here is X, so that is where I would read conditions first."
3. "The smallest safe action is Y, because it limits blast radius to one pool."
4. "I would validate against the customer-visible symptom, not just resource status."
5. "The trade-off is A versus B; I would choose A because of this constraint."
6. "To prevent recurrence I would add this alert and this guardrail."

Three habits that reliably lose offers

- Restarting things before reading anything. It sometimes works, and it always sounds junior.
- Answering a design question with a product name instead of a requirement and a trade-off.
- Claiming a backup, a policy or a failover works without ever having tested the restore.

Where to check current behaviour

Product behaviour moves. Before an interview, spot-check anything version-sensitive against the Red Hat OpenShift Container Platform documentation, the Red Hat Advanced Cluster Management documentation, the Red Hat Ansible Automation Platform documentation and the upstream Kubernetes concepts pages. If you are unsure in the room, say which release you are describing and offer to confirm - that is how senior engineers actually talk.